AWS · CLOUD · DEVOPS PROJECT

Deploy a Three-Tier Web Application on AWS

Highly Available · Highly Secured · Highly Scalable · Fault Tolerant

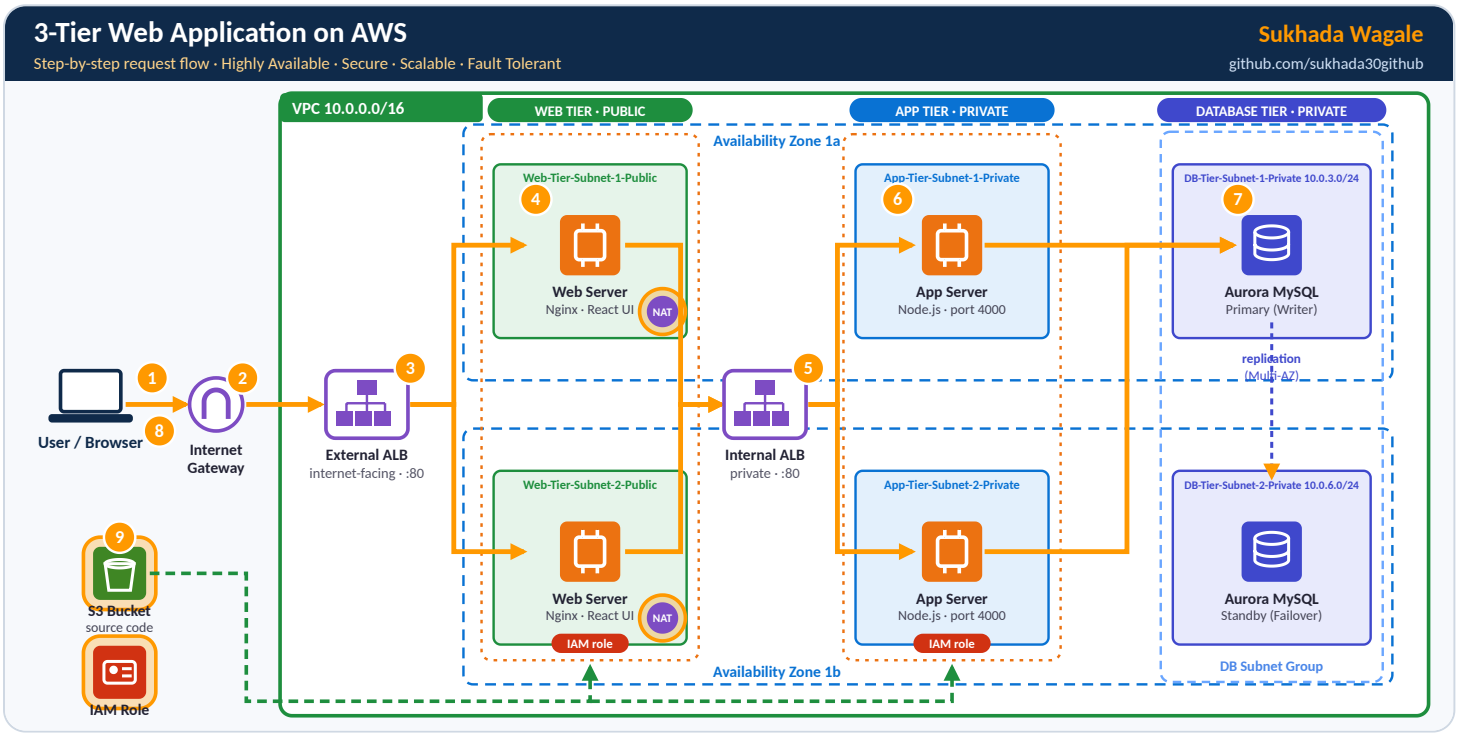
Sukhada Wagale

Associate Cloud Engineer · AWS Certified Cloud Practitioner

GitHub: github.com/sukhada30github

LinkedIn: linkedin.com/in/sukhada-wagale-0961002b0

Architecture at a glance · follow the numbered flow (animated version: 3tier_architecture_flow.gif)



1. **User sends a request**
   A user opens the app URL in a browser (HTTP, port 80).
2. **Internet Gateway**
   The VPC's front door; the Web-Tier route table sends 0.0.0.0/0 to the IGW.
3. **External ALB**
   Internet-facing load balancer spreads requests across both AZs.
4. **Web tier (Nginx)**
   Public-subnet EC2 servers in an Auto Scaling Group serve the UI.
5. **Internal ALB**
   A private load balancer routes API calls to the app tier.
6. **App tier (Node.js)**
   Private-subnet EC2 servers run the logic on port 4000.
7. **Database tier (Aurora)**
   Reads/writes on port 3306; Multi-AZ standby for failover.
8. **Response returns**
   Data flows back DB → App → ALB → Web → ALB → IGW → user.

Project at a Glance

REGION **Mumbai (ap-south-1)**	VPC CIDR **10.0.0.0/16**	AVAILABILITY ZONES **2 (1a and 1b)**
SUBNETS **6 (2 public, 4 private)**	NAT GATEWAYS **2 (one per AZ)**	ROUTE TABLES **3**
SECURITY GROUPS **5 (chained tier by tier)**	DATABASE **Amazon Aurora (MySQL compatible)**	WEB TIER **Nginx on EC2 + Auto Scaling**
APP TIER **Node.js 16 + PM2, port 4000**	LOAD BALANCERS **1 external + 1 internal ALB**	AWS COMPONENTS **14 services**

Table of Contents

Introduction 3
Project Overview 3
Project Implementation 5
1. Create VPC and Subnets 5
2. Create Internet Gateway, NAT Gateways and Route Tables 10
3. Create Security Groups 22
4. Create Database Subnet Group and RDS 26
5. Create IAM Role 32
6. Create an S3 Bucket and Upload the Source Code 34
7. Create Test App Server and Set It Up 39
8. Create AMI of the App Server 48
9. Create Target Group, Launch Template and Internal Load Balancer for App Tier 49
10. Create Test Web Server and Set It Up 57
11. Create AMI of the Web Server 62
12. Create Target Group, Launch Template and External Load Balancer for Web Tier 63
13. Create Auto Scaling Groups 72
14. Access the Application 82
15. Cleaning the Infrastructure 84

Prepared by Sukhada Wagale

Introduction

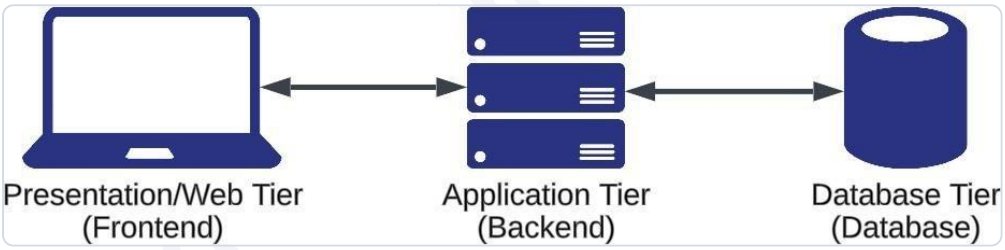
The three-tier web application architecture is a robust and scalable design pattern that separates an application into three distinct layers: the presentation/web tier, the application tier, and the database tier. This architecture enhances security, manageability, and flexibility by isolating each layer's responsibilities.

In this documentation, let's see how to deploy a three-tier web application on AWS.

A three-tier architecture is a common architectural pattern used in software development and deployment, particularly in cloud environments like AWS. As I mentioned before, it separates an application into three main layers or tiers: the presentation/Web tier, the application tier, and the database tier.

- **Presentation/Web Tier (Frontend)** - This is the user interface layer where users interact with the application. It's responsible for presenting information to the user and collecting user inputs.
- **Application Tier (Backend)** - This is the logic layer where the actual application logic, processing, and control flow take place. It processes user inputs, performs calculations, and makes decisions.
- **Database Tier (Database)** - This layer is responsible for data storage and management. It stores, retrieves, and manages data that the application tier needs.

Users interact with the application via the presentation tier. These requests are sent to the application tier, where the logic processes the request. The application tier interacts with the database tier to retrieve or store data. The processed data or output is sent back to the presentation tier, which displays the results to the user.



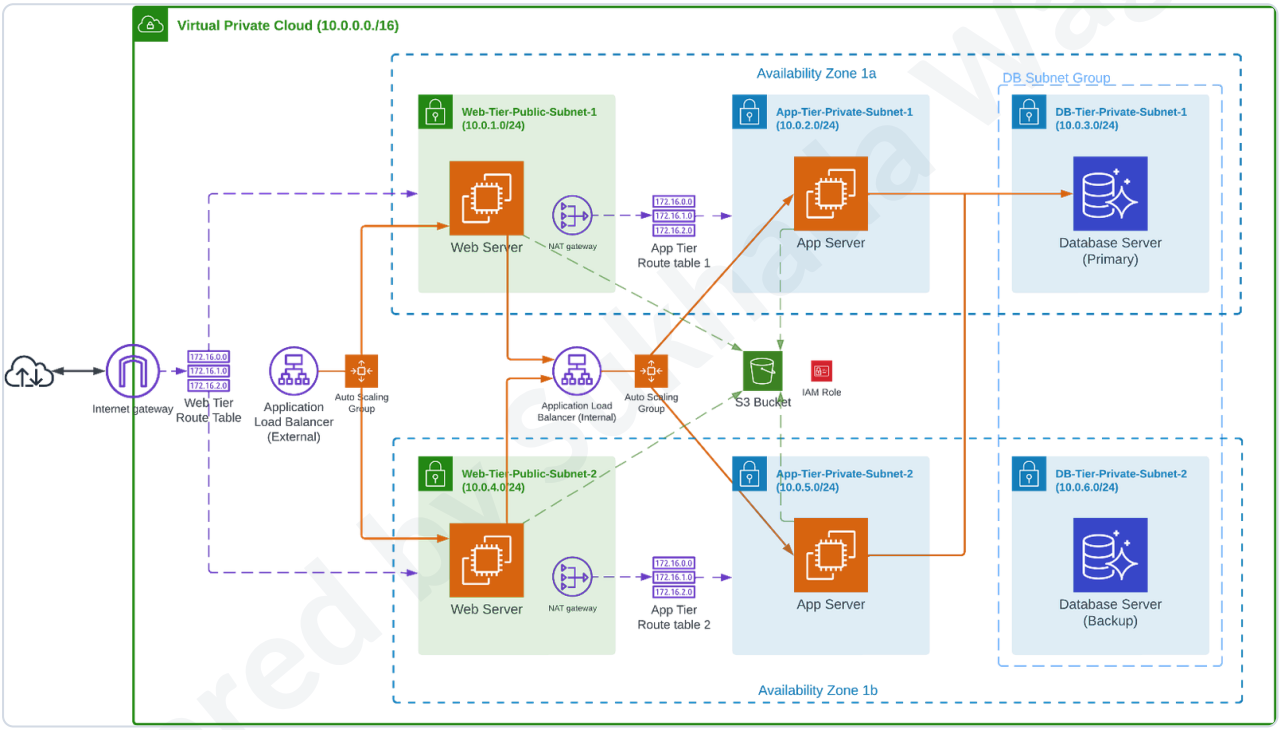
Project Overview

In this project we are implementing a **Highly Available, Highly Secured, Highly Scalable** and **Fault tolerant** three tier application on AWS.

Throughout this project, we have to interact with the following components on AWS.

- Amazon Elastic Compute Cloud (EC2)
- Amazon Simple Storage Service (S3)
- Virtual Private Cloud (VPC)
- Auto Scaling Group (ASG)
- Application Load Balancer (ALB)
- Identity and Access Management (IAM)
- Amazon Relational Database Service (RDS)
- Amazon Machine Image (AMI)
- Launch Templates
- Target Groups
- Internet Gateway (IGW)
- NAT Gateway
- Route Table
- Security Groups

The following diagram shows the overview of the project. It shows connection between each component and how they are distributed in two availability zones Let's have a look at architecture diagram



As you can see, we use public subnets for web tier because it should be open to the internet to let users interact with the application. App tier and Database tier should be secured. Therefore, private subnets are assigned for them. Resources are distributed in two availability zones, and all are included in our custom VPC. Observe how CIDR blocks are assigned for each subnet.

An S3 bucket is used to store the source code of the application. It makes the server configuration process easier.

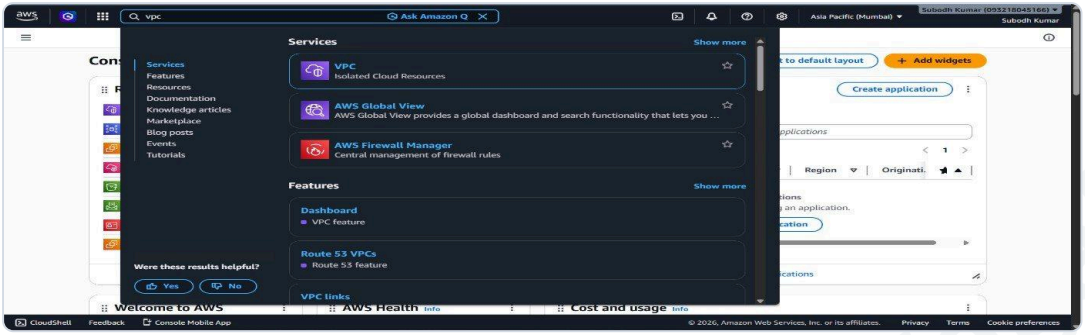
Source code: https://github.com/sukhada30github/Three-Tier-Project.git

Let's get started with the implementation.

Prepared by Sukhada Wagale

Project Implementation

Go to the **AWS Console** and **choose a region** you like (I chose the Mumbai region) and create the following services as shown in the figures. To reach each service, use the search bar and search for it.

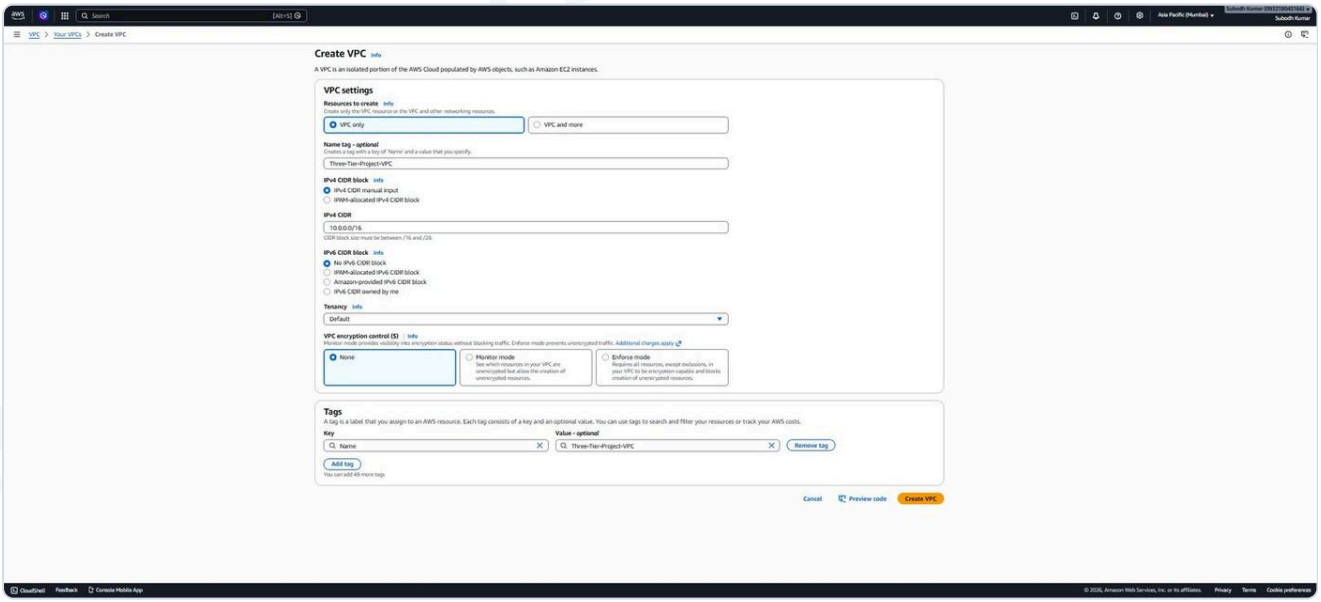


Note: When naming components you create, you can always use names of your choice.

Step 1 Create VPC and Subnets

Create VPC

We have to create a VPC. Search for VPC and go to **Your VPCs > Create VPC**. Setup it as follows. Use **10.0.0.0/16** as CIDR block.



Create Subnets

Next, we Have to create six subnets in this VPC. (Refer the diagram)

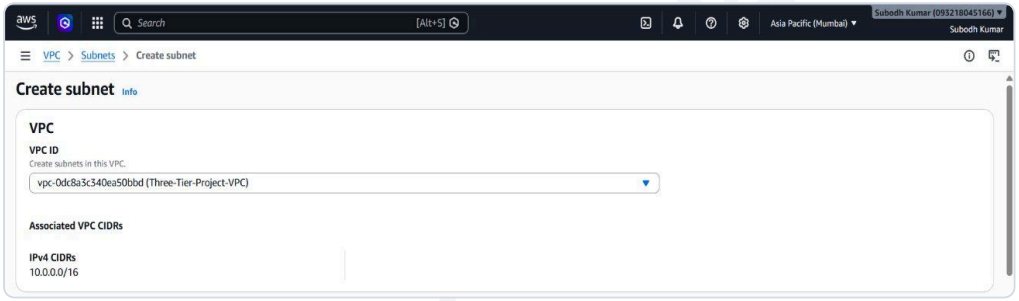
In Availability Zone 1a

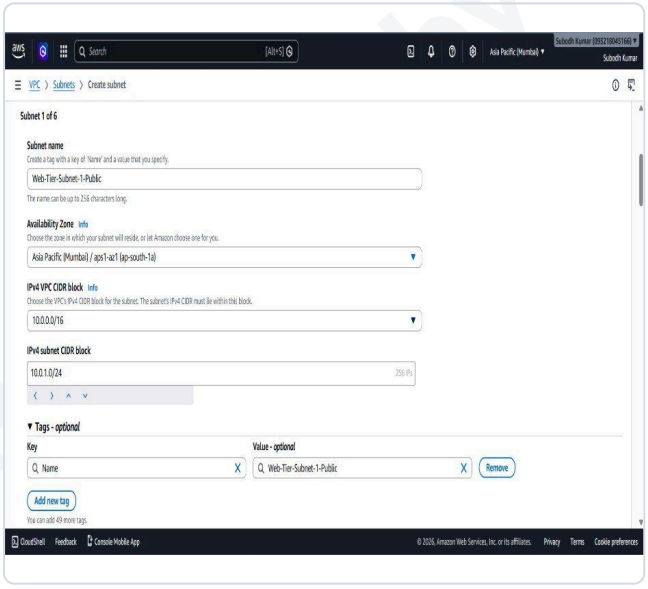
- Web-Tier-Subnet-1-Public
- App-Tier-Subnet-1-Private
- DB-Tier-Subnet-1-Private

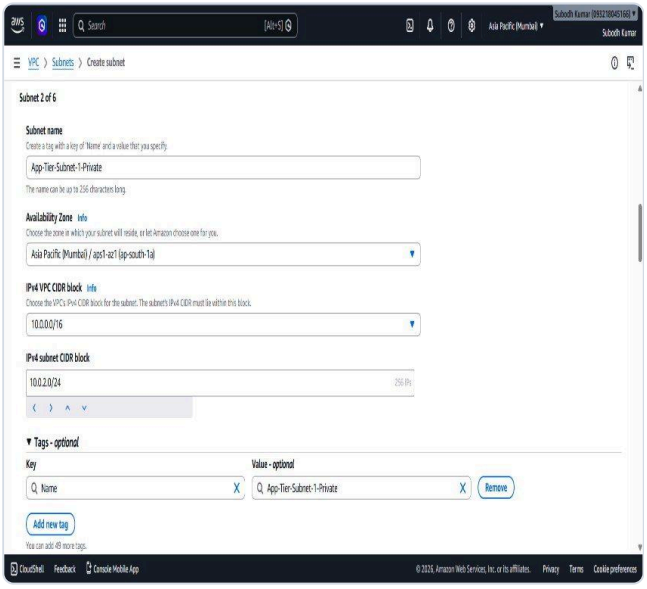
In Availability Zone 1b

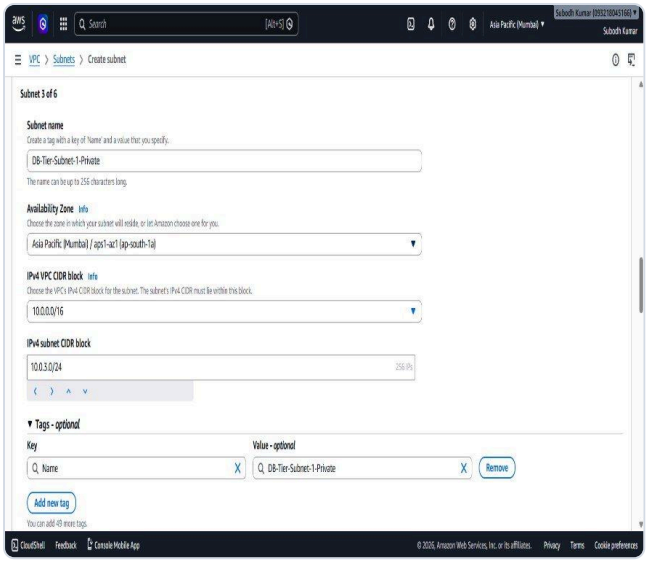
- Web-Tier-Subnet-2-Public
- App-Tier-Subnet-2-Private
- DB-Tier-Subnet-2-Private

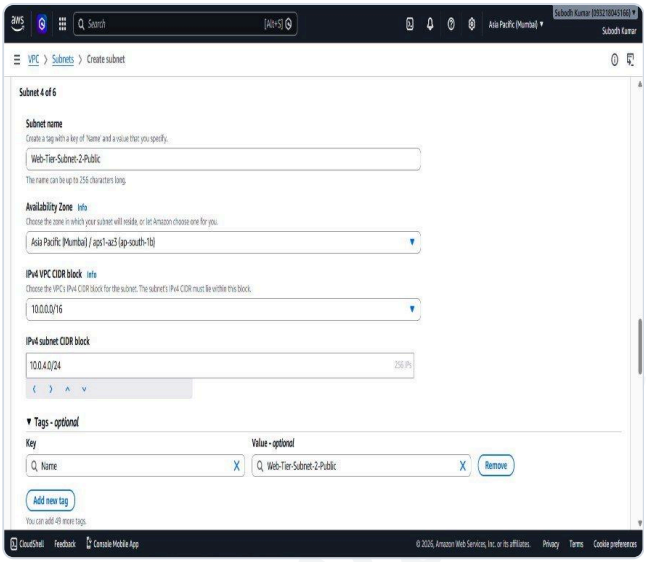
Go to **VPC dashboard > Subnets > Create subnets.** And setup them as follows. And Click create subnets

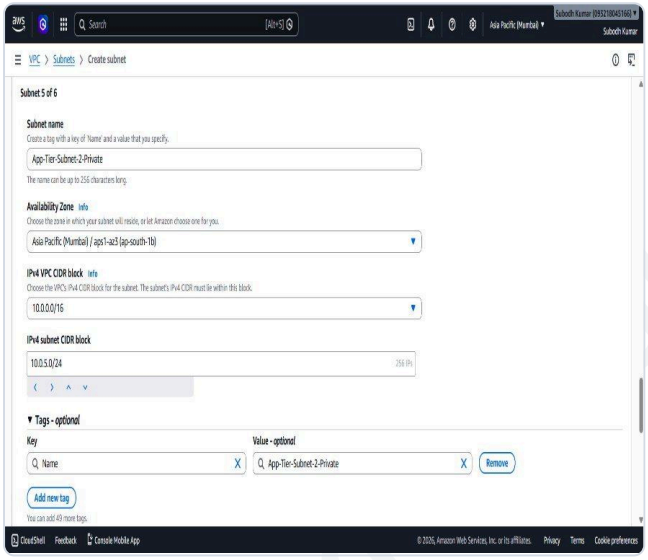


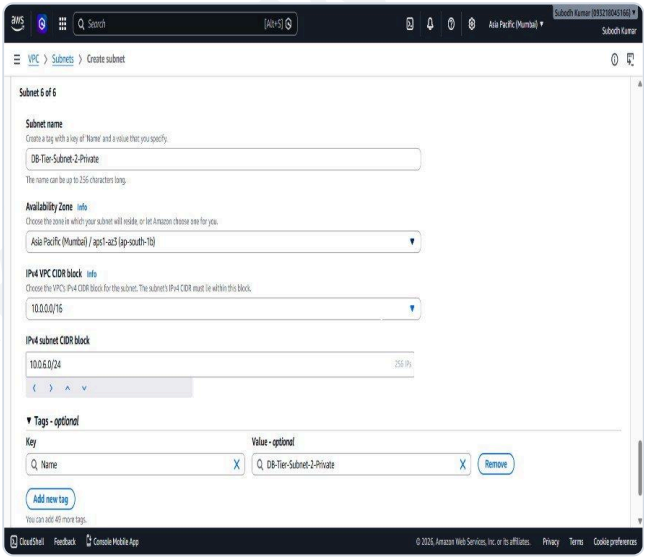




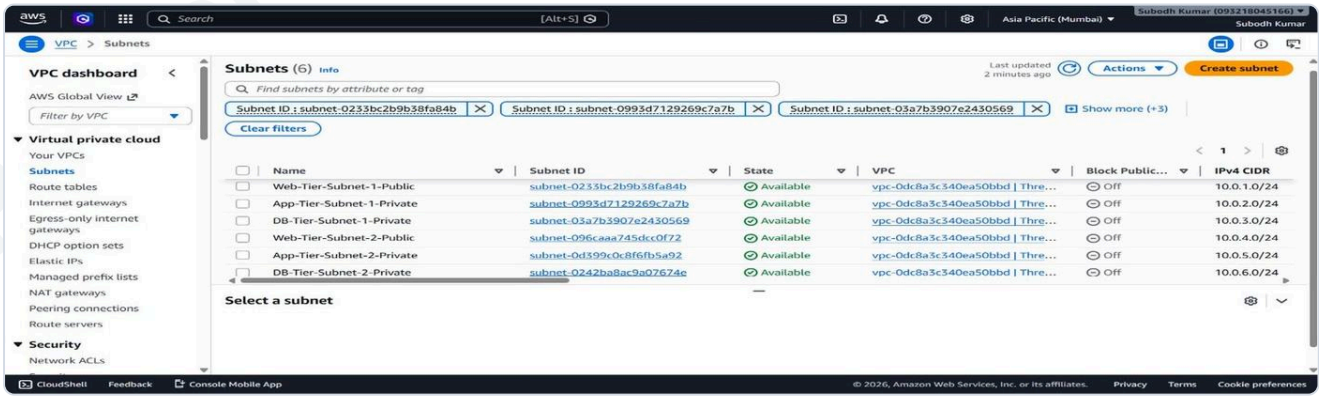




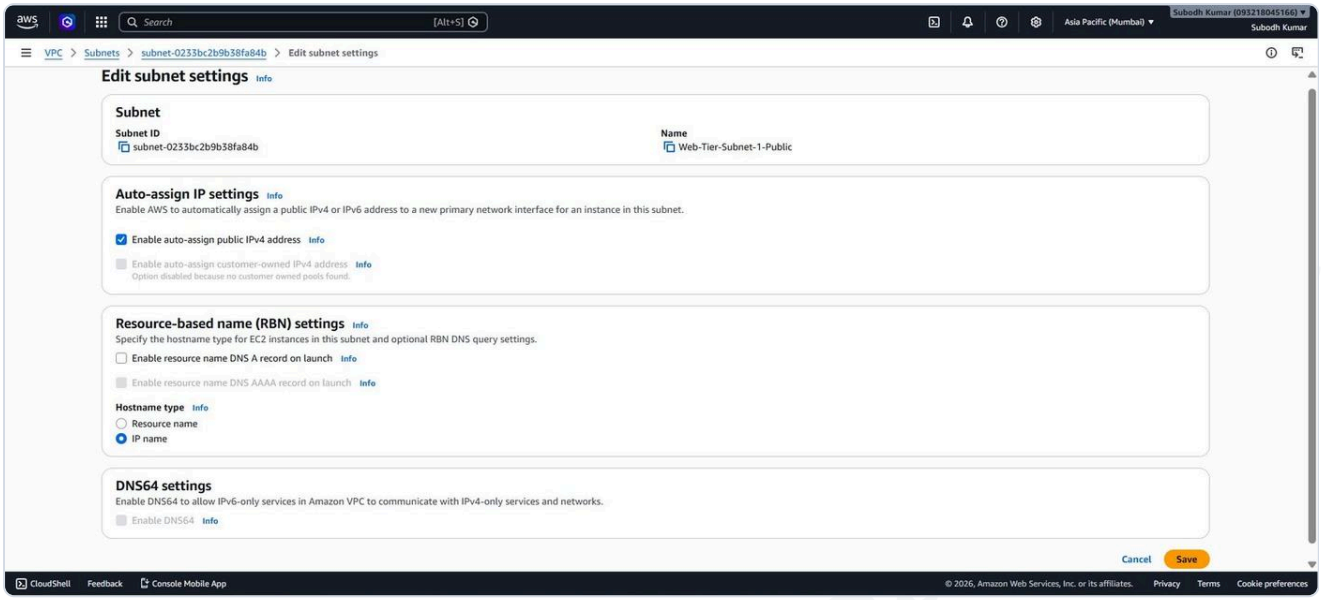


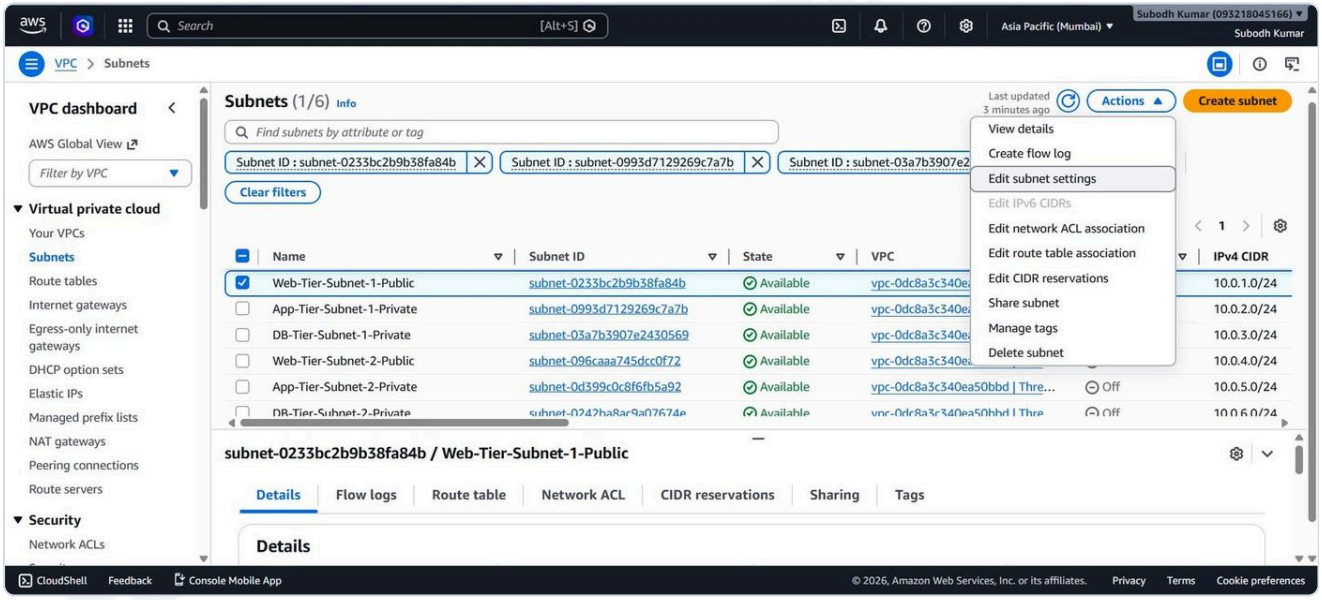


They will create as follows

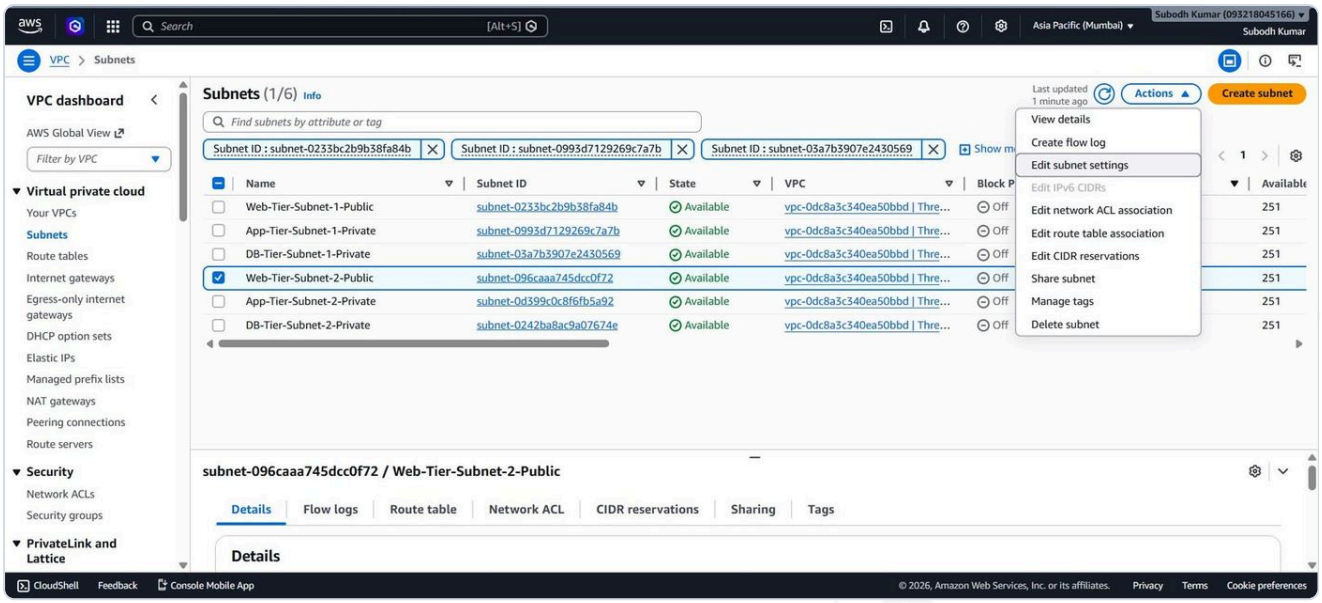


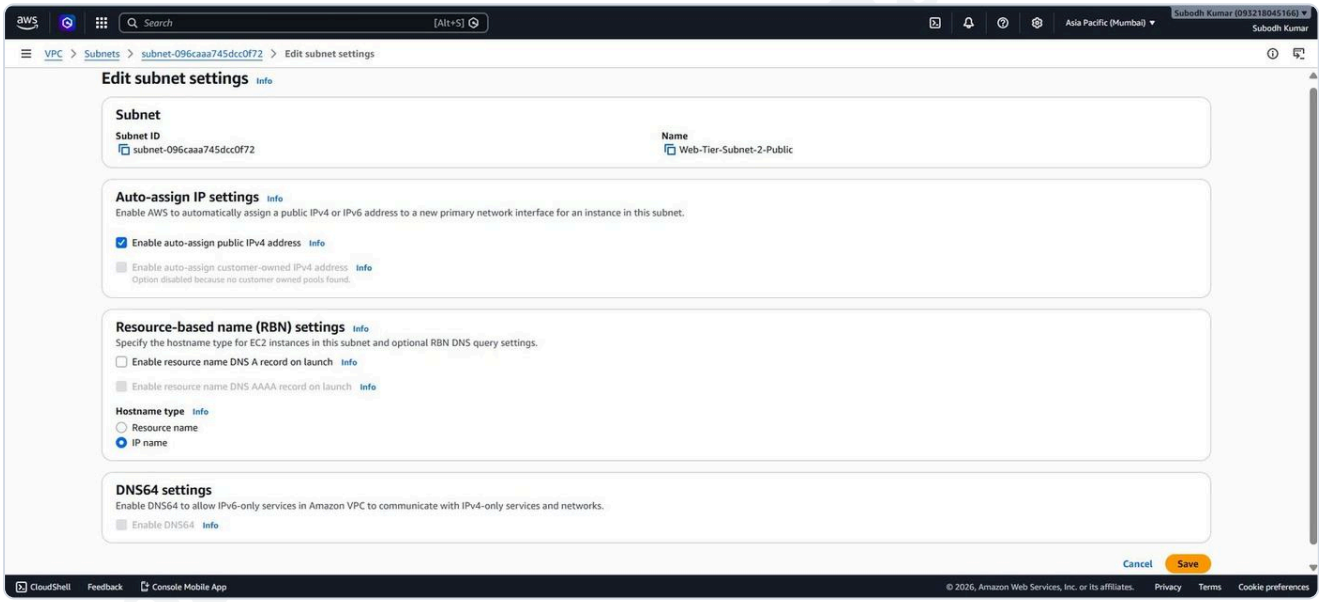
Now select **Web-Tier-Subnet-1-Public.** Go to **Action > Edit subnet settings.** Under Auto-assign IP settings, select **Enable auto-assign public IPv4 address** and save**.**





Do the same for **Web-Tier-Subnet-2-Public** as well.

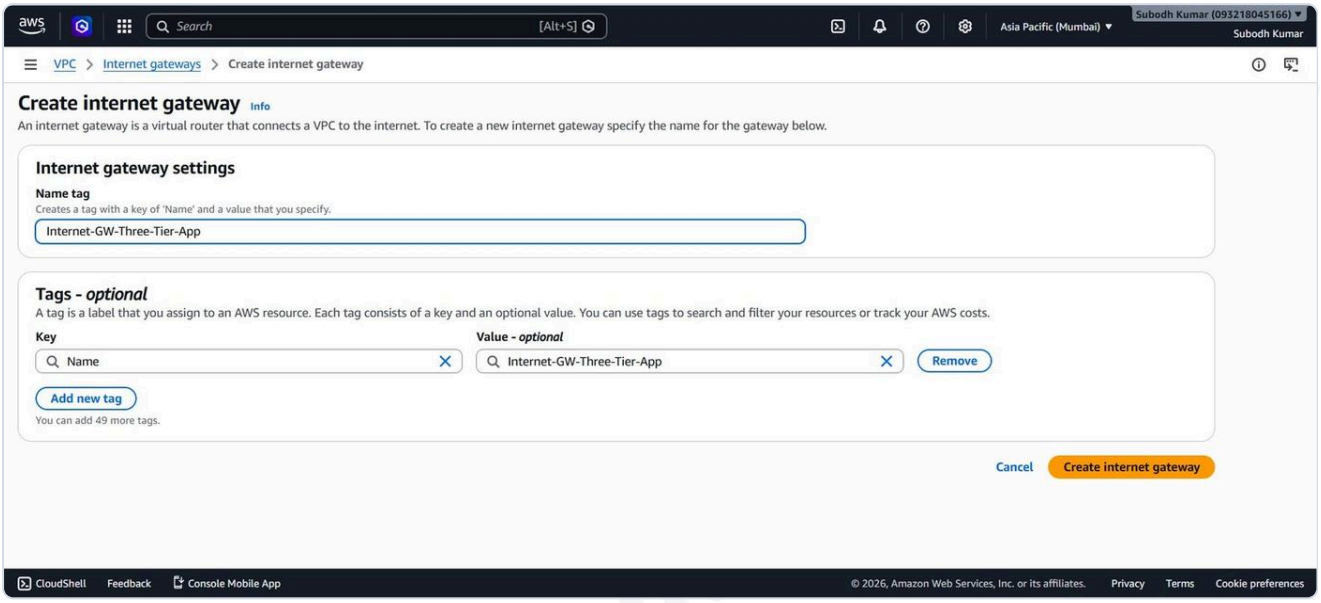




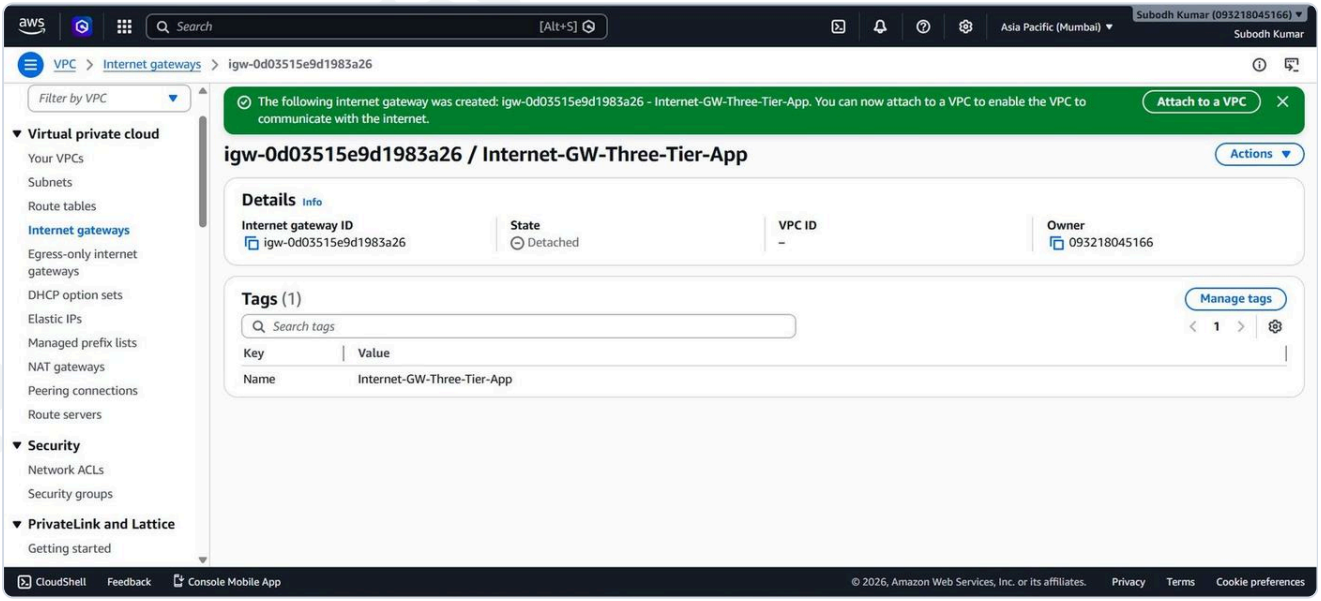
Step 2 Create Internet Gateway, NAT Gateways and Route Tables

Create Internet Gateway

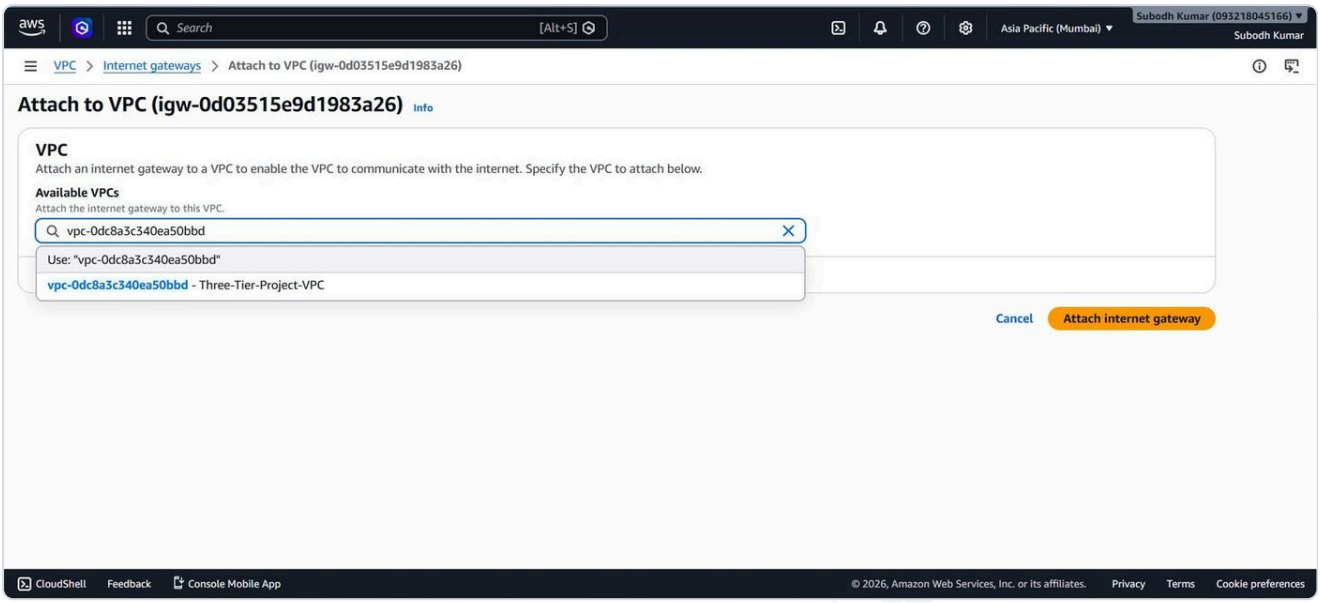
Go to VPC Dashboard **Internet gateways > Create internet Gateway.** Enter a name then click on Create internet gateway icon.



Then Go to internet gateways list and select our internet gateway and go to **Actions > Attach to VPC or Click directly on pop-up > Attach to VPC or Directly click on Pop-up menu.**

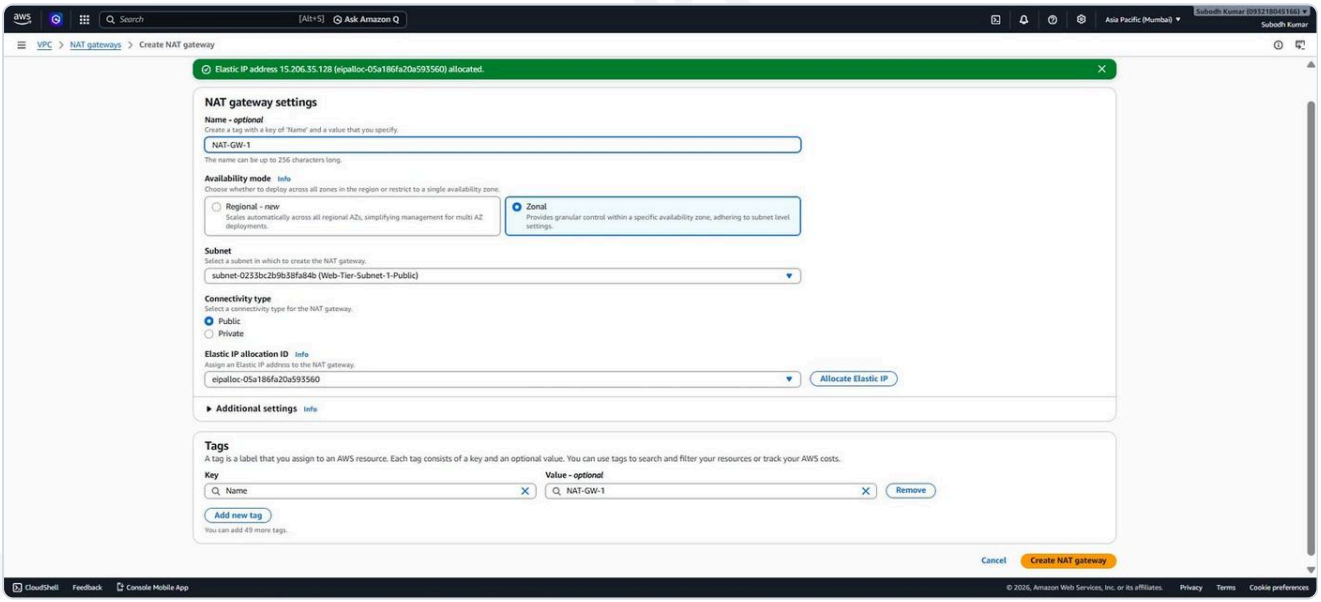


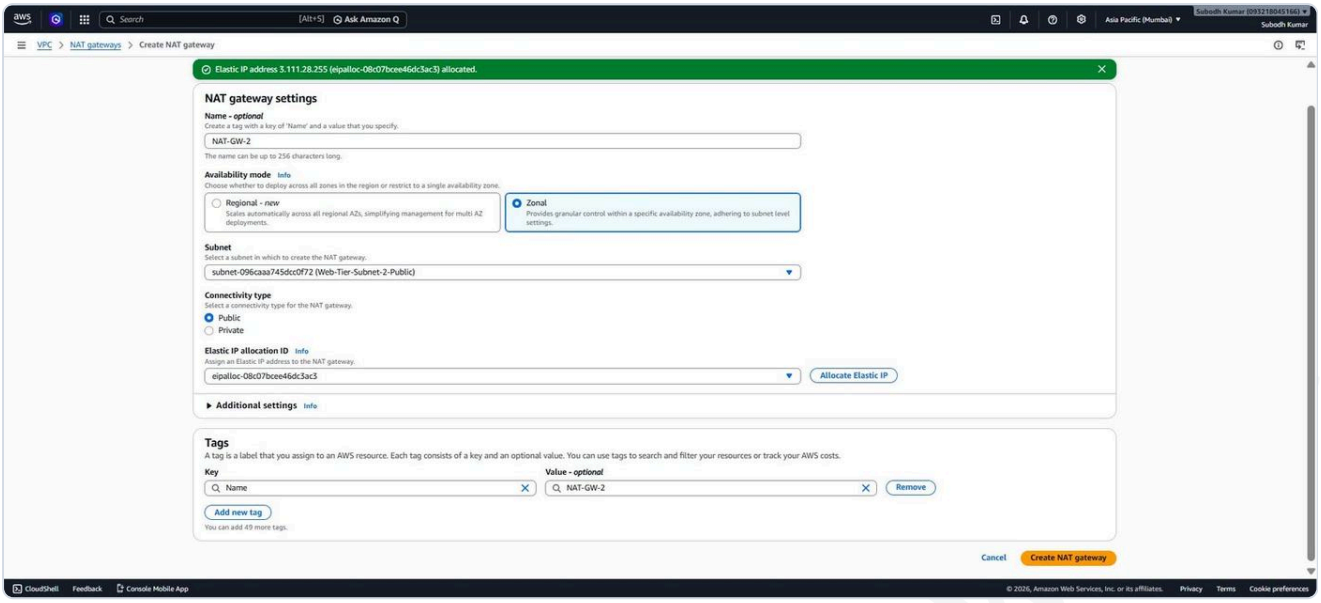
Then select our **VPC**. Finally, click on **Attach internet gateway** icon.



Create NAT Gateways

Go to VPC Dashboard **NAT gateways > Create NAT Gateway.** Create two NAT Gateways. Create it as follows.



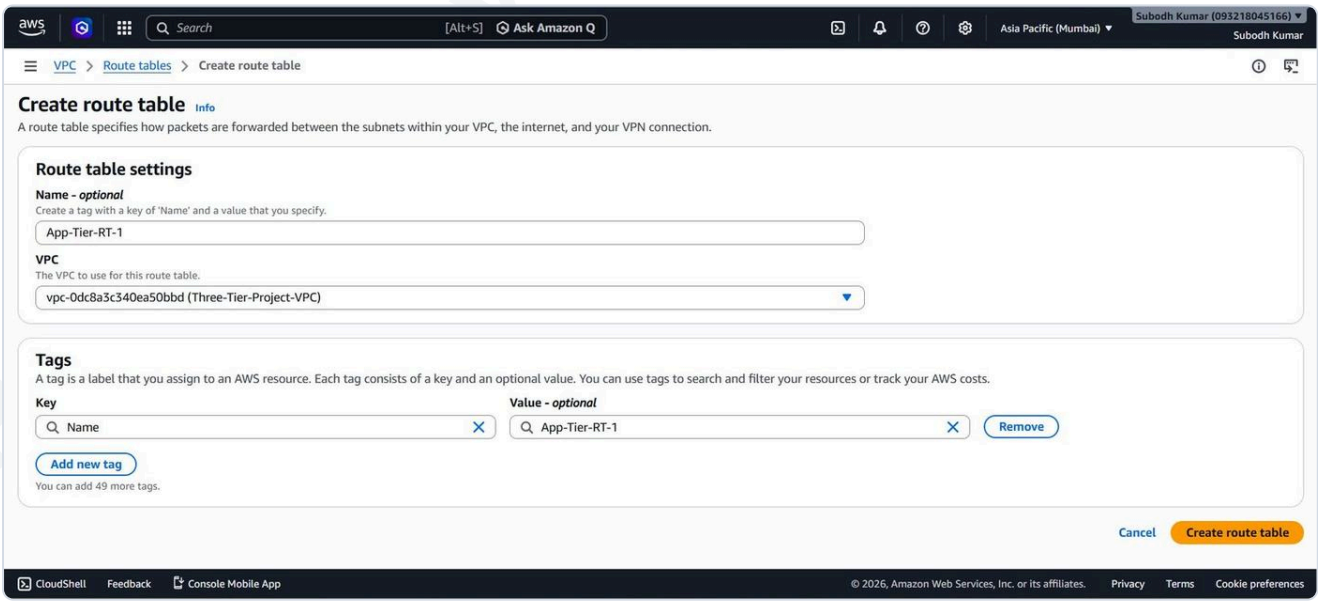


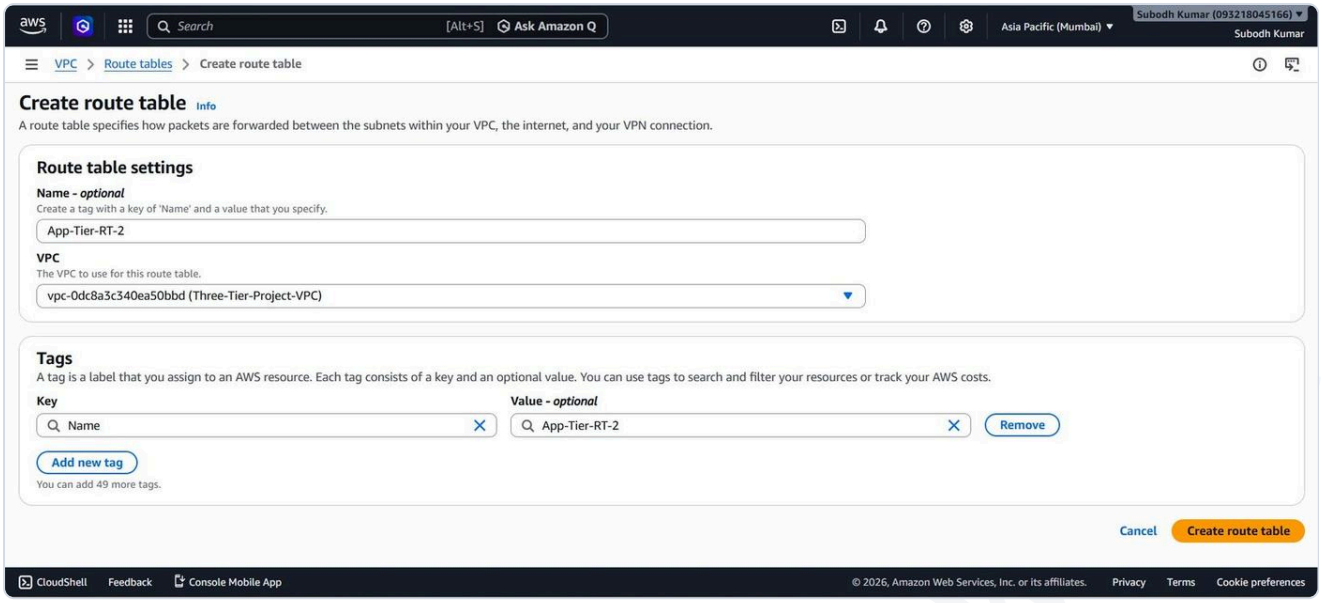
Create Route Tables

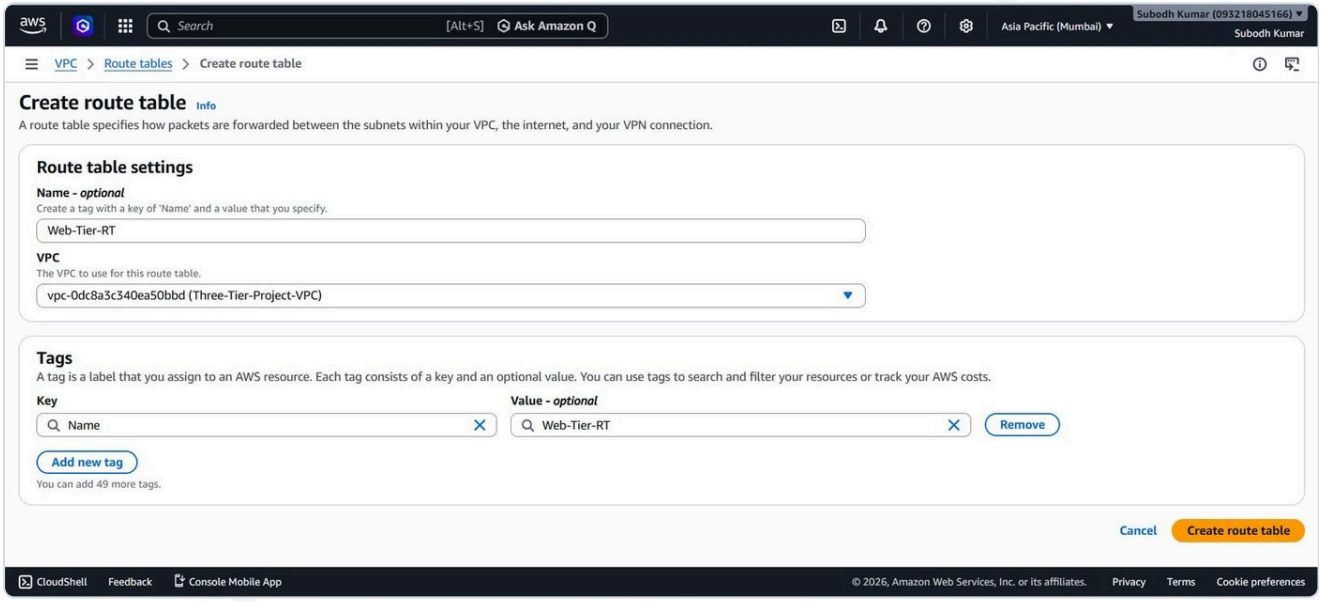
Now Let's create **Route tables**. According to our diagram we have to create **three route tables**.

- **App-Tier-RT-1**
- **App-Tier-RT-2**
- **Web-Tier-RT**

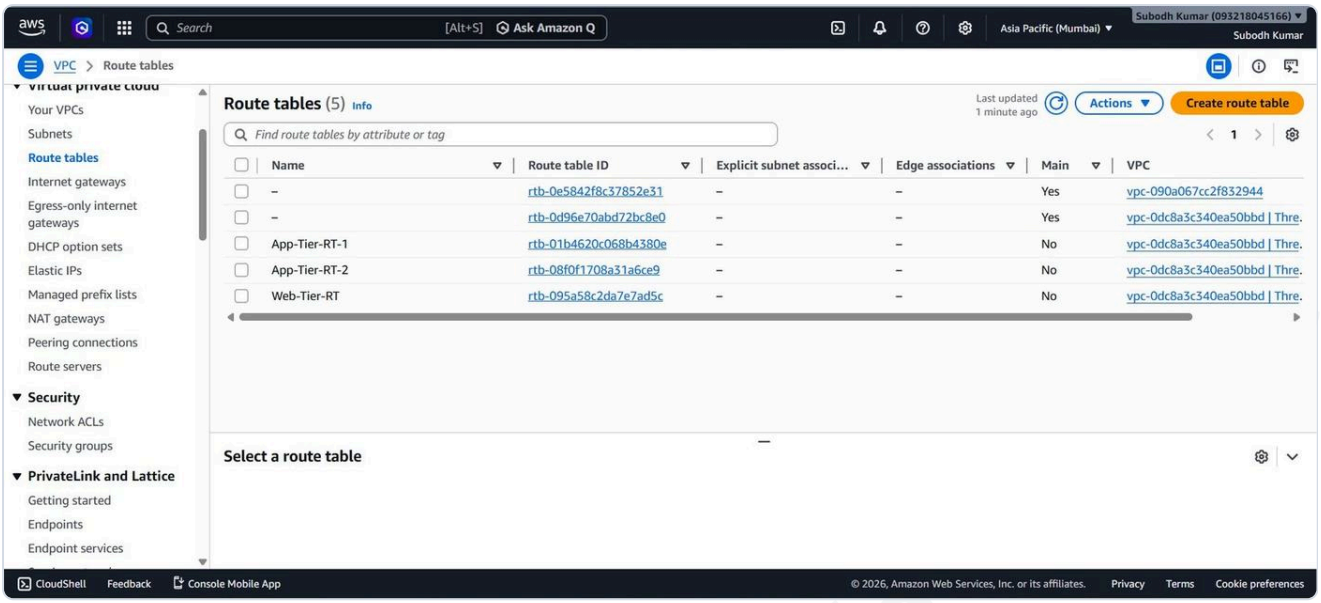
Now got to VPC dashboard **Route tables > Create route table**. Create Route tables as follows.



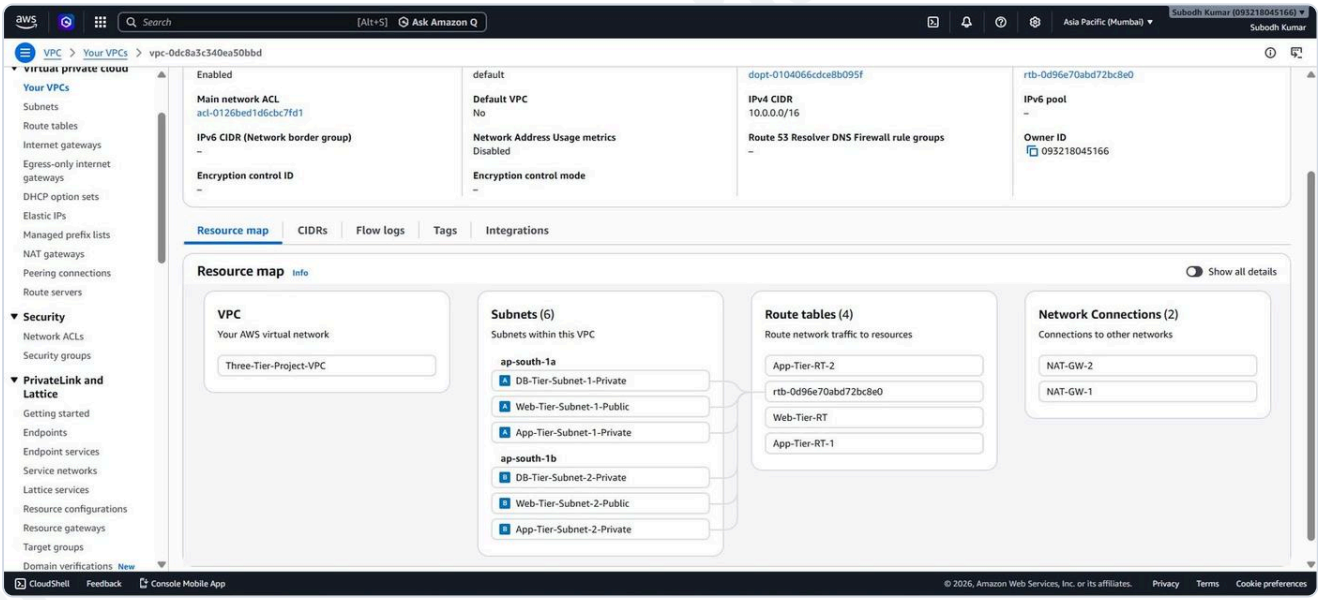

Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
VPC > Route tables > Create route table
Create route table Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.
Route table settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
App-Tier-RT-2
VPC
The VPC to use for this route table.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Value - optional
Name
App-Tier-RT-2
Remove
Add new tag
You can add 49 more tags.
Cancel
Create route table
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
VPC > Route tables > Create route table
Create route table Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.
Route table settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
Web-Tier-RT
VPC
The VPC to use for this route table.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Value - optional
Name
Web-Tier-RT
Remove
Add new tag
You can add 49 more tags.
Cancel
Create route table
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Now you will see all the route tables in the menu.

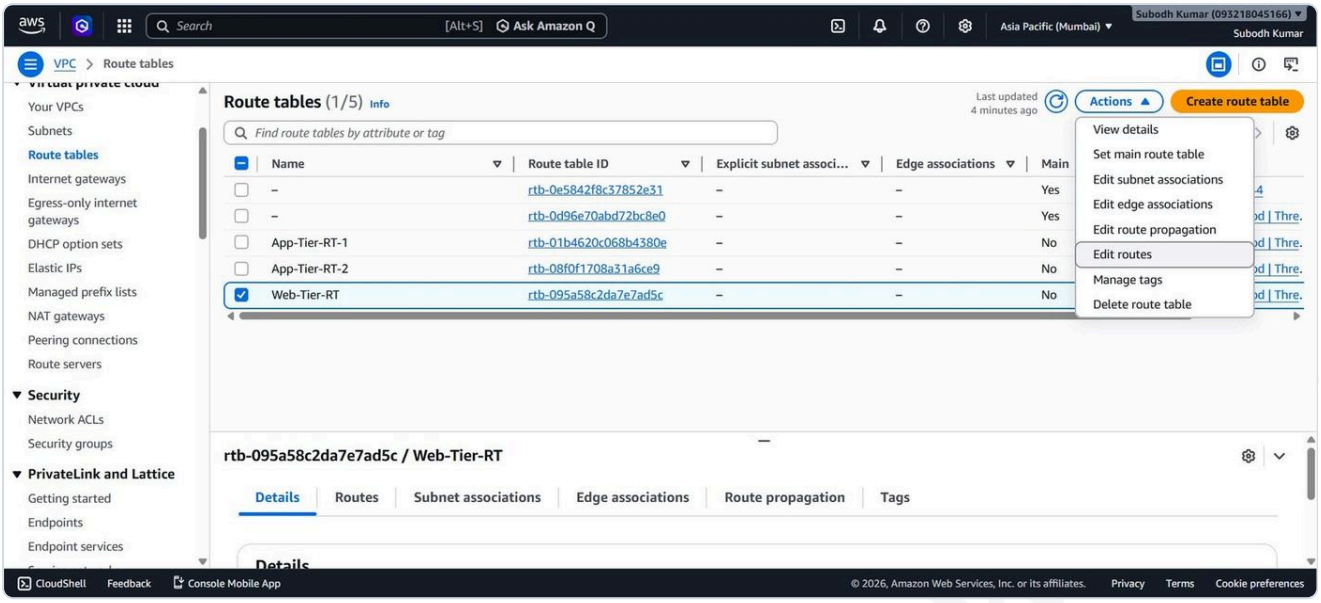


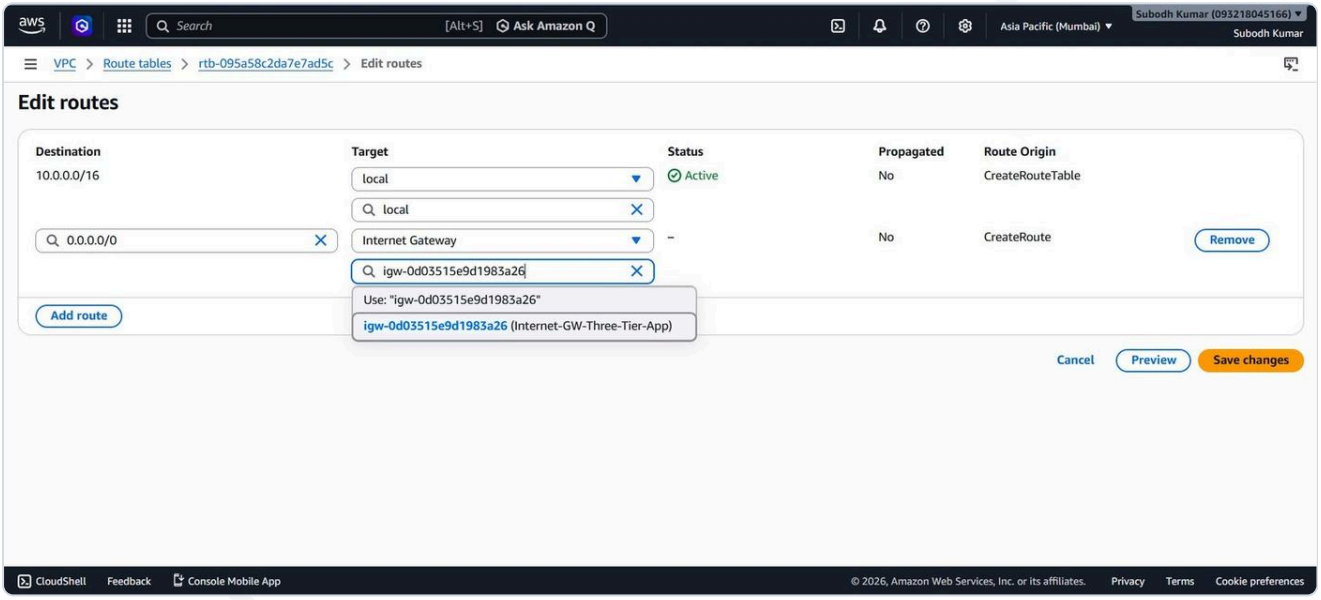
If you go to **VPC** dashboard **> Your VPCs**. And select **Three-Tier-Project-VPC**. Then if you go to **Resource map** you can see **Route tables** are **created** but there are **no connections.**



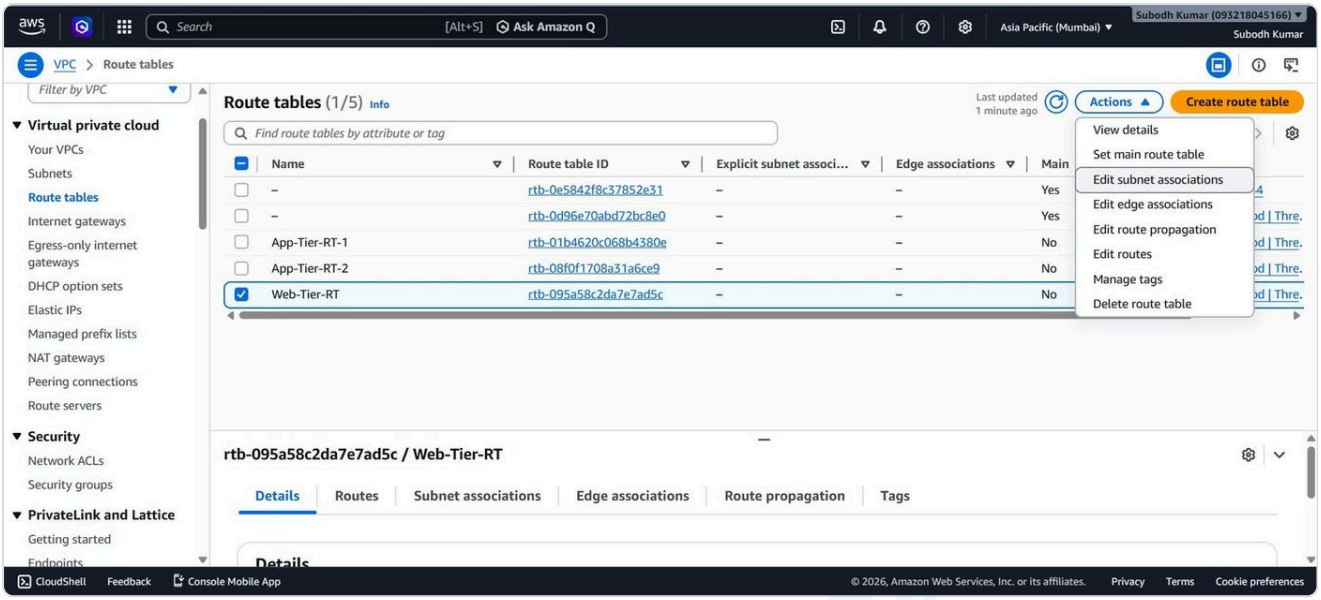
Edit Routes

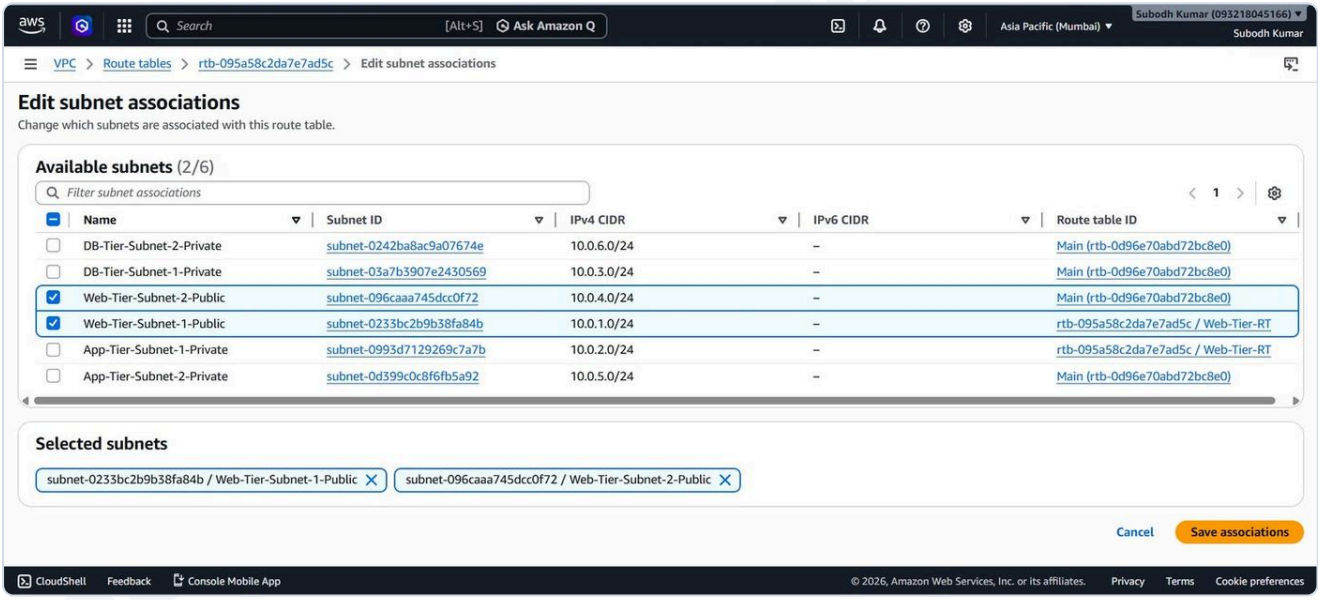
- Now Select Web-Tier-RT Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target internet gateway > select our igw. Then Save changes.


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
VPC > Route tables
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
PrivateLink and Lattice
Getting started
Endpoints
Endpoint services
Route tables (1/5) Info
Last updated 4 minutes ago
Actions
Create route table
Find route tables by attribute or tag
Name
Route table ID
Explicit subnet associ...
Edge associations
Main
rtb-0e5842f8c37852e31
rtb-0d96e70abd72bc8e0
App-Tier-RT-1
rtb-01b4620c068b4380e
App-Tier-RT-2
rtb-08f0f1708a31a6ce9
Web-Tier-RT
rtb-095a58c2da7e7ad5c
View details
Set main route table
Edit subnet associations
Edit edge associations
Edit route propagation
Edit routes
Manage tags
Delete route table
rtb-095a58c2da7e7ad5c / Web-Tier-RT
Details
Routes
Subnet associations
Edge associations
Route propagation
Tags
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

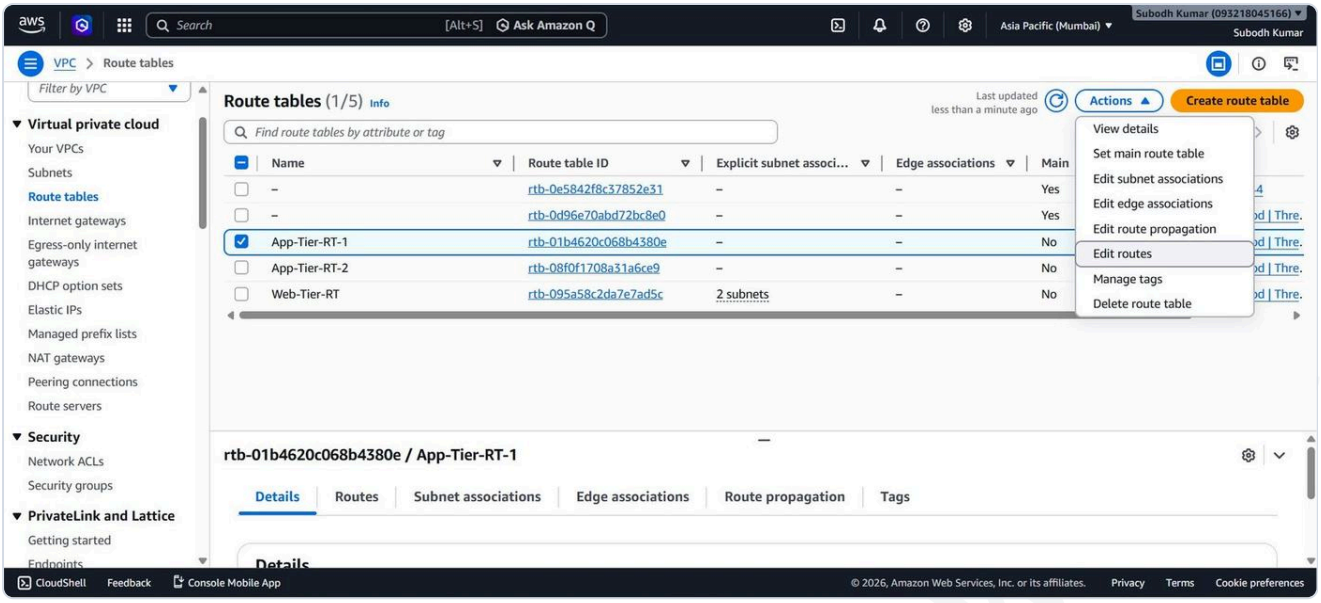

Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
VPC > Route tables > rtb-095a58c2da7e7ad5c > Edit routes
Edit routes
Destination
Target
Status
Propagated
Route Origin
10.0.0.0/16
local
Active
No
CreateRouteTable
local
0.0.0.0/0
Internet Gateway
-
No
CreateRoute
Remove
igw-0d03515e9d1983a26
Use: "igw-0d03515e9d1983a26"
igw-0d03515e9d1983a26 (Internet-GW-Three-Tier-App)
Add route
Cancel
Preview
Save changes
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

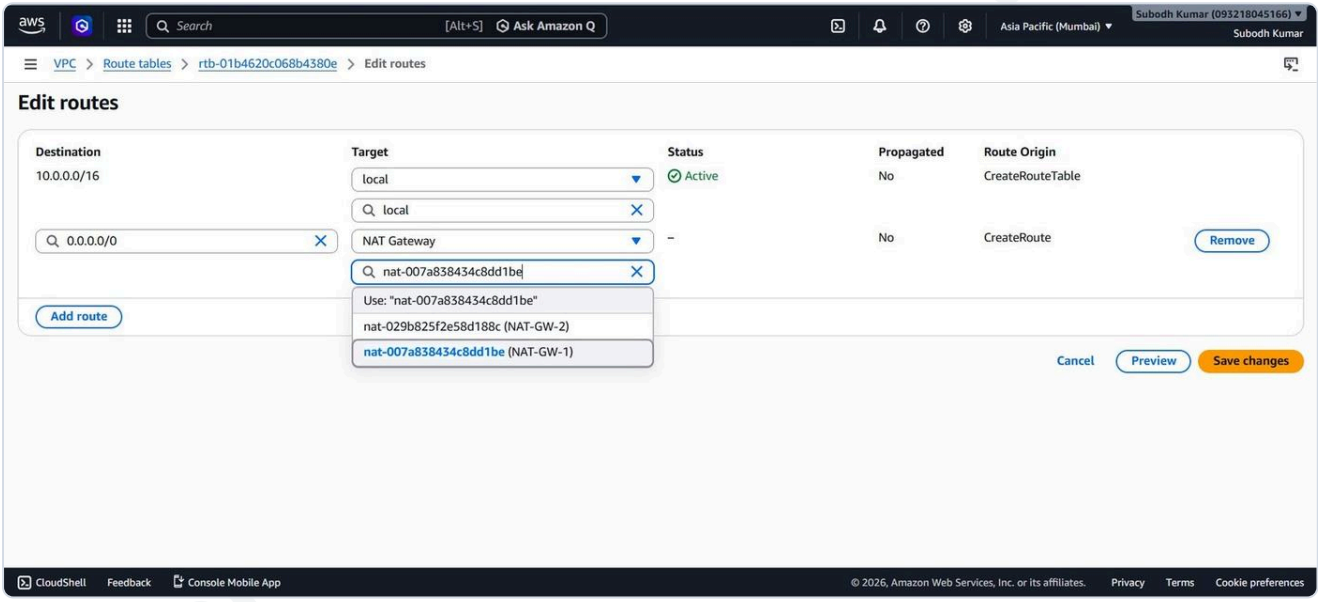
Again, select the Web-Tier-RT and go to **Actions > Edit subnet associations.** Select Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public and Save associations.





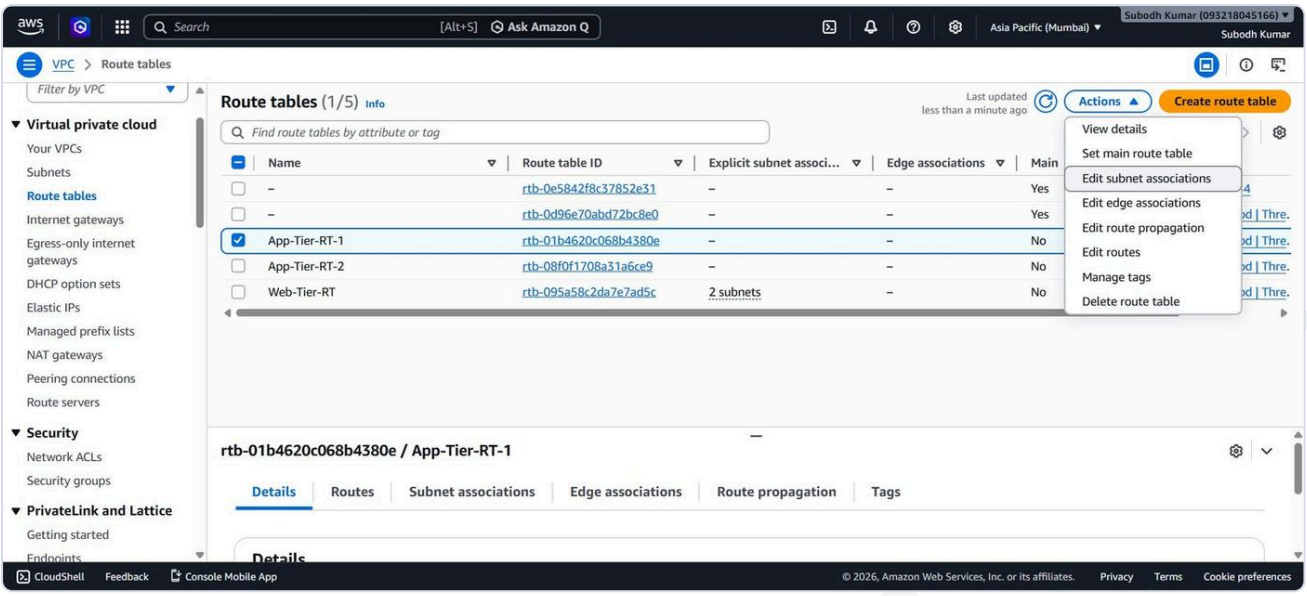
- Now Select **App-Tier-RT-1** Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-1. Then Save changes.

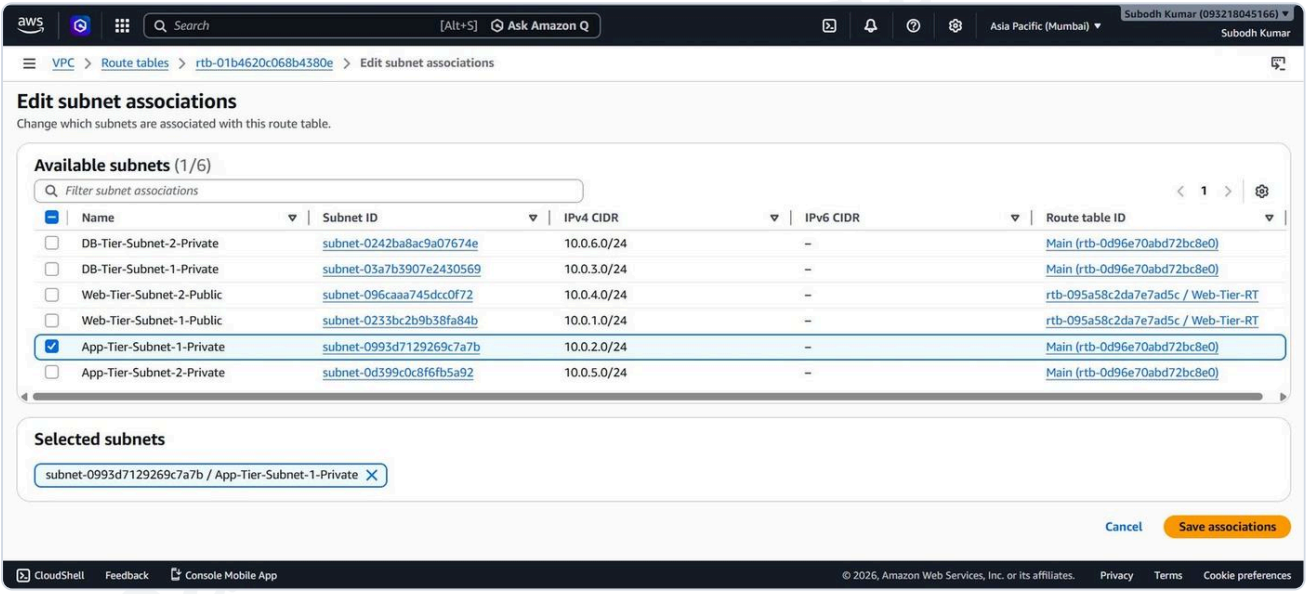




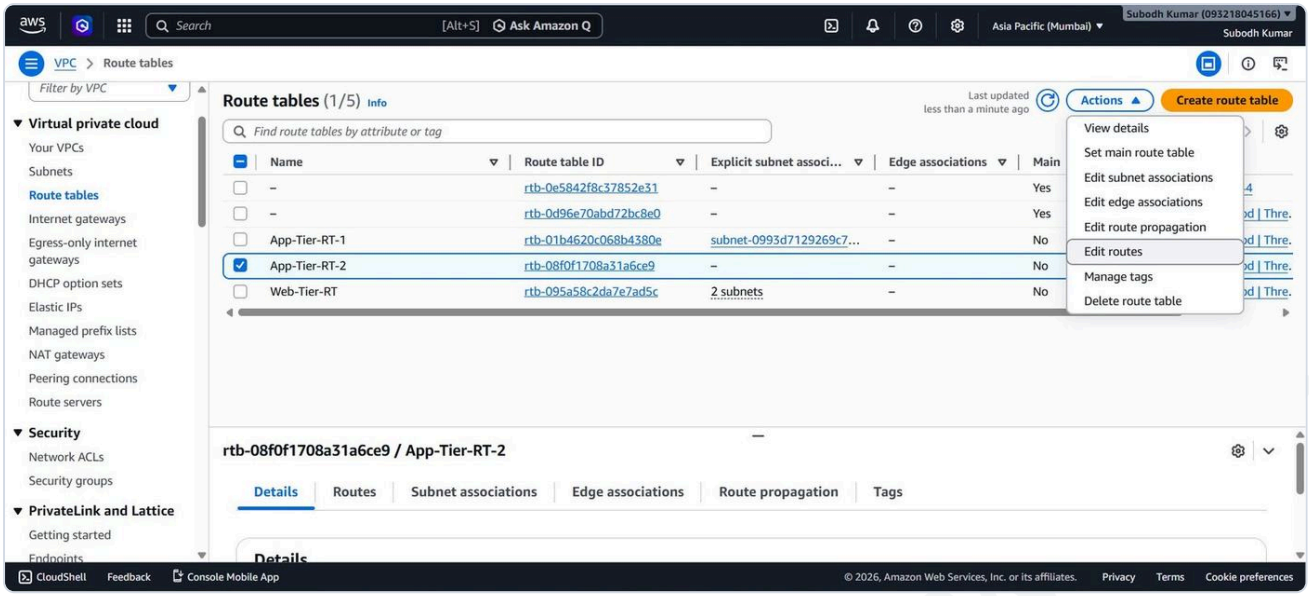
Again, select the App-Tier-RT-1 and go to **Actions > Edit subnet associations.**

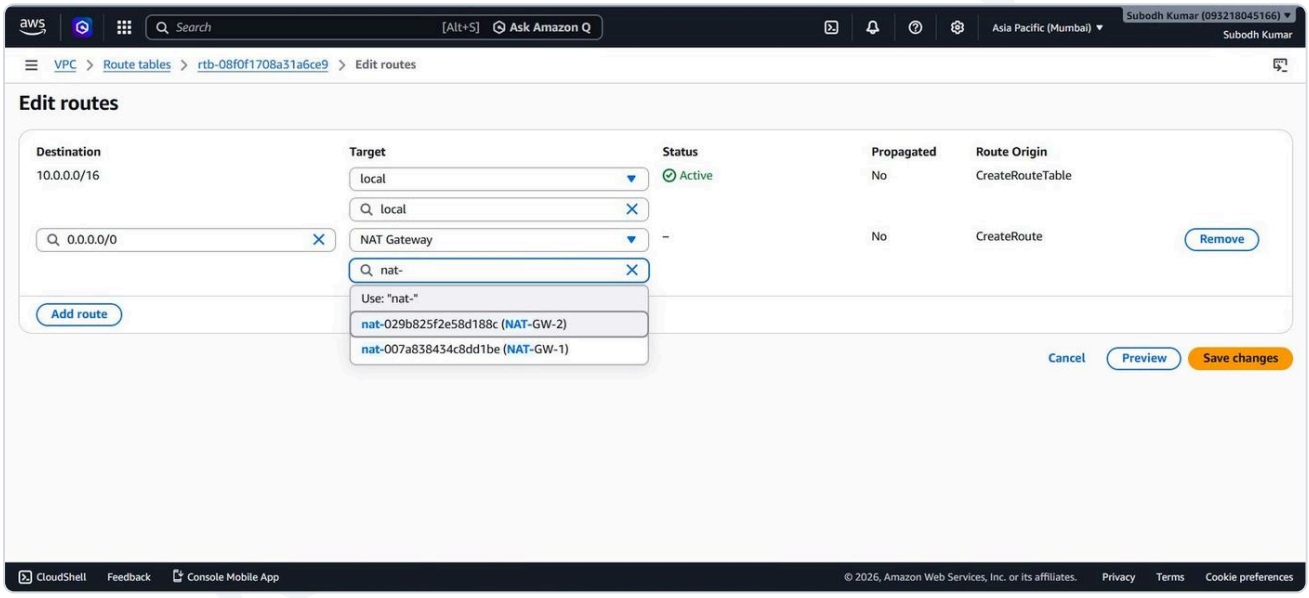
Select App-Tier-Subnet-1-Private and Save associations.





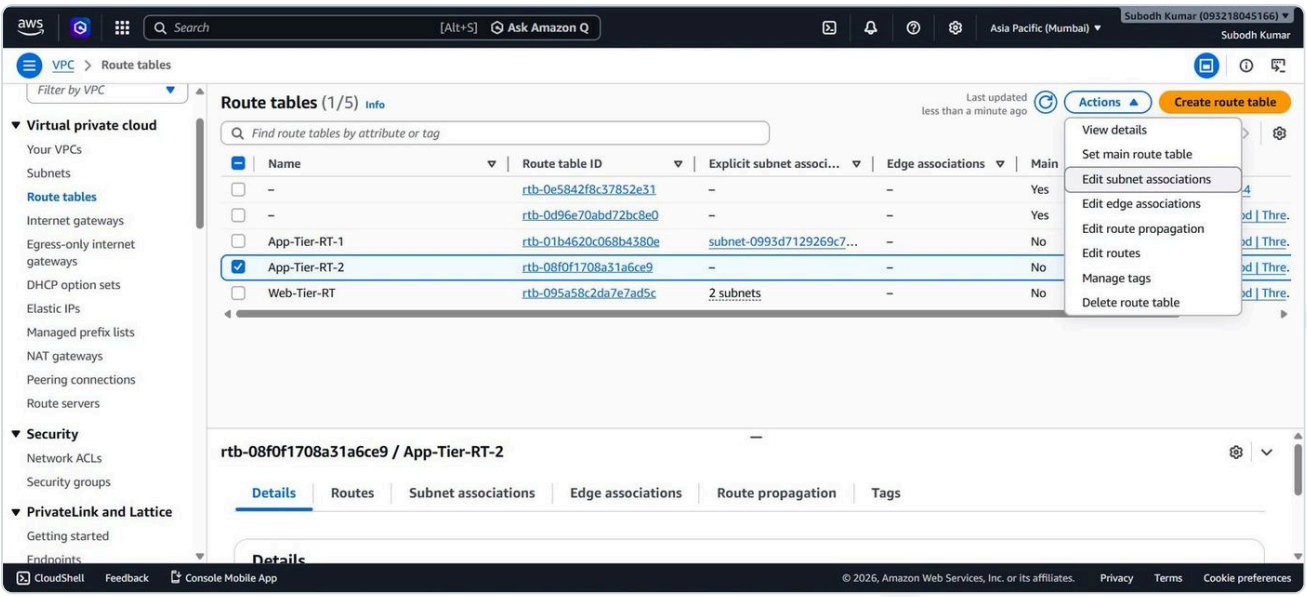
- Now Select **App-Tier-RT-2** Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-2. Then Save changes.

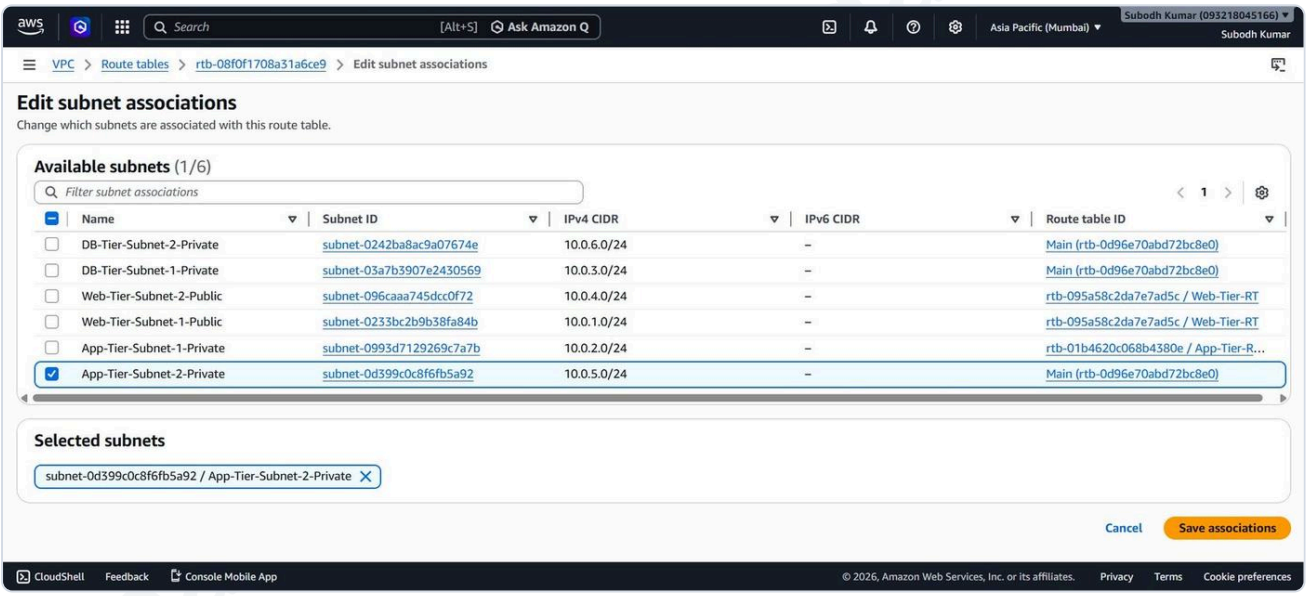




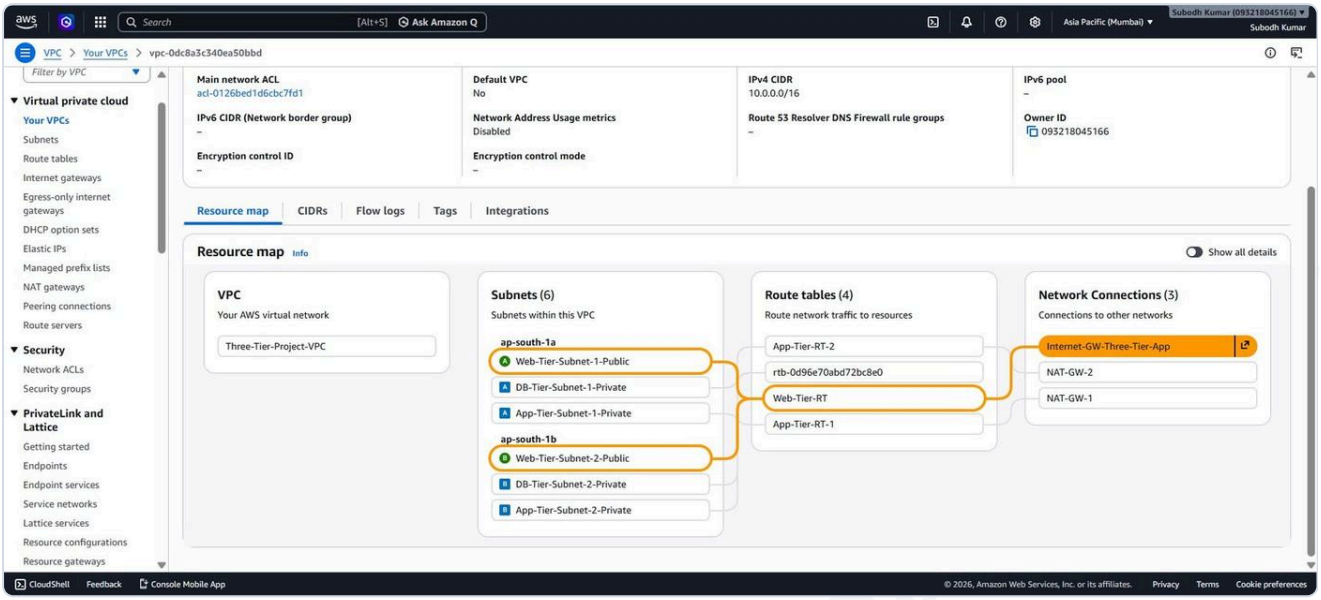
Again, select the App-Tier-RT-2 and go to **Actions > Edit subnet associations.**

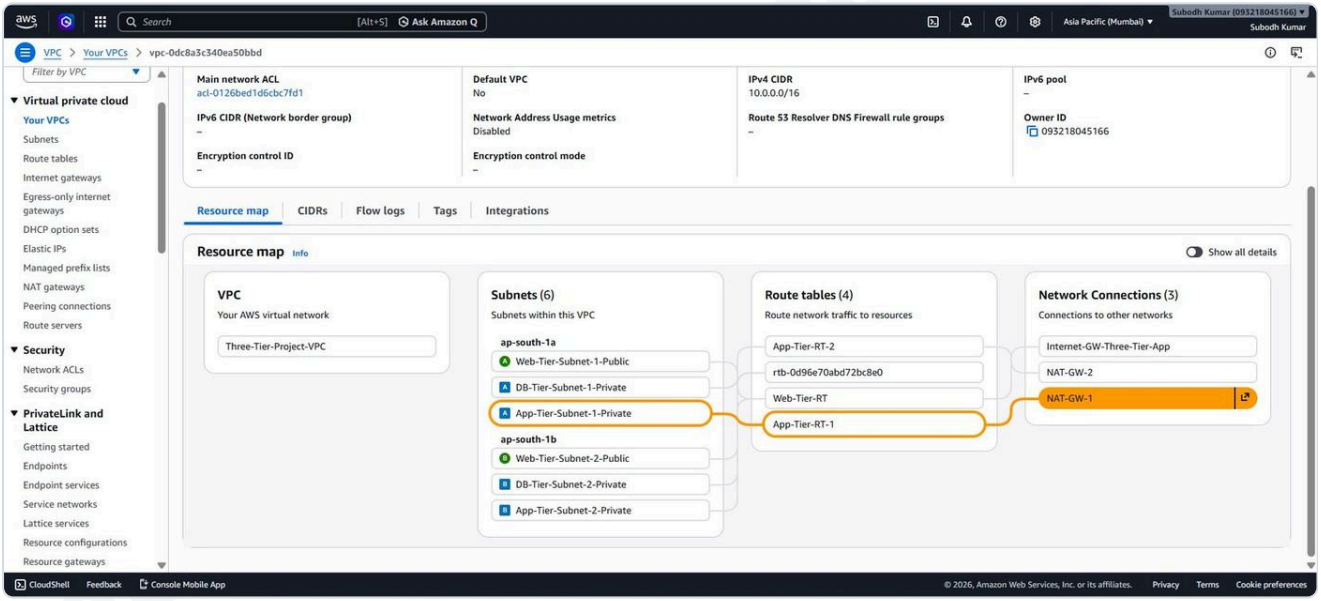
Select App-Tier-Subnet-2-Private and Save associations.

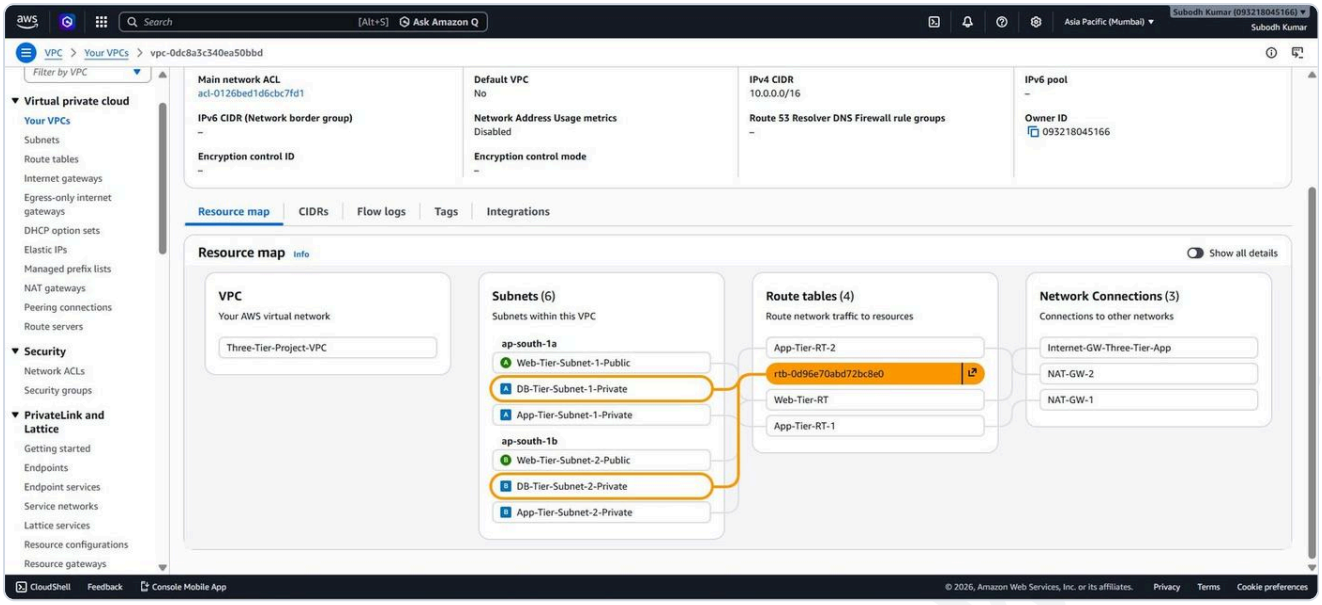




Now if you **check** the **Resource map** you can see that connections have been made. **Check** if there are **any mistakes** according to our architecture diagram by hovering over each element.







Good job! Now let's create the Security Groups.

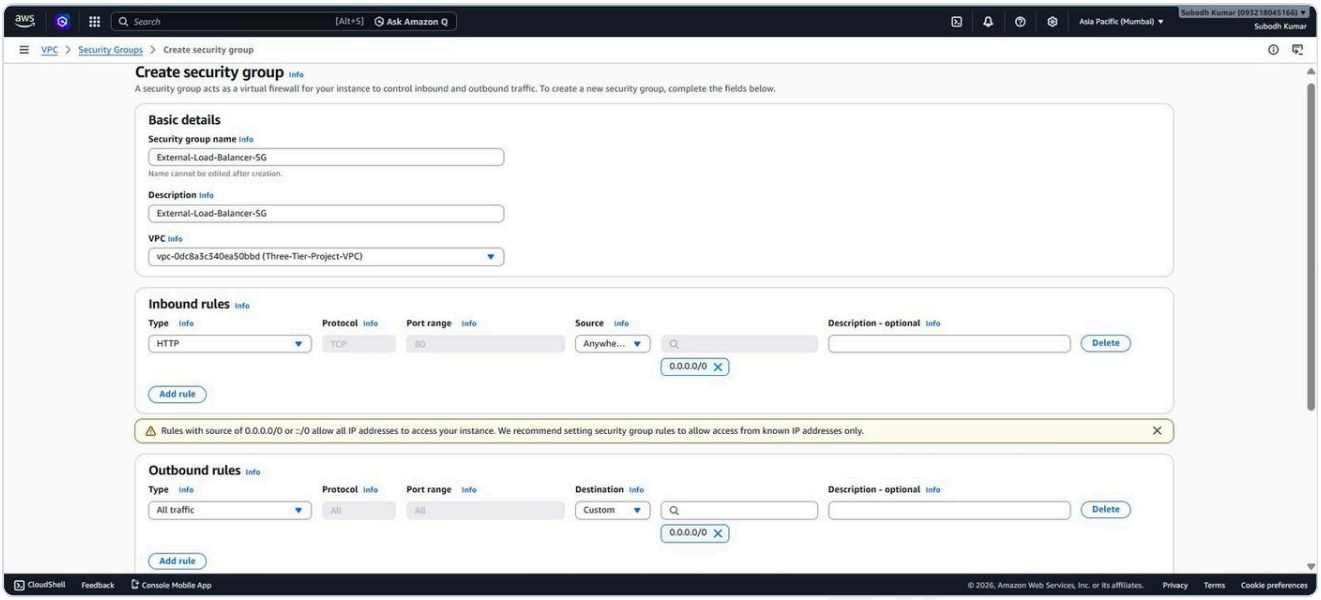
Step 3 Create Security Groups

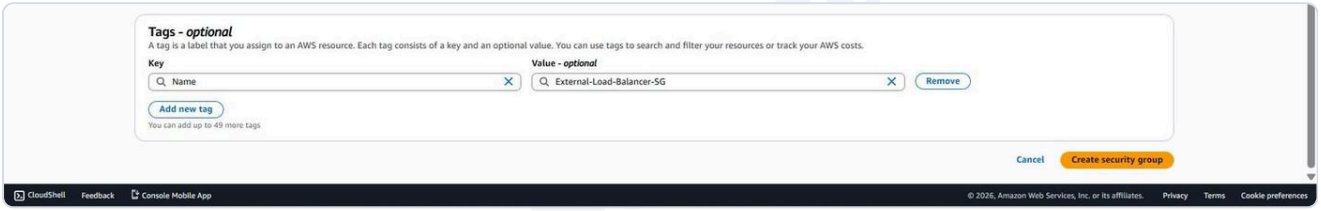
We have to create the following Security Groups.

Security Group	Type (Port Range)	Source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL (3306)	App-Tier-SG

Prepared by Sukhada W

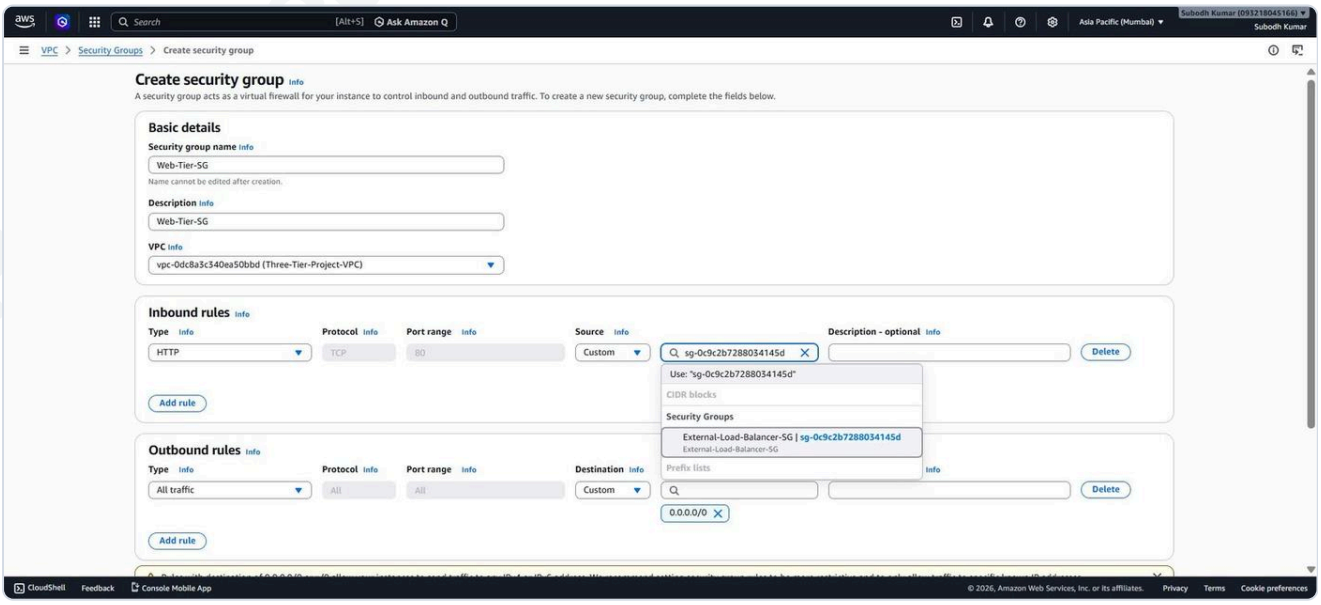
Go to VPC dashboard **Security Groups > Create security group**. Create **External-Load-Balancer-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.

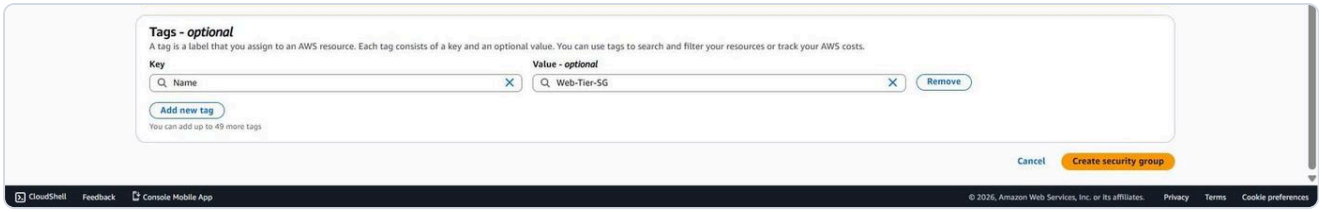




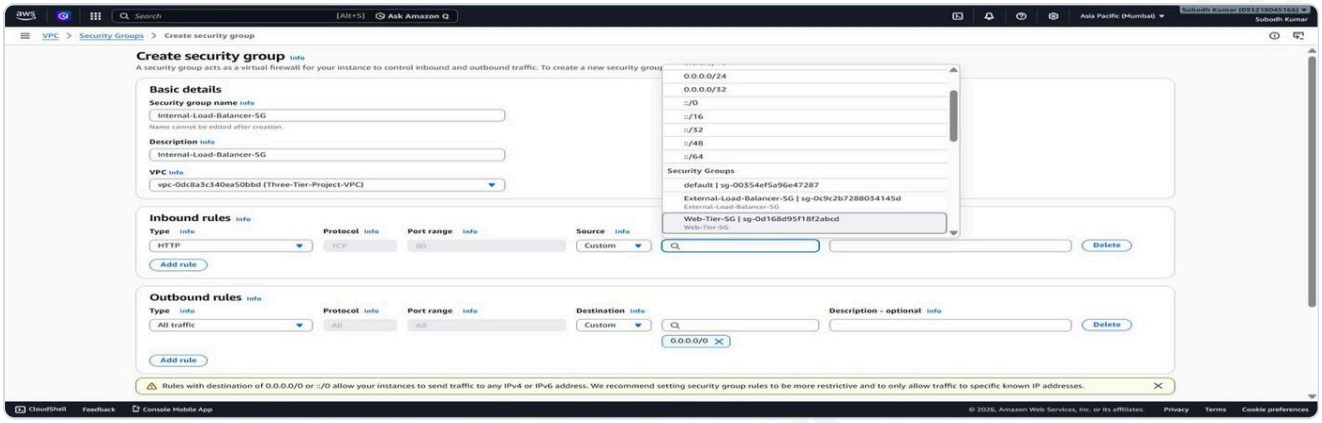
Creating other Security groups as well as shown in the table. The process is same.

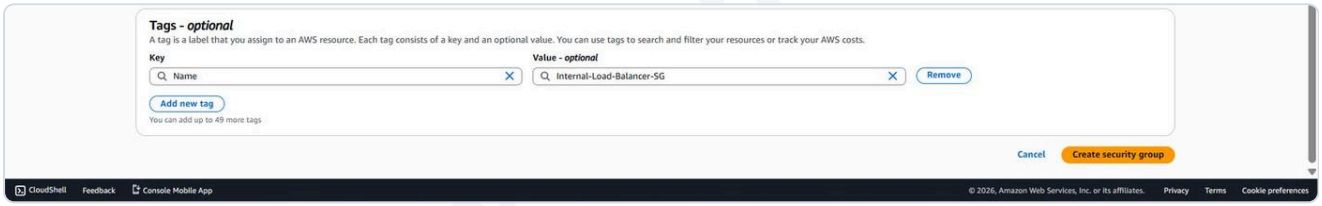
Go to VPC dashboard **Security Groups > Create security group**. Create **Web-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.



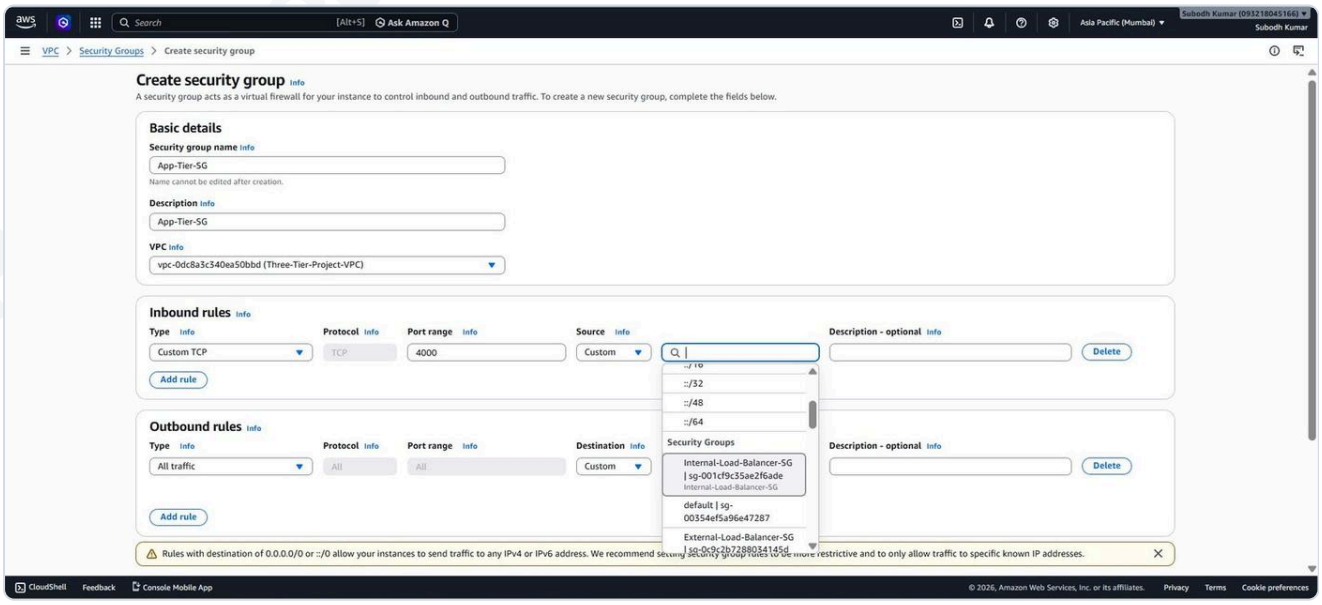


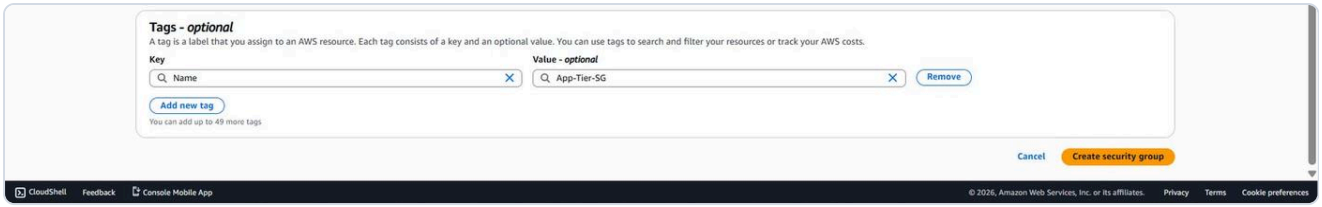
Go to VPC dashboard **Security Groups > Create security group**. Create **Internal- Load-Balancer-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.



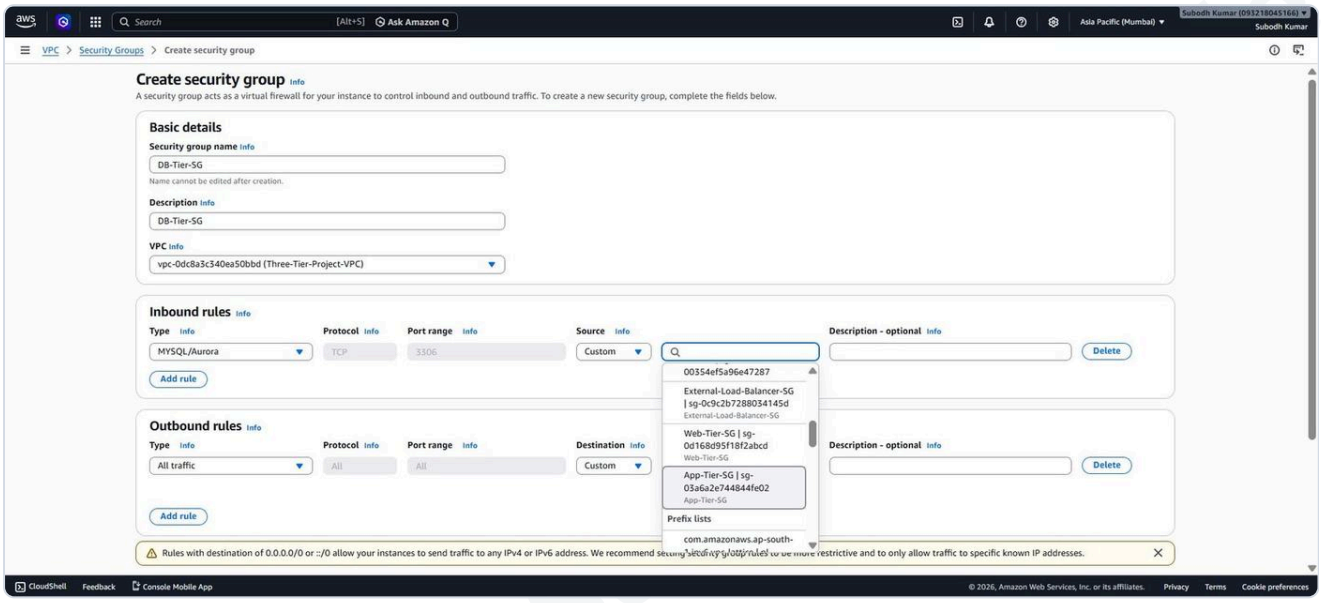


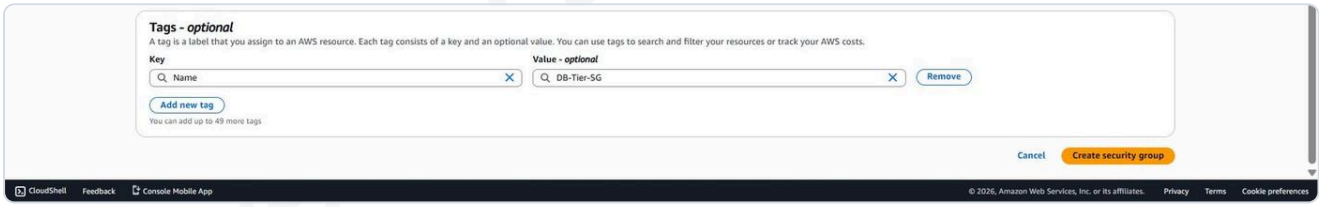
Go to VPC dashboard **Security Groups > Create security group**. Create **App-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click on **Create security group** icon.



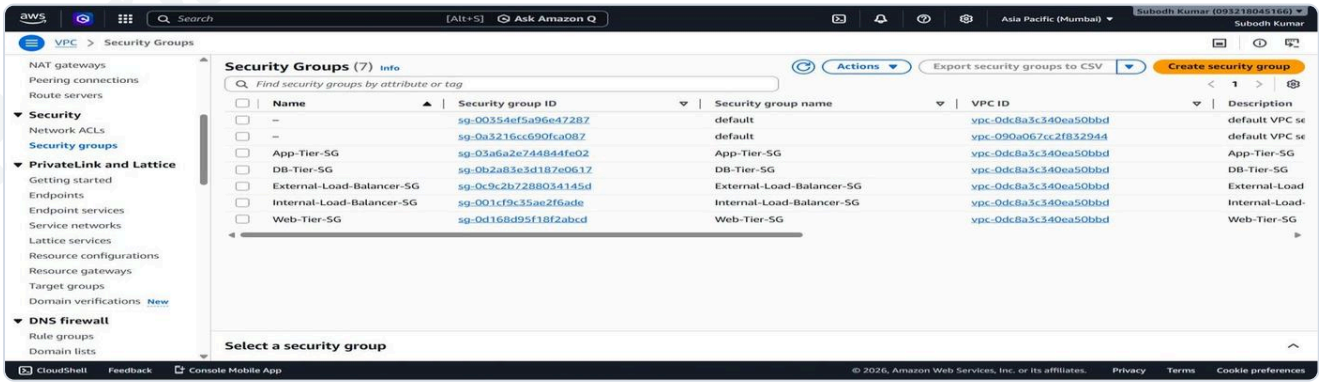


Go to VPC dashboard **Security Groups > Create security group**. Create **DB-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click on **Create security group** icon.





After creating all Security groups you can see them like this.



Well done! Now let’s move on to setting up the database servers.

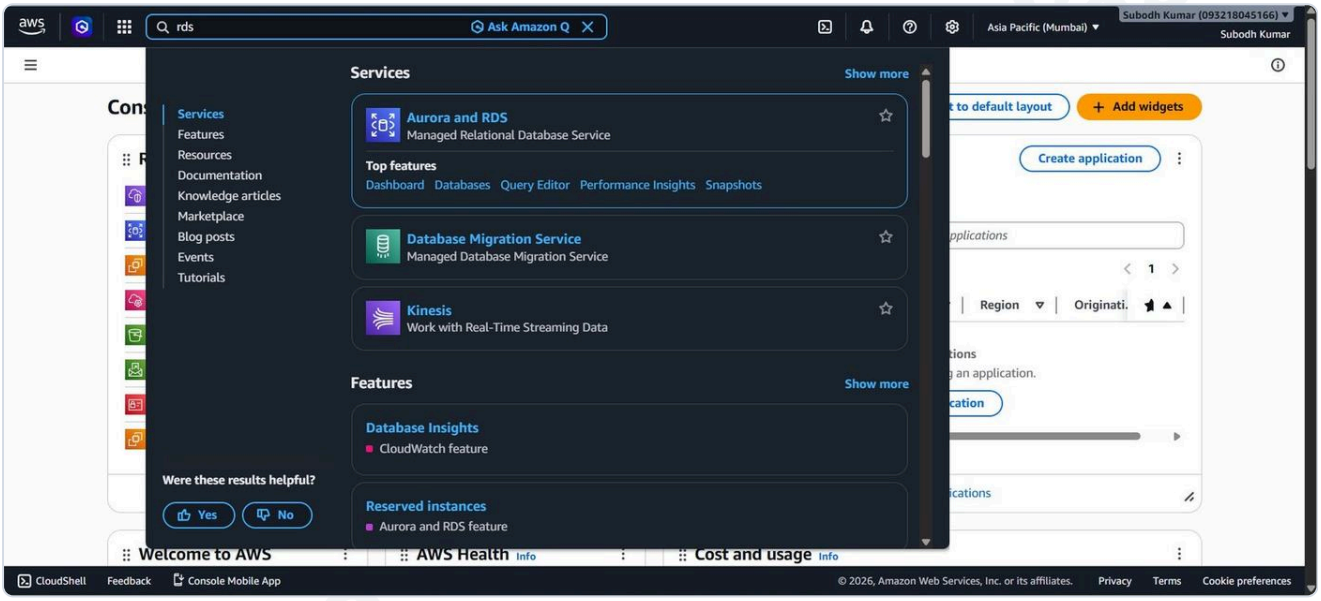
Step 4 Create Database Subnet Group and RDS

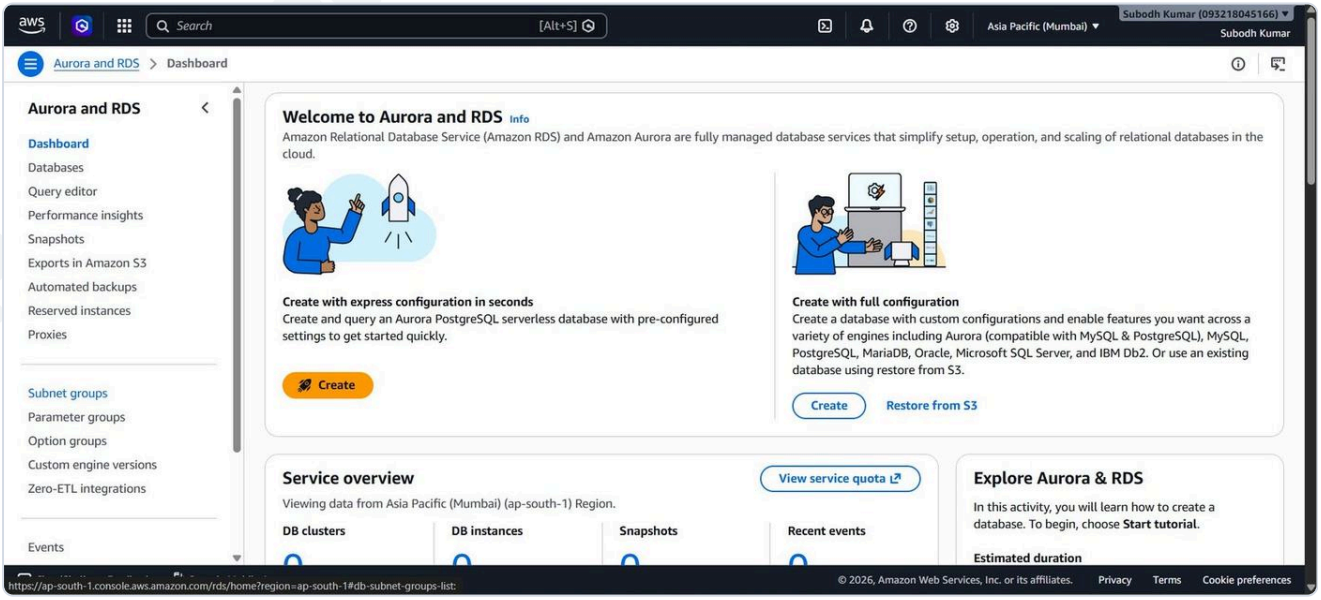
First, we need to **create** a **subnet group**. But **why?**

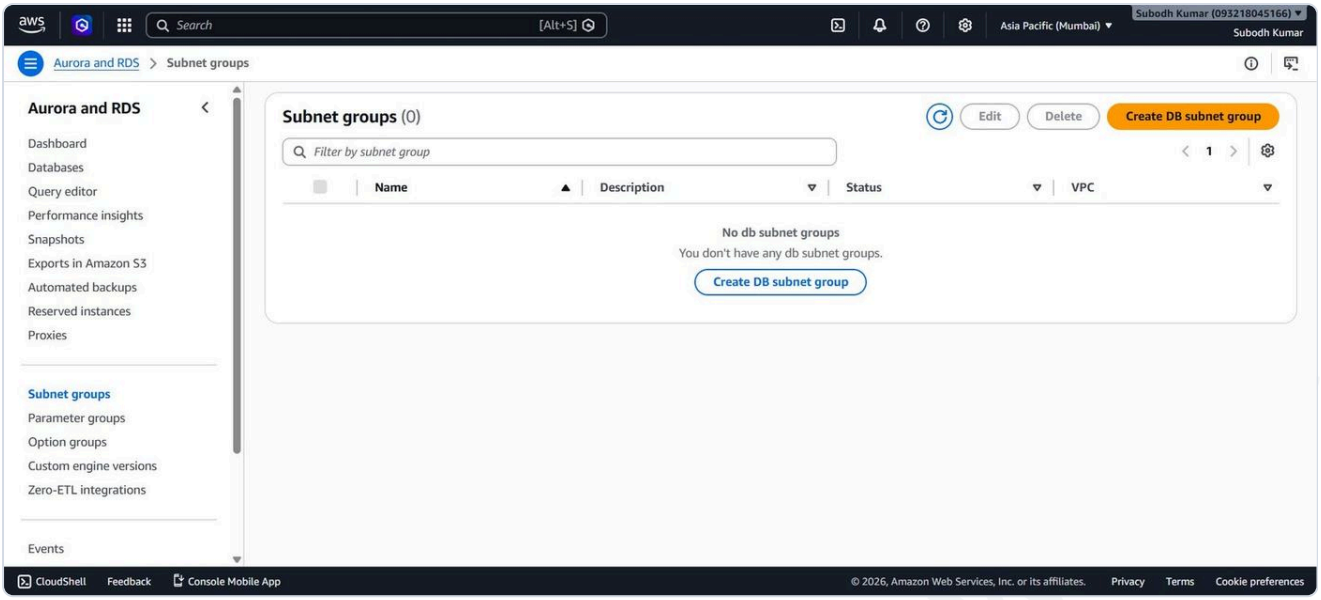
Because, when we **deploy** an **RDS instance** in a **Multi-AZ configuration**, AWS **automatically creates** a **primary database instance** in **one Availability Zone (AZ)** and **a standby replica** in **another AZ** within the **same region.** This setup **requires subnet groups** because **each subnet** in the **group** must belong to a **different AZ**, **ensuring** that your **database instances** are **spread across multiple AZs** for **redundancy** and **failover protection**.

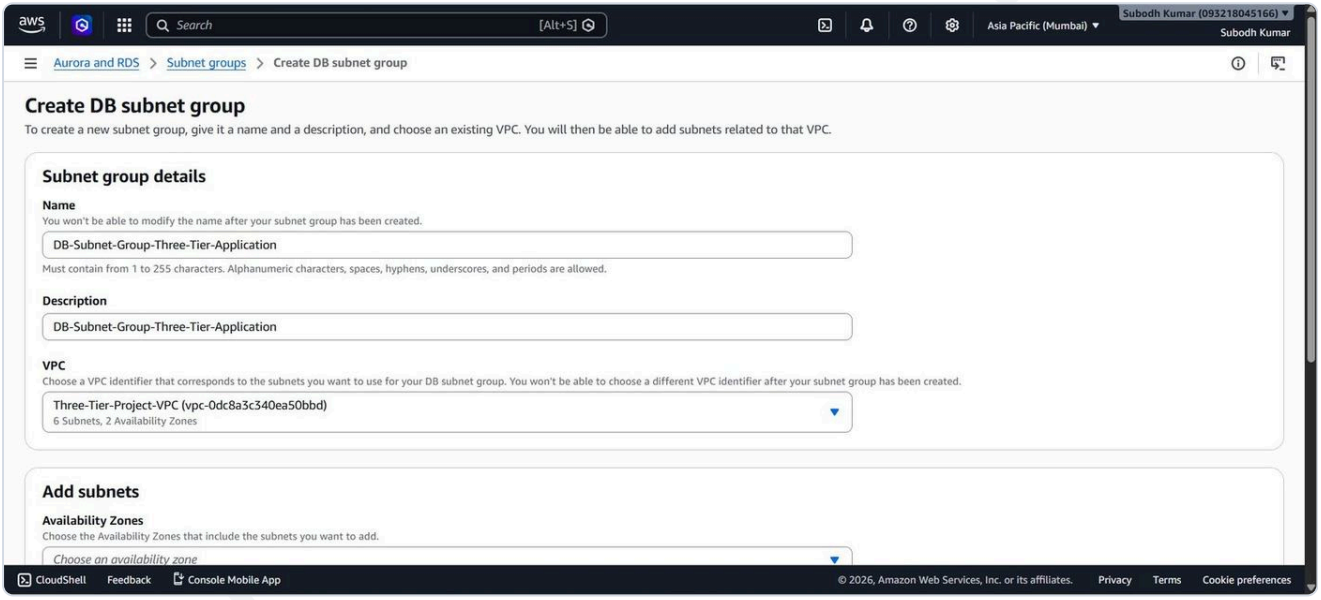
Cool!

Now search for **RDS** in the **search bar**. In the Amazon RDS go to **Subnet groups > Create DB subnet group.** Setup it as follows.



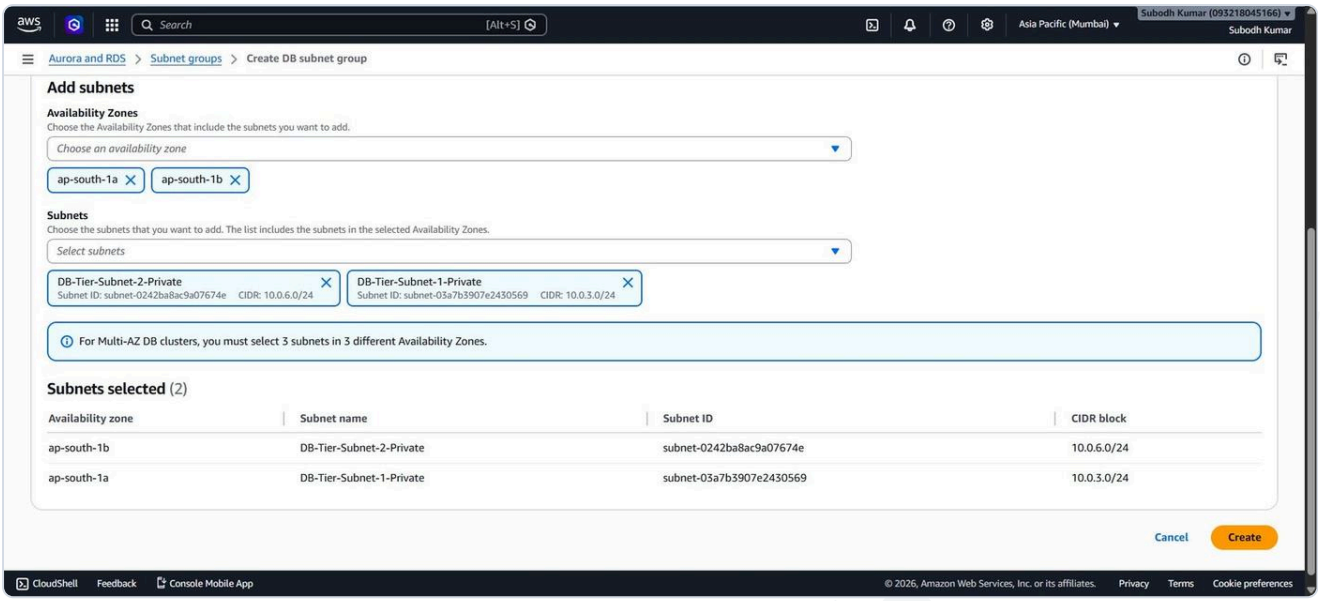




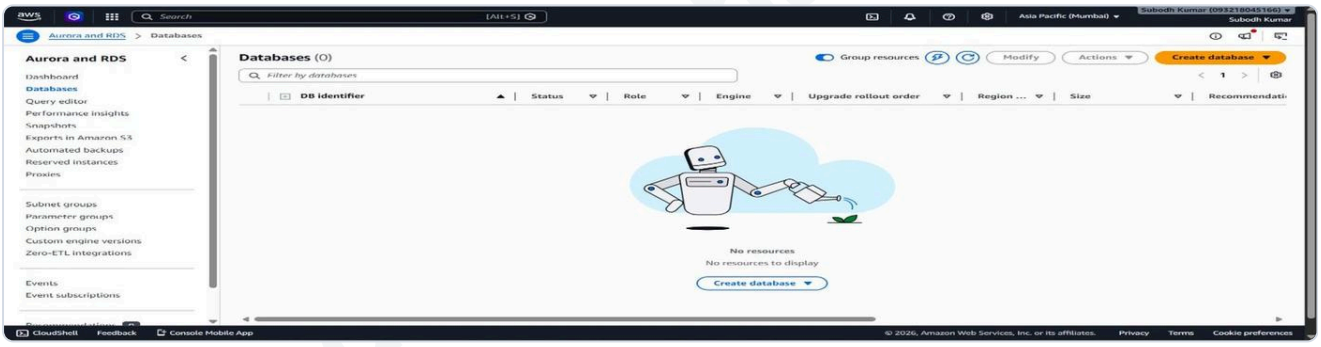


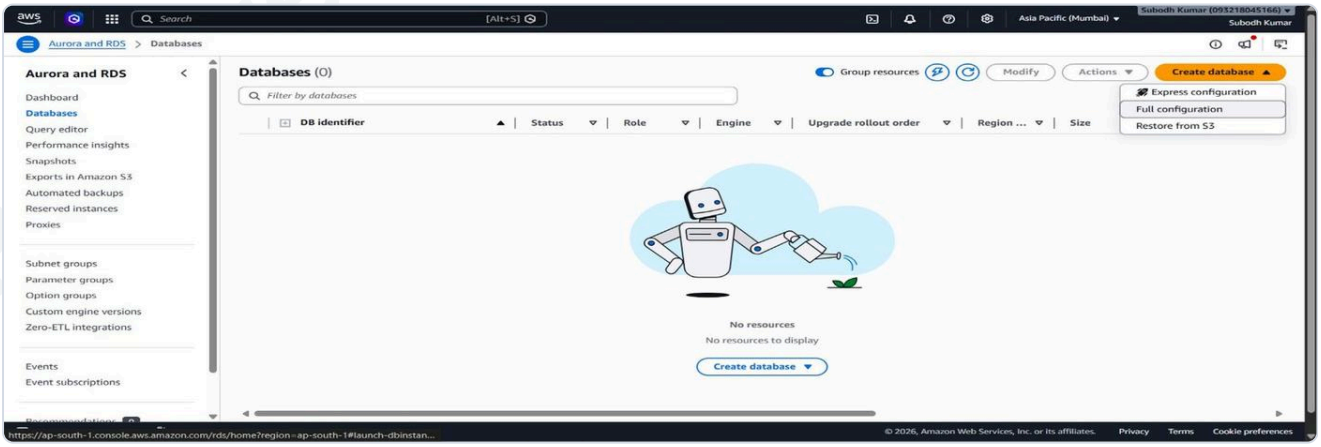
Choose **Availability Zones** that show in the diagram (**1a, 1b**).

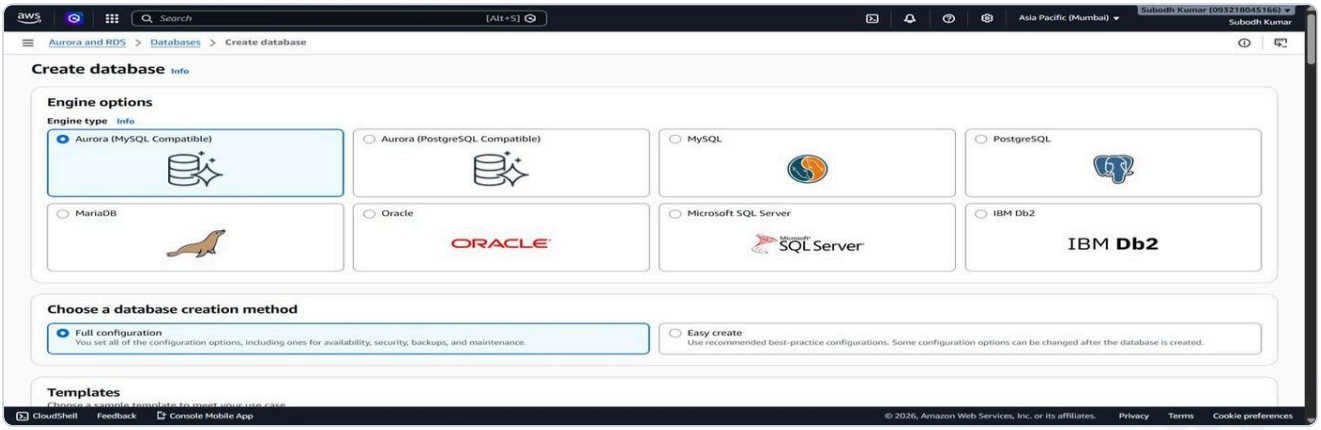
Choose **Subnet** which are **assigned** to the **DB** (**10.0.3.0/24, 10.0.6.0/24**).

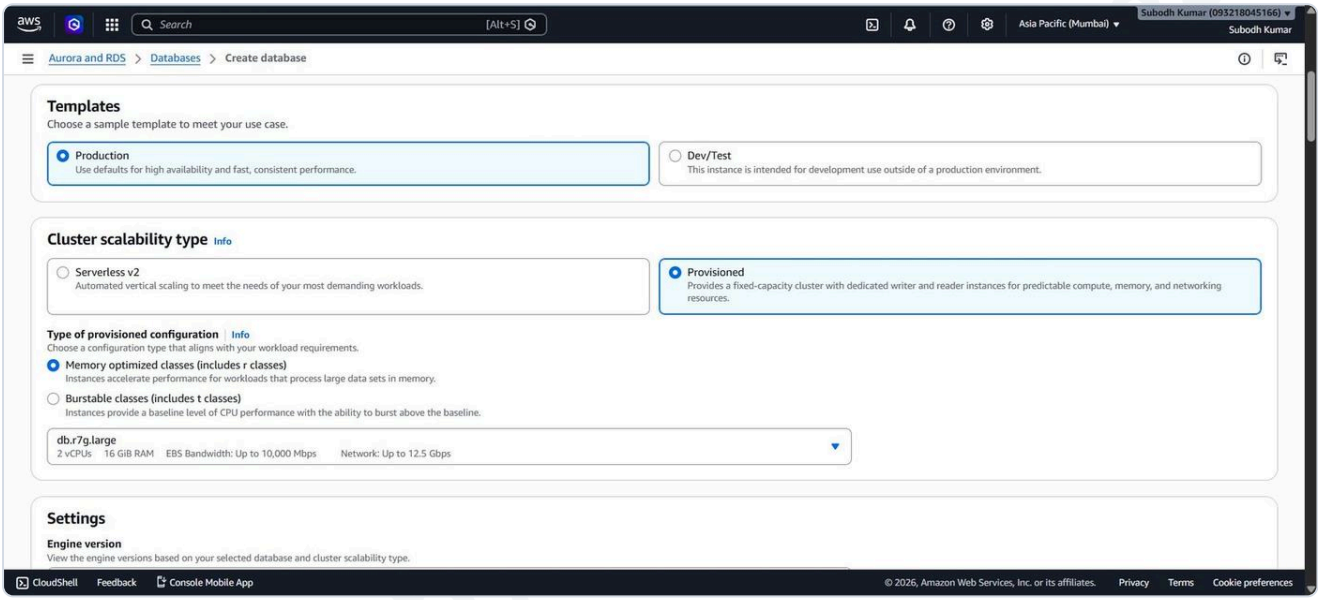


Now go to Amazon RDS **Databases > Create database**. Setup it as follows (In our application we have use MySQL therefore we use **Aurora (MySQL Compatible)** option. We can use MySQL as well).



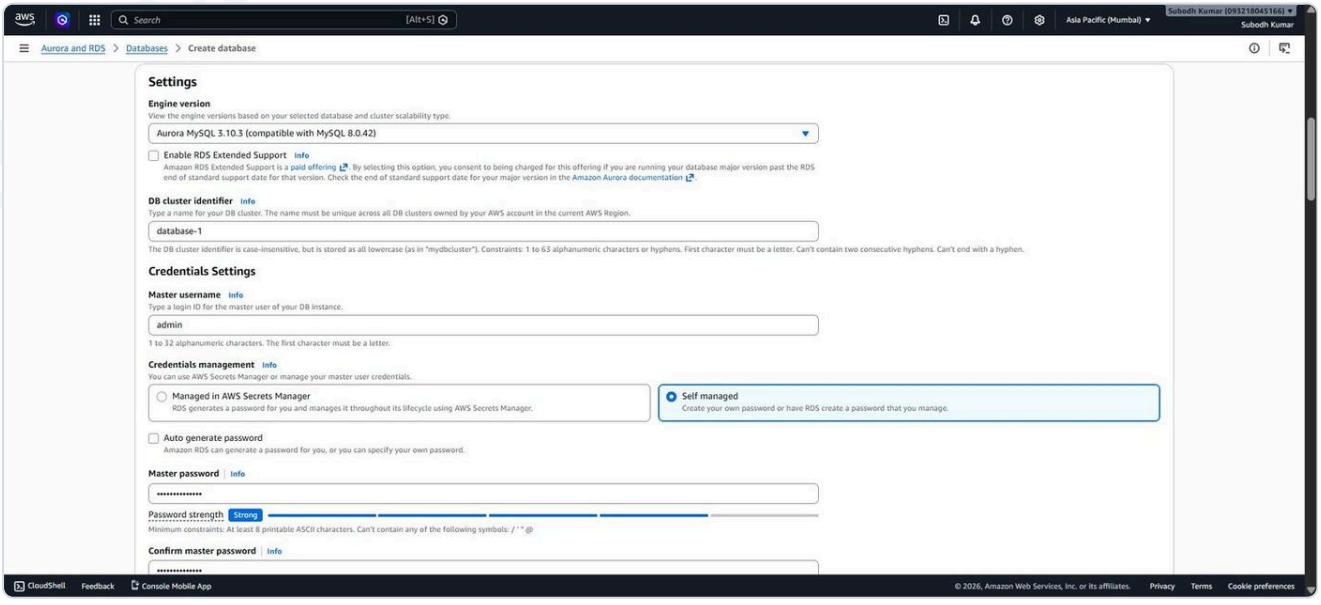


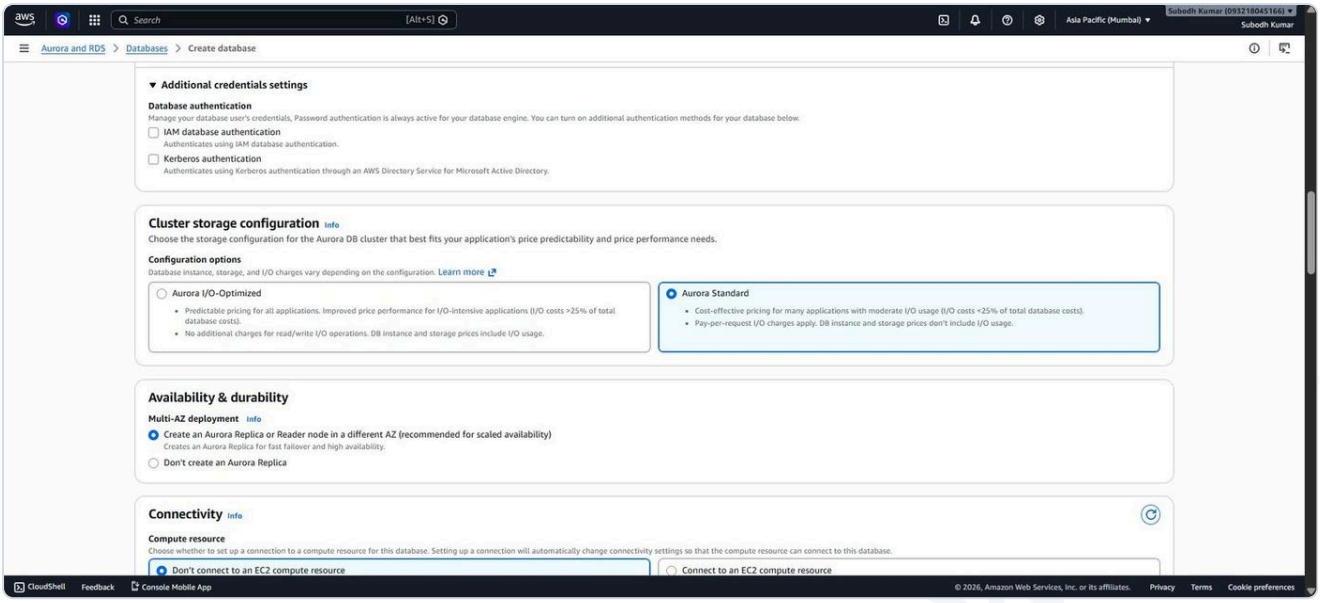


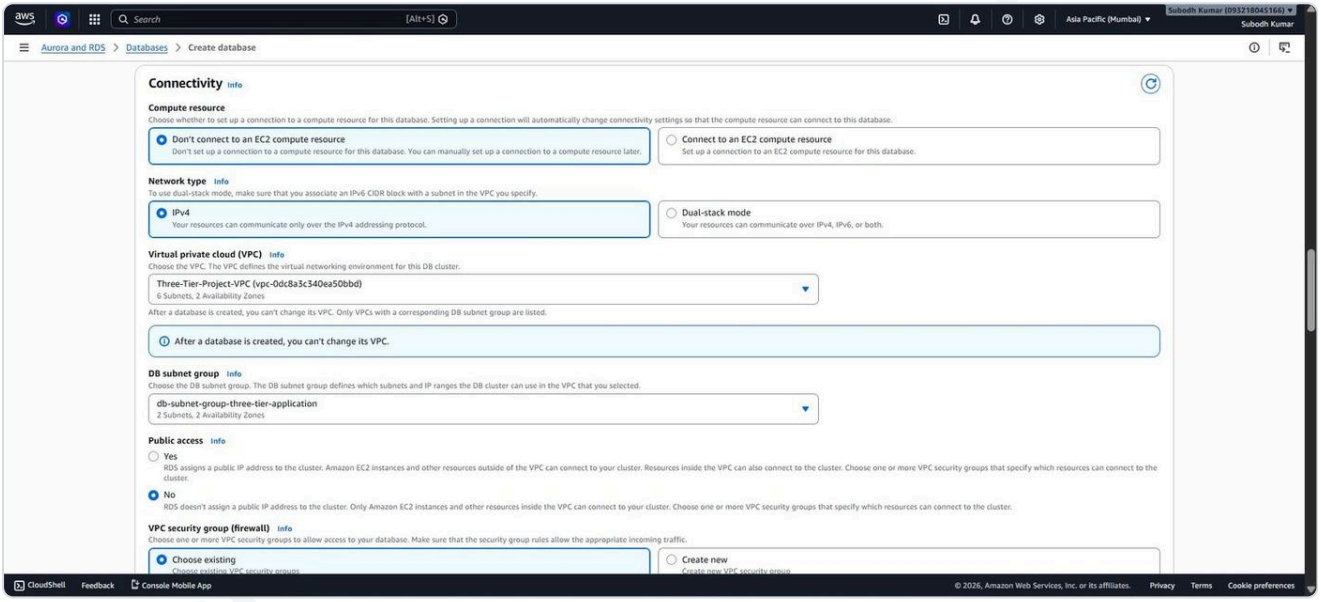


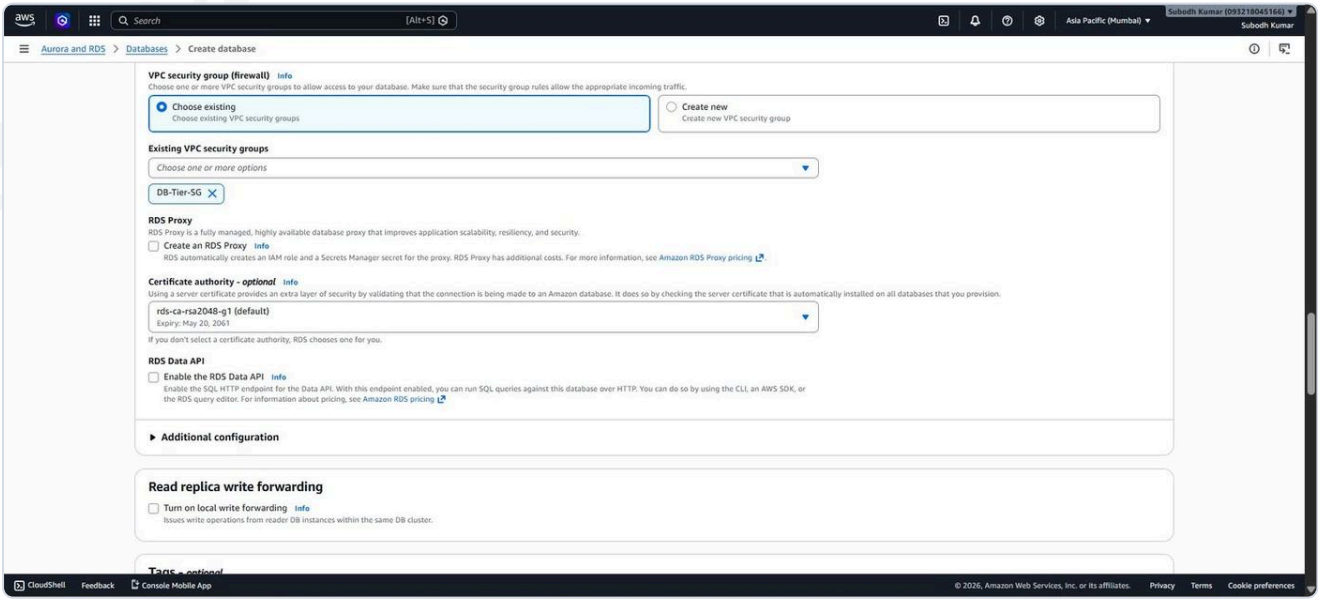
In the following step, remember the username and password. We require them when we access the DB server in the upcoming steps.

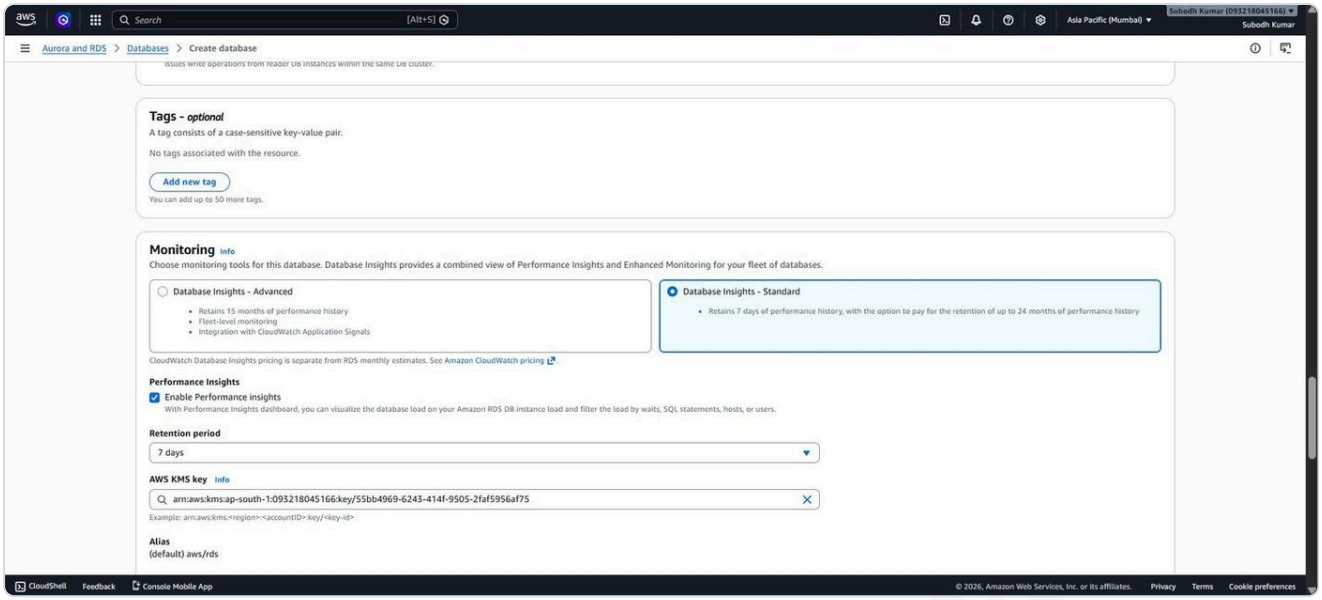
(Note down your master username and master Password)

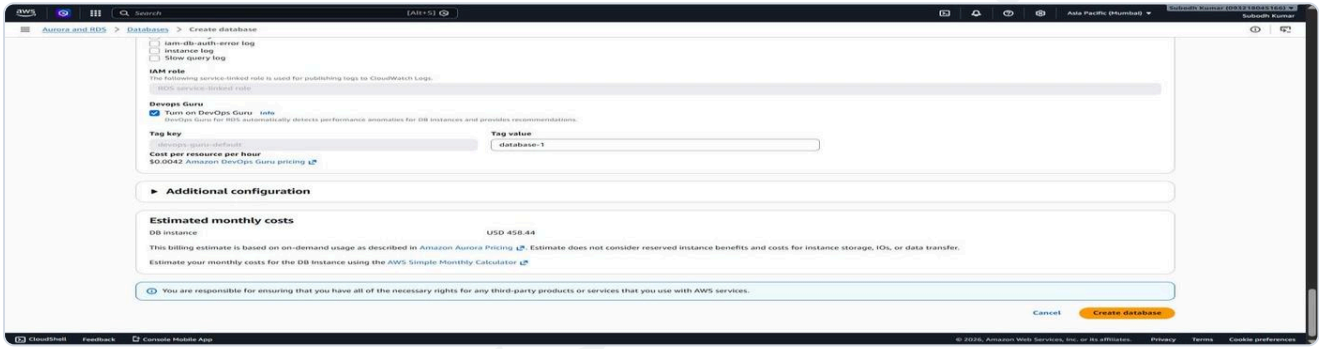



Aurora and RDS > Databases > Create database
Additional credentials settings
Database authentication
IAM database authentication
Kerberos authentication
Cluster storage configuration Info
Configuration options
Aurora I/O-Optimized
Aurora Standard
Availability & durability
Multi-AZ deployment Info
Create an Aurora Replica or Reader node in a different AZ (recommended for scaled availability)
Don't create an Aurora Replica
Connectivity Info
Compute resource
Don't connect to an EC2 compute resource
Connect to an EC2 compute resource


Aurora and RDS > Databases > Create database
Connectivity Info
Compute resource
Don't connect to an EC2 compute resource
Connect to an EC2 compute resource
Network type Info
IPv4
Dual-stack mode
Virtual private cloud (VPC) Info
Three-Tier-Project-VPC (vpc-0dc8a3c340ea50bbd)
6 Subnets, 2 Availability Zones
After a database is created, you can't change its VPC.
DB subnet group Info
db-subnet-group-three-tier-application
2 Subnets, 2 Availability Zones
Public access Info
Yes
No
VPC security group (firewall) Info
Choose existing
Create new

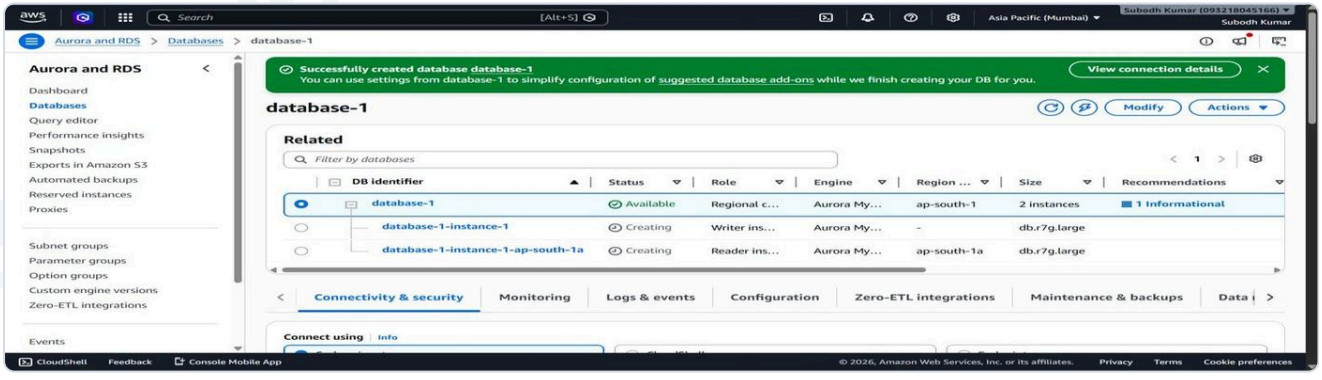

Aurora and RDS > Databases > Create database
VPC security group (firewall) Info
Choose existing
Create new
Existing VPC security groups
DB-Tier-SG
RDS Proxy
Create an RDS Proxy Info
Certificate authority - optional Info
rds-ca-rsa2048-g1 (default)
RDS Data API
Enable the RDS Data API Info
Additional configuration
Read replica write forwarding
Turn on local write forwarding Info





Keep other settings as default. This DB Server creation will cost you some money. But since we delete as soon as project is implemented, it will be few dollars.

Now Click **Create database**. It will take some time to create the server.



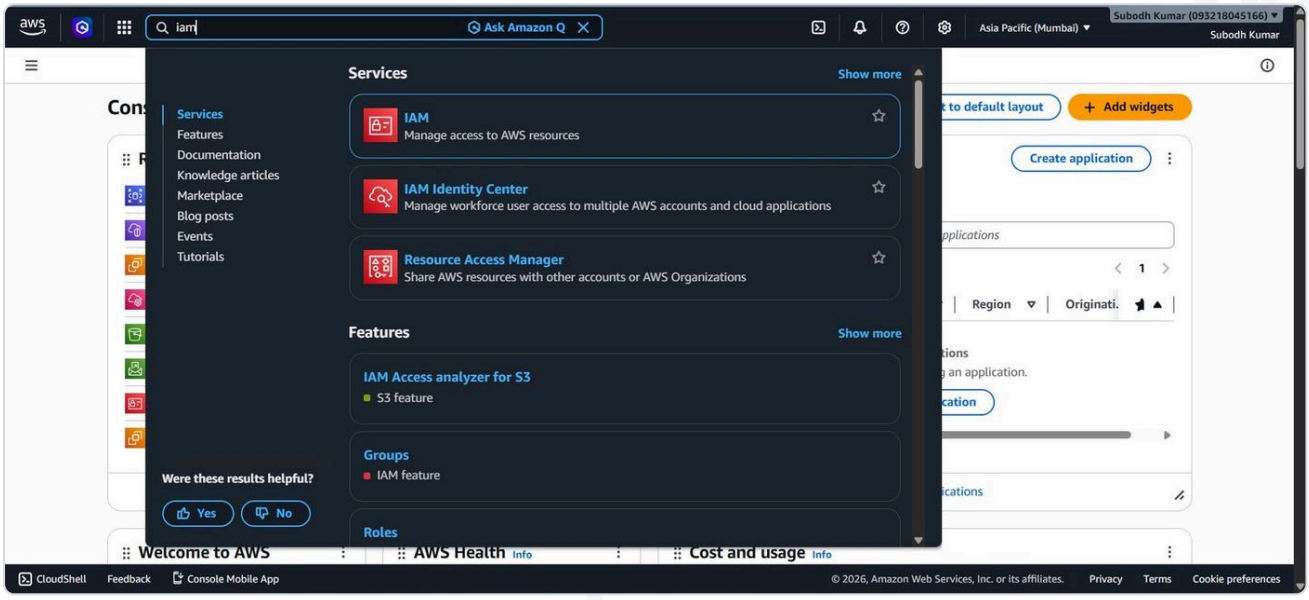
Great we created the database server

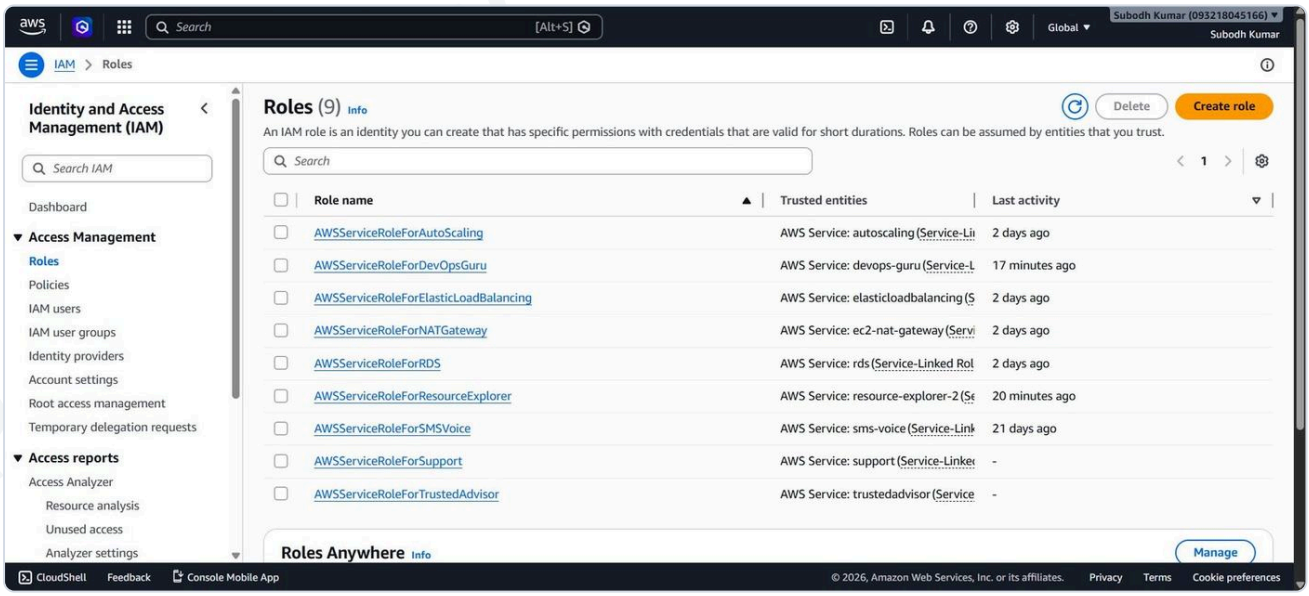
Step 5 Create IAM Role

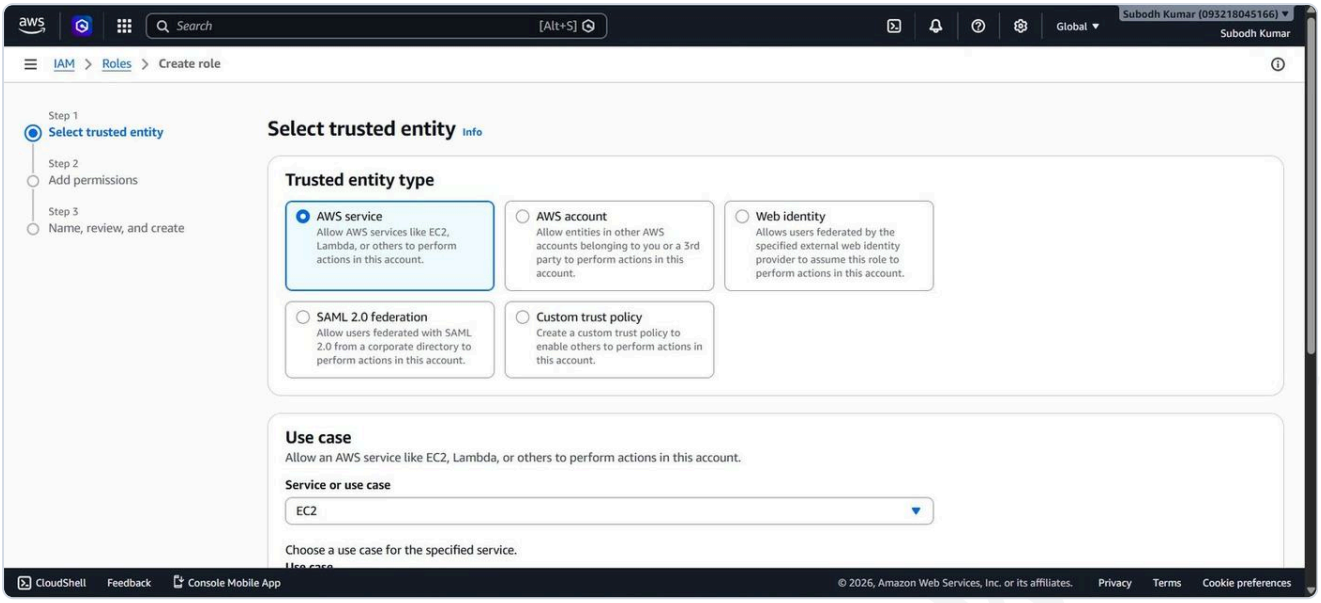
Our application should be Highly secured. In IAM Role we have to use two permission policies in upcoming steps. **S3 read only Access** for access the S3 bucket and **SSM managed instance core** for grants the necessary permissions for an EC2 instance to communicate with the Systems Manager service.

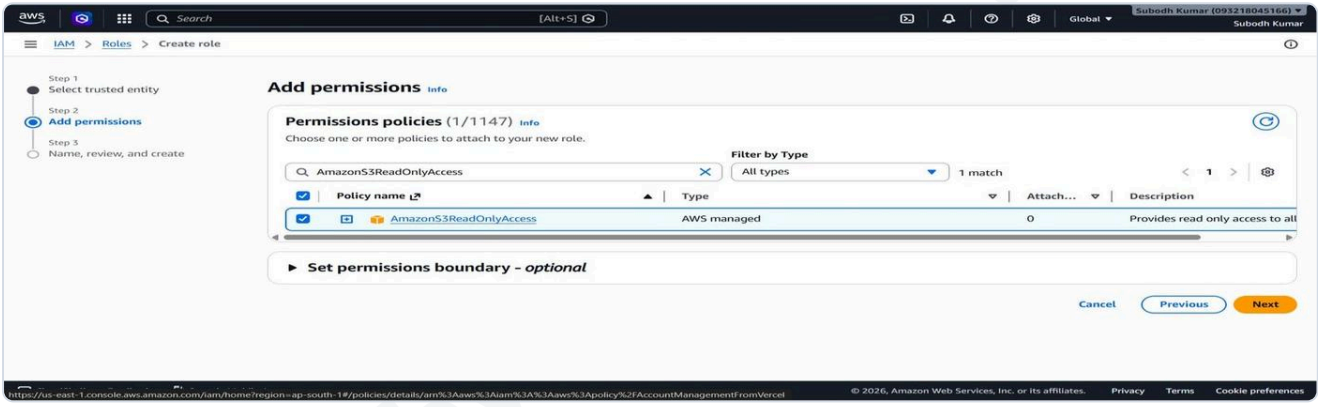
Let's create them

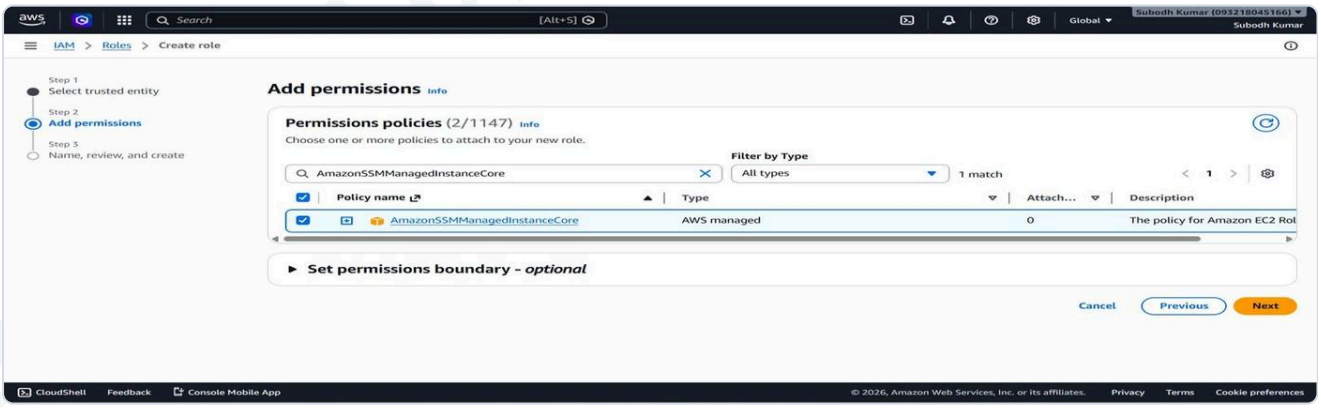
Search for IAM in the search bar. Go to **Roles > Create role**

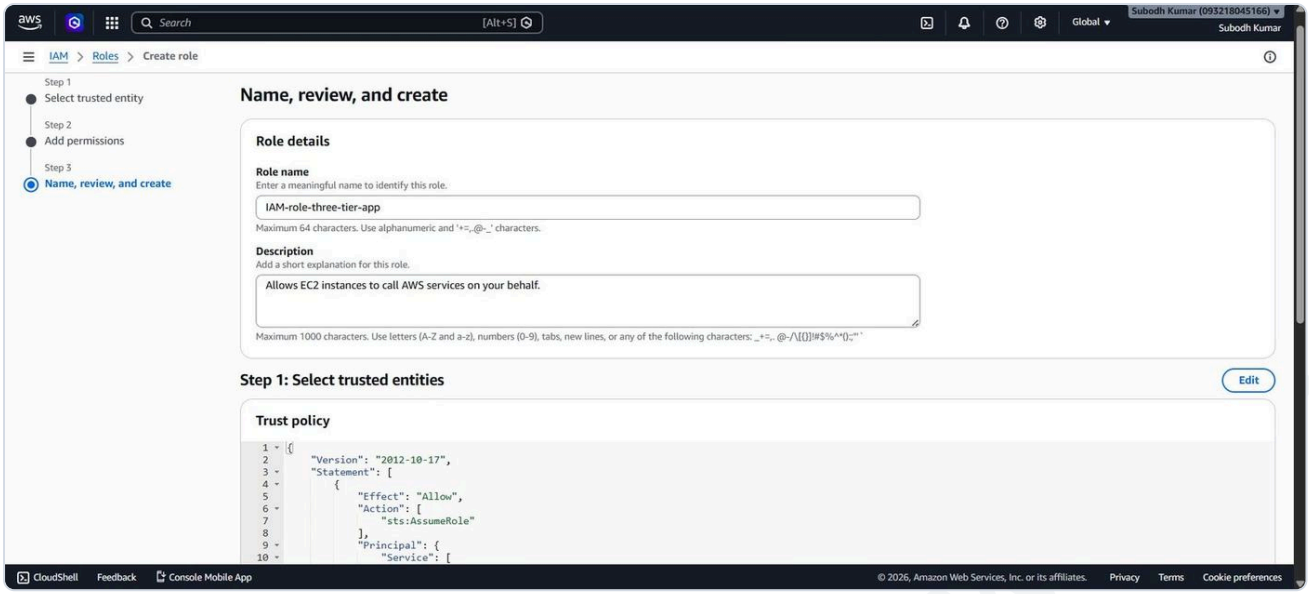


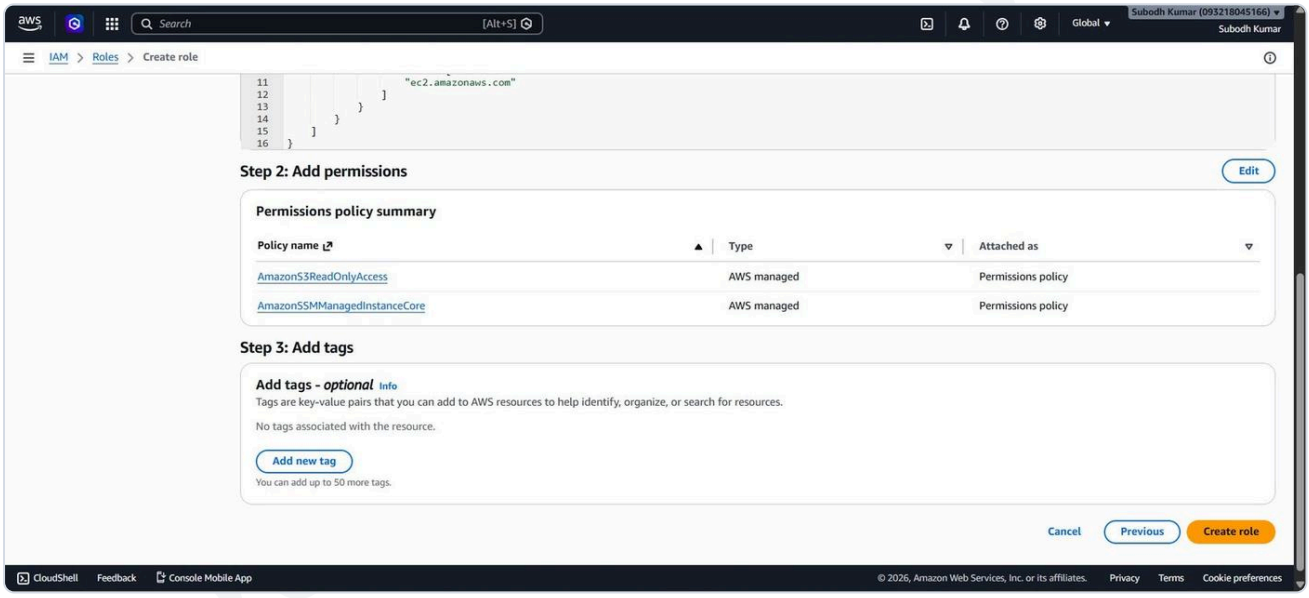



Search [Alt+S]
Global
Subodh Kumar (093218045166)
Subodh Kumar
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Select trusted entity Info
Trusted entity type
AWS service
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
AWS account
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
Web identity
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
SAML 2.0 federation
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
Custom trust policy
Create a custom trust policy to enable others to perform actions in this account.
Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.
Service or use case
EC2
Choose a use case for the specified service.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Global
Subodh Kumar (093218045166)
Subodh Kumar
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (1/1147) Info
Choose one or more policies to attach to your new role.
Filter by Type
AmazonS3ReadOnlyAccess
All types
1 match
Policy name
Type
Attach...
Description
AmazonS3ReadOnlyAccess
AWS managed
0
Provides read only access to all
Set permissions boundary - optional
Cancel
Previous
Next
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Global
Subodh Kumar (093218045166)
Subodh Kumar
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (2/1147) Info
Choose one or more policies to attach to your new role.
Filter by Type
AmazonSSMManagedInstanceCore
All types
1 match
Policy name
Type
Attach...
Description
AmazonSSMManagedInstanceCore
AWS managed
0
The policy for Amazon EC2 Rol
Set permissions boundary - optional
Cancel
Previous
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

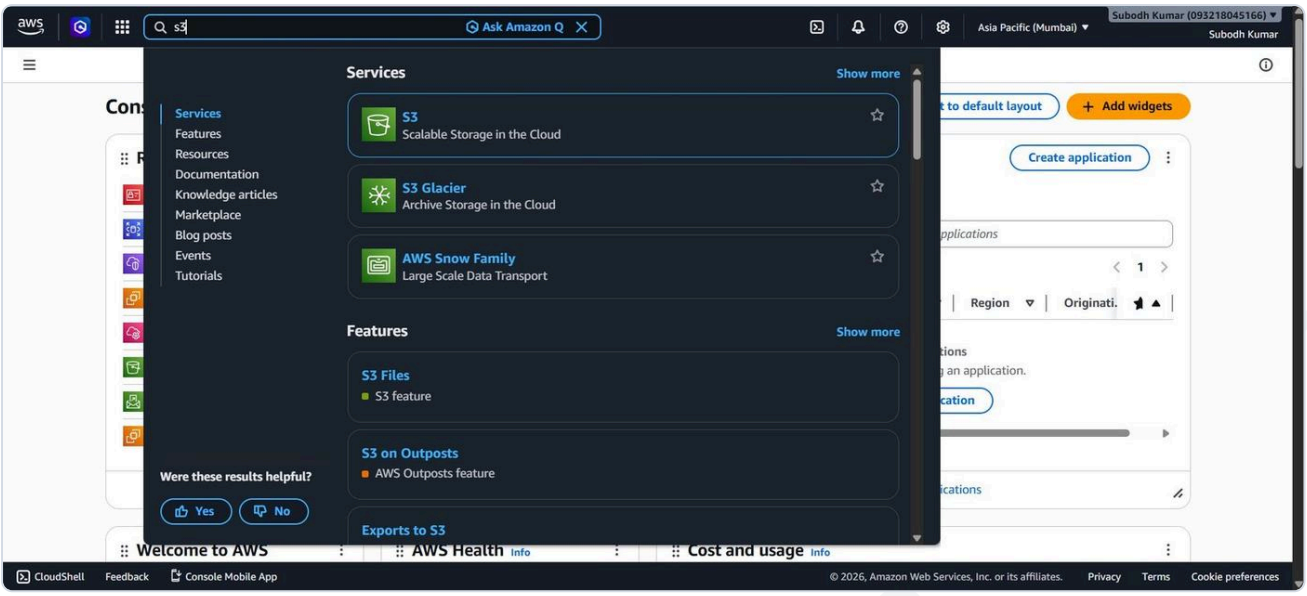


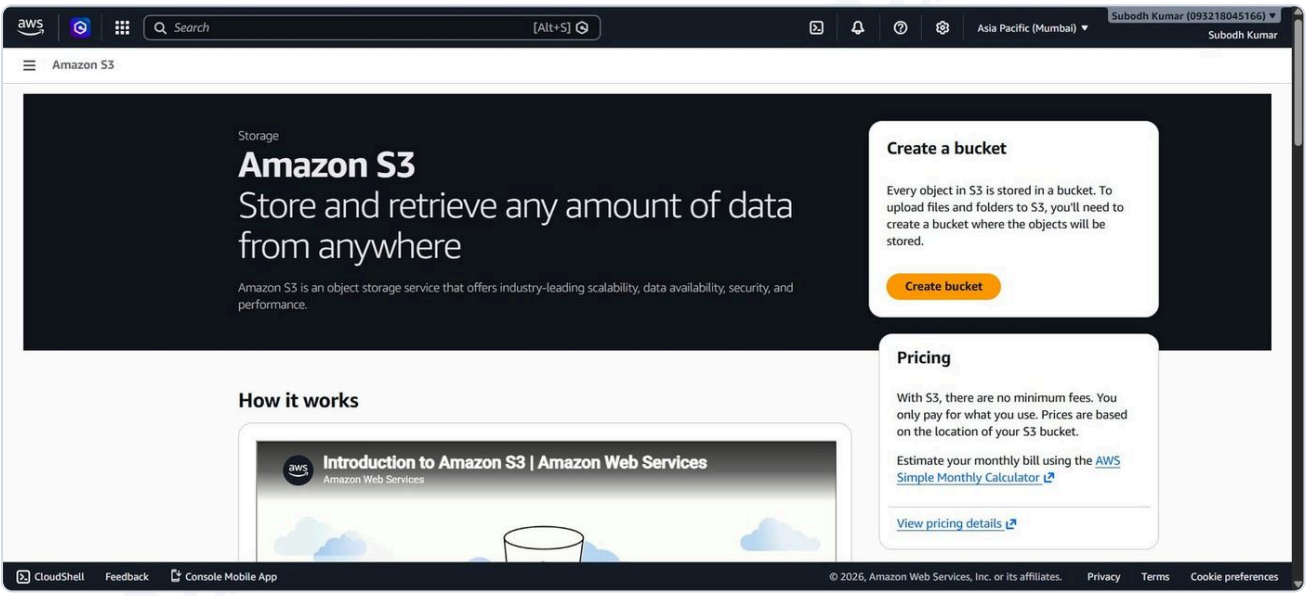


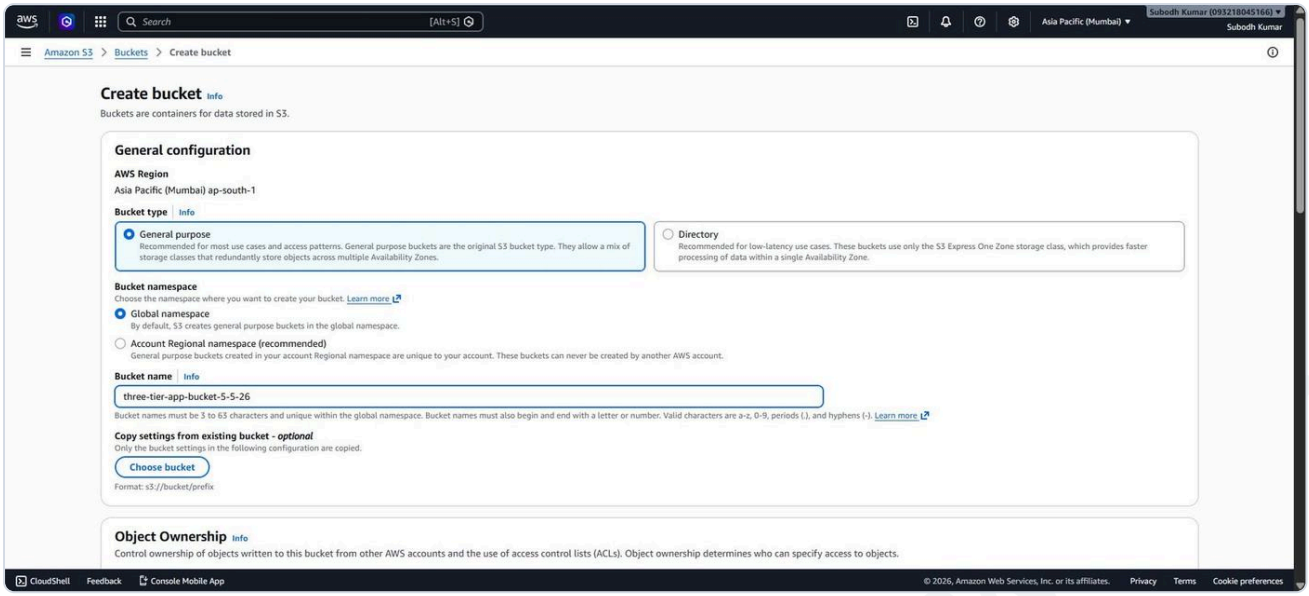
Step 6 Create an S3 Bucket and Upload the Source Code

When we setup application on servers it is easy to if the source codes are in a s3 bucket

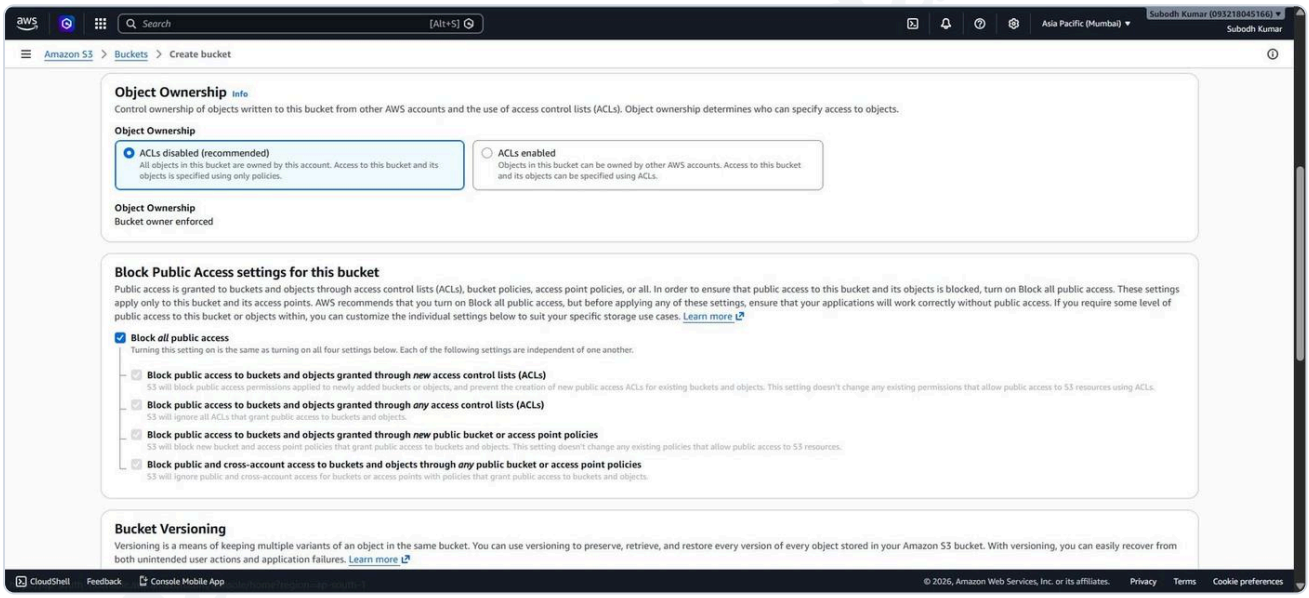
Go to **search bar** and search for **S3**. Then click **create bucket.** Bucket name should be unique one.

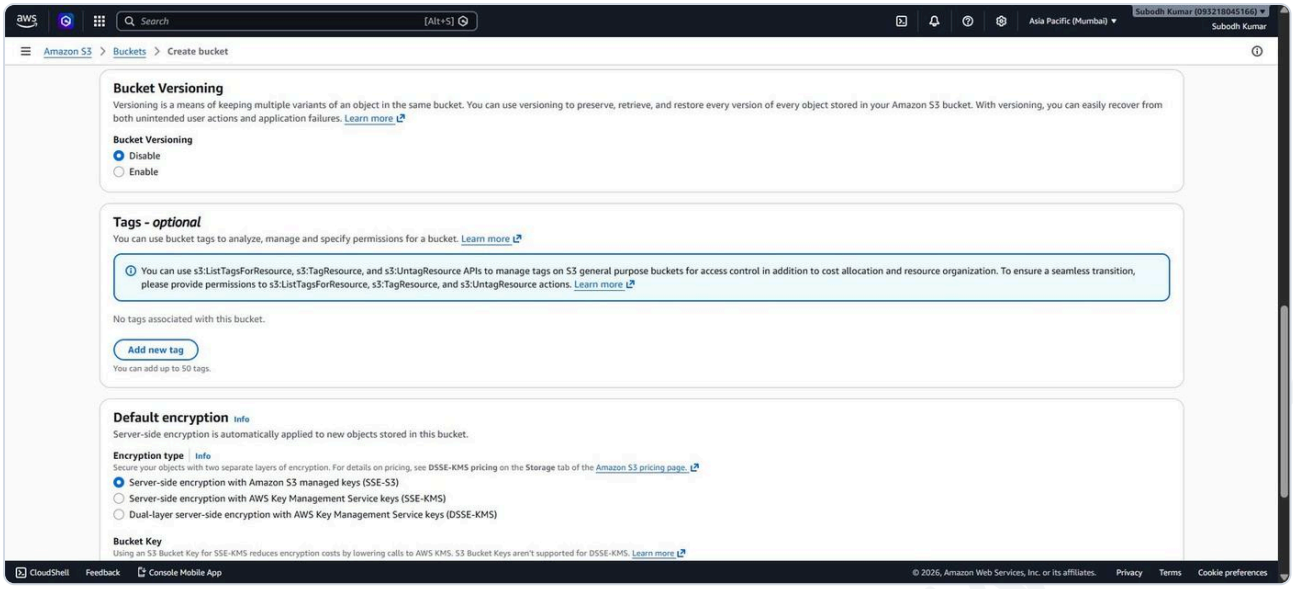


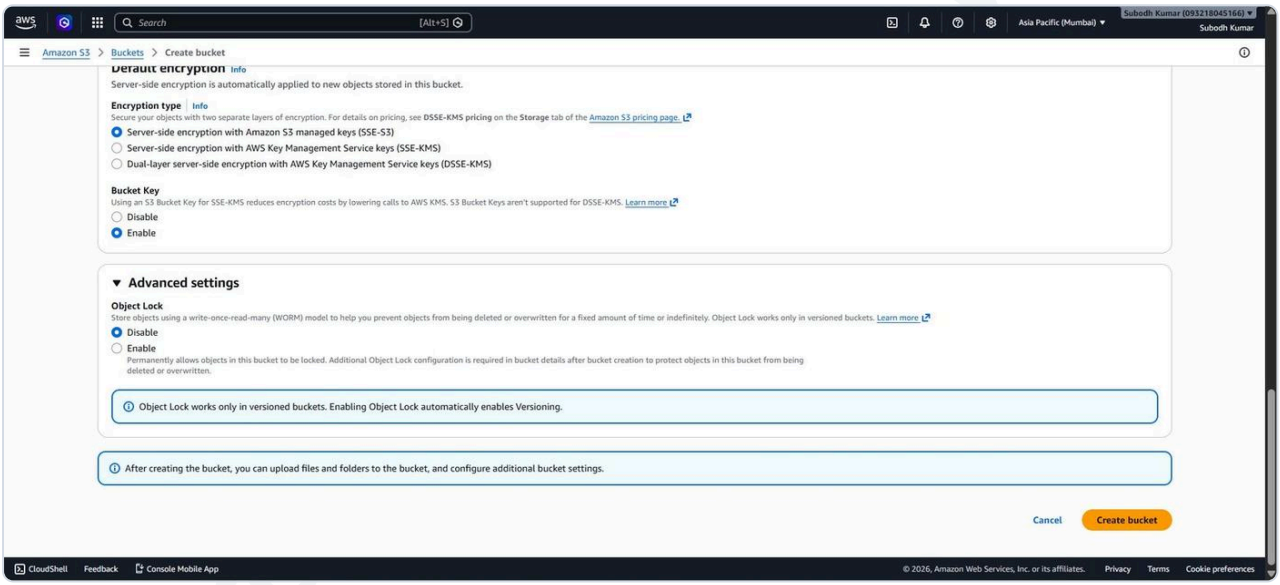




Let all other options default and click create bucket.



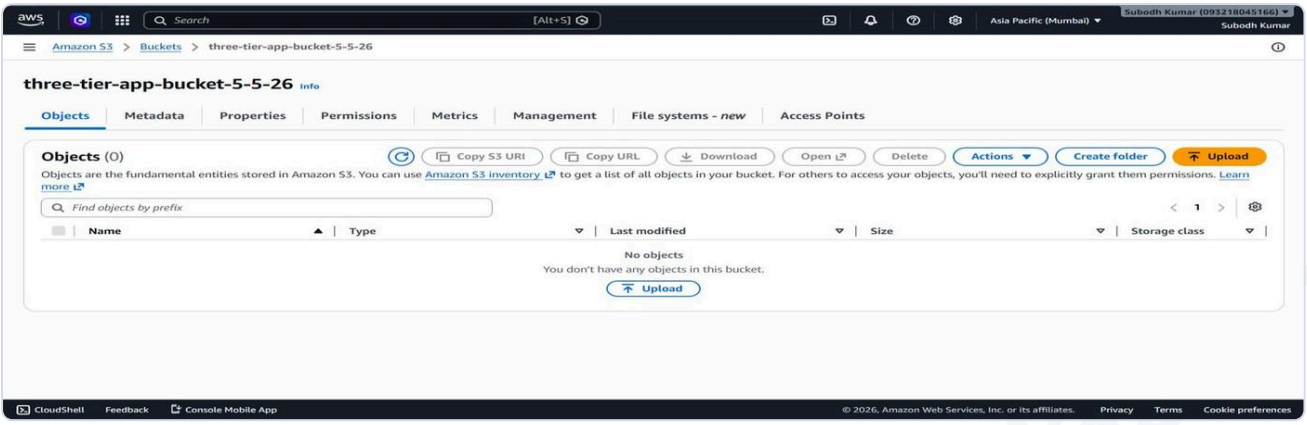


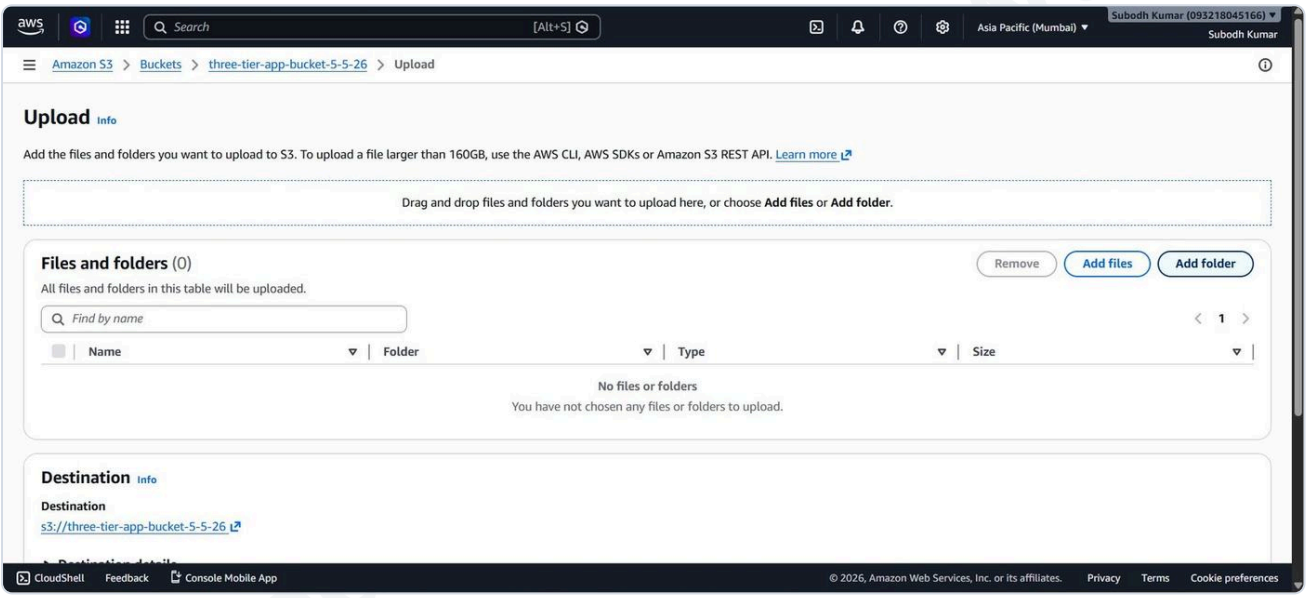


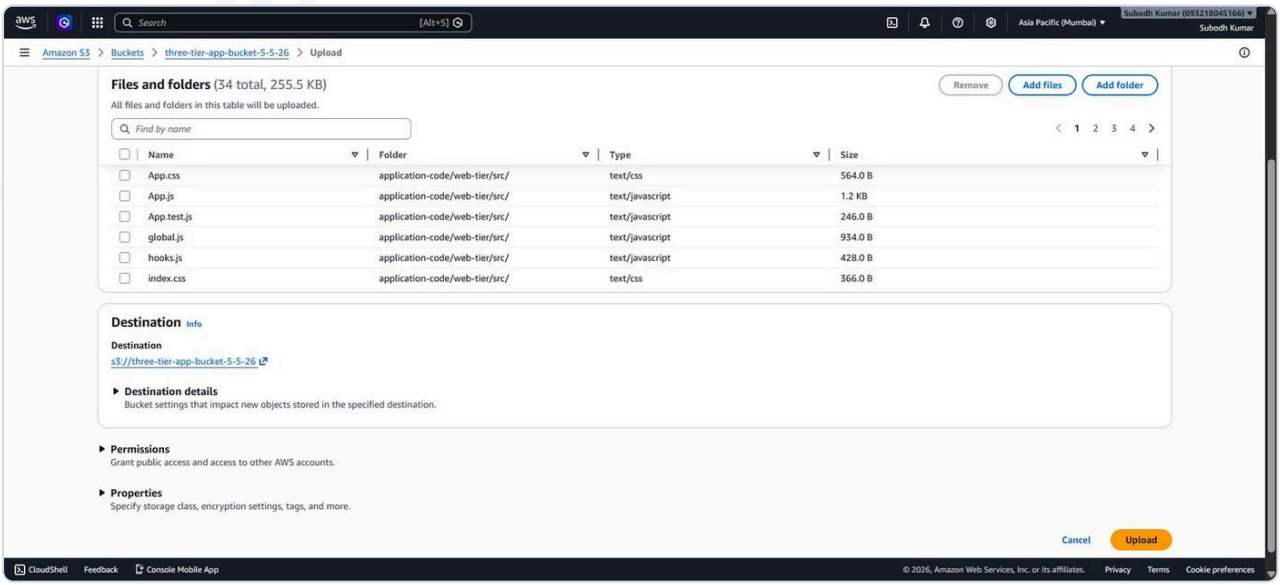
Now go to the following repository and clone it in your pc. It contains all source code.

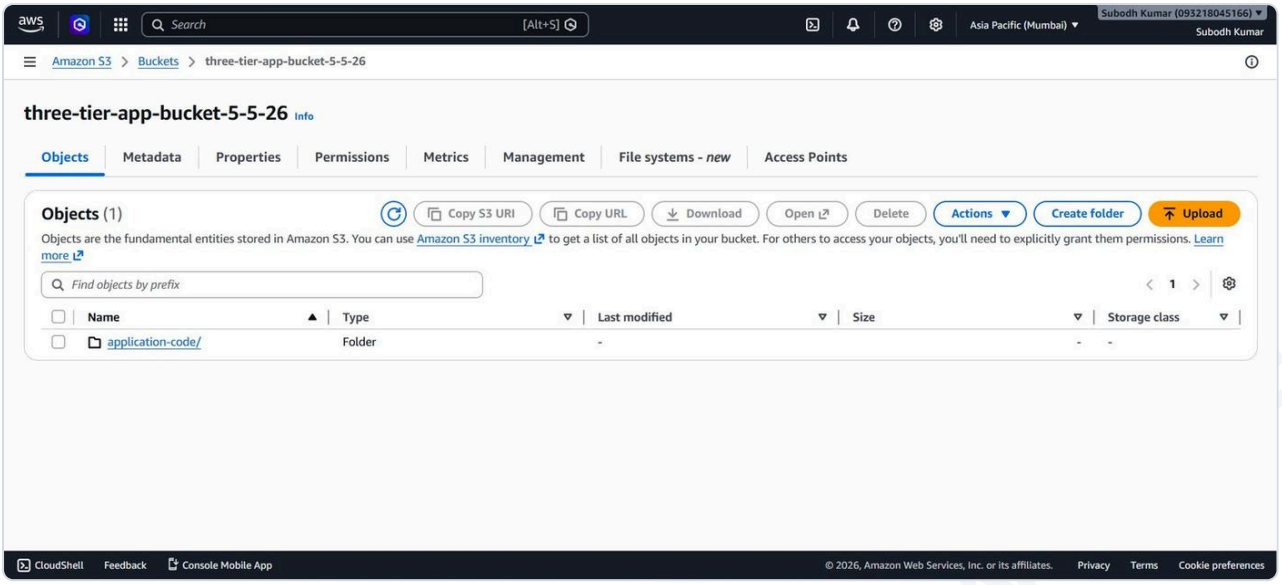
https://github.com/sukhada30github/Three-Tier-Project.git

Now go to the bucket and upload the application-code folder. There are some changes to make in the files. We will do it in the upcoming steps.







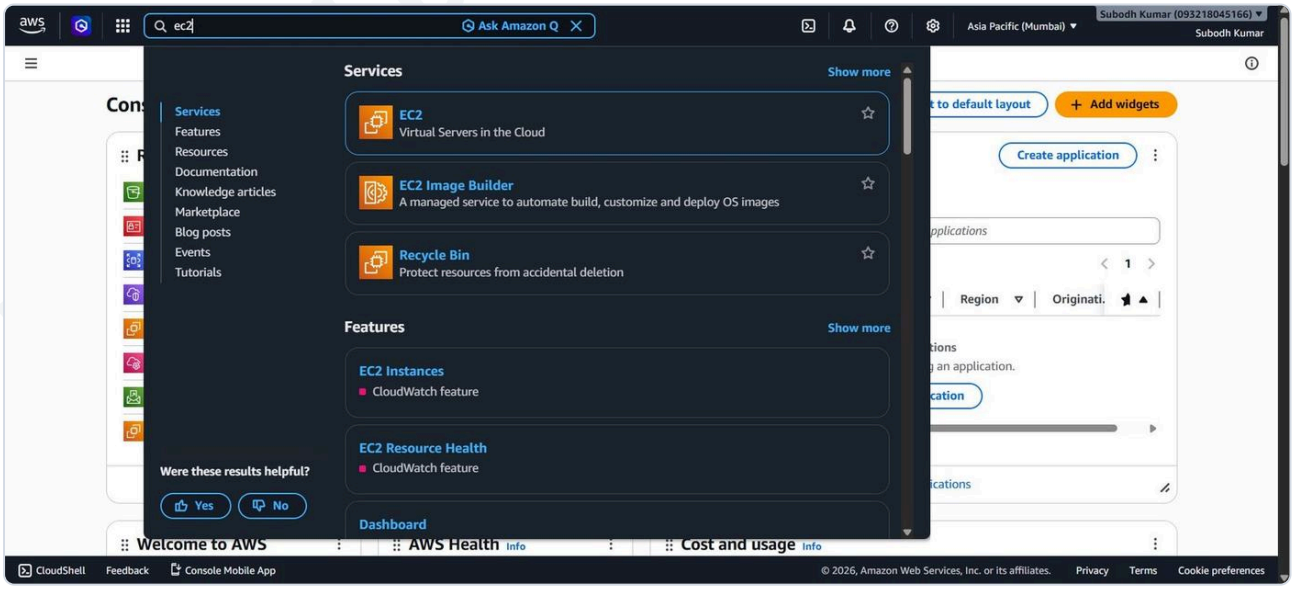


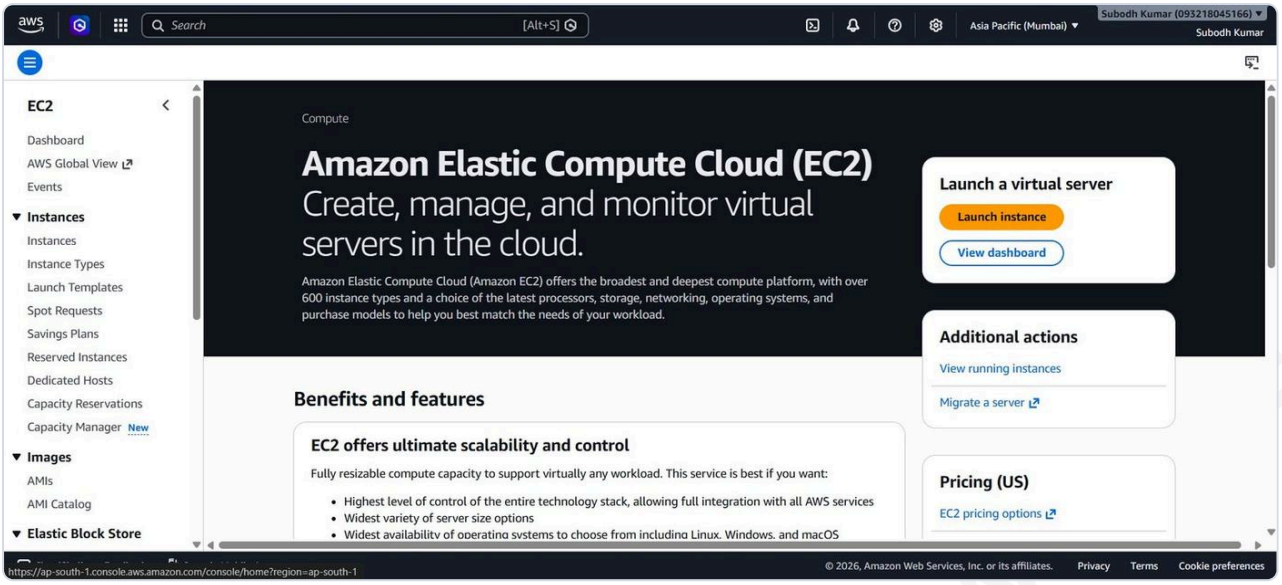
Our application should be highly scalable. To achieve this we use Auto Scaling Groups. To create servers and their configuration automatically we use Launch Templates, so we need Amazon Machine Images (AMIs) of our servers. For that we create a test server using an EC2 instance, configure the application on it and then create an AMI from it. In this way we implement our Web Servers and App Servers.

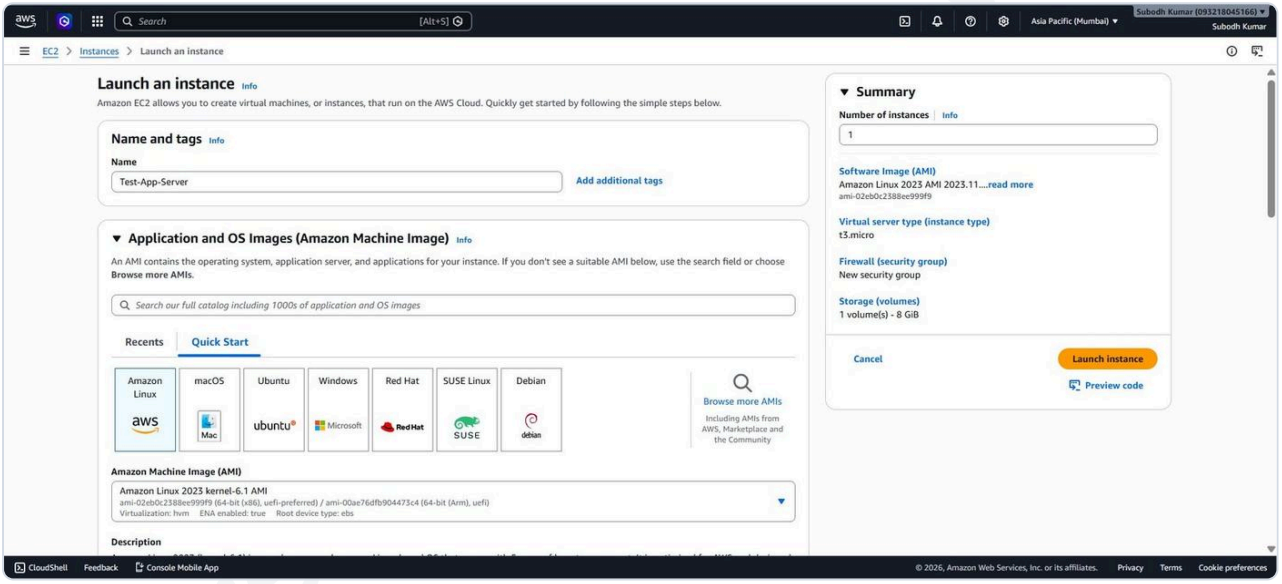
Let's do

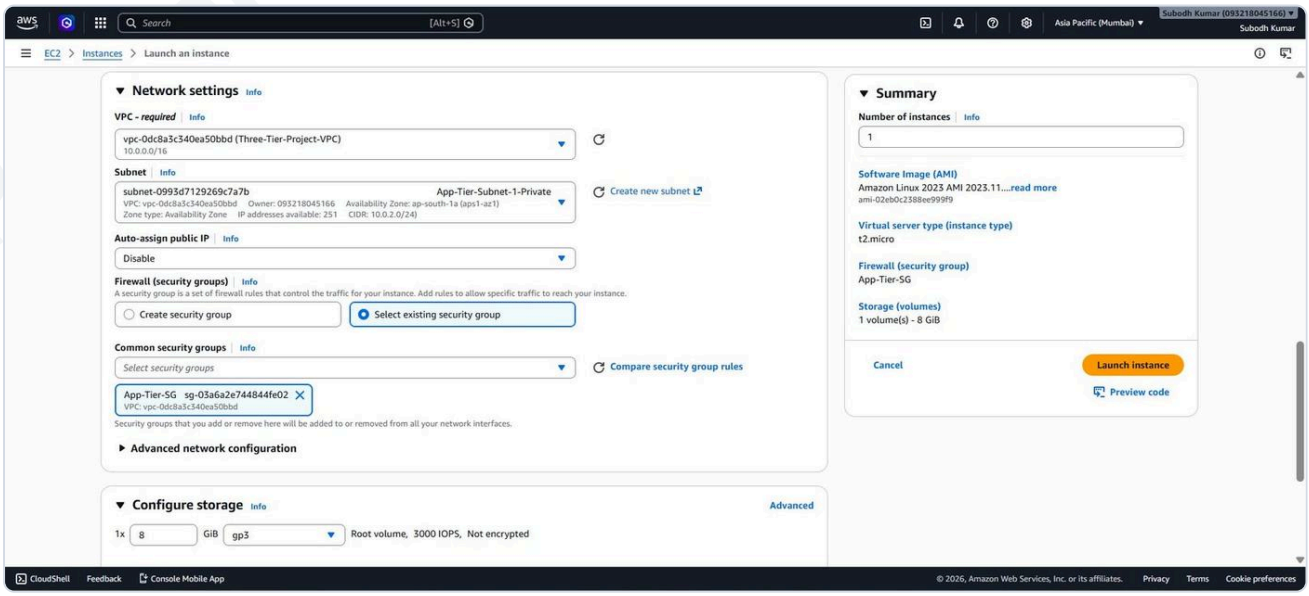
Step 7 Create Test App Server and Set It Up

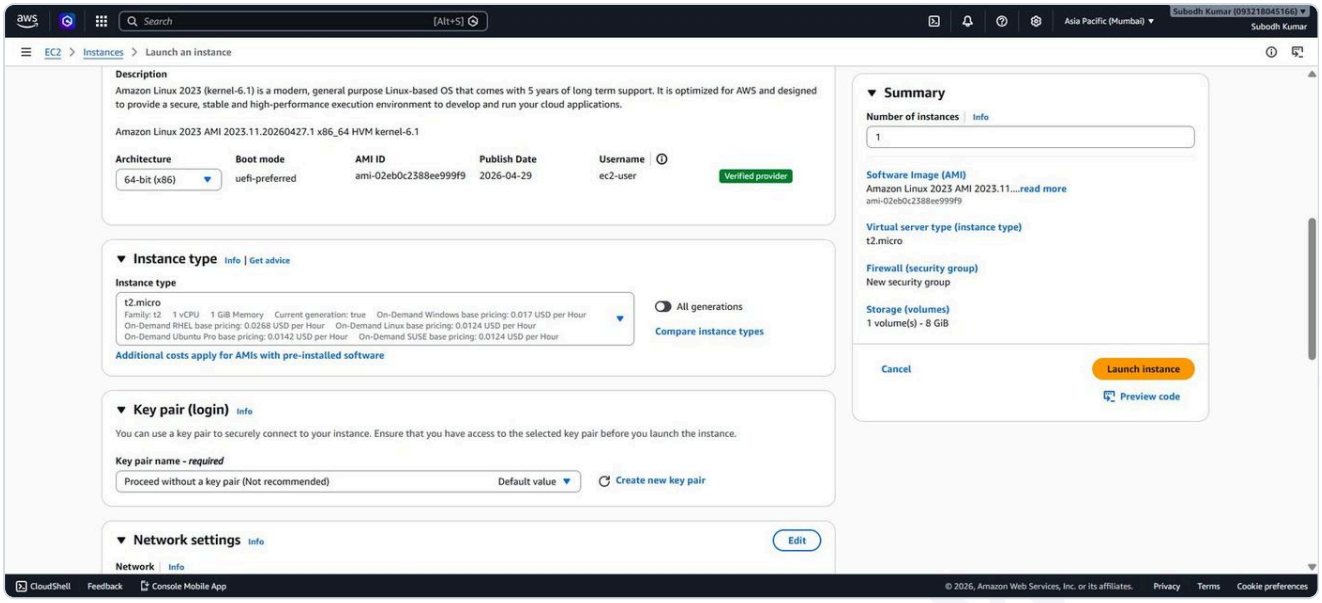
Search for EC2 in the search bar. Go to **Instances > Launch instances.** Setup it as follows.

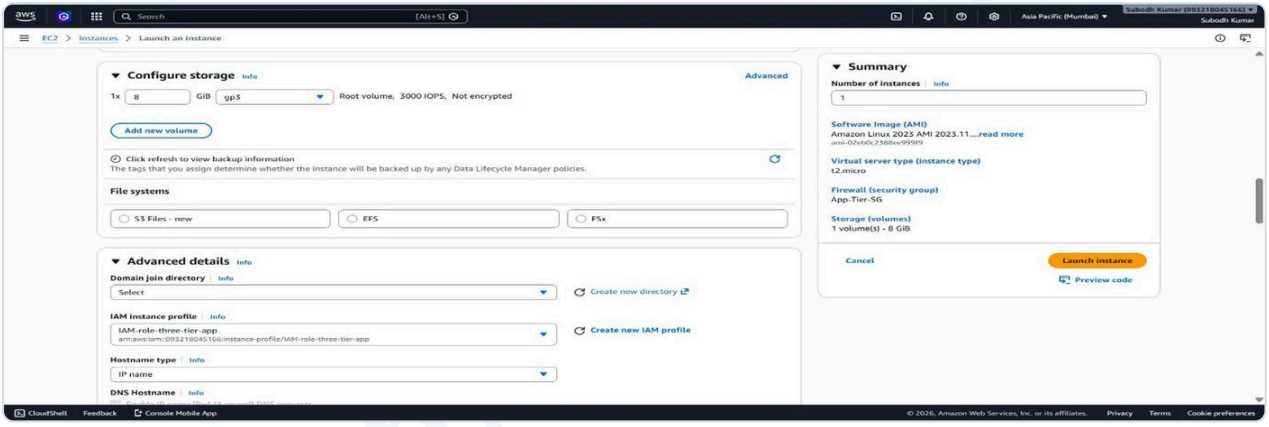



Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Compute
Amazon Elastic Compute Cloud (EC2)
Create, manage, and monitor virtual servers in the cloud.
Amazon Elastic Compute Cloud (Amazon EC2) offers the broadest and deepest compute platform, with over 600 instance types and a choice of the latest processors, storage, networking, operating systems, and purchase models to help you best match the needs of your workload.
Launch a virtual server
Launch instance
View dashboard
Additional actions
View running instances
Migrate a server
Benefits and features
EC2 offers ultimate scalability and control
Fully resizable compute capacity to support virtually any workload. This service is best if you want:
Highest level of control of the entire technology stack, allowing full integration with all AWS services
Widest variety of server size options
Widest availability of operating systems to choose from including Linux, Windows, and macOS
Pricing (US)
EC2 pricing options
https://ap-south-1.console.aws.amazon.com/console/home?region=ap-south-1
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


EC2 > Instances > Launch an instance
Launch an instance Info
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
Test-App-Server
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
SUSE Linux
Debian
Browse more AMIs
Including AMIs from AWS, Marketplace and the Community
Amazon Machine Image (AMI)
Amazon Linux 2023 kernel-6.1 AMI
Description
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
Virtual server type (instance type)
t3.micro
Firewall (security group)
New security group
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


EC2 > Instances > Launch an instance
Network settings Info
VPC - required Info
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Subnet Info
subnet-0993d7129269c7a7b App-Tier-Subnet-1-Private
Create new subnet
Auto-assign public IP Info
Disable
Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
Create security group
Select existing security group
Common security groups Info
Select security groups
Compare security group rules
App-Tier-SG sg-03a6a2e744844fe02
Security groups that you add or remove here will be added to or removed from all your network interfaces.
Advanced network configuration
Configure storage Info
Advanced
1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
Virtual server type (instance type)
t2.micro
Firewall (security group)
App-Tier-SG
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

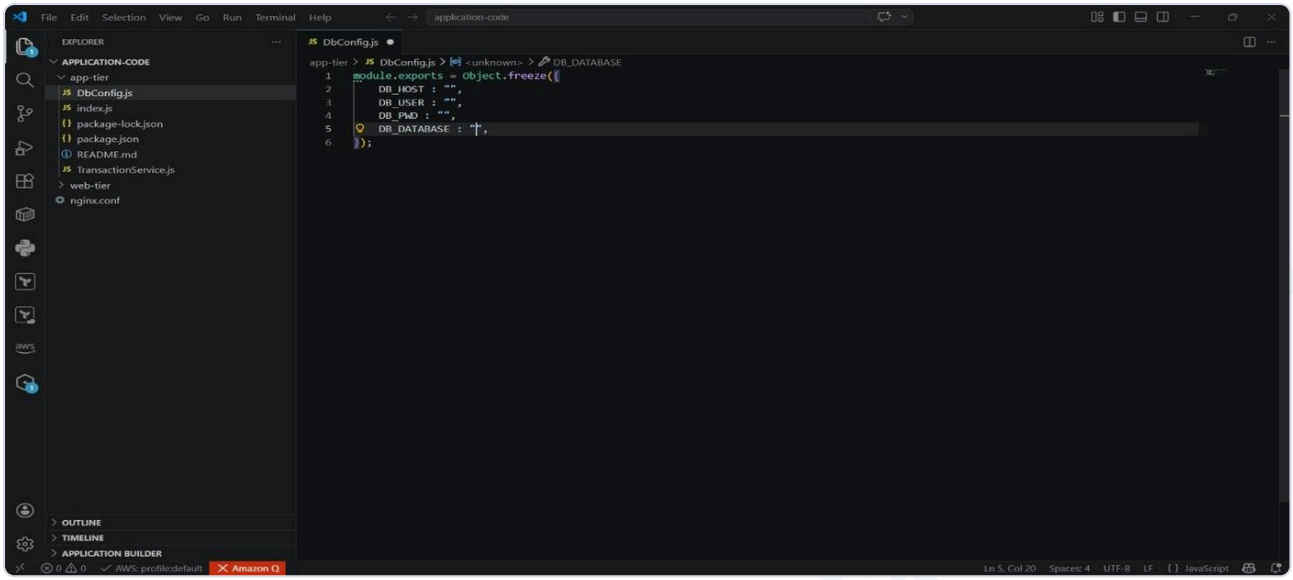




Keep other settings as default and click Launch Instance.

Now we have to connect to the server and configure the application tier.

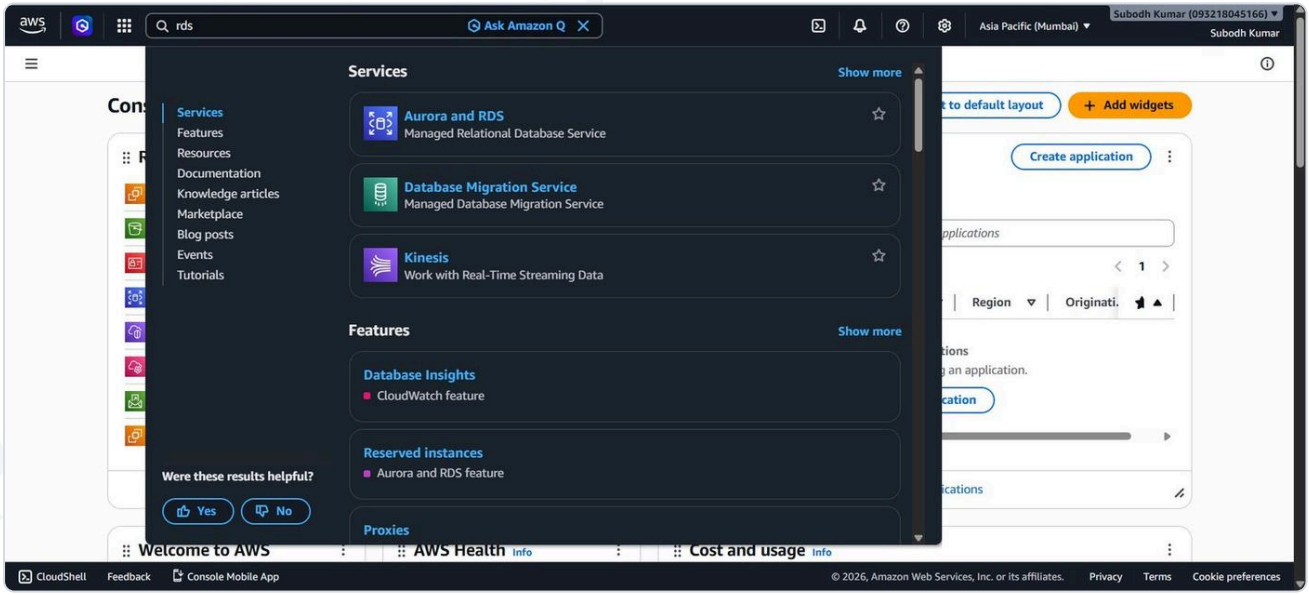
Before that let's do some changes in the source code. Go to the source code folder on your pc. Go to **application-code > app-tier.** Open **DbConfig.js** with your favorite editor. I use vs code.

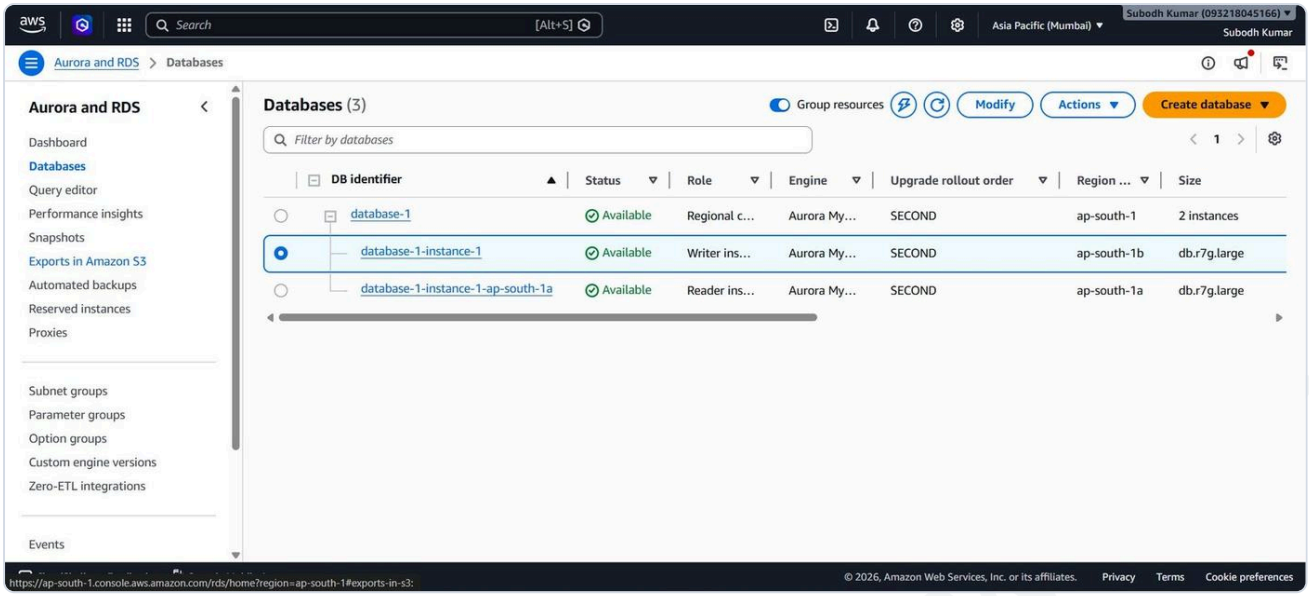


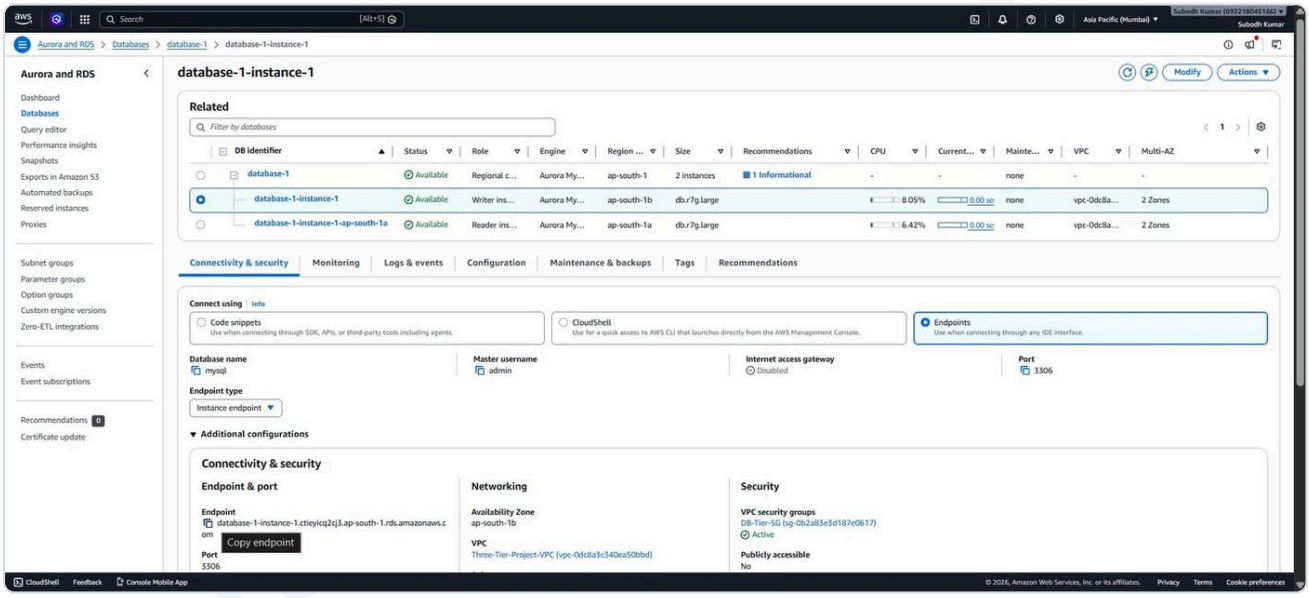
We have to change these values.

Let's see how to get the **DB_HOST**

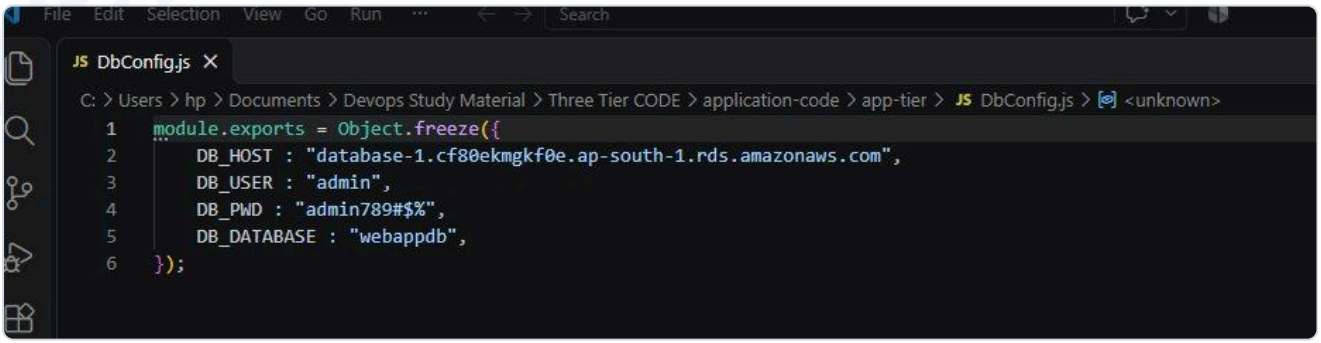
Go to search and go to **RDS > Databases**. Select Writer instance. Under connectivity and security, you can find a value called Endpoint. Copy it and paste it as the value for DB_HOST.







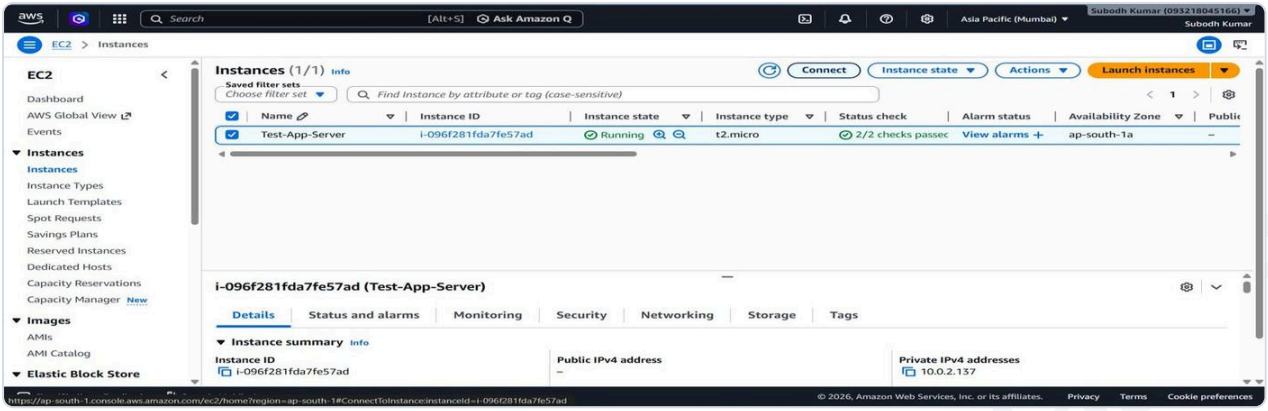
DB_USER is the username for DB server (**admin**). **DB_PWD** is the password we assigned (**Your master password**). **DB_DATABASE** is **webappdb**.



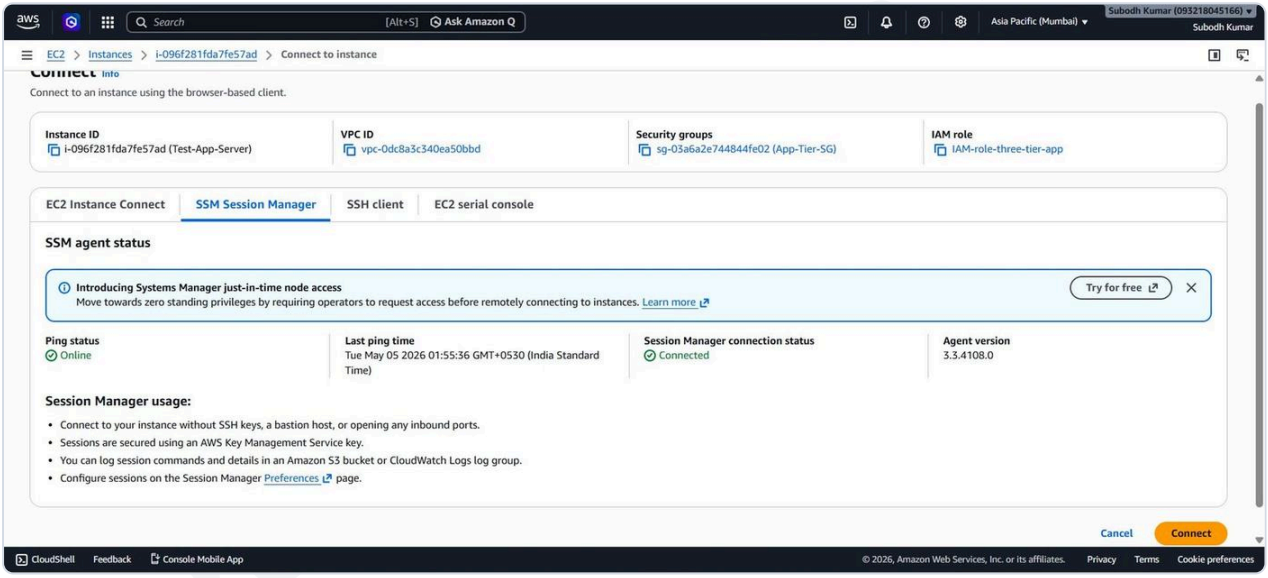
Now go to the S3 Bucket we created. Go to **application-code > app-tier** in the s3 bucket**.** Delete the DbConfig.js and then upload our newly updated DbConfig.js file.

Cool!

Now again go to EC2 instance and select Test-APP-Server we created click connect.



Go to Session Manager tab and click connect.



It will open a new tab like this. Which means you have connected to EC2 instance. IAM role we created let us connect to this EC2 instance even if it is in a private subnet.

Now let's configure the app-tier

First Switch user to the ec2-user by running sudo -su ec2-user

Now install the MySQL using following commands.

```
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y
```

Run mysql --version to confirm it installed correctly

```
[ec2-user@ip-10-0-2-137 bin]$ mysql --version
mysql  Ver 8.0.46 for Linux on x86_64 (MySQL Community Server - GPL)
[ec2-user@ip-10-0-2-137 bin]$
```

Now let's connect to Database server by running following command

```
mysql -h <RDS-Endpoint> -u <username> -p
```

RDS-Endpoint: Endpoint value of Writer instance copied from the RDS>Databases.

Username: admin

```
mysql -h database-1-instance-1.ctieyicq2cj3.ap-south-1.rds.amazonaws.com -u admin -p
```

Hit enter and input the password (**Your master password**). Enter. You will see the following output.

```
[ec2-user@ip-10-0-2-137 bin]$ mysql -h database-1-instance-1.ctieyicq2cj3.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 294
Server version: 8.0.42 6252a59a

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Let's Create the database. Run following commands.

```
create DATABASE webappdb;
show databases;
use webappdb;
```

```
mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.00 sec)

mysql> use webappdb;
Database changed
mysql>
```

Now Create a table by running following command

```
CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL AUTO_INCREMENT, amount DECIMAL(10,2),
description VARCHAR(100), PRIMARY KEY(id));
```

```
mysql> CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL
    -> AUTO_INCREMENT, amount DECIMAL(10,2), description
    -> VARCHAR(100), PRIMARY KEY(id));
Query OK, 0 rows affected (0.02 sec)

mysql> show tables;
+-------------------+
| Tables_in_webappdb |
+-------------------+
| transactions      |
+-------------------+
1 row in set (0.00 sec)

mysql>
```

Let's create insert a sample value.

```
INSERT INTO transactions (amount, description) VALUES ('400', 'groceries');
```

```
SELECT * FROM transactions;
```

```
mysql> INSERT INTO transactions (amount,description) VALUES ('400','groceries');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.00 sec)

mysql>
```

Type exit and hit Enter

Next we copy application tier codes from s3 bucket. Replace <S3_BUCKET_NAME> with your s3 bucket name.

```
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier --recursive
```

```
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

```
[ec2-user@ip-10-0-2-137 bin]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-137 ~]$ sudo aws s3 cp s3://three-tier-app-bucket-5-5-26/application-code/app-tier app-tier --recursive
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/README.md to app-tier/README.md
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/package-lock.json to app-tier/package-lock.json
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/package.json to app-tier/package.json
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/index.js to app-tier/index.js
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/TransactionService.js to app-tier/TransactionService.js
[ec2-user@ip-10-0-2-137 ~]$ cd app-tier
[ec2-user@ip-10-0-2-137 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-137 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-137 app-tier]$
```

Now let's install Node.js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2
npm install
npm audit fix
```

After that, let's run the index.js file and then save the configuration.

```
pm2 start index.js
pm2 logs
```

```
[ec2-user@ip-10-0-2-137 app-tier]$ pm2 logs
[TAILING] Tailing last 15 lines for [all] processes (change the value with --lines option)
/home/ec2-user/.pm2/pm2.log last 15 lines:
PM2        | 2026-05-04T20:44:38: PM2 log: Node.js version      : 16.20.2
PM2        | 2026-05-04T20:44:38: PM2 log: Current arch         : x64
PM2        | 2026-05-04T20:44:38: PM2 log: PM2 home             : /home/ec2-user/.pm2
PM2        | 2026-05-04T20:44:38: PM2 log: PM2 PID file         : /home/ec2-user/.pm2/pm2.pid
PM2        | 2026-05-04T20:44:38: PM2 log: RPC socket file      : /home/ec2-user/.pm2/rpc.sock
PM2        | 2026-05-04T20:44:38: PM2 log: BUS socket file      : /home/ec2-user/.pm2/pub.sock
PM2        | 2026-05-04T20:44:38: PM2 log: Application log path : /home/ec2-user/.pm2/logs
PM2        | 2026-05-04T20:44:38: PM2 log: Worker Interval      : 30000
PM2        | 2026-05-04T20:44:38: PM2 log: Process dump file    : /home/ec2-user/.pm2/dump.pm2
PM2        | 2026-05-04T20:44:38: PM2 log: Concurrent actions   : 2
PM2        | 2026-05-04T20:44:38: PM2 log: SIGTERM timeout      : 1600
PM2        | 2026-05-04T20:44:38: PM2 log: Runtime Binary       : /home/ec2-user/.nvm/versions/node/v16.20.2/bin/node
PM2        | 2026-05-04T20:44:38: PM2 log: ===============================================================================
PM2        | 2026-05-04T20:44:38: PM2 log: App [index:0] starting in -fork mode-
PM2        | 2026-05-04T20:44:38: PM2 log: App [index:0] online

/home/ec2-user/.pm2/logs/index-error.log last 15 lines:
/home/ec2-user/.pm2/logs/index-out.log last 15 lines:
0|index    | AB3 backend app listening at http://localhost:4000
```

If you got output like this. The backend is connected to port 4000. Press Ctrl+c to exit the loop. Then run the following commands.

```
pm2 startup
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin /home/ec2-
user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd -u ec2-user --hp
/home/ec2-user
pm2 save
curl http://localhost:4000/health
```

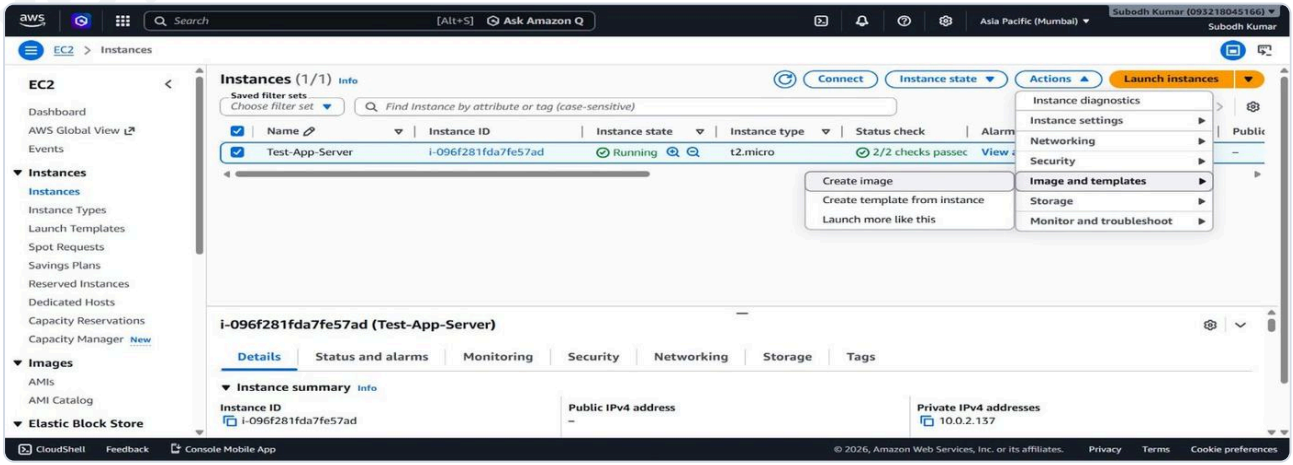
```
[PM2] Remove init script via:
$ pm2 unstartup systemd
[ec2-user@ip-10-0-2-137 app-tier]$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ec2-user/.pm2/dump.pm2
[ec2-user@ip-10-0-2-137 app-tier]$ curl http://localhost:4000/health
"This is the health check"[ec2-user@ip-10-0-2-137 app-tier]$
```

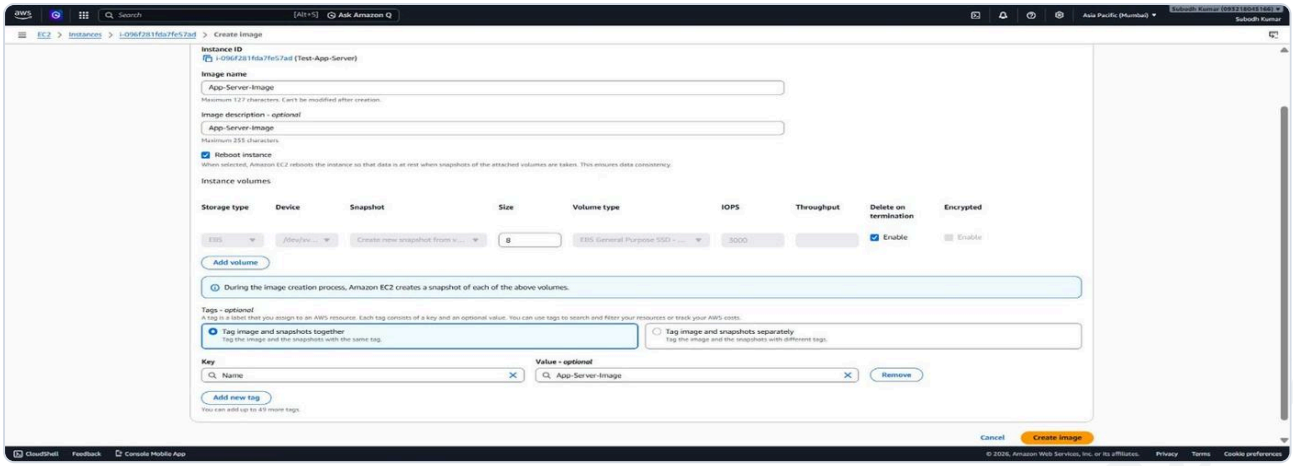
Cool! You successfully set up the app server.

Step 8 Create AMI of the App Server

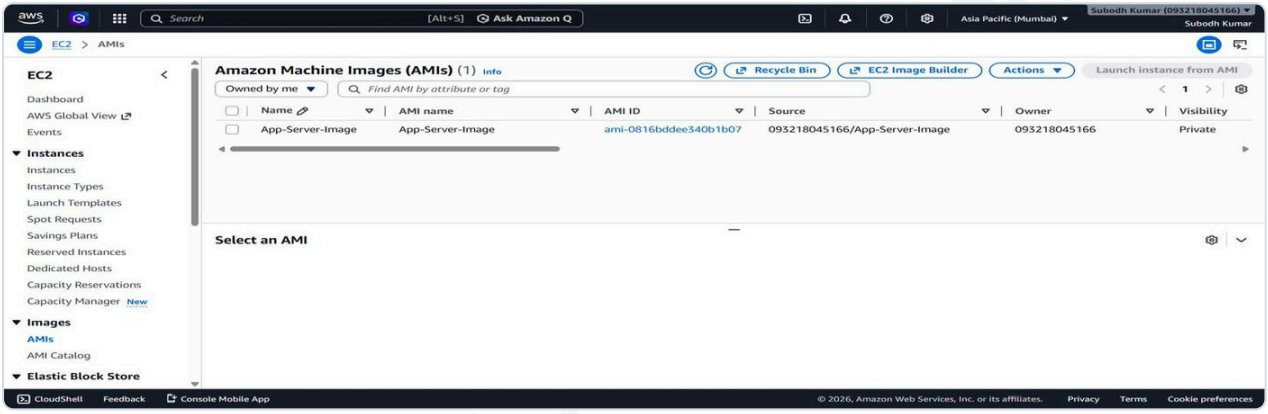
Next Let's create an image from this Test-App-Server instance.

Go to instances choose **Test-App-Server** go to **Actions > Image and templates > Create image.**





If you go to **AMIs section**, you can see that App-Server AMI is created

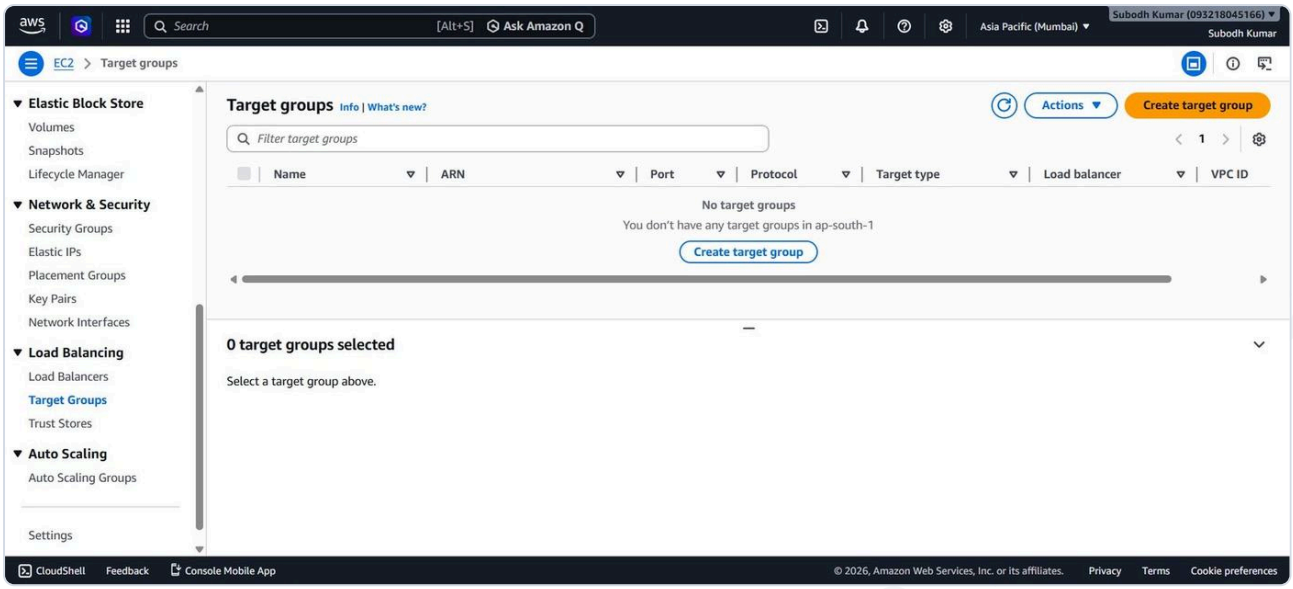


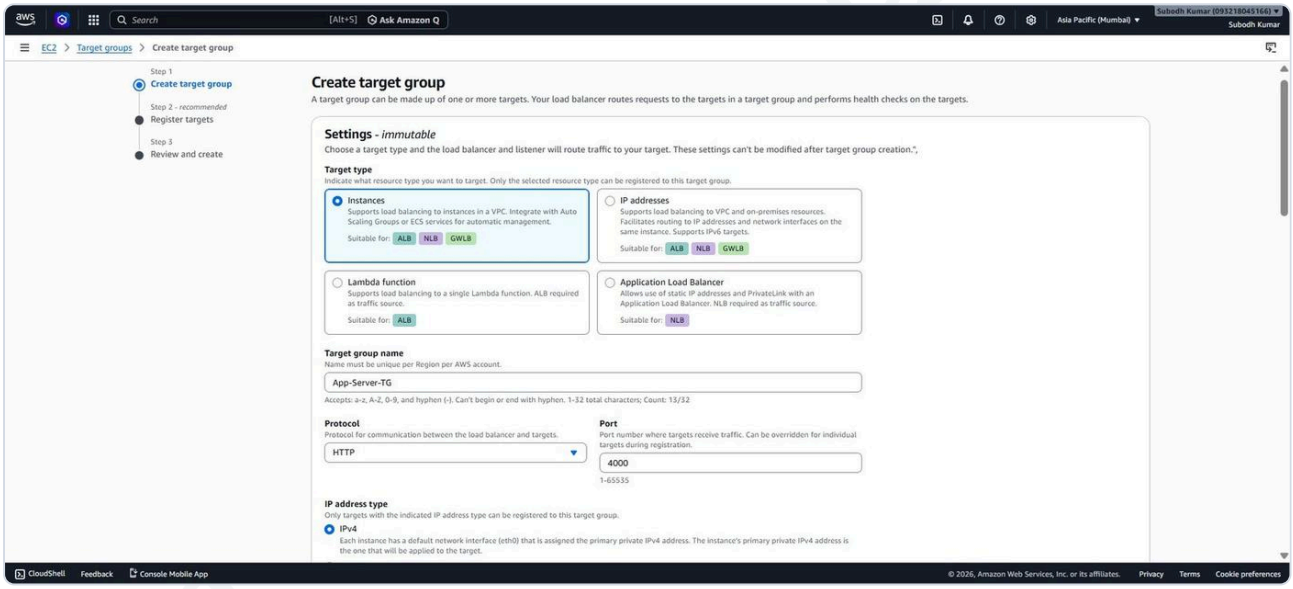
Step 9 Create Target Group, Launch Template and Internal Load Balancer for App Tier

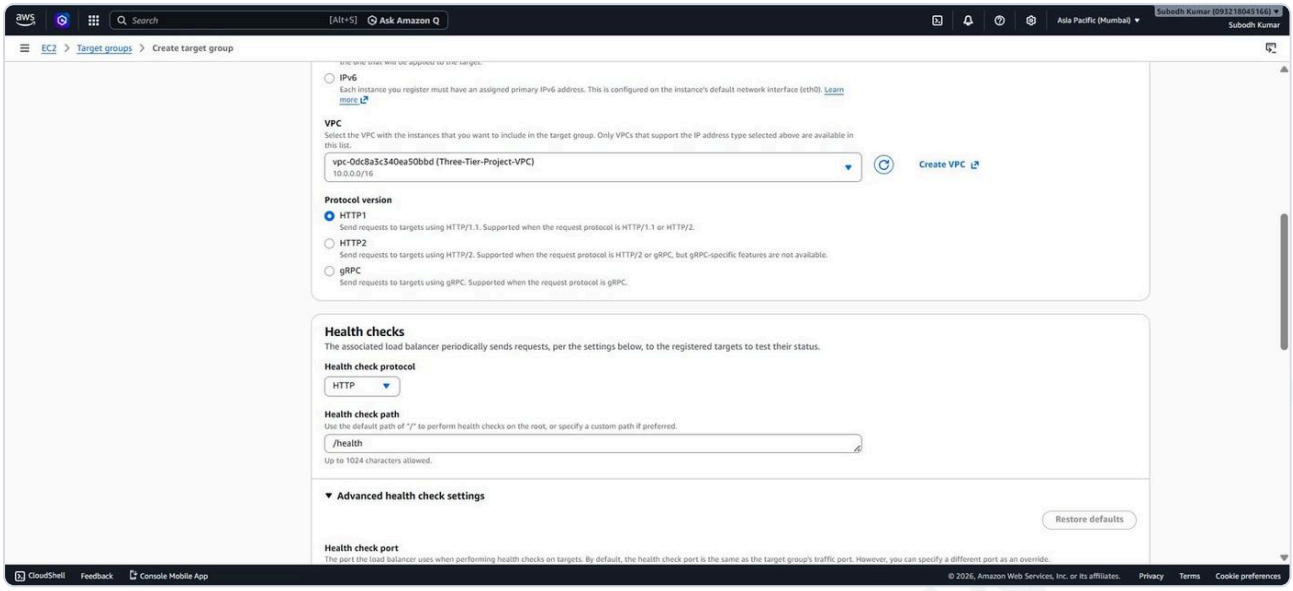
Create Target Group

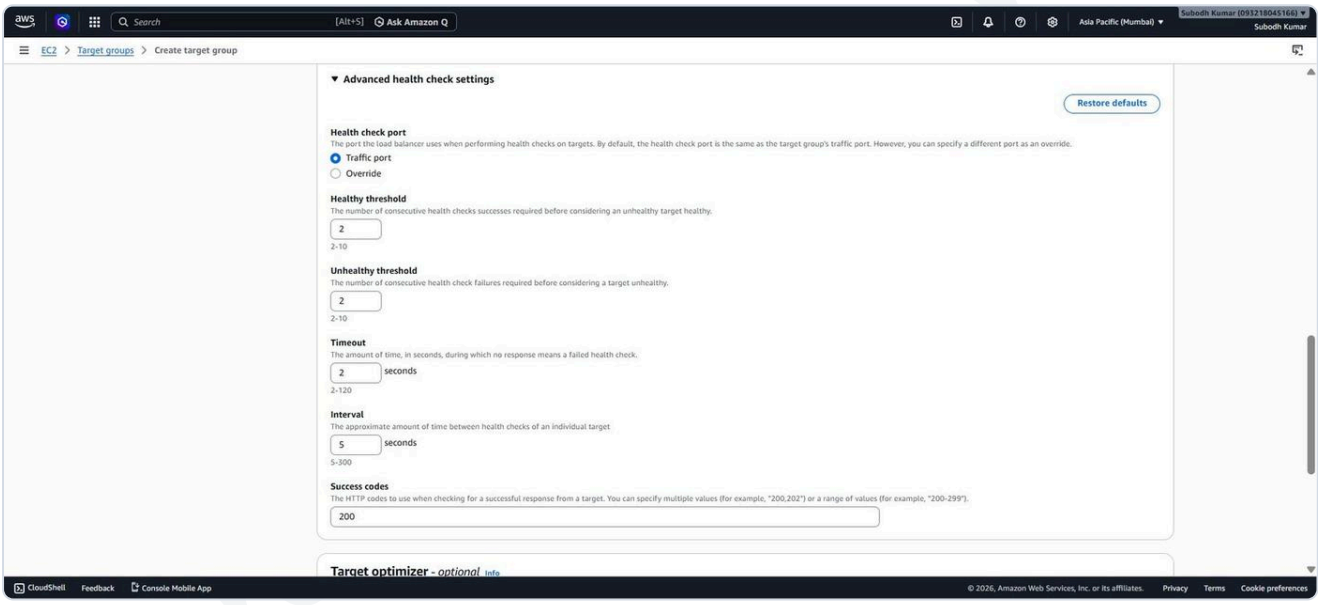
Go to EC2 > Target Groups > Create Target Group change only following settings. Click Next.

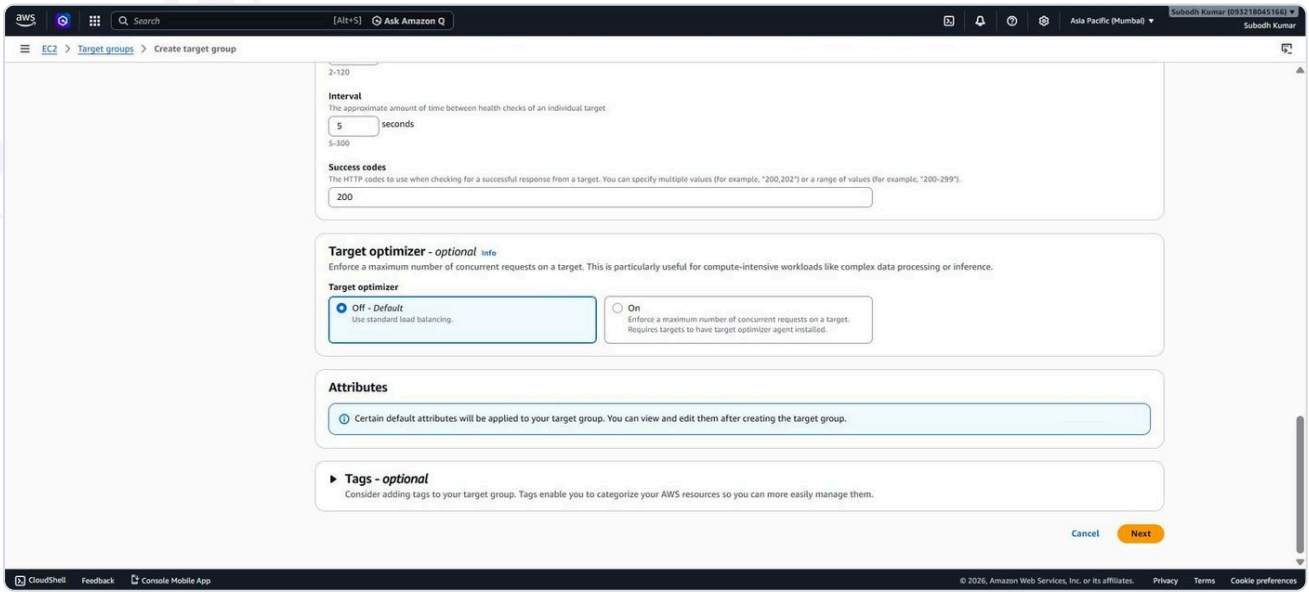
Click Create target group.

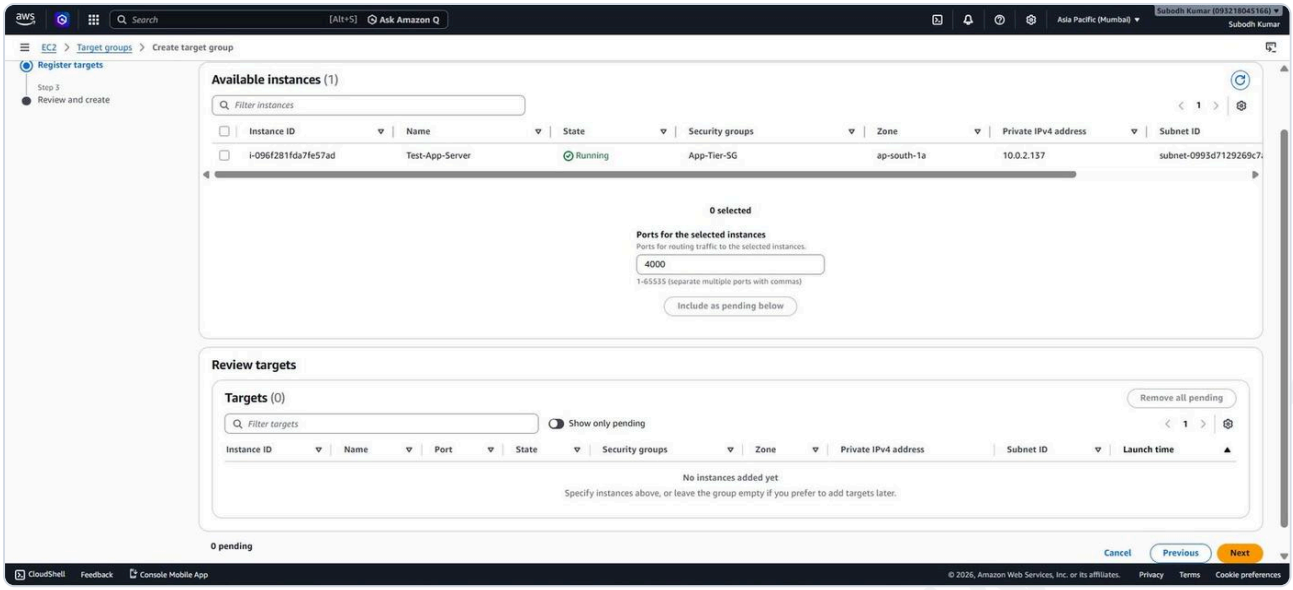


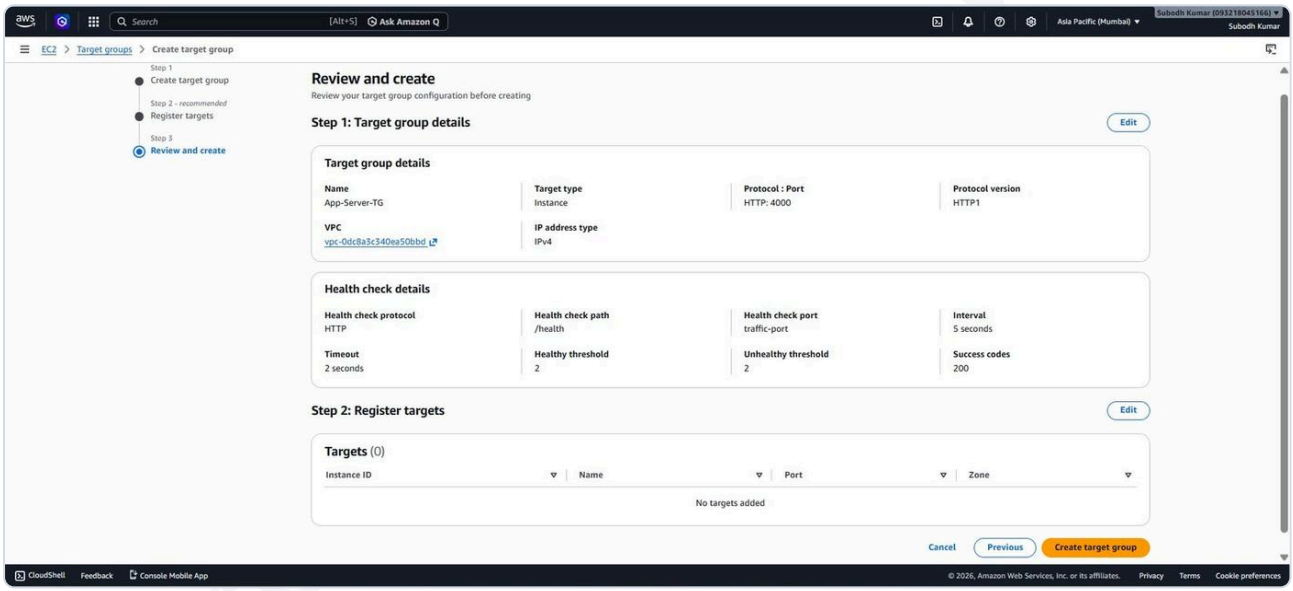



EC2 > Target groups > Create target group
IPv6
VPC
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create VPC
Protocol version
HTTP1
HTTP2
gRPC
Health checks
The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.
Health check protocol
HTTP
Health check path
Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.
/health
Up to 1024 characters allowed.
Advanced health check settings
Restore defaults
Health check port


EC2 > Target groups > Create target group
Advanced health check settings
Restore defaults
Health check port
Traffic port
Override
Healthy threshold
The number of consecutive health checks successes required before considering an unhealthy target healthy.
2
2-10
Unhealthy threshold
The number of consecutive health check failures required before considering a target unhealthy.
2
2-10
Timeout
The amount of time, in seconds, during which no response means a failed health check.
2
seconds
2-120
Interval
The approximate amount of time between health checks of an individual target
5
seconds
5-300
Success codes
The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").
200
Target optimizer - optional Info

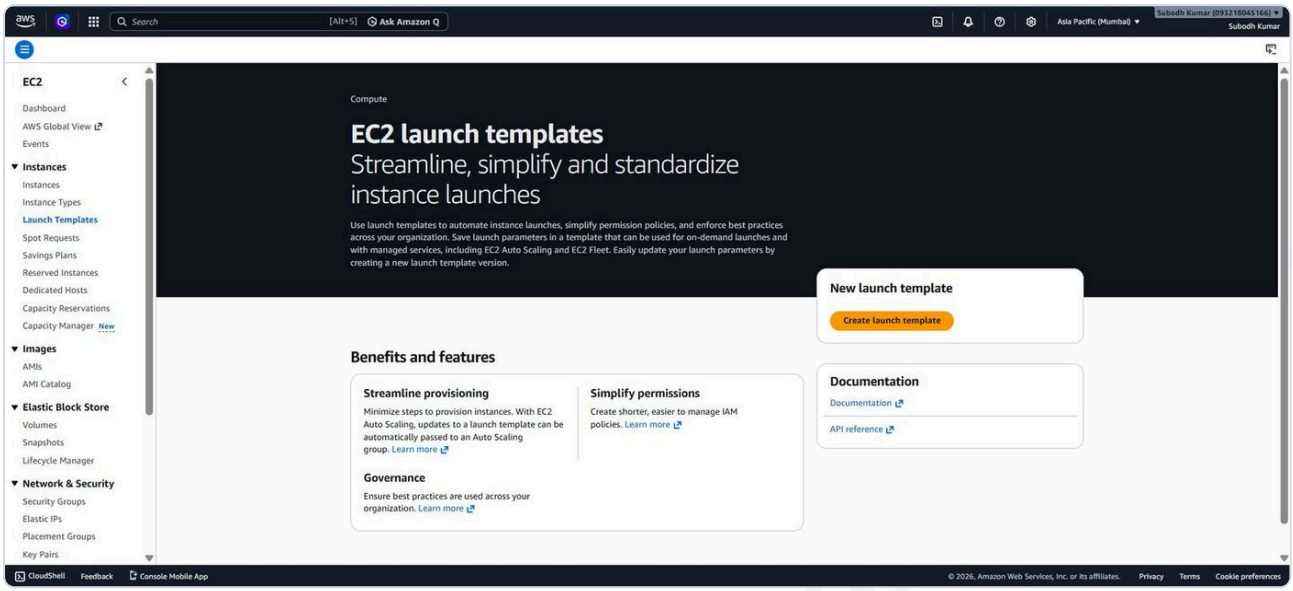

EC2 > Target groups > Create target group
2-120
Interval
The approximate amount of time between health checks of an individual target
5
seconds
5-300
Success codes
The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").
200
Target optimizer - optional Info
Enforce a maximum number of concurrent requests on a target. This is particularly useful for compute-intensive workloads like complex data processing or inference.
Target optimizer
Off - Default
Use standard load balancing.
On
Enforce a maximum number of concurrent requests on a target.
Requires targets to have target optimizer agent installed.
Attributes
Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.
Tags - optional
Consider adding tags to your target group. Tags enable you to categorize your AWS resources so you can more easily manage them.
Cancel
Next

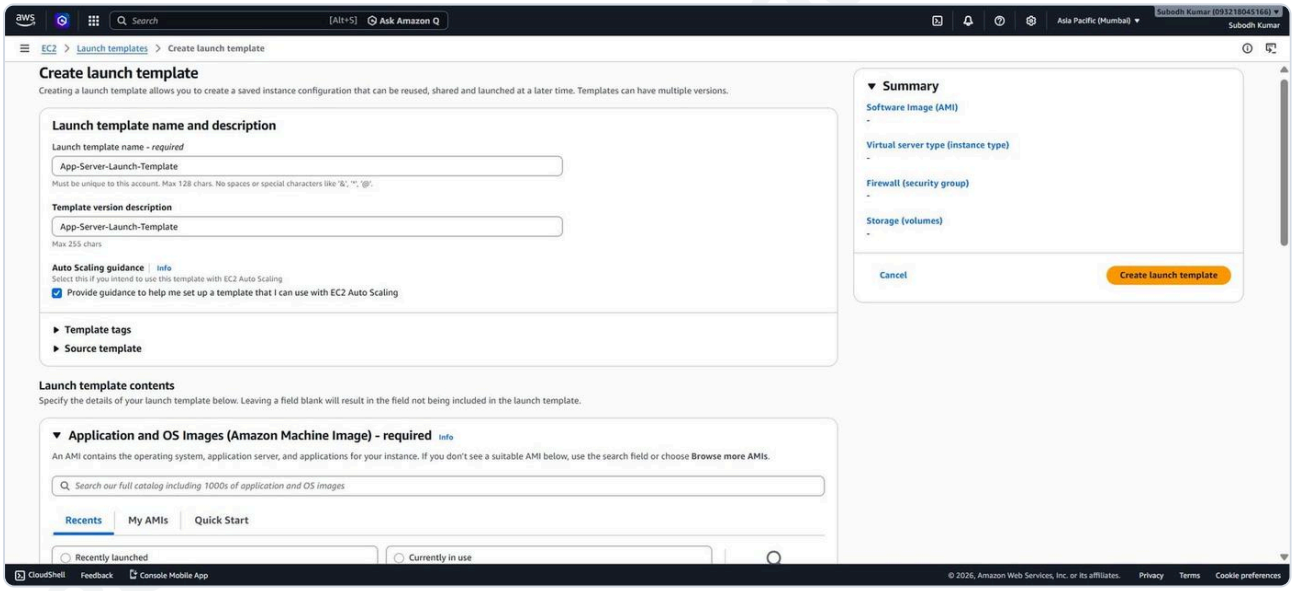

EC2 > Target groups > Create target group
Register targets
Step 3
Review and create
Available instances (1)
Filter instances
Instance ID | Name | State | Security groups | Zone | Private IPv4 address | Subnet ID
i-096f281fda7fe57ad | Test-App-Server | Running | App-Tier-SG | ap-south-1a | 10.0.2.137
0 selected
Ports for the selected instances
Ports for routing traffic to the selected instances.
4000
1-65535 (separate multiple ports with commas)
Include as pending below
Review targets
Targets (0)
Remove all pending
Filter targets
Show only pending
Instance ID | Name | Port | State | Security groups | Zone | Private IPv4 address | Subnet ID | Launch time
No instances added yet
Specify instances above, or leave the group empty if you prefer to add targets later.
0 pending
Cancel
Previous
Next

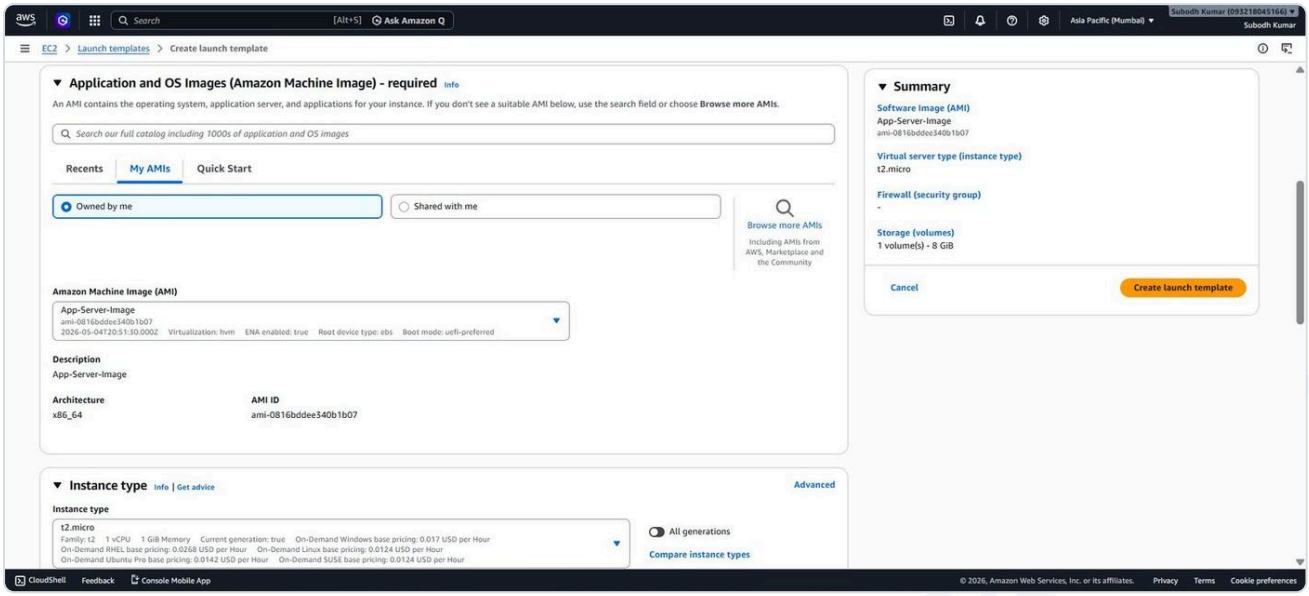

EC2 > Target groups > Create target group
Step 1
Create target group
Step 2 - recommended
Register targets
Step 3
Review and create
Review and create
Review your target group configuration before creating
Step 1: Target group details
Edit
Target group details
Name
App-Server-TG
Target type
Instance
Protocol : Port
HTTP: 4000
Protocol version
HTTP1
VPC
vpc-0dc8a3c340ea50bbd
IP address type
IPv4
Health check details
Health check protocol
HTTP
Health check path
/health
Health check port
traffic-port
Interval
5 seconds
Timeout
2 seconds
Healthy threshold
2
Unhealthy threshold
2
Success codes
200
Step 2: Register targets
Edit
Targets (0)
Instance ID | Name | Port | Zone
No targets added
Cancel
Previous
Create target group

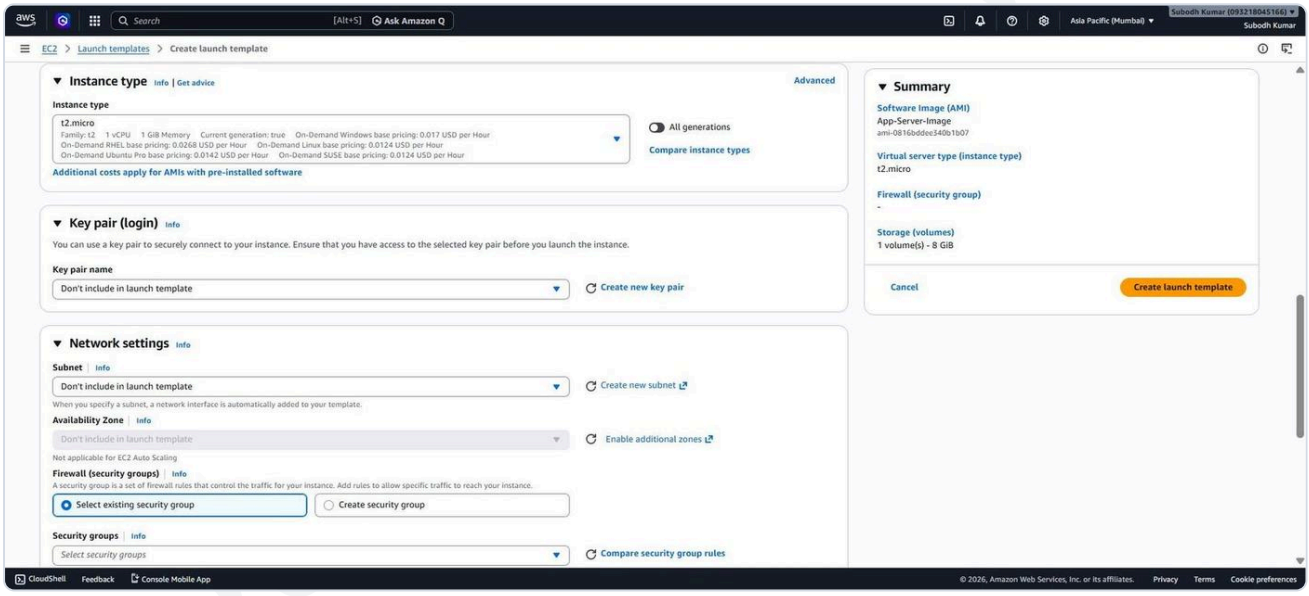
Create Launch Template

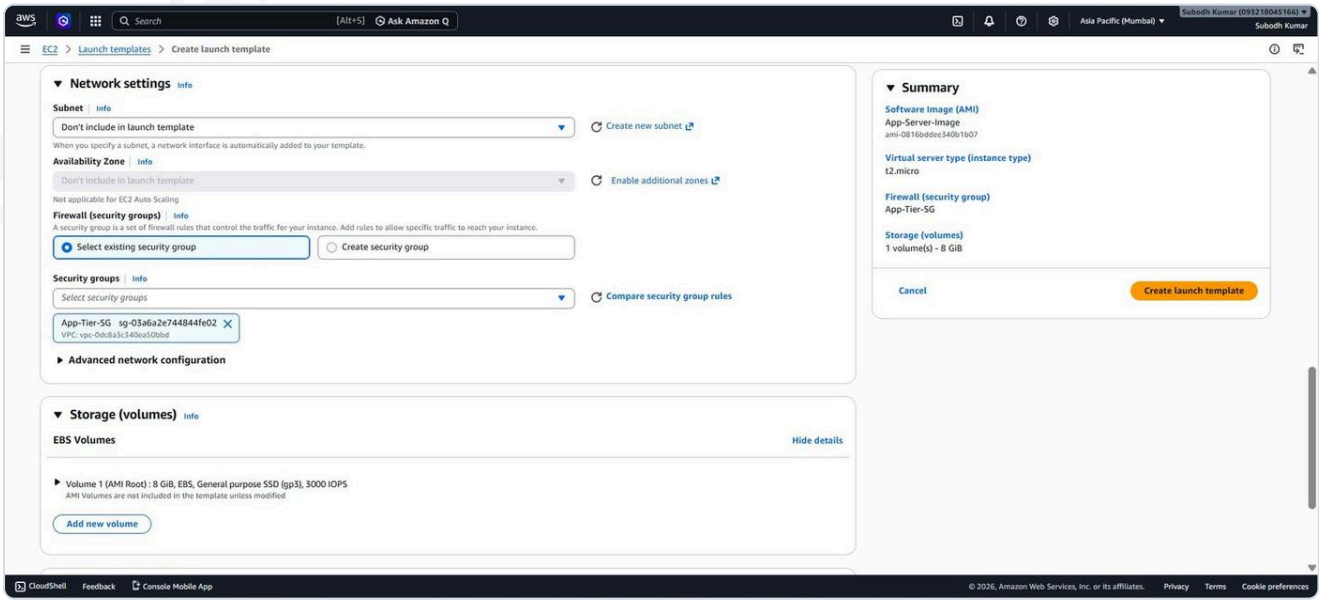
Go to EC2 > Launch Templates > Create launch template.

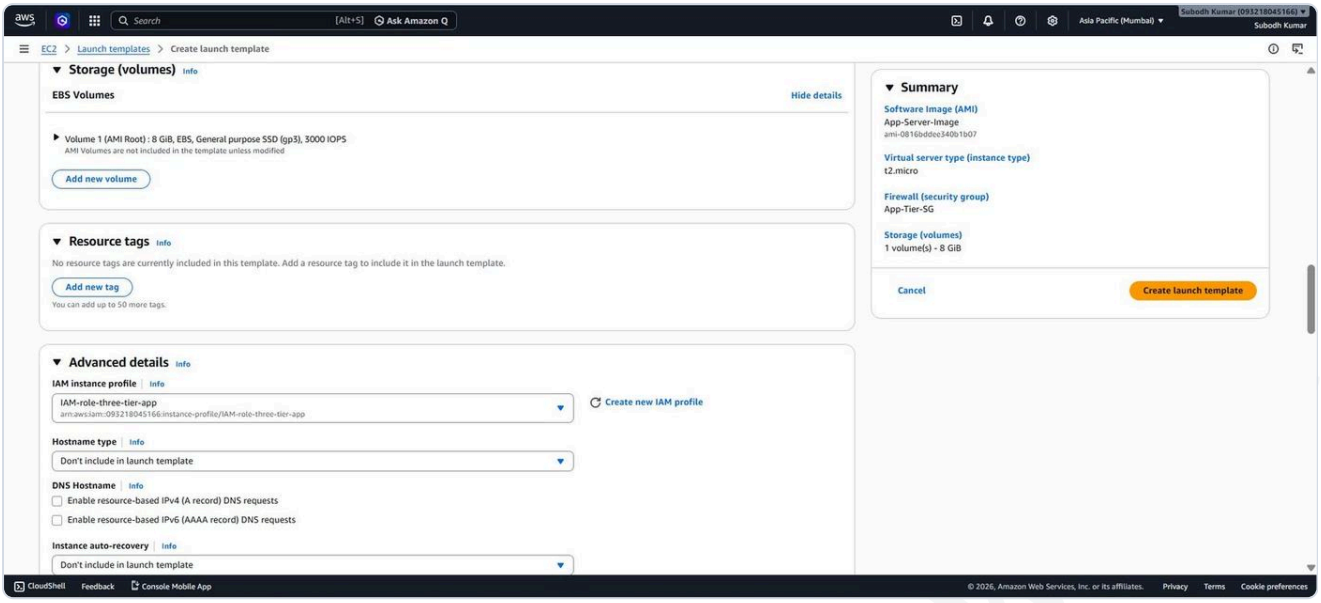





Application and OS Images (Amazon Machine Image) - required
Summary
Instance type
Advanced
Create launch template

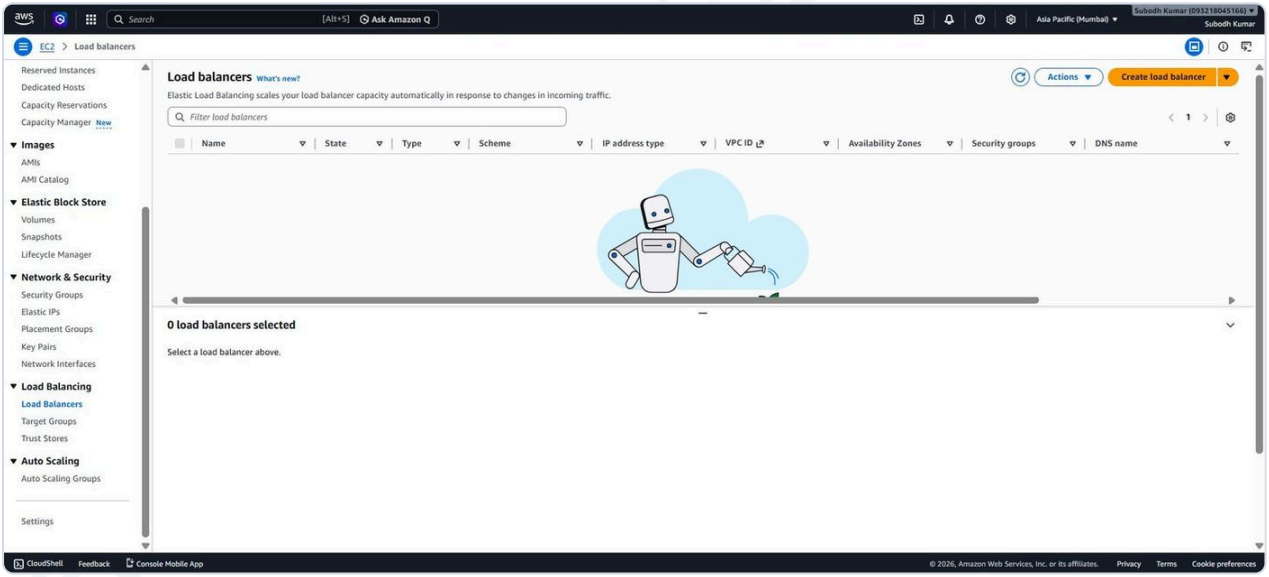

Instance type
Key pair (login)
Network settings
Summary
Create launch template

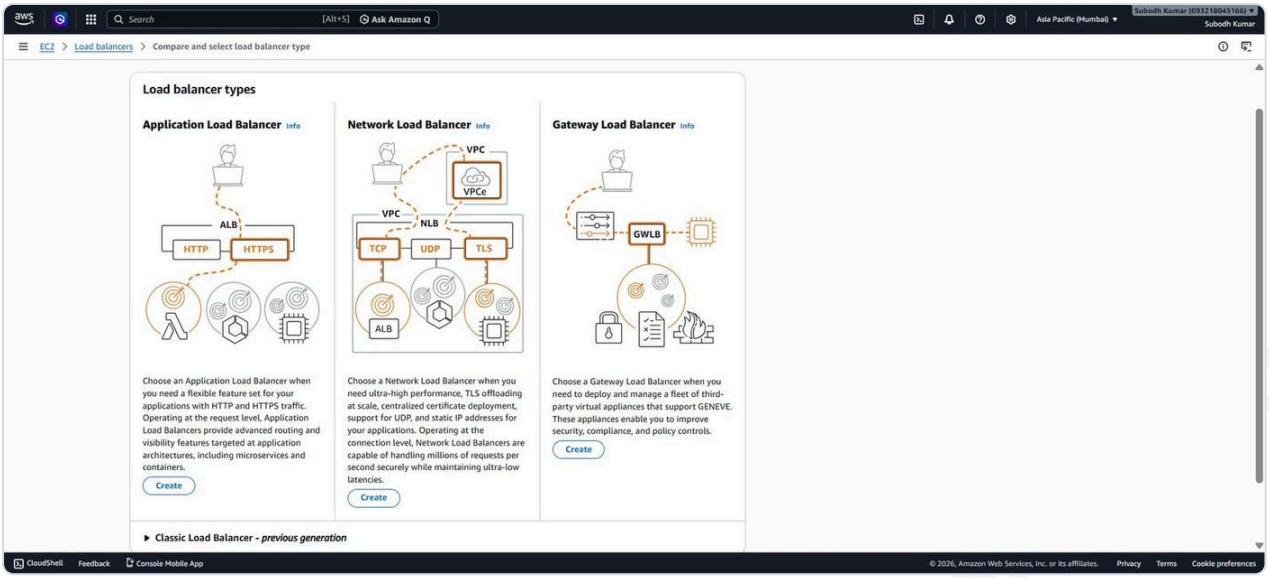

Network settings
Storage (volumes)
Summary
Create launch template

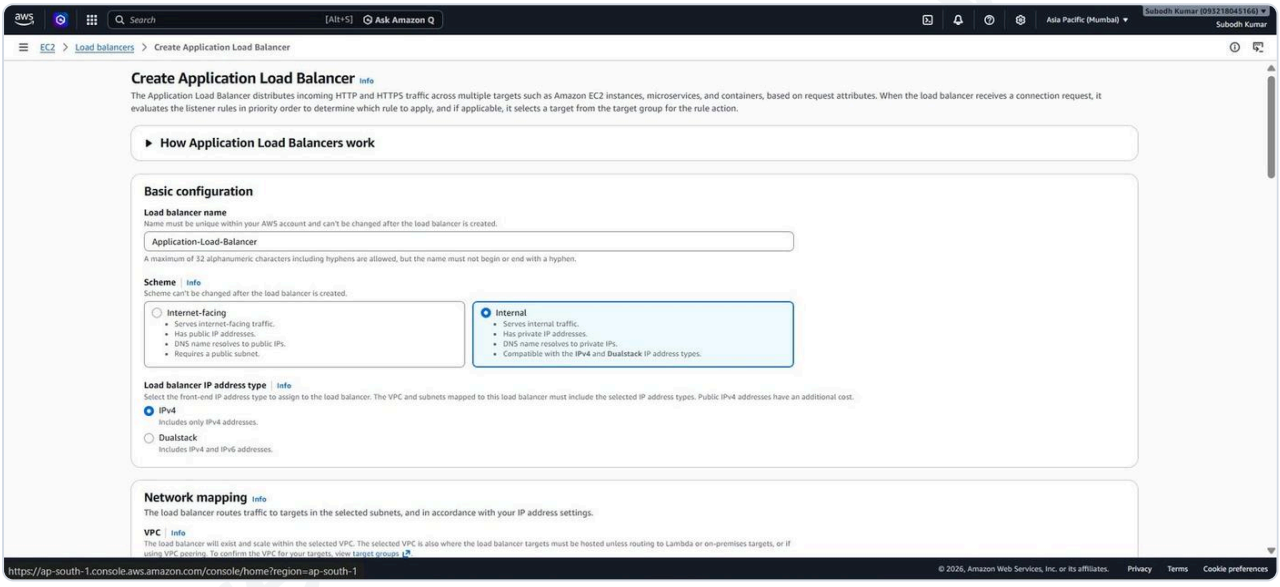


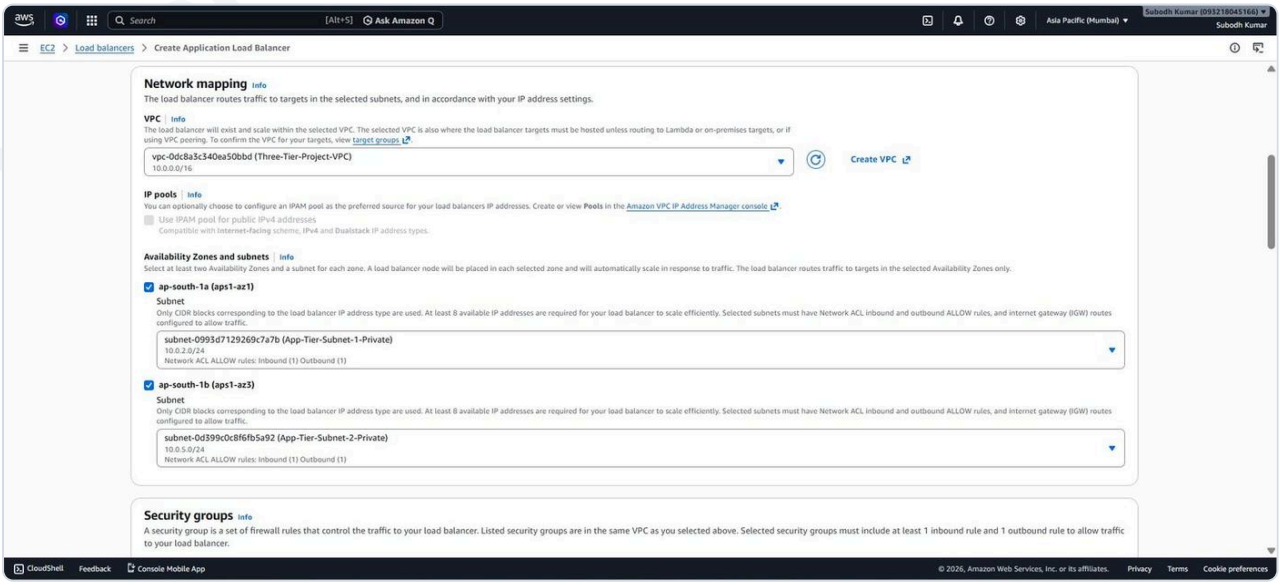
Create Application Load Balancer (Internal)

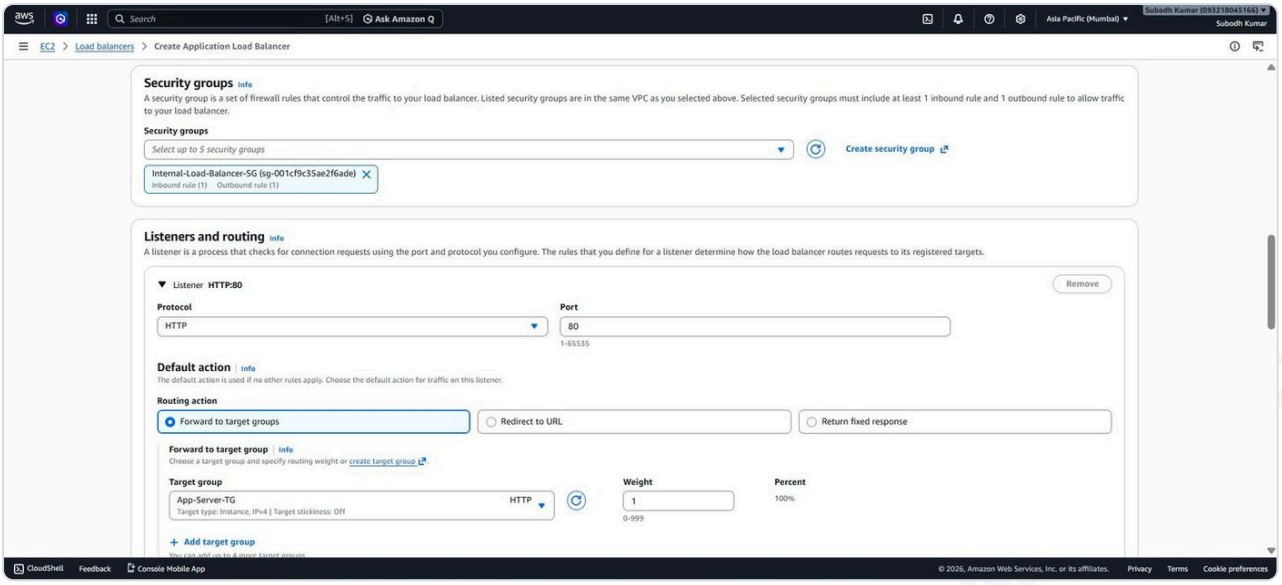
Go to EC2 > Load Balancer > Create Load Balancer create application load balancer.

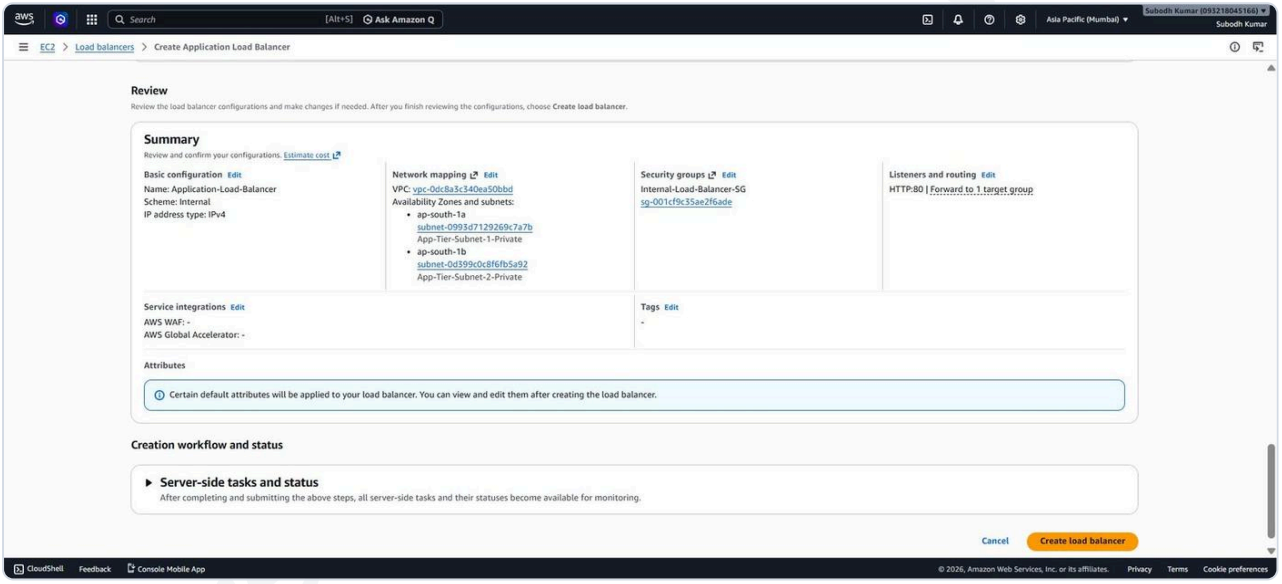



Load balancer types
Application Load Balancer Info
Network Load Balancer Info
Gateway Load Balancer Info
ALB
HTTP
HTTPS
VPC
VPCe
NLB
TCP
UDP
TLS
GWLB
Choose an Application Load Balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic. Operating at the request level, Application Load Balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.
Create
Choose a Network Load Balancer when you need ultra-high performance, TLS offloading at scale, centralized certificate deployment, support for UDP, and static IP addresses for your applications. Operating at the connection level, Network Load Balancers are capable of handling millions of requests per second securely while maintaining ultra-low latencies.
Create
Choose a Gateway Load Balancer when you need to deploy and manage a fleet of third-party virtual appliances that support GENEVE. These appliances enable you to improve security, compliance, and policy controls.
Create
Classic Load Balancer - previous generation


Create Application Load Balancer Info
How Application Load Balancers work
Basic configuration
Load balancer name
Application-Load-Balancer
Scheme Info
Internet-facing
Internal
Load balancer IP address type Info
IPv4
Dualstack
Network mapping Info
VPC Info


Network mapping Info
VPC Info
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create VPC
IP pools Info
Availability Zones and subnets Info
ap-south-1a (aps1-az1)
Subnet
subnet-0993d7129269c7a7b (App-Tier-Subnet-1-Private)
10.0.2.0/24
ap-south-1b (aps1-az3)
Subnet
subnet-0d399c0c8f6fb5a92 (App-Tier-Subnet-2-Private)
10.0.5.0/24
Security groups Info

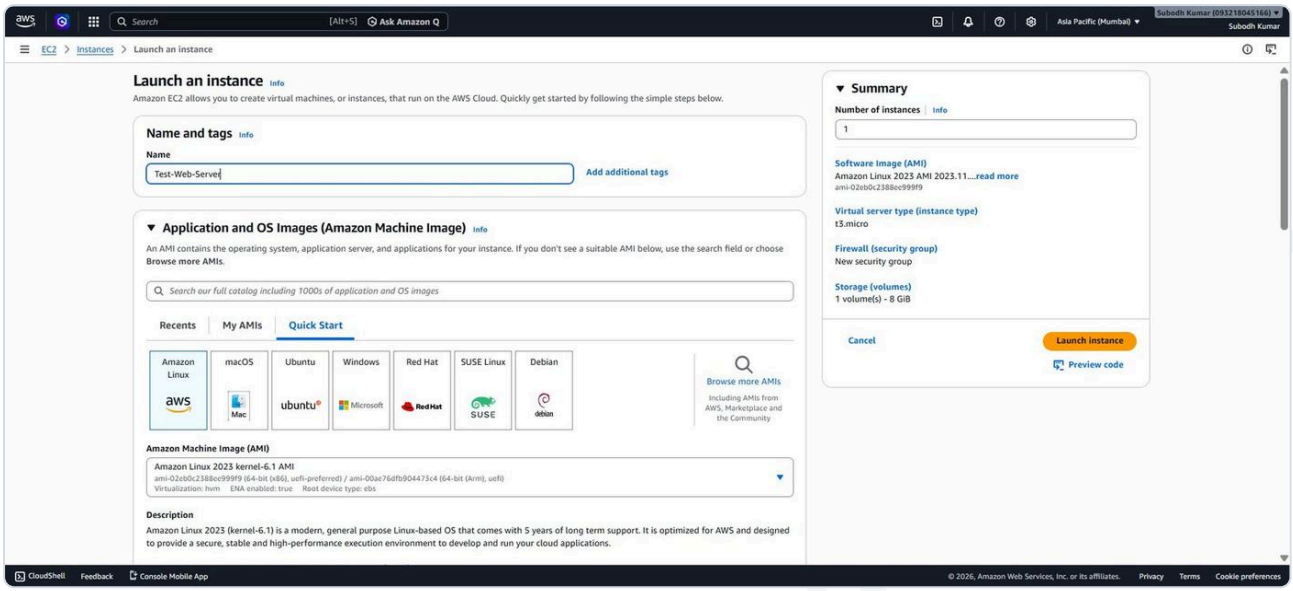


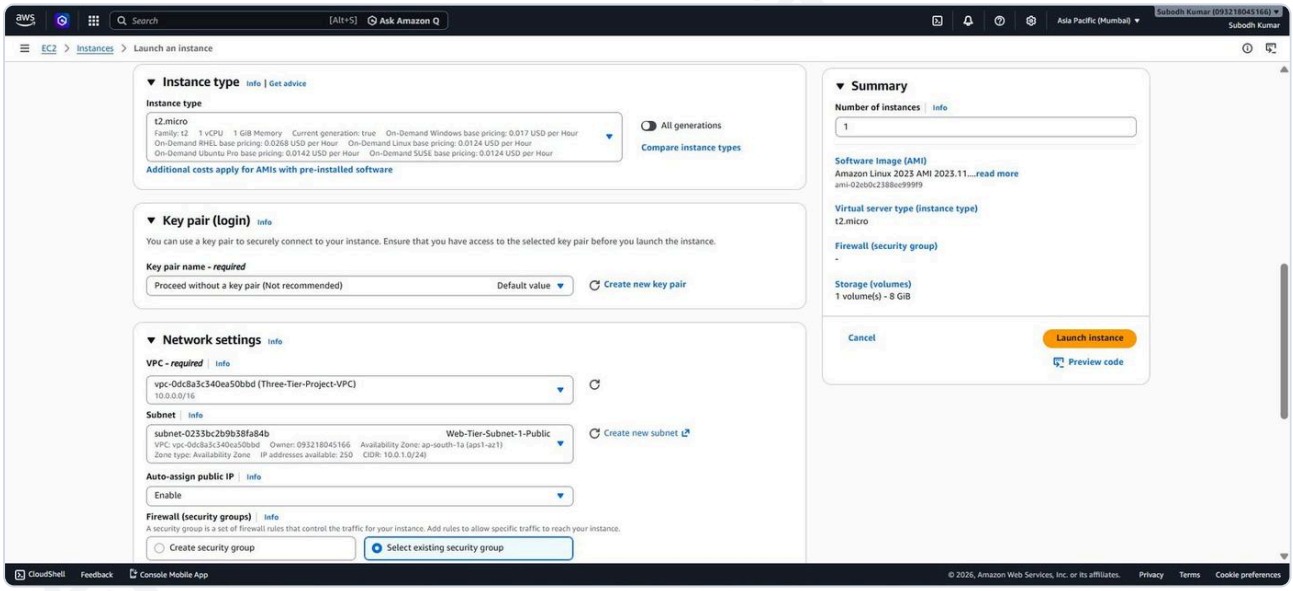


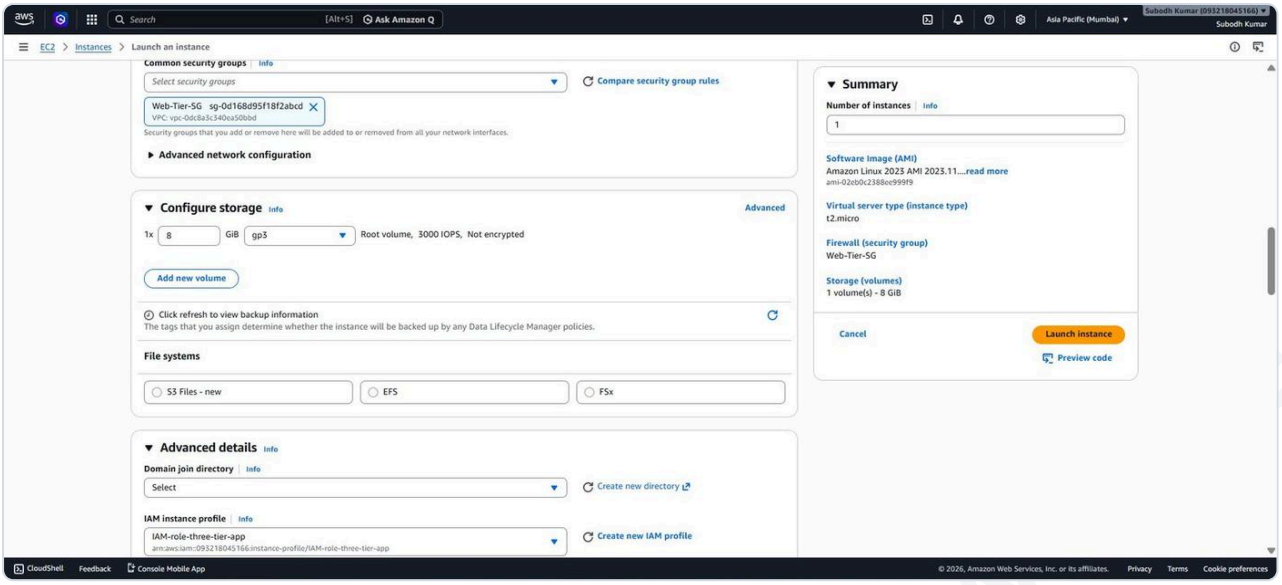
Step 10 Create Test Web Server and Set It Up

We have to follow the same process as followed in App server. Only **EC2 configuration** part **is different**. Configurations will be given below.

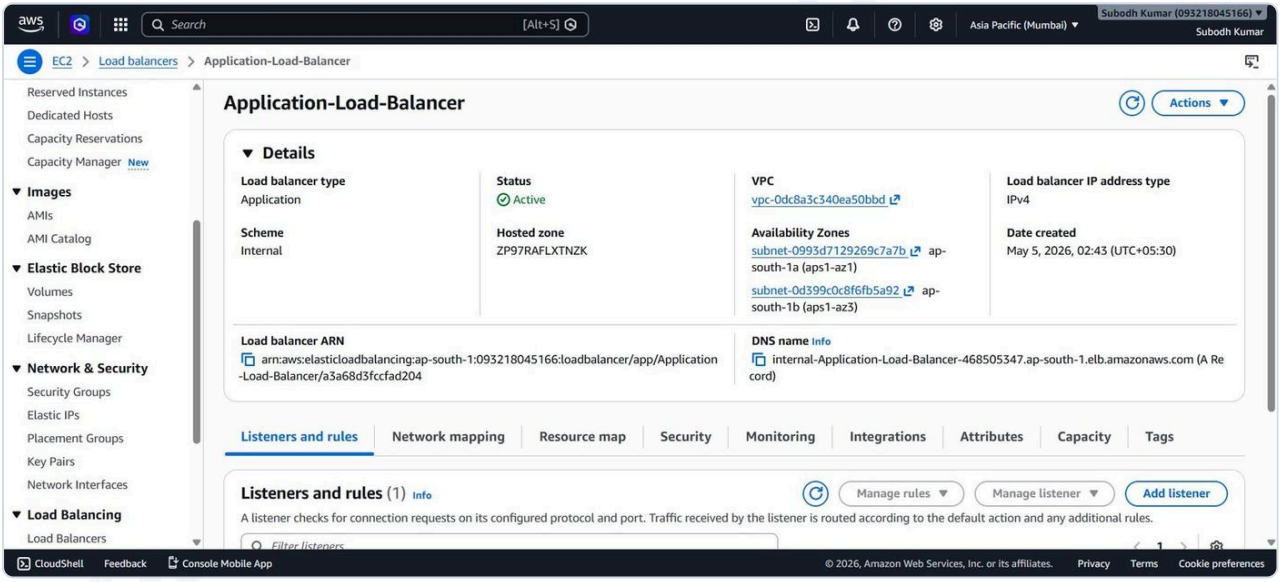
Now got EC2 instance > Launch instance and create a instance for Test Web Server. Change name (Test-Web-Server), VPC, Subnet(Web-Tier-Subnet-1-Public), Security group (Web-Tier-SG) and also select IAM role under advanced details. Finally, Launch instance.



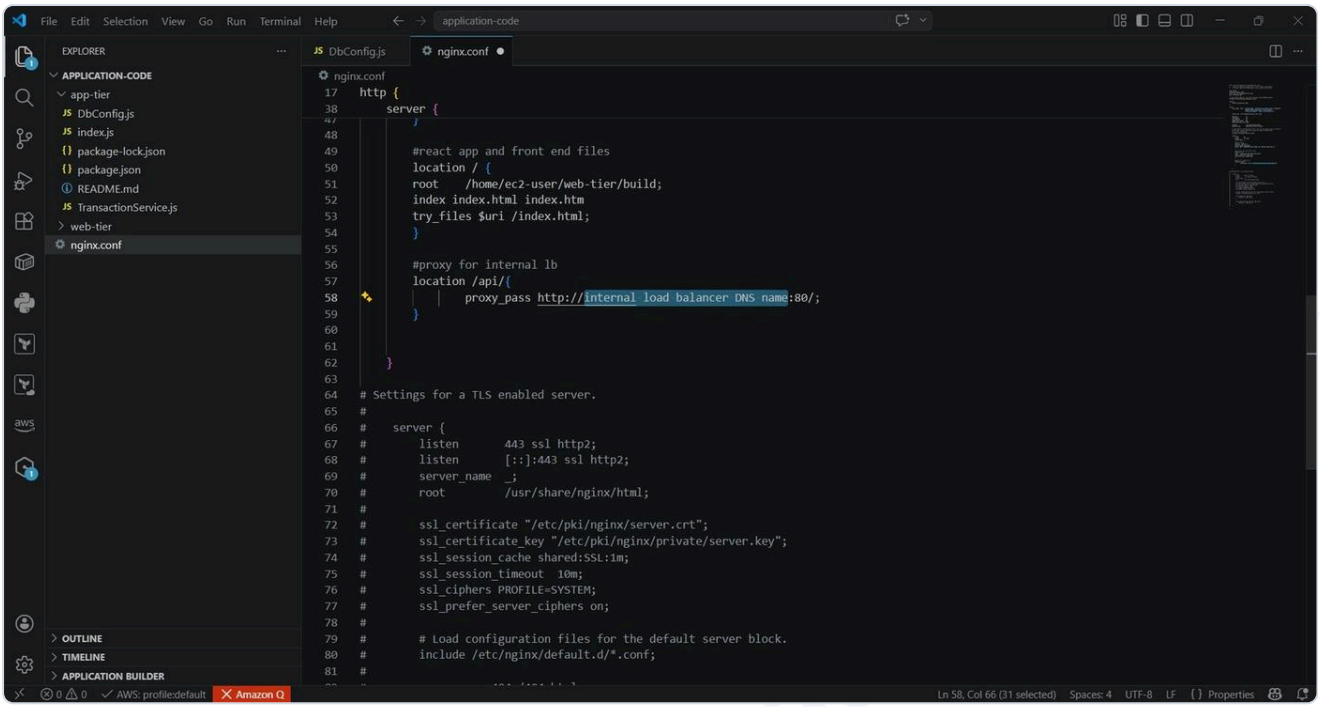




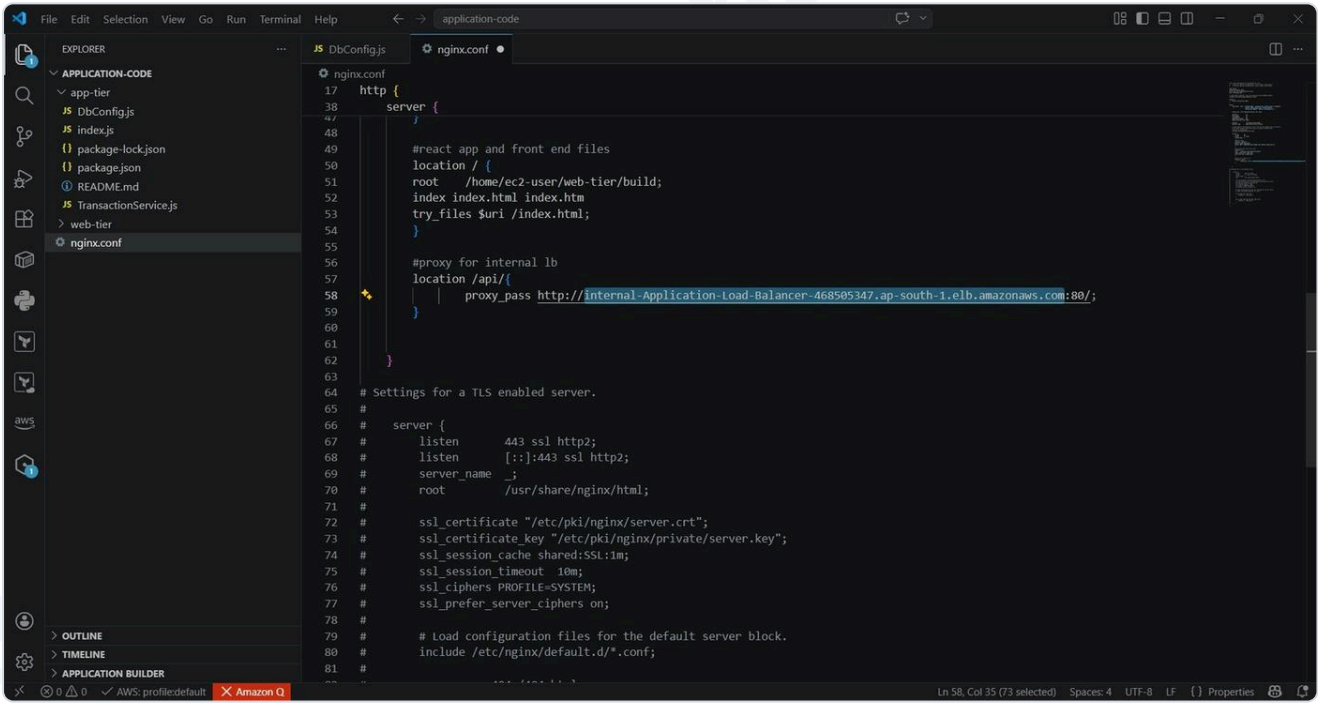
First go to the source code folder in your PC. Open **nginx.conf** file in an editor Now go to the **internal load balancer** we created and copy the **DNS name** of it.



Go to the **nginx.conf** file and paste it in the following section.



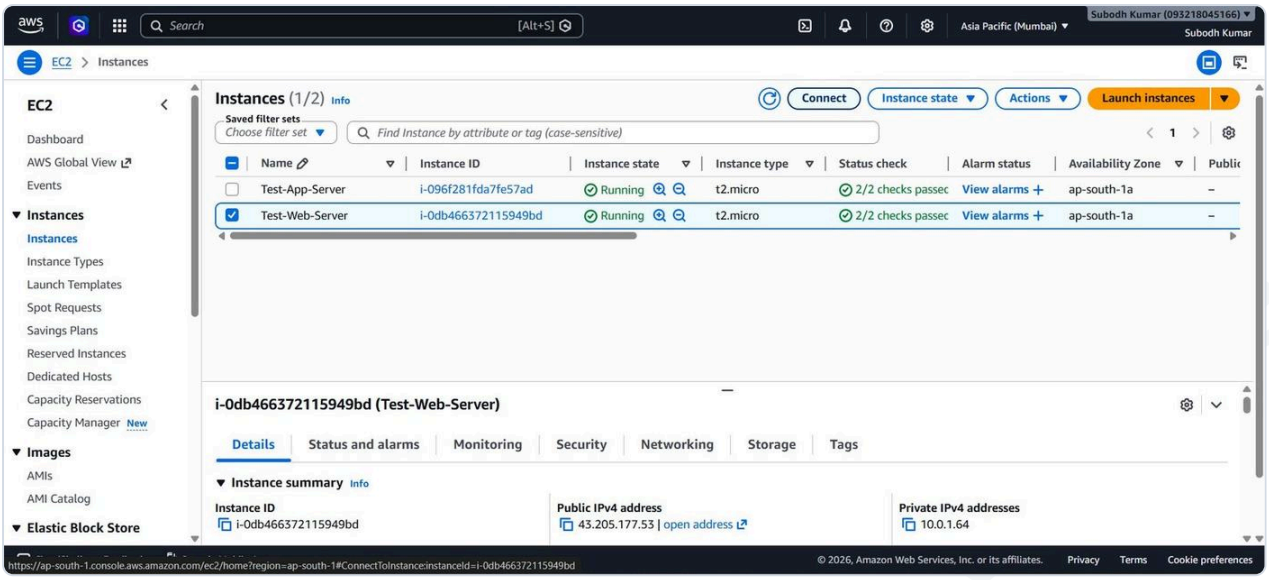

```
http {
    server {

        #react app and front end files
        location / {
        root    /home/ec2-user/web-tier/build;
        index index.html index.htm
        try_files $uri /index.html;
        }

        #proxy for internal lb
        location /api/{
            proxy_pass http://internal load balancer DNS name:80/;
        }


    }
# Settings for a TLS enabled server.
#
#    server {
#        listen       443 ssl http2;
#        listen       [::]:443 ssl http2;
#        server_name  _;
#        root         /usr/share/nginx/html;
#
#        ssl_certificate "/etc/pki/nginx/server.crt";
#        ssl_certificate_key "/etc/pki/nginx/private/server.key";
#        ssl_session_cache shared:SSL:1m;
#        ssl_session_timeout  10m;
#        ssl_ciphers PROFILE=SYSTEM;
#        ssl_prefer_server_ciphers on;
#
#        # Load configuration files for the default server block.
#        include /etc/nginx/default.d/*.conf;
#
```

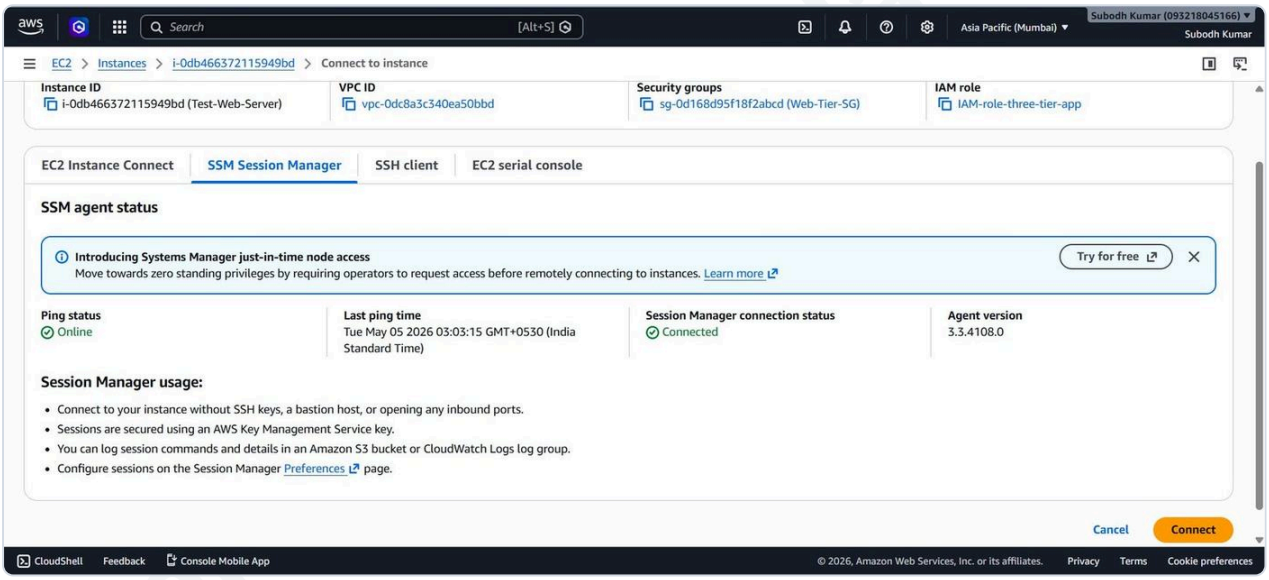



```
http {
    server {

        #react app and front end files
        location / {
        root    /home/ec2-user/web-tier/build;
        index index.html index.htm
        try_files $uri /index.html;
        }

        #proxy for internal lb
        location /api/{
            proxy_pass http://internal-Application-Load-Balancer-468505347.ap-south-1.elb.amazonaws.com:80/;
        }


    }
# Settings for a TLS enabled server.
#
#    server {
#        listen       443 ssl http2;
#        listen       [::]:443 ssl http2;
#        server_name  _;
#        root         /usr/share/nginx/html;
#
#        ssl_certificate "/etc/pki/nginx/server.crt";
#        ssl_certificate_key "/etc/pki/nginx/private/server.key";
#        ssl_session_cache shared:SSL:1m;
#        ssl_session_timeout  10m;
#        ssl_ciphers PROFILE=SYSTEM;
#        ssl_prefer_server_ciphers on;
#
#        # Load configuration files for the default server block.
#        include /etc/nginx/default.d/*.conf;
#
```

Now go to S3 bucket and delete the **nginx.conf** file and upload the new one.

Now Connect to the instance (**Test-Web-Server**) and do configurations as follows.





Switch users

```
sudo -su ec2-user
cd /home/ec2-user
```

Copy content from the S3 bucket

```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/web-tier web-tier --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user
sudo chmod -R 755 /home/ec2-user
```

Install Node js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
```

```
nvm use 16
```

```
cd /home/ec2-user/web-tier
npm install
```

Build the app

```
cd /home/ec2-user/web-tier
npm run build
```

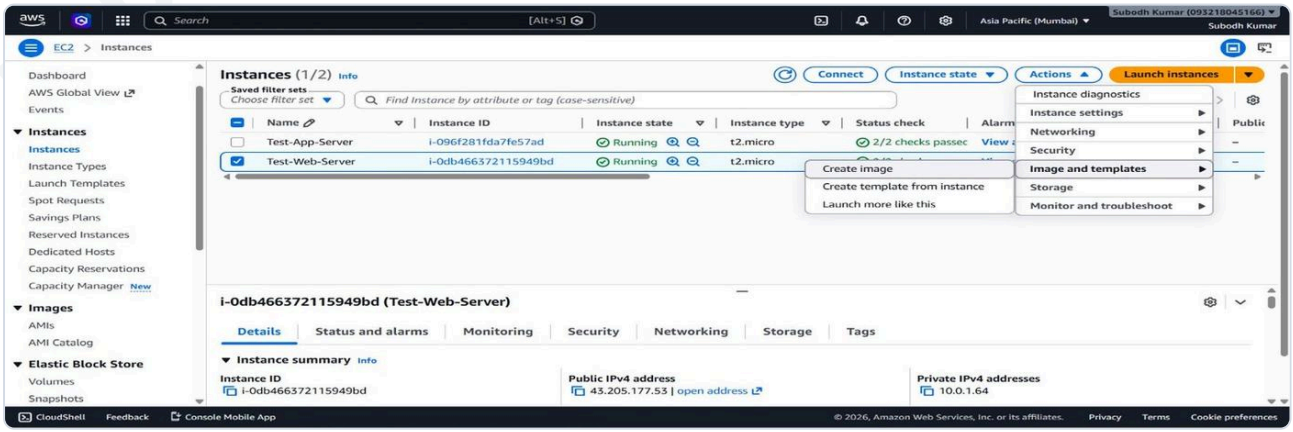
Install nginx (Webserver)

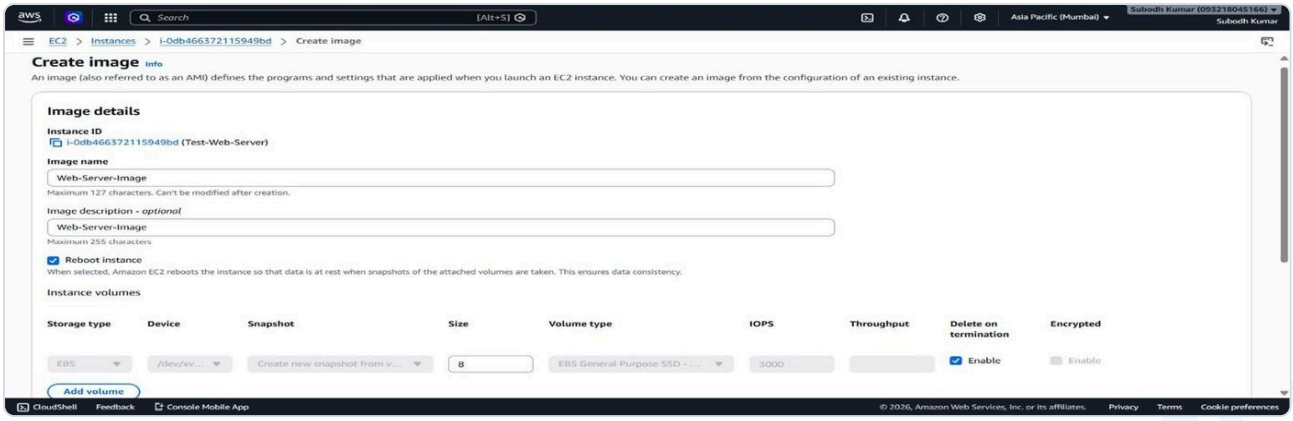
```
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf
```

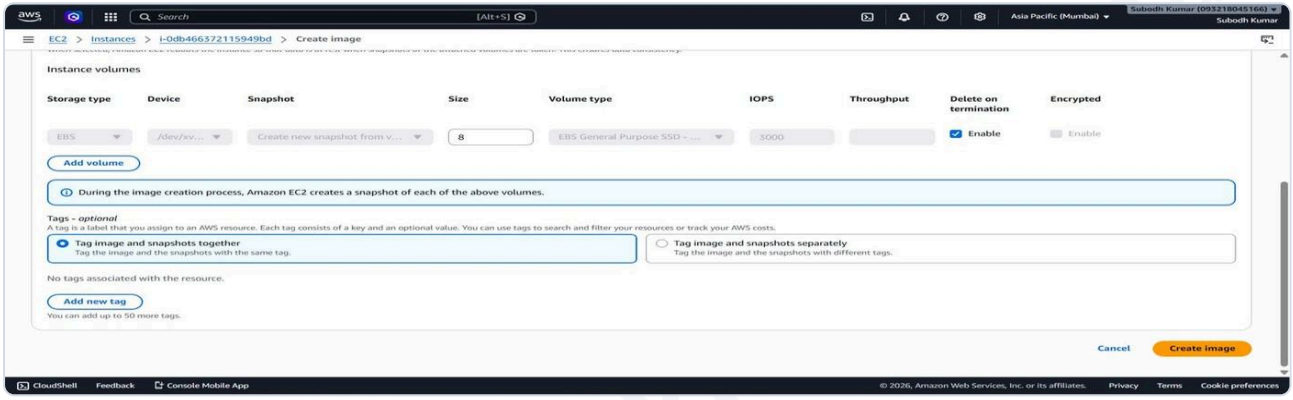
```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

Step 11 Create AMI of the Web Server

Go to instances and select Test-Web-Server > Actions > Image and templates > Create image.







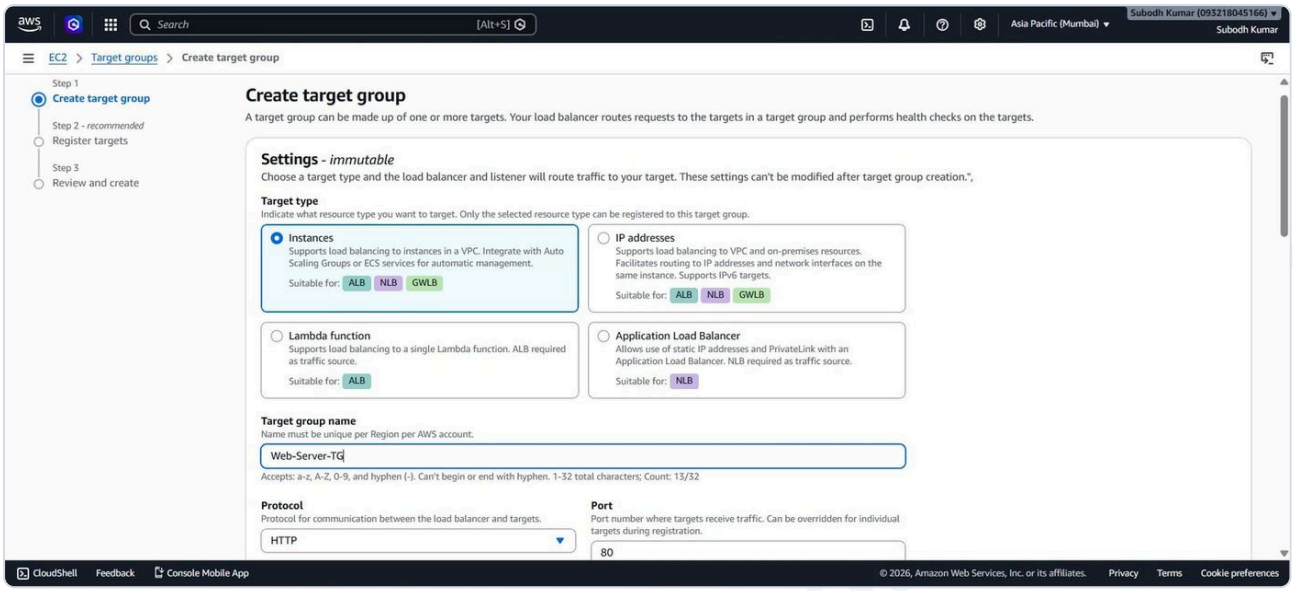
Step 12 Create Target Group, Launch Template and External Load Balancer for Web Tier

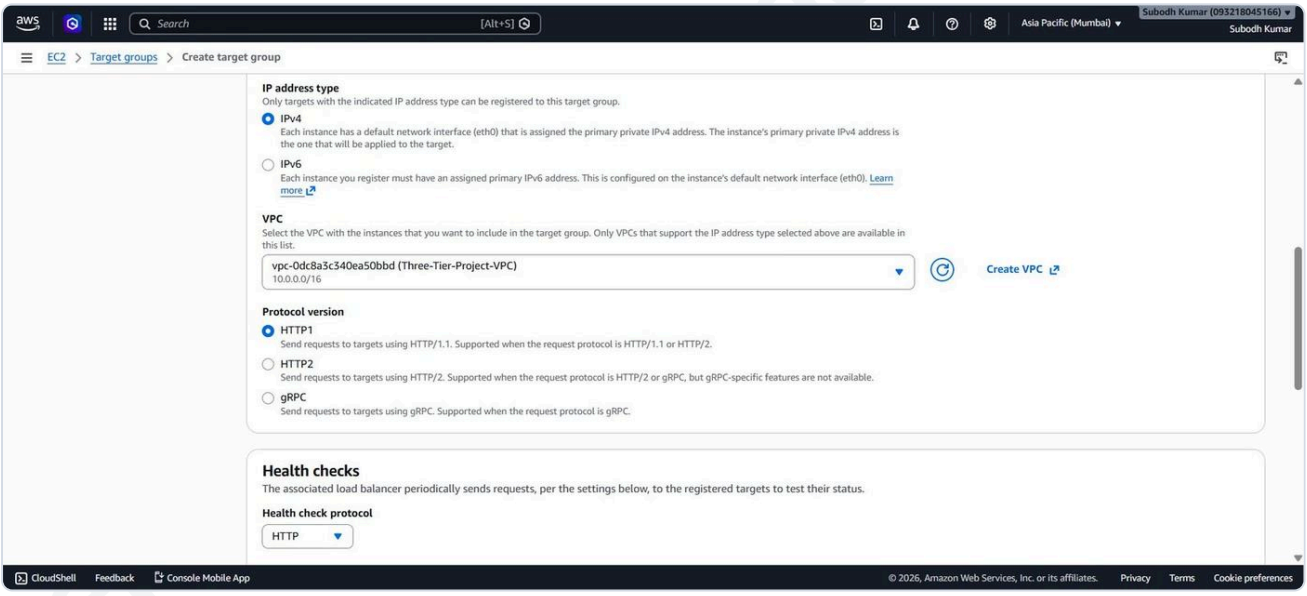
Same process

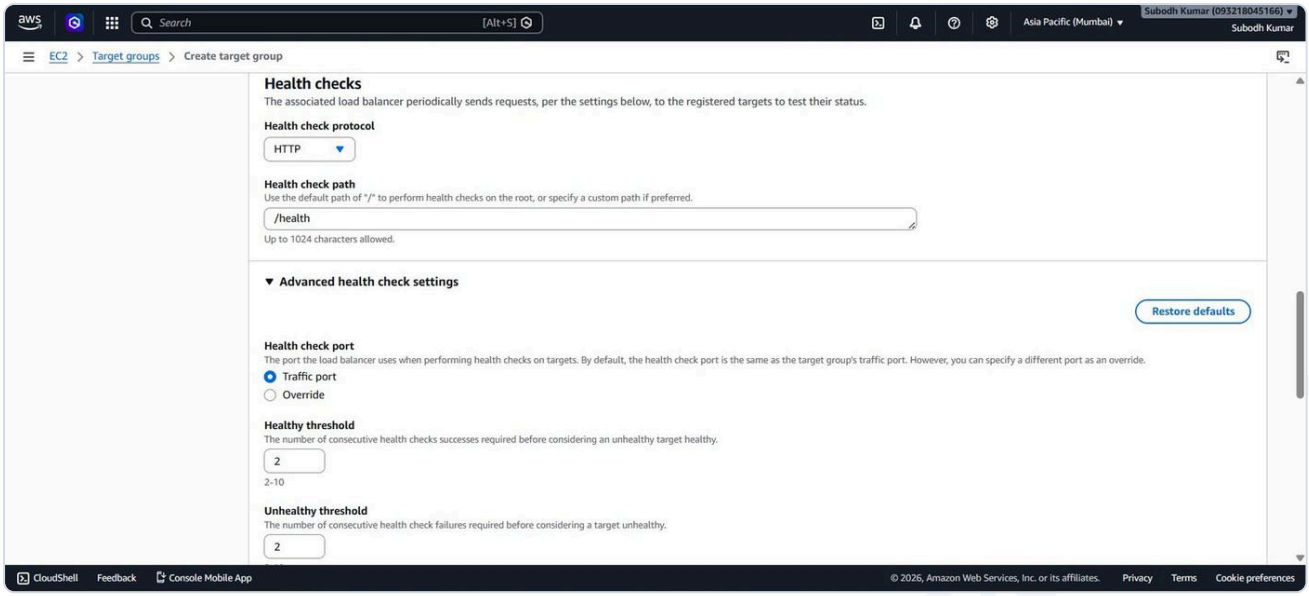
Prepared by Sukhada Wagale

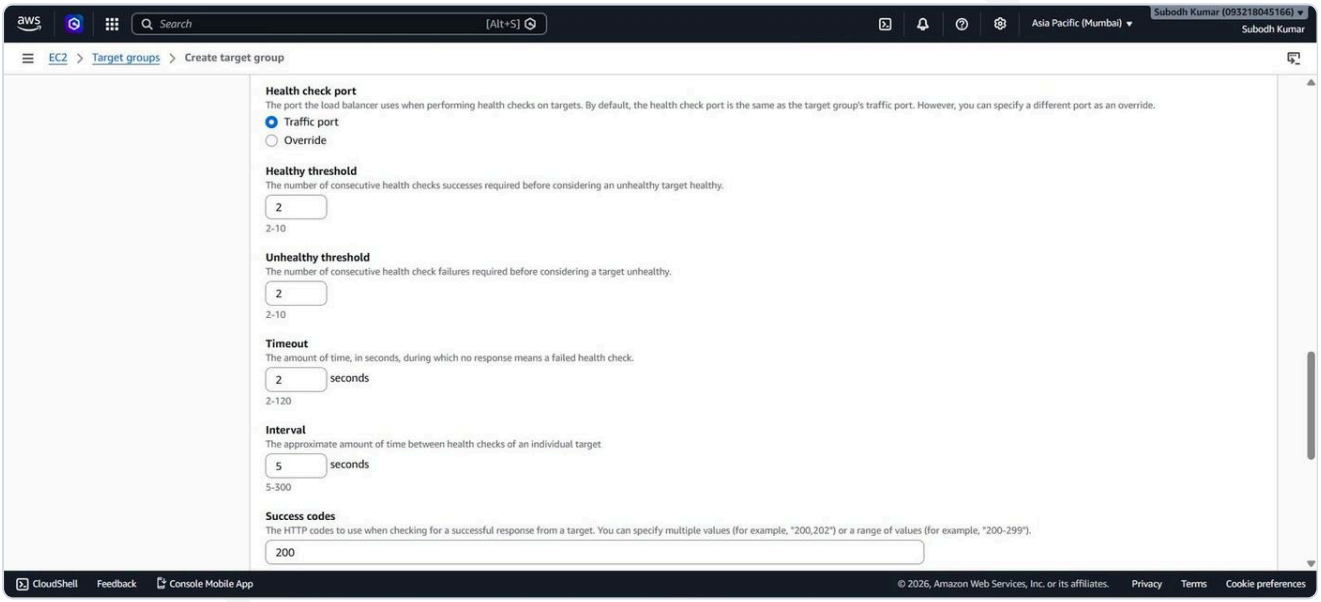
Create Target Group

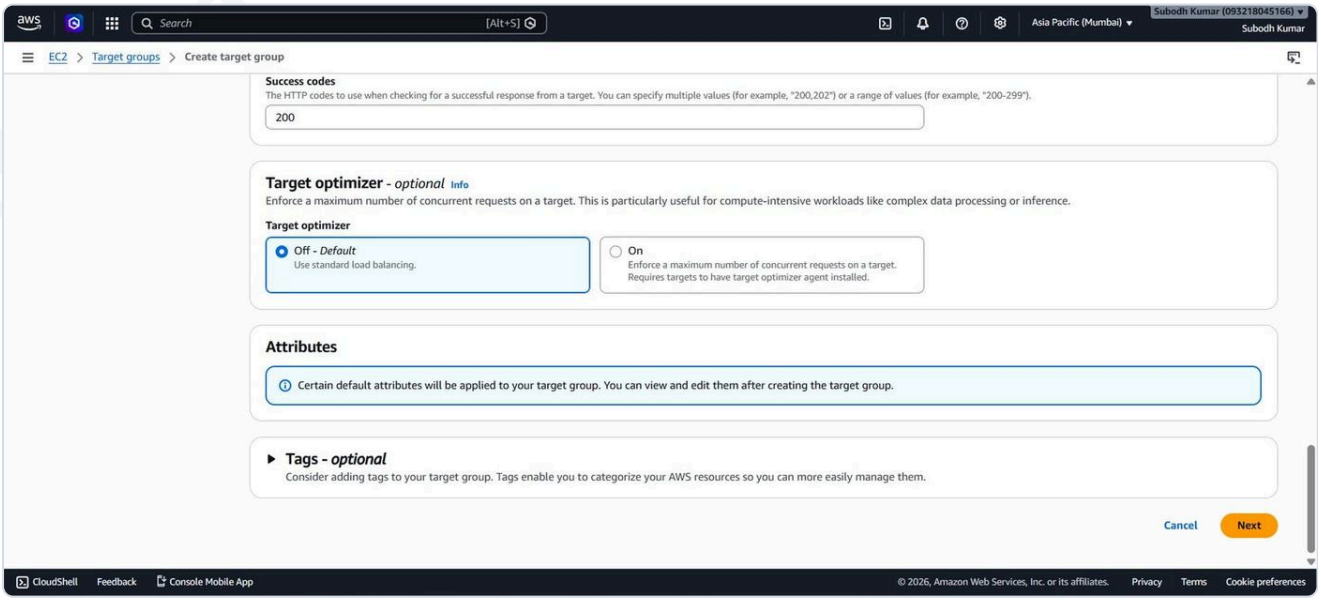
Change Protocol: Port to 80. Select VPC. Give name. Create.

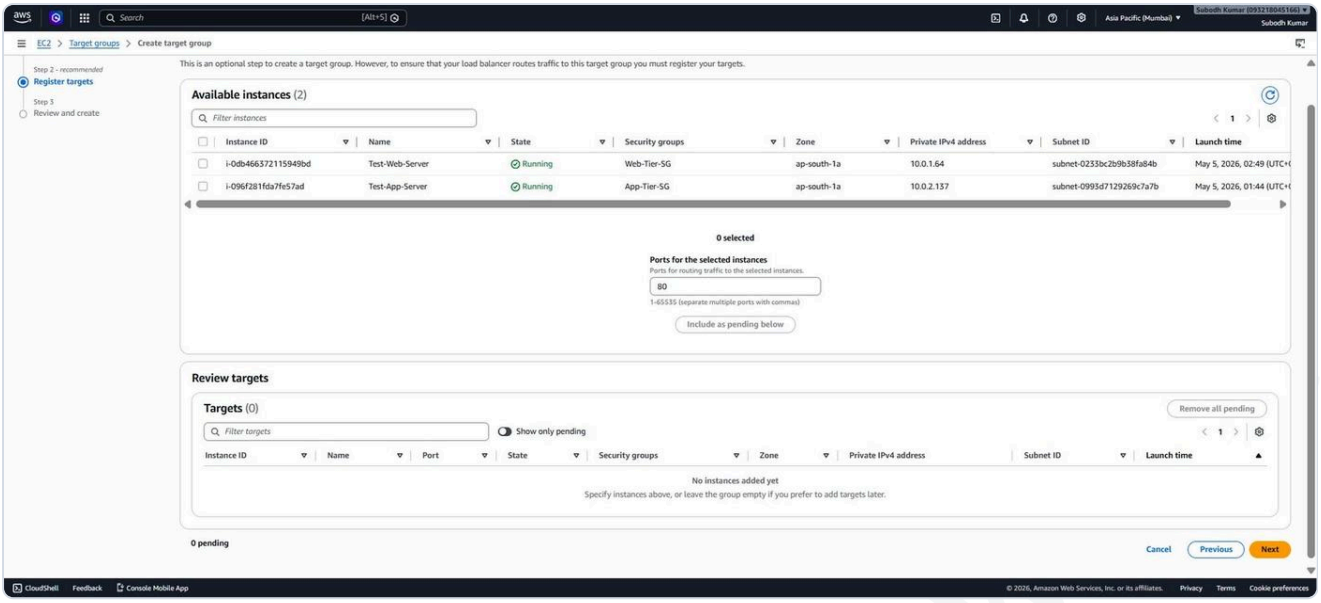


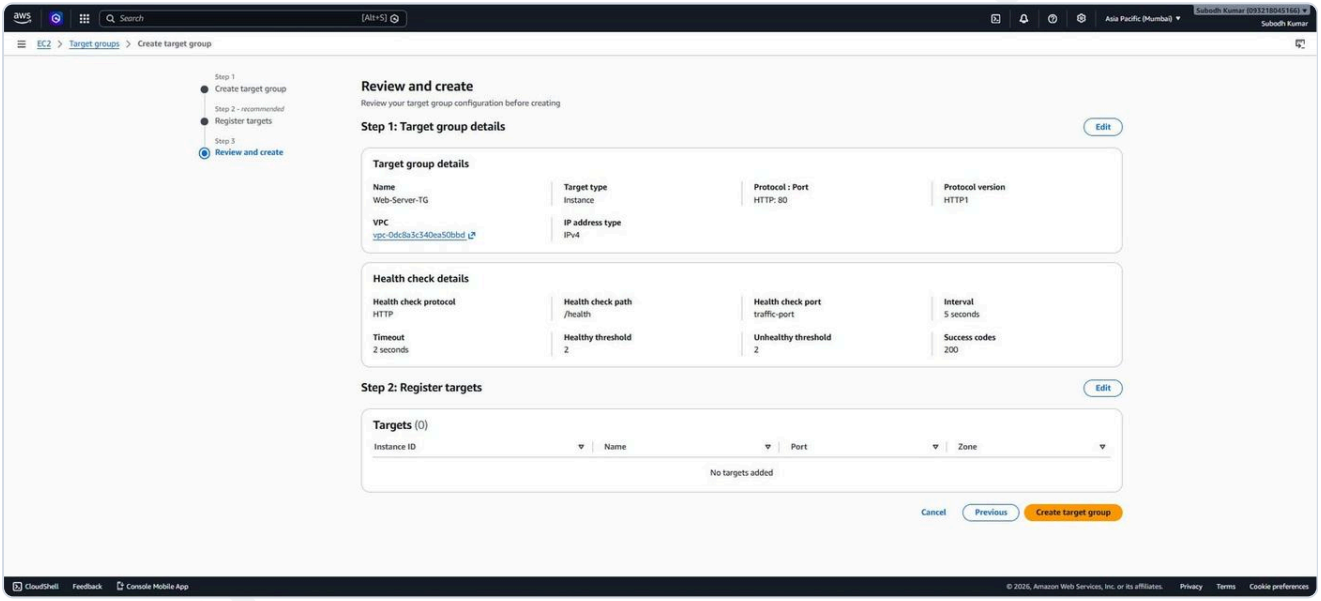



Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Target groups > Create target group
Health checks
The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.
Health check protocol
HTTP
Health check path
Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.
/health
Up to 1024 characters allowed.
Advanced health check settings
Restore defaults
Health check port
The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.
Traffic port
Override
Healthy threshold
The number of consecutive health checks successes required before considering an unhealthy target healthy.
2
2-10
Unhealthy threshold
The number of consecutive health check failures required before considering a target unhealthy.
2
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Target groups > Create target group
Health check port
The port the load balancer uses when performing health checks on targets. By default, the health check port is the same as the target group's traffic port. However, you can specify a different port as an override.
Traffic port
Override
Healthy threshold
The number of consecutive health checks successes required before considering an unhealthy target healthy.
2
2-10
Unhealthy threshold
The number of consecutive health check failures required before considering a target unhealthy.
2
2-10
Timeout
The amount of time, in seconds, during which no response means a failed health check.
2
seconds
2-120
Interval
The approximate amount of time between health checks of an individual target
5
seconds
5-300
Success codes
The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").
200
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

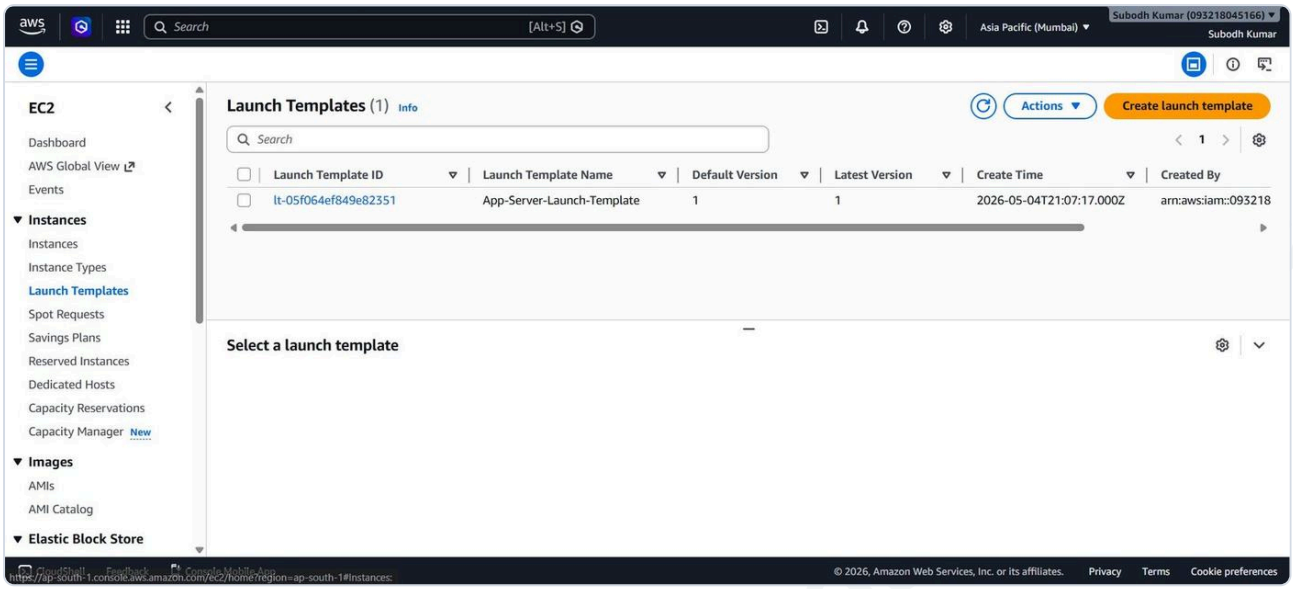

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Target groups > Create target group
Success codes
The HTTP codes to use when checking for a successful response from a target. You can specify multiple values (for example, "200,202") or a range of values (for example, "200-299").
200
Target optimizer - optional Info
Enforce a maximum number of concurrent requests on a target. This is particularly useful for compute-intensive workloads like complex data processing or inference.
Target optimizer
Off - Default
Use standard load balancing.
On
Enforce a maximum number of concurrent requests on a target.
Requires targets to have target optimizer agent installed.
Attributes
Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.
Tags - optional
Consider adding tags to your target group. Tags enable you to categorize your AWS resources so you can more easily manage them.
Cancel
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

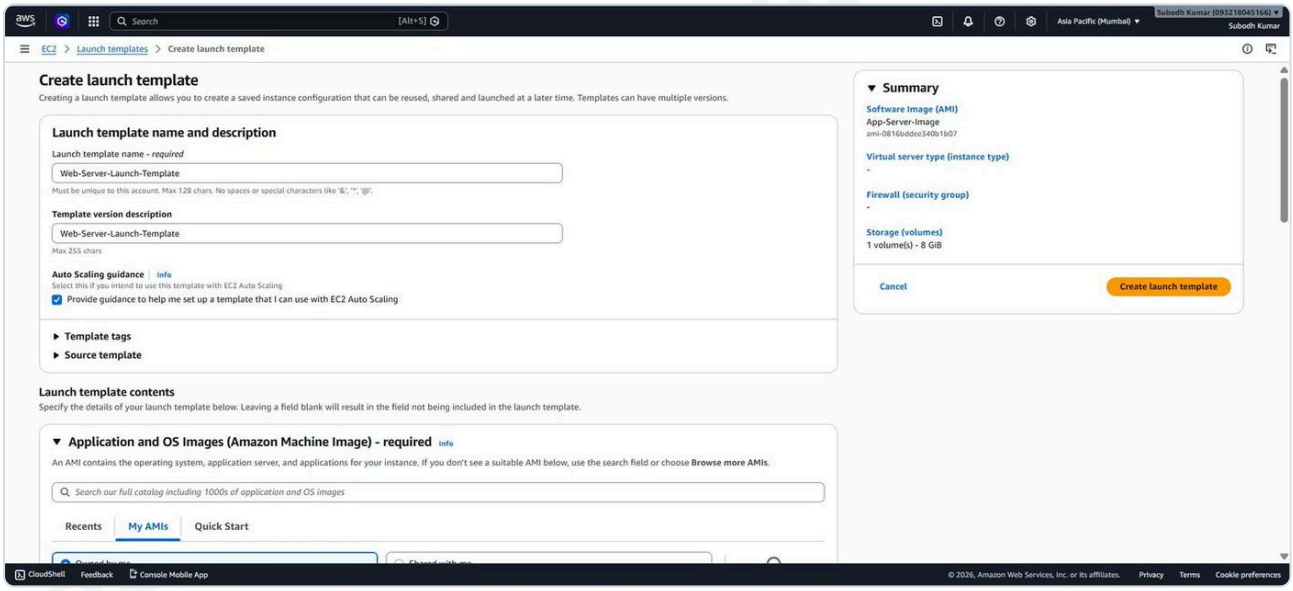

EC2 > Target groups > Create target group
Step 2 - recommended
Register targets
Step 3
Review and create
This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.
Available instances (2)
Filter instances
Instance ID
Name
State
Security groups
Zone
Private IPv4 address
Subnet ID
Launch time
i-0db466372115949bd
Test-Web-Server
Running
Web-Tier-SG
ap-south-1a
10.0.1.64
subnet-0233bc2b9b38fa84b
May 5, 2026, 02:49 (UTC+
i-096f281fda7fe57ad
Test-App-Server
Running
App-Tier-SG
ap-south-1a
10.0.2.137
subnet-0993d7129269c7a7b
May 5, 2026, 01:44 (UTC+
0 selected
Ports for the selected instances
Ports for routing traffic to the selected instances.
80
1-65535 (separate multiple ports with commas)
Include as pending below
Review targets
Targets (0)
Remove all pending
Filter targets
Show only pending
Instance ID
Name
Port
State
Security groups
Zone
Private IPv4 address
Subnet ID
Launch time
No instances added yet
Specify instances above, or leave the group empty if you prefer to add targets later.
0 pending
Cancel
Previous
Next
CloudShell
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences

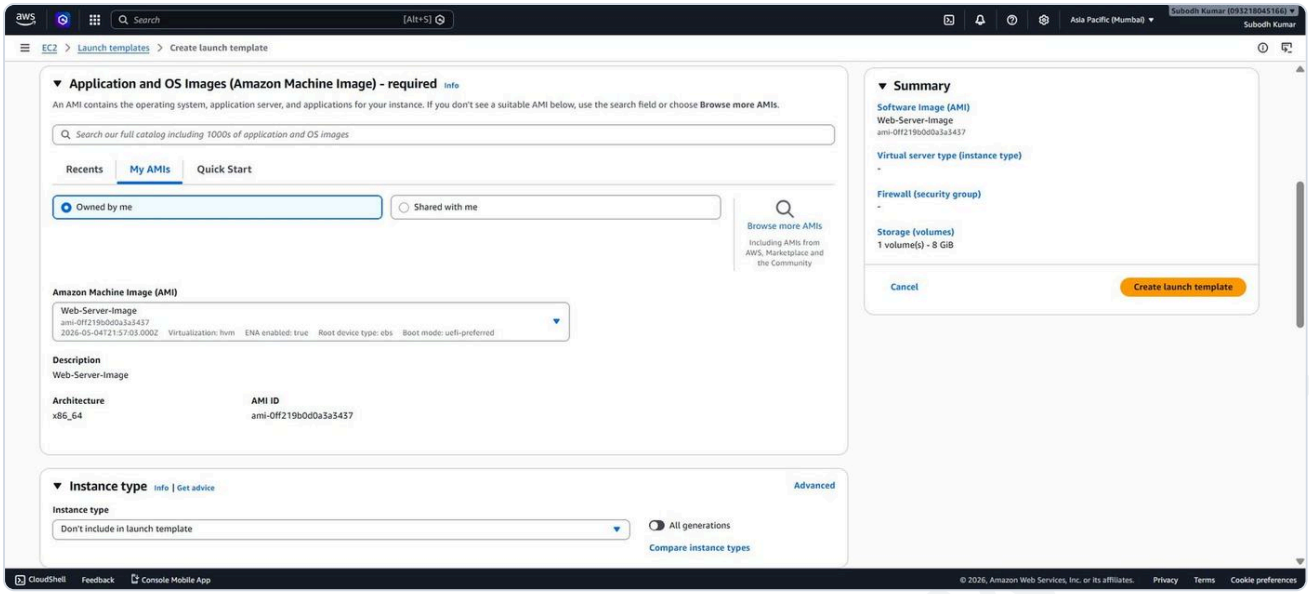

EC2 > Target groups > Create target group
Step 1
Create target group
Step 2 - recommended
Register targets
Step 3
Review and create
Review and create
Review your target group configuration before creating
Step 1: Target group details
Edit
Target group details
Name
Web-Server-TG
Target type
Instance
Protocol : Port
HTTP: 80
Protocol version
HTTP1
VPC
vpc-0dc8a3c340ea50bbd
IP address type
IPv4
Health check details
Health check protocol
HTTP
Health check path
/health
Health check port
traffic-port
Interval
5 seconds
Timeout
2 seconds
Healthy threshold
2
Unhealthy threshold
2
Success codes
200
Step 2: Register targets
Edit
Targets (0)
Instance ID
Name
Port
Zone
No targets added
Cancel
Previous
Create target group
CloudShell
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences

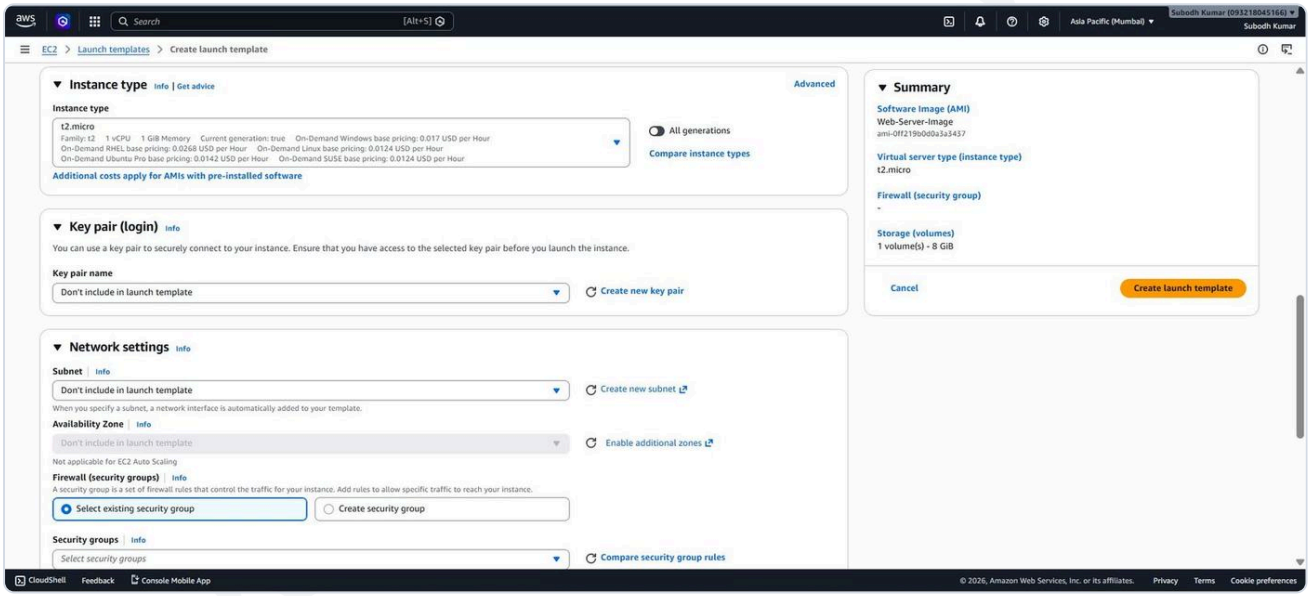
Create Launch Template

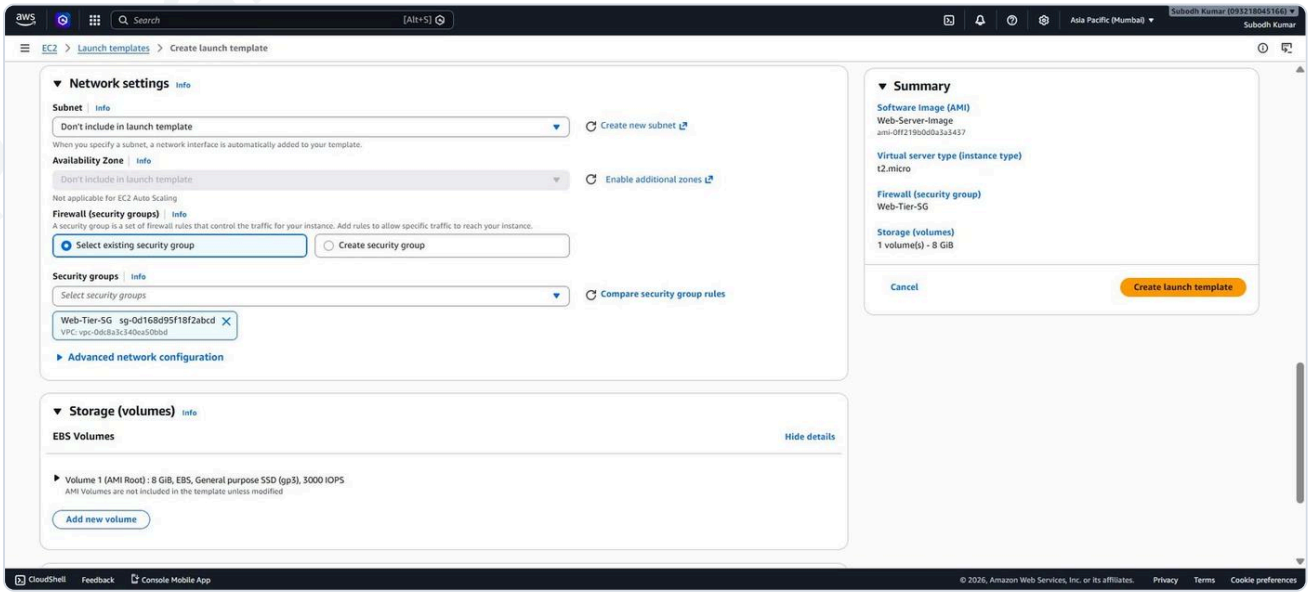
Choose Web-Server-Image for AMI and choose Web-Tier-SG. Other settings are same as above app tier.

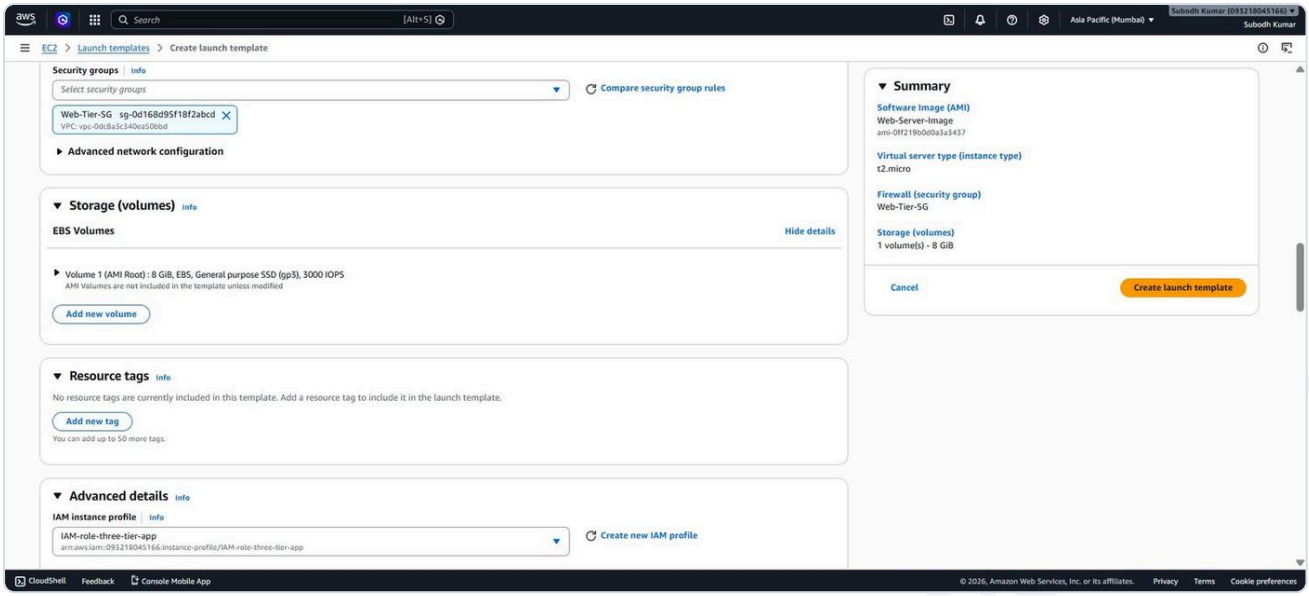





EC2 > Launch templates > Create launch template
Application and OS Images (Amazon Machine Image) - required
Recents
My AMIs
Quick Start
Owned by me
Shared with me
Browse more AMIs
Amazon Machine Image (AMI)
Web-Server-Image
Description
Web-Server-Image
Architecture
x86_64
AMI ID
ami-0ff219b0d0a3a3437
Instance type
Don't include in launch template
All generations
Compare instance types
Advanced
Summary
Software Image (AMI)
Web-Server-Image
Virtual server type (instance type)
Firewall (security group)
Storage (volumes)
1 volume(s) - 8 GiB
Cancel
Create launch template

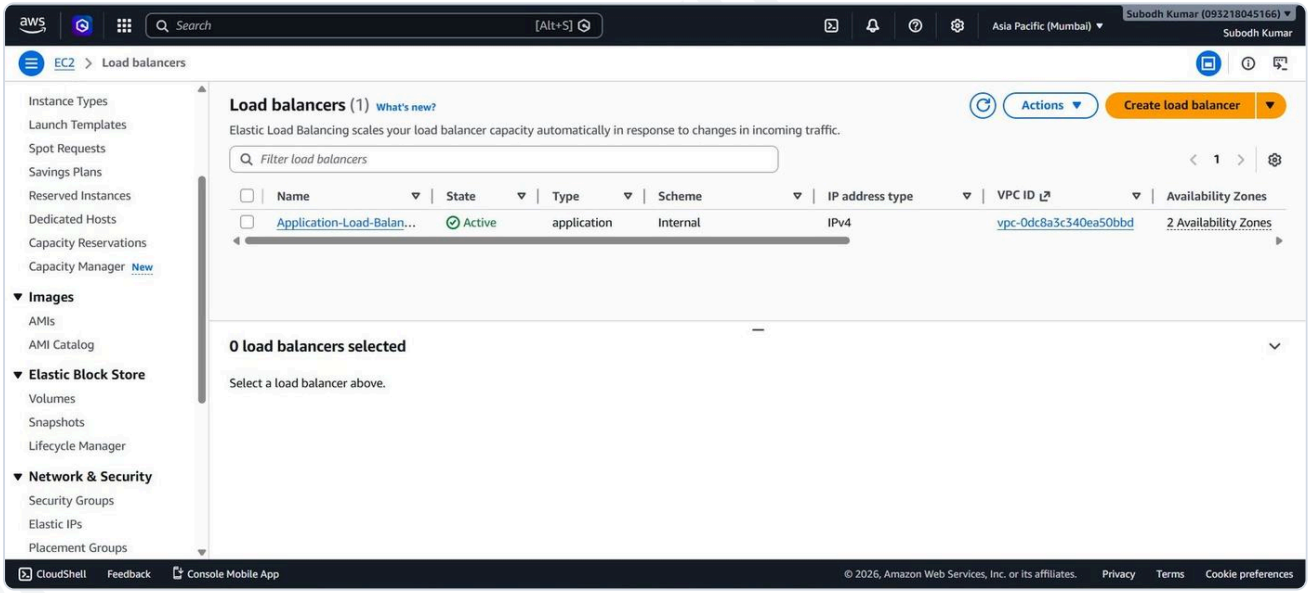

EC2 > Launch templates > Create launch template
Instance type
t2.micro
Additional costs apply for AMIs with pre-installed software
Key pair (login)
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name
Don't include in launch template
Create new key pair
Network settings
Subnet
Create new subnet
Availability Zone
Enable additional zones
Firewall (security groups)
Select existing security group
Create security group
Security groups
Compare security group rules
Summary
Virtual server type (instance type)
t2.micro
Cancel
Create launch template

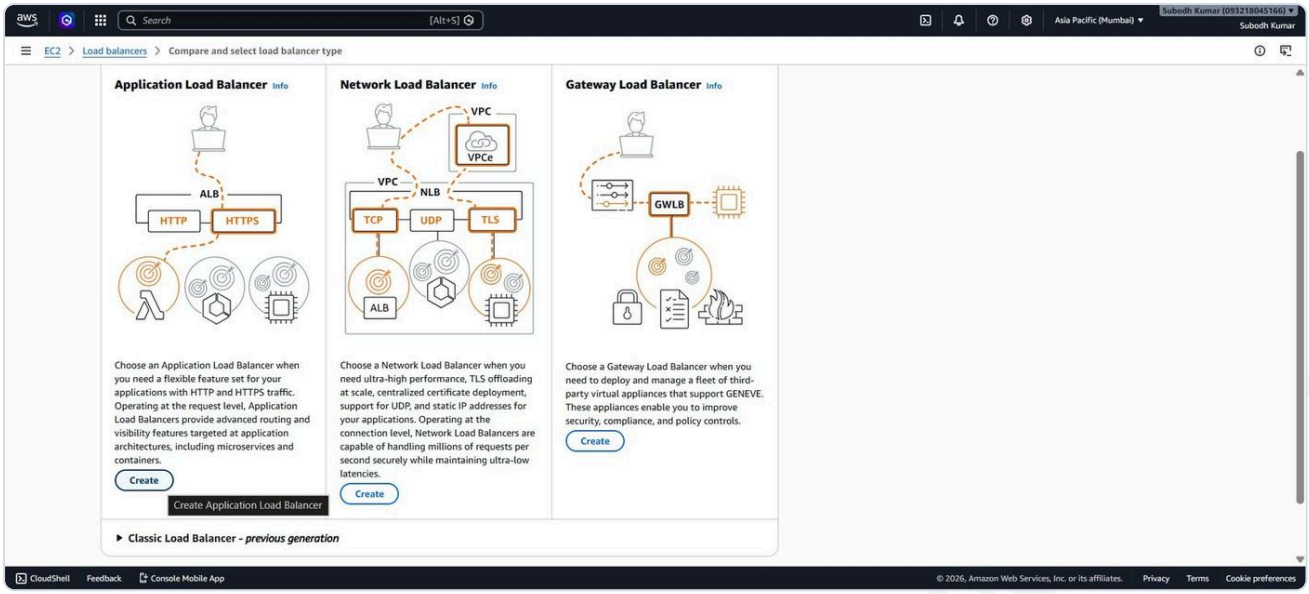

EC2 > Launch templates > Create launch template
Network settings
Web-Tier-SG sg-0d168d95f18f2abcd
Advanced network configuration
Storage (volumes)
EBS Volumes
Hide details
Volume 1 (AMI Root) : 8 GiB, EBS, General purpose SSD (gp3), 3000 IOPS
Add new volume
Summary
Firewall (security group)
Web-Tier-SG
Cancel
Create launch template

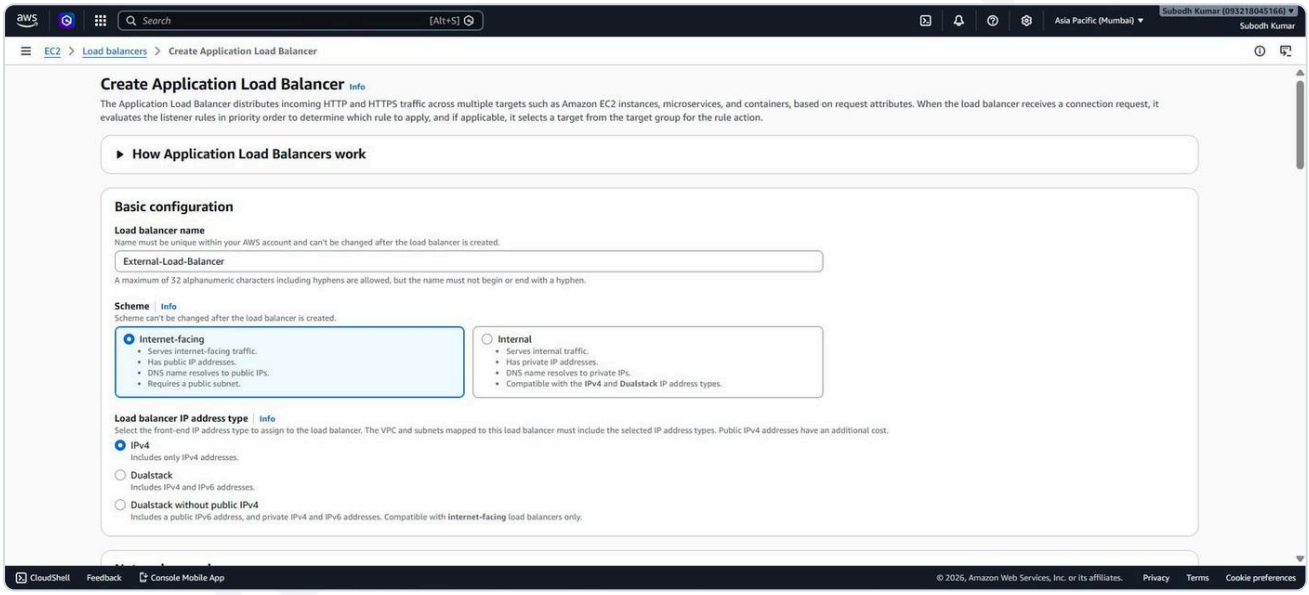


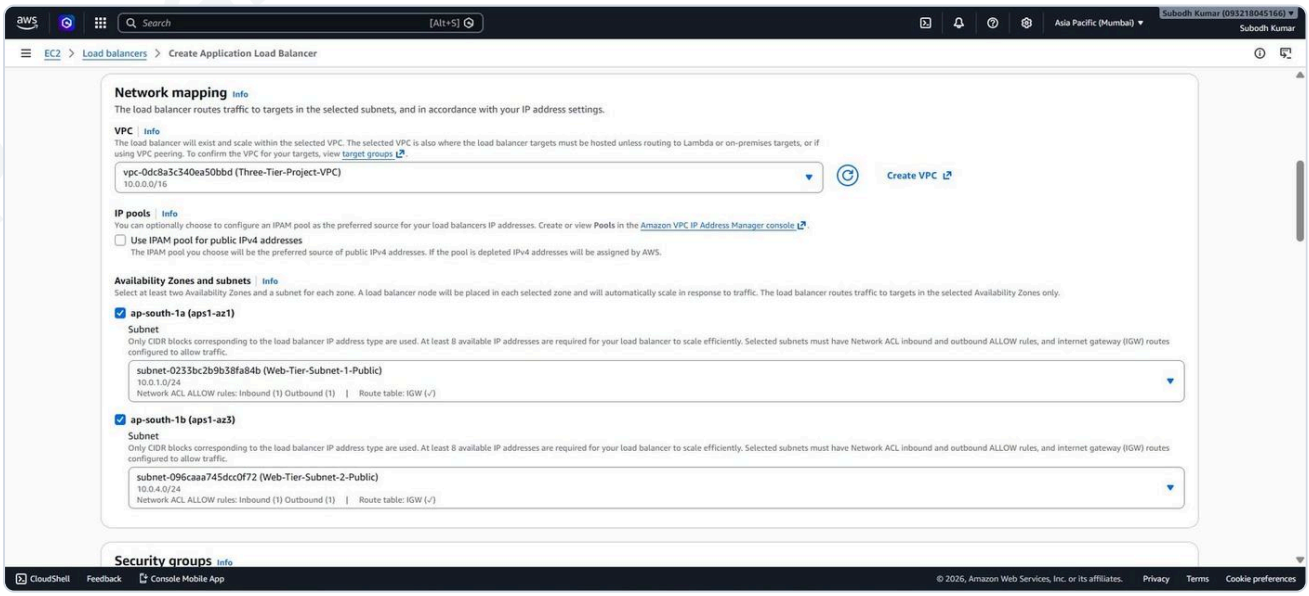
Create External Load Balancer

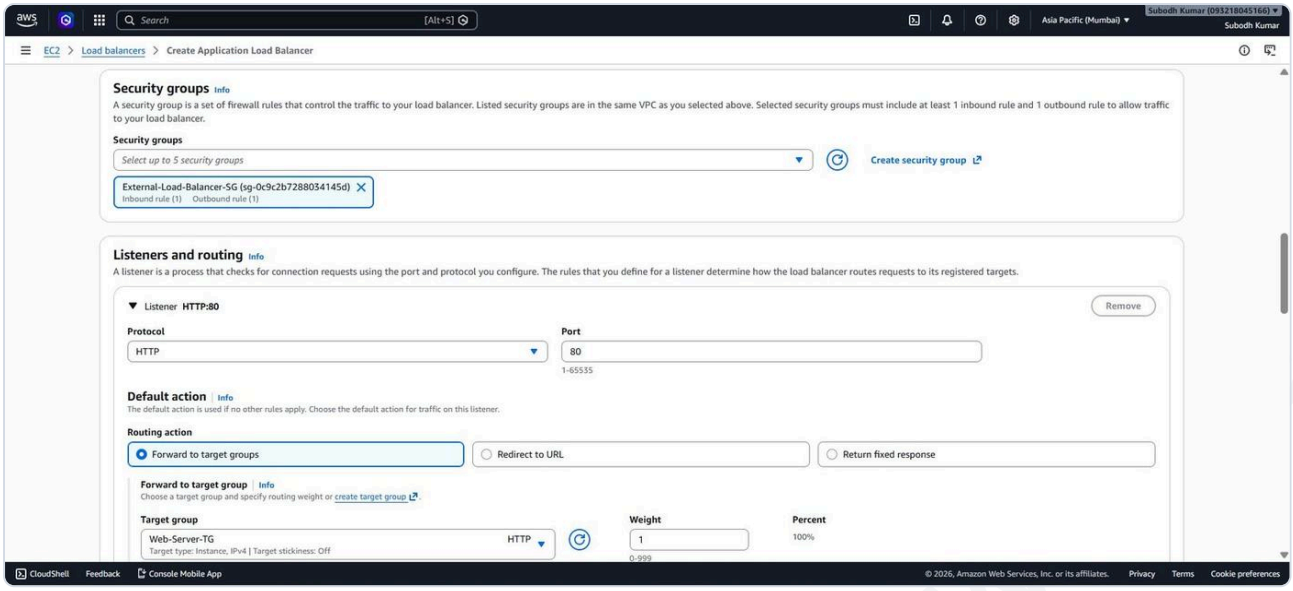
Choose **Scheme** as **Internet-facing**. Choose vpc. Choose **Availability Zones** of **web- tier**. Choose **External load balancer Security Group**. Choose target group in Listeners and routing. Create load balancer.

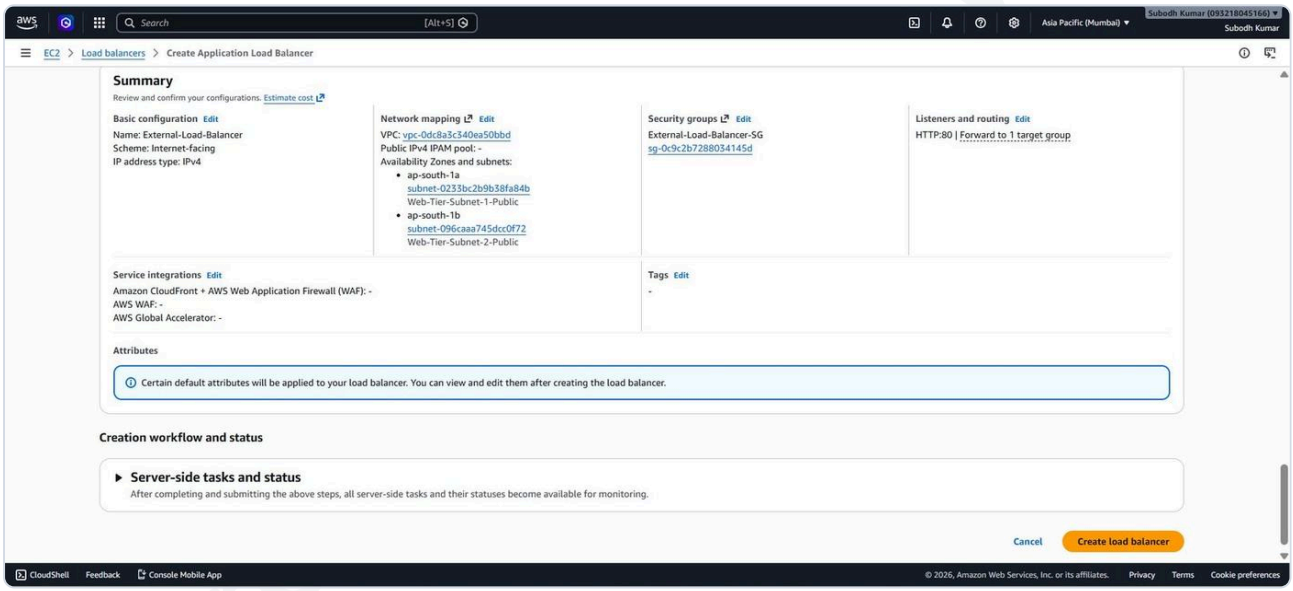



Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Load balancers > Compare and select load balancer type
Application Load Balancer Info
ALB
HTTP
HTTPS
Choose an Application Load Balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic. Operating at the request level, Application Load Balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.
Create
Create Application Load Balancer
Network Load Balancer Info
VPC
VPCe
VPC
NLB
TCP
UDP
TLS
ALB
Choose a Network Load Balancer when you need ultra-high performance, TLS offloading at scale, centralized certificate deployment, support for UDP, and static IP addresses for your applications. Operating at the connection level, Network Load Balancers are capable of handling millions of requests per second securely while maintaining ultra-low latencies.
Create
Gateway Load Balancer Info
GWLB
Choose a Gateway Load Balancer when you need to deploy and manage a fleet of third-party virtual appliances that support GENEVE. These appliances enable you to improve security, compliance, and policy controls.
Create
Classic Load Balancer - previous generation
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Load balancers > Create Application Load Balancer
Create Application Load Balancer Info
The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.
How Application Load Balancers work
Basic configuration
Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.
External-Load-Balancer
A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.
Scheme Info
Scheme can't be changed after the load balancer is created.
Internet-facing
Serves internet-facing traffic.
Has public IP addresses.
DNS name resolves to public IPs.
Requires a public subnet.
Internal
Serves internal traffic.
Has private IP addresses.
DNS name resolves to private IPs.
Compatible with the IPv4 and Dualstack IP address types.
Load balancer IP address type Info
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.
IPv4
Includes only IPv4 addresses.
Dualstack
Includes IPv4 and IPv6 addresses.
Dualstack without public IPv4
Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with internet-facing load balancers only.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar (093218045166)
Subodh Kumar
EC2 > Load balancers > Create Application Load Balancer
Network mapping Info
The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.
VPC Info
The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view target groups.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create VPC
IP pools Info
You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the Amazon VPC IP Address Manager console.
Use IPAM pool for public IPv4 addresses
The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.
Availability Zones and subnets Info
Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.
ap-south-1a (aps1-az1)
Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.
subnet-0233bc2b9b38fa84b (Web-Tier-Subnet-1-Public)
10.0.1.0/24
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)
ap-south-1b (aps1-az3)
Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.
subnet-096caaa745dcc0f72 (Web-Tier-Subnet-2-Public)
10.0.4.0/24
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)
Security groups Info
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

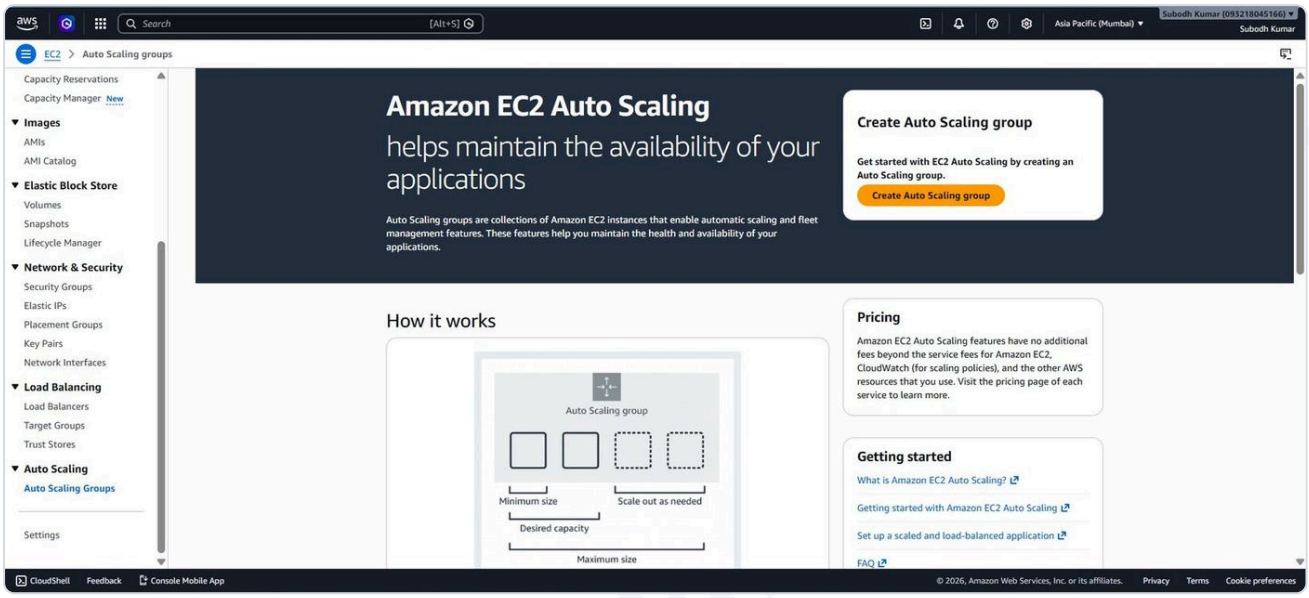

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Load balancers > Create Application Load Balancer
Security groups Info
A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.
Security groups
Select up to 5 security groups
Create security group
External-Load-Balancer-SG (sg-0c9c2b7288034145d)
Inbound rule (1) Outbound rule (1)
Listeners and routing Info
A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.
Listener HTTP:80
Remove
Protocol
HTTP
Port
80
1-65535
Default action Info
The default action is used if no other rules apply. Choose the default action for traffic on this listener.
Routing action
Forward to target groups
Redirect to URL
Return fixed response
Forward to target group Info
Choose a target group and specify routing weight or create target group
Target group
Web-Server-TG
HTTP
Target type: Instance, IPv4 | Target stickiness: Off
Weight
1
0-999
Percent
100%
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

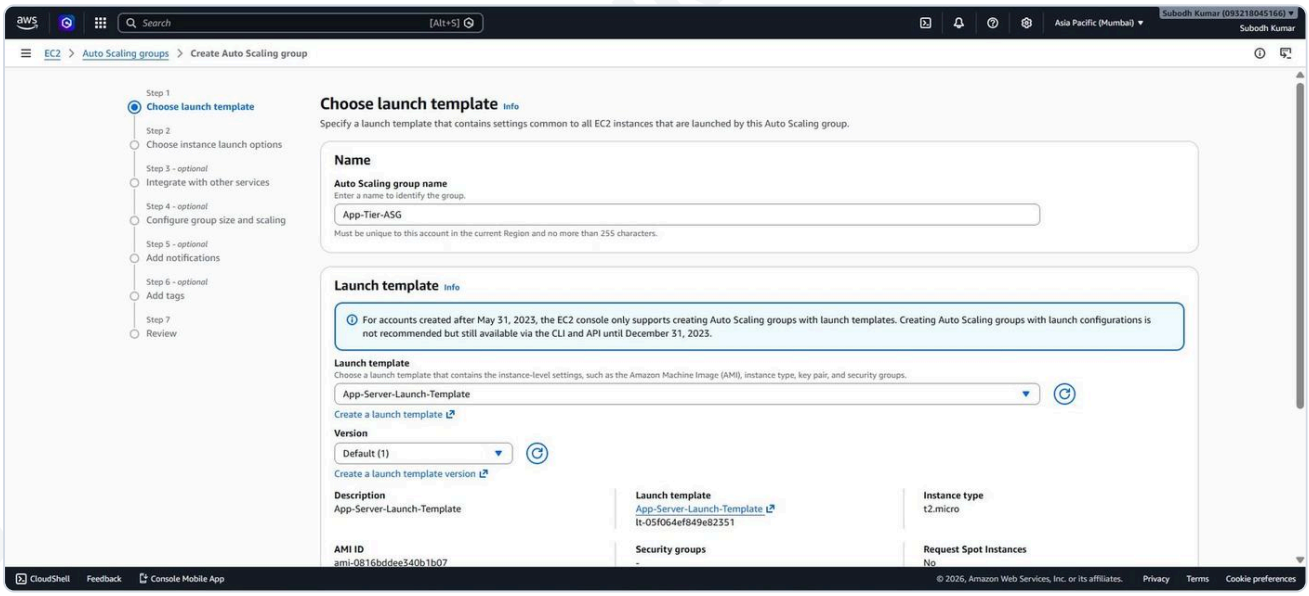

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Load balancers > Create Application Load Balancer
Summary
Review and confirm your configurations. Estimate cost
Basic configuration Edit
Name: External-Load-Balancer
Scheme: Internet-facing
IP address type: IPv4
Network mapping Edit
VPC: vpc-0dc8a3c340ea50bbd
Public IPv4 IPAM pool: -
Availability Zones and subnets:
ap-south-1a
subnet-0233bc2b9b38fa84b
Web-Tier-Subnet-1-Public
ap-south-1b
subnet-096caaa745dcc0f72
Web-Tier-Subnet-2-Public
Security groups Edit
External-Load-Balancer-SG
sg-0c9c2b7288034145d
Listeners and routing Edit
HTTP:80 | Forward to 1 target group
Service integrations Edit
Amazon CloudFront + AWS Web Application Firewall (WAF): -
AWS WAF: -
AWS Global Accelerator: -
Tags Edit
-
Attributes
Certain default attributes will be applied to your load balancer. You can view and edit them after creating the load balancer.
Creation workflow and status
Server-side tasks and status
After completing and submitting the above steps, all server-side tasks and their statuses become available for monitoring.
Cancel
Create load balancer
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

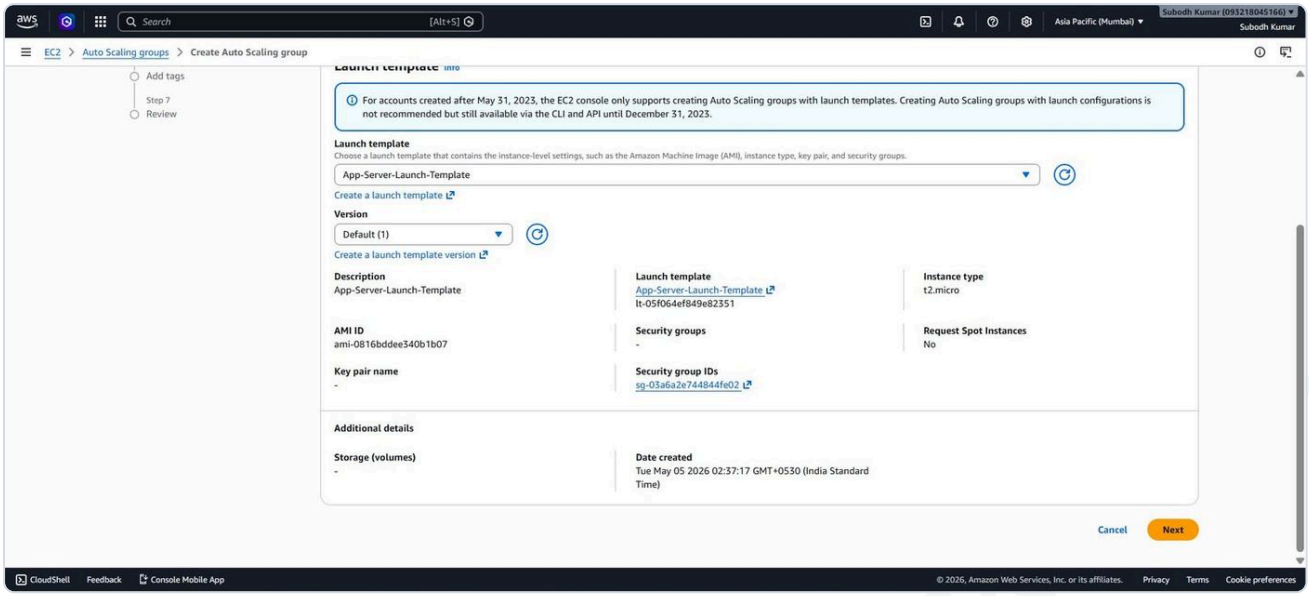
Step 13 Create Auto Scaling Groups

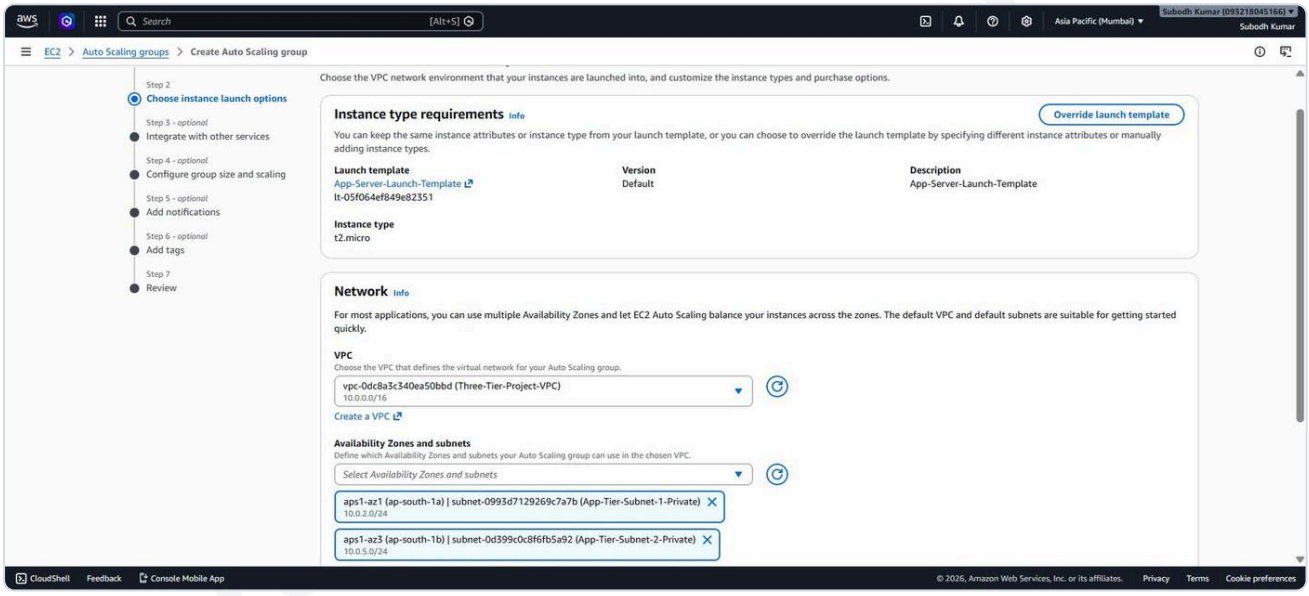
Auto Scaling Group for App Tier

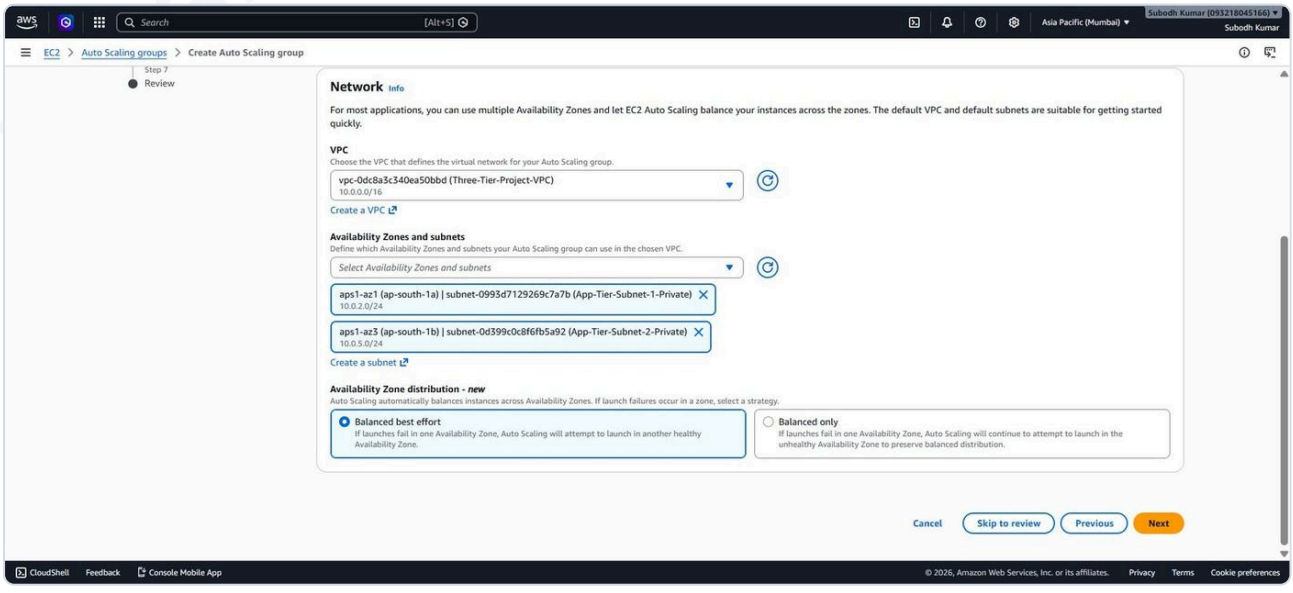
Go to EC2 > Create Auto Scaling group

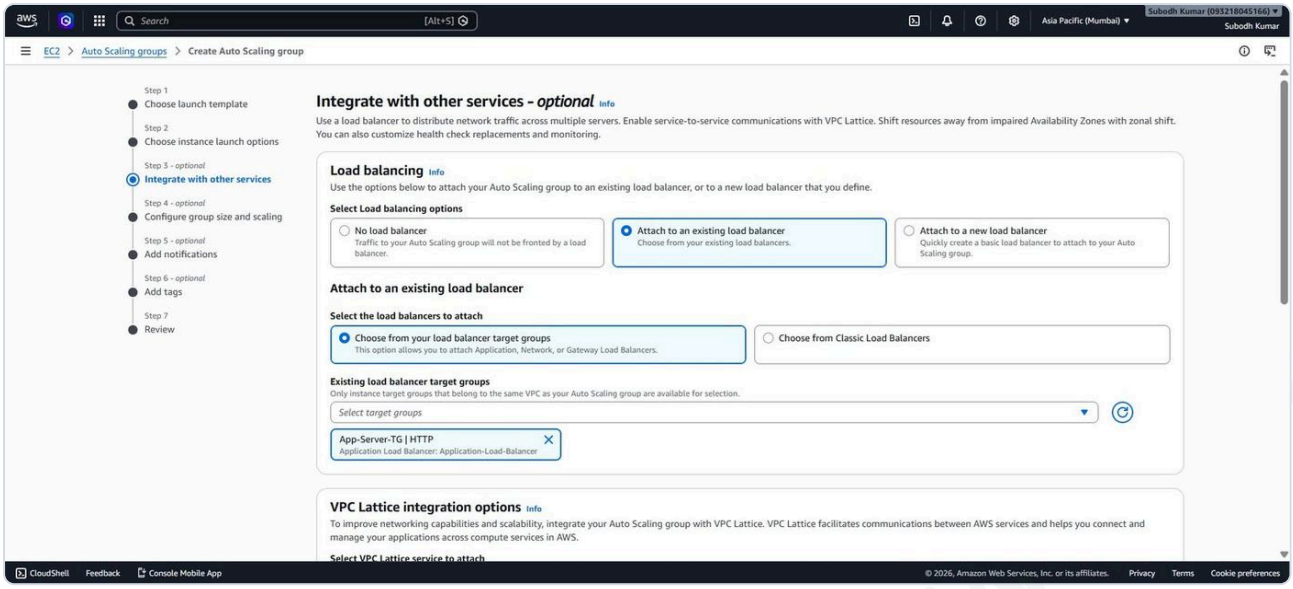


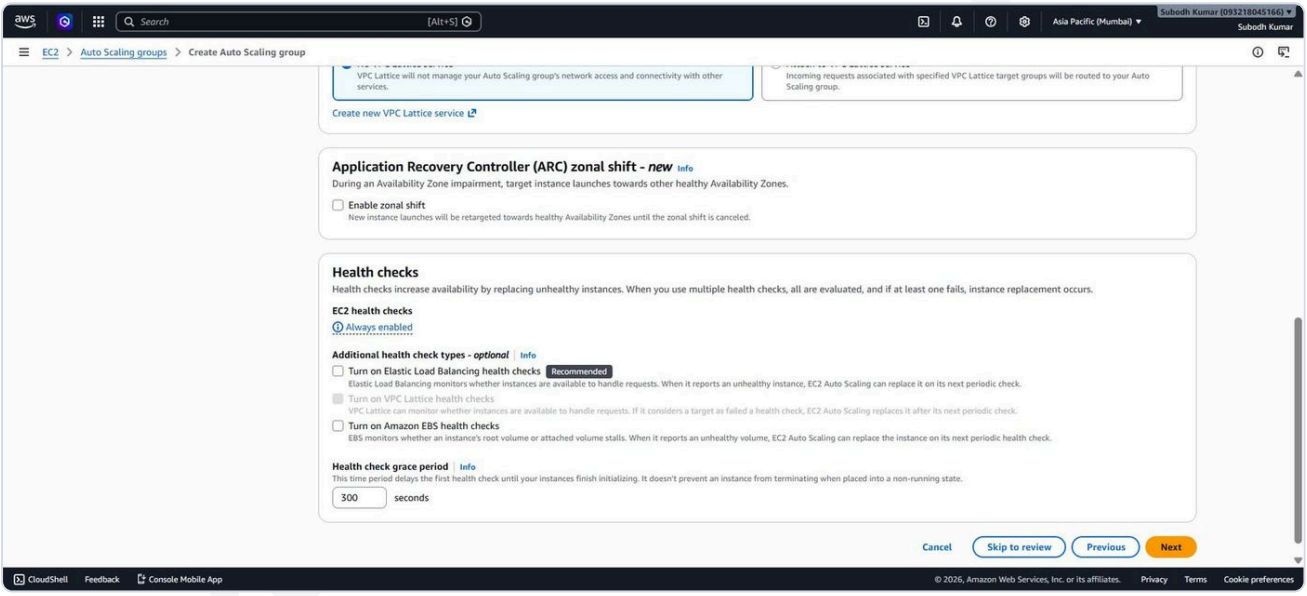


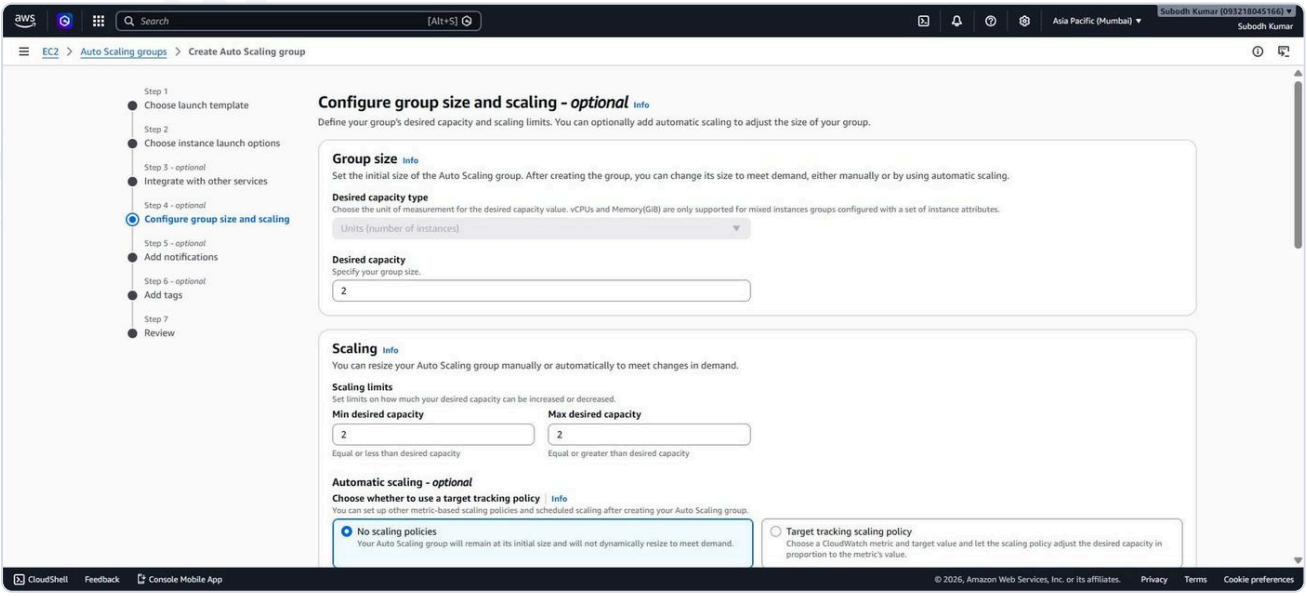

EC2 > Auto Scaling groups > Create Auto Scaling group
Add tags
Step 7
Review
For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.
Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
App-Server-Launch-Template
Create a launch template
Version
Default (1)
Create a launch template version
Description
App-Server-Launch-Template
Launch template
App-Server-Launch-Template
lt-05f064ef849e82351
Instance type
t2.micro
AMI ID
ami-0816bddee340b1b07
Security groups
-
Request Spot Instances
No
Key pair name
-
Security group IDs
sg-03a6a2e744844fe02
Additional details
Storage (volumes)
-
Date created
Tue May 05 2026 02:37:17 GMT+0530 (India Standard Time)
Cancel
Next

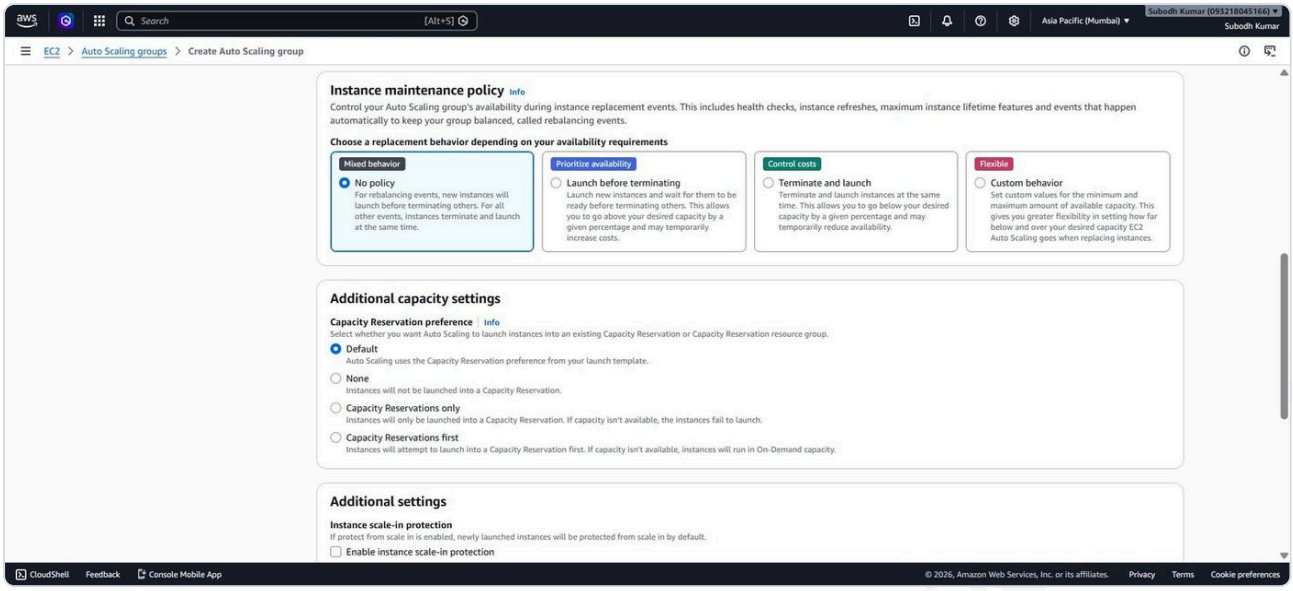

EC2 > Auto Scaling groups > Create Auto Scaling group
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Choose the VPC network environment that your instances are launched into, and customize the instance types and purchase options.
Instance type requirements
Override launch template
You can keep the same instance attributes or instance type from your launch template, or you can choose to override the launch template by specifying different instance attributes or manually adding instance types.
Launch template
App-Server-Launch-Template
lt-05f064ef849e82351
Version
Default
Description
App-Server-Launch-Template
Instance type
t2.micro
Network
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
aps1-az1 (ap-south-1a) | subnet-0993d7129269c7a7b (App-Tier-Subnet-1-Private)
10.0.2.0/24
aps1-az3 (ap-south-1b) | subnet-0d399c0c8f6fb5a92 (App-Tier-Subnet-2-Private)
10.0.5.0/24

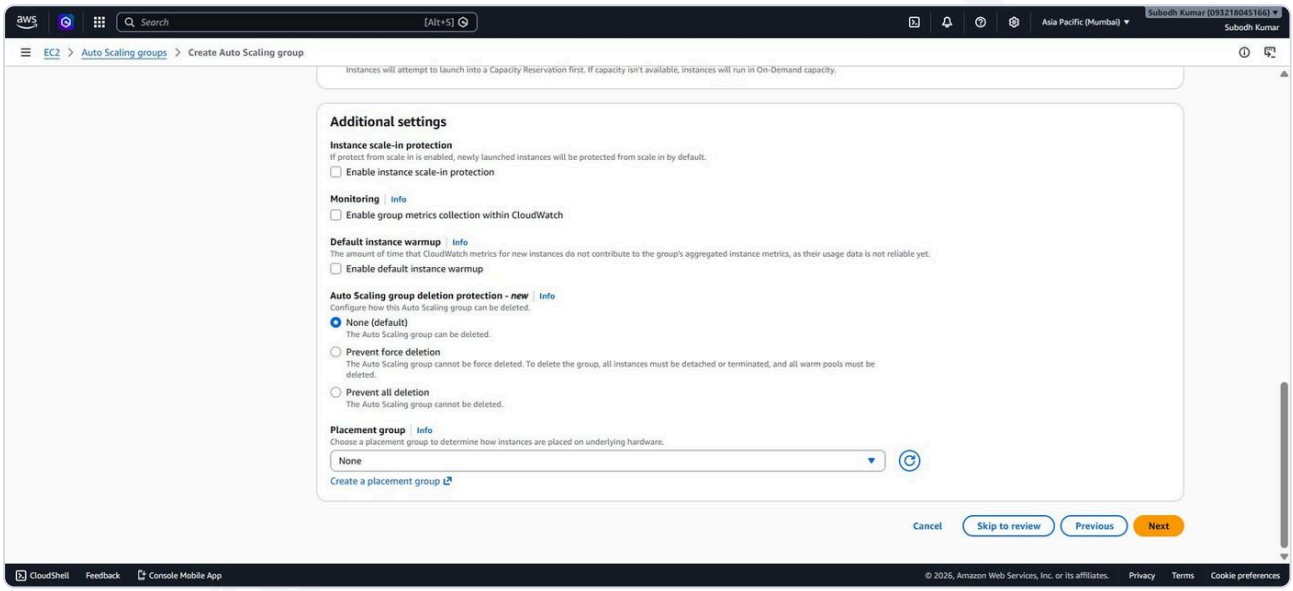

EC2 > Auto Scaling groups > Create Auto Scaling group
Step 7
Review
Network
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
aps1-az1 (ap-south-1a) | subnet-0993d7129269c7a7b (App-Tier-Subnet-1-Private)
10.0.2.0/24
aps1-az3 (ap-south-1b) | subnet-0d399c0c8f6fb5a92 (App-Tier-Subnet-2-Private)
10.0.5.0/24
Create a subnet
Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.
Balanced best effort
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.
Balanced only
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.
Cancel
Skip to review
Previous
Next

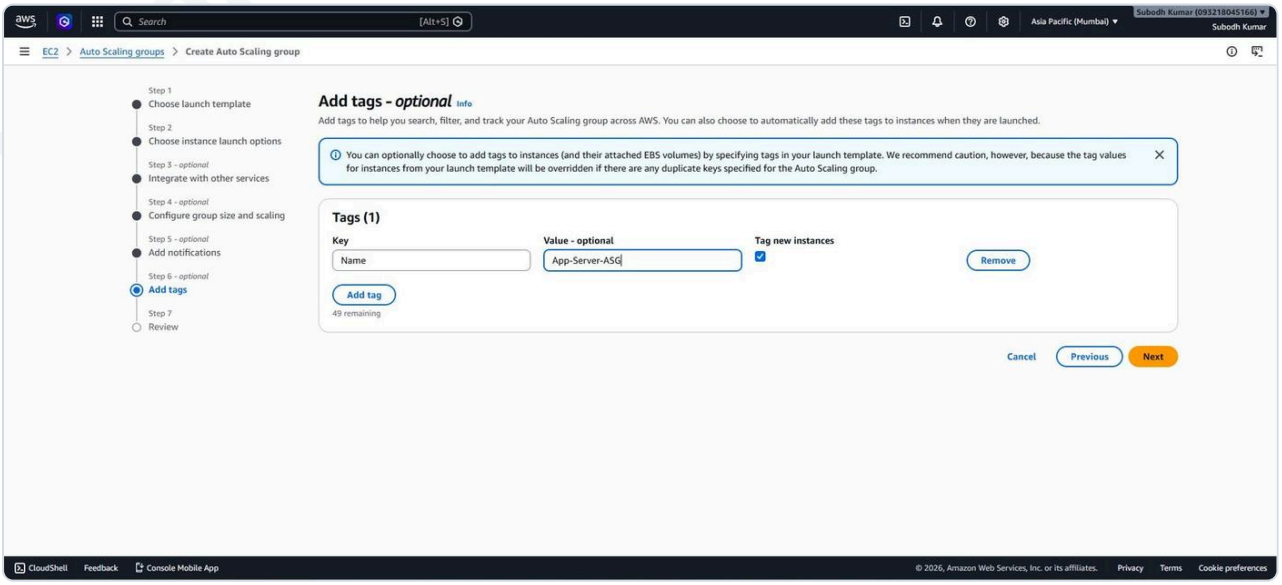

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review
Integrate with other services - optional Info
Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.
Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
Select Load balancing options
No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
Attach to an existing load balancer
Choose from your existing load balancers.
Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.
Choose from Classic Load Balancers
Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.
Select target groups
App-Server-TG | HTTP
Application Load Balancer: Application-Load-Balancer
VPC Lattice integration options Info
To improve networking capabilities and scalability, integrate your Auto Scaling group with VPC Lattice. VPC Lattice facilitates communications between AWS services and helps you connect and manage your applications across compute services in AWS.
Select VPC Lattice service to attach
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

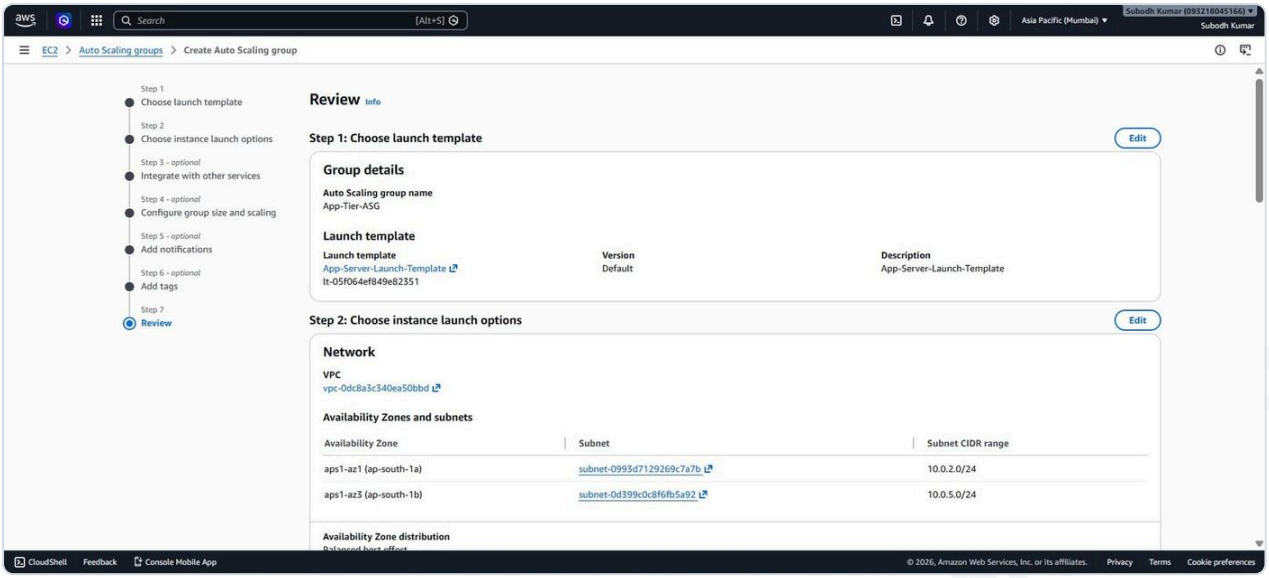

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
VPC Lattice will not manage your Auto Scaling group's network access and connectivity with other services.
Incoming requests associated with specified VPC Lattice target groups will be routed to your Auto Scaling group.
Create new VPC Lattice service
Application Recovery Controller (ARC) zonal shift - new Info
During an Availability Zone impairment, target instance launches towards other healthy Availability Zones.
Enable zonal shift
New instance launches will be retargeted towards healthy Availability Zones until the zonal shift is canceled.
Health checks
Health checks increase availability by replacing unhealthy instances. When you use multiple health checks, all are evaluated, and if at least one fails, instance replacement occurs.
EC2 health checks
Always enabled
Additional health check types - optional Info
Turn on Elastic Load Balancing health checks Recommended
Elastic Load Balancing monitors whether instances are available to handle requests. When it reports an unhealthy instance, EC2 Auto Scaling can replace it on its next periodic check.
Turn on VPC Lattice health checks
VPC Lattice can monitor whether instances are available to handle requests. If it considers a target as failed a health check, EC2 Auto Scaling replaces it after its next periodic check.
Turn on Amazon EBS health checks
EBS monitors whether an instance's root volume or attached volume stalls. When it reports an unhealthy volume, EC2 Auto Scaling can replace the instance on its next periodic health check.
Health check grace period Info
This time period delays the first health check until your instances finish initializing. It doesn't prevent an instance from terminating when placed into a non-running state.
300 seconds
Cancel Skip to review Previous Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

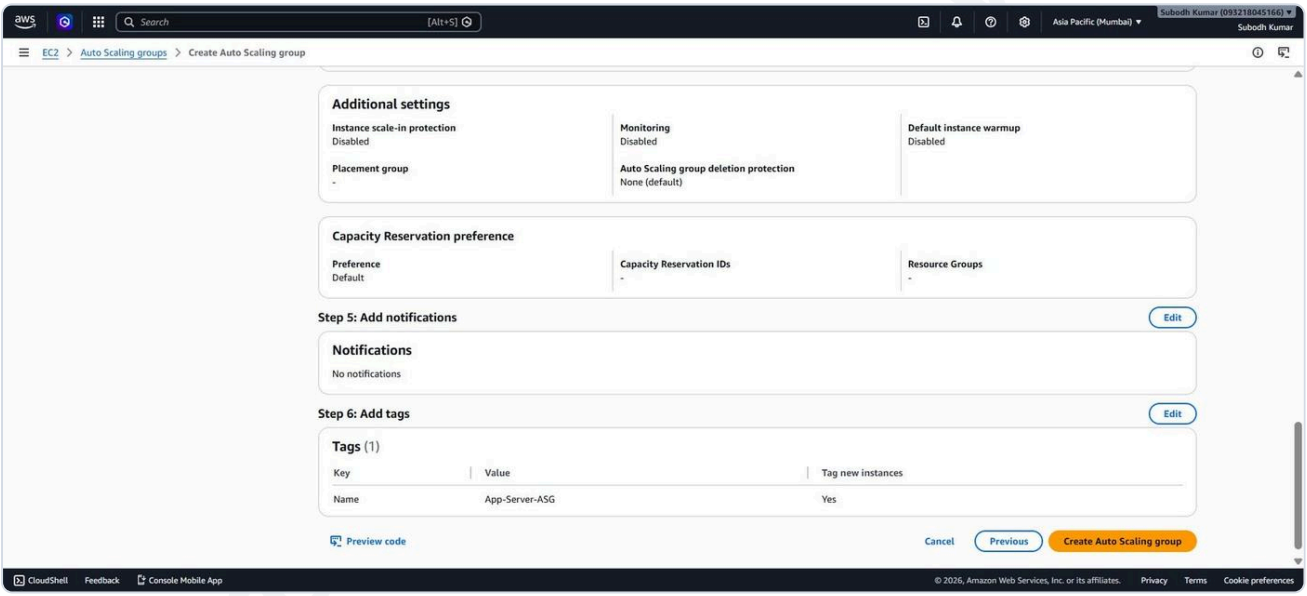

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review
Configure group size and scaling - optional Info
Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.
Group size Info
Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.
Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.
Units (number of instances)
Desired capacity
Specify your group size.
2
Scaling Info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.
Scaling limits
Set limits on how much your desired capacity can be increased or decreased.
Min desired capacity
2
Equal or less than desired capacity
Max desired capacity
2
Equal or greater than desired capacity
Automatic scaling - optional
Choose whether to use a target tracking policy Info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.
No scaling policies
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.
Target tracking scaling policy
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
Instance maintenance policy Info
Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.
Choose a replacement behavior depending on your availability requirements
Mixed behavior
No policy
For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.
Prioritize availability
Launch before terminating
Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.
Control costs
Terminate and launch
Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.
Flexible
Custom behavior
Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.
Additional capacity settings
Capacity Reservation preference Info
Select whether you want Auto Scaling to launch instances into an existing Capacity Reservation or Capacity Reservation resource group.
Default
Auto Scaling uses the Capacity Reservation preference from your launch template.
None
Instances will not be launched into a Capacity Reservation.
Capacity Reservations only
Instances will only be launched into a Capacity Reservation. If capacity isn't available, the instances fail to launch.
Capacity Reservations first
Instances will attempt to launch into a Capacity Reservation first. If capacity isn't available, instances will run in On-Demand capacity.
Additional settings
Instance scale-in protection
If protect from scale in is enabled, newly launched instances will be protected from scale in by default.
Enable instance scale-in protection
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
Additional settings
Instance scale-in protection
If protect from scale in is enabled, newly launched instances will be protected from scale in by default.
Enable instance scale-in protection
Monitoring Info
Enable group metrics collection within CloudWatch
Default instance warmup Info
The amount of time that CloudWatch metrics for new instances do not contribute to the group's aggregated instance metrics, as their usage data is not reliable yet.
Enable default instance warmup
Auto Scaling group deletion protection - new Info
Configure how this Auto Scaling group can be deleted.
None (default)
The Auto Scaling group can be deleted.
Prevent force deletion
The Auto Scaling group cannot be force deleted. To delete the group, all instances must be detached or terminated, and all warm pools must be deleted.
Prevent all deletion
The Auto Scaling group cannot be deleted.
Placement group Info
Choose a placement group to determine how instances are placed on underlying hardware.
None
Create a placement group
Cancel
Skip to review
Previous
Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

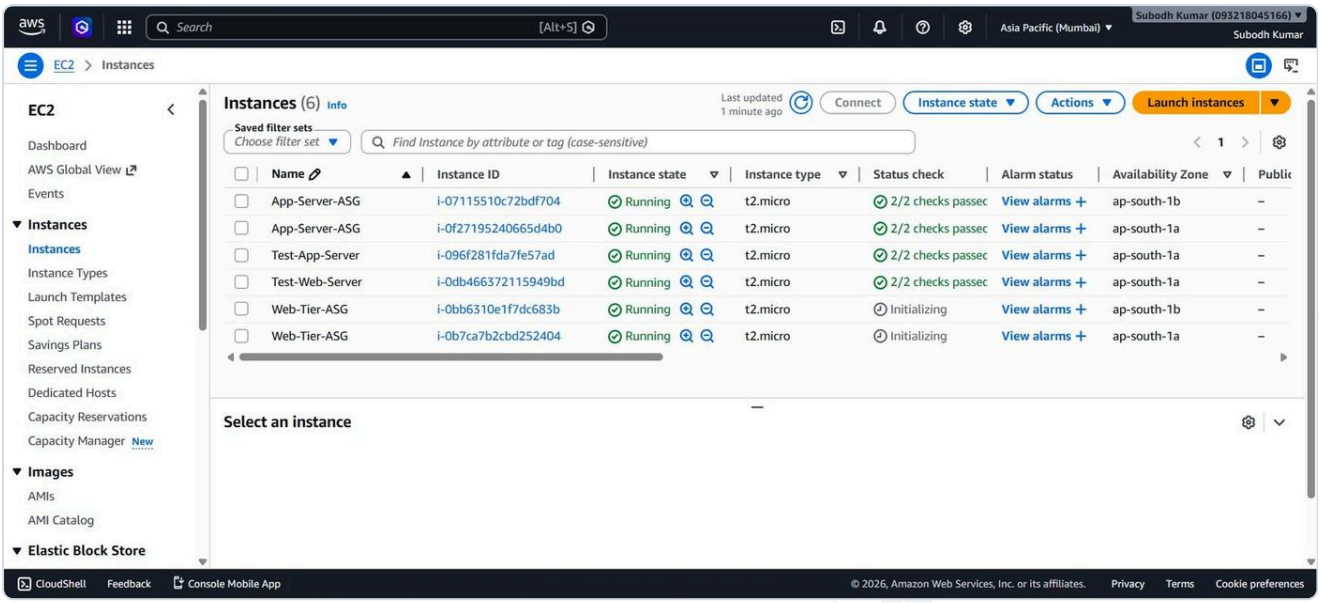

EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1
Choose launch template
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Add tags - optional Info
Add tags to help you search, filter, and track your Auto Scaling group across AWS. You can also choose to automatically add these tags to instances when they are launched.
You can optionally choose to add tags to instances (and their attached EBS volumes) by specifying tags in your launch template. We recommend caution, however, because the tag values for instances from your launch template will be overridden if there are any duplicate keys specified for the Auto Scaling group.
Tags (1)
Key
Name
Value - optional
App-Server-ASG
Tag new instances
Remove
Add tag
49 remaining
Cancel
Previous
Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences





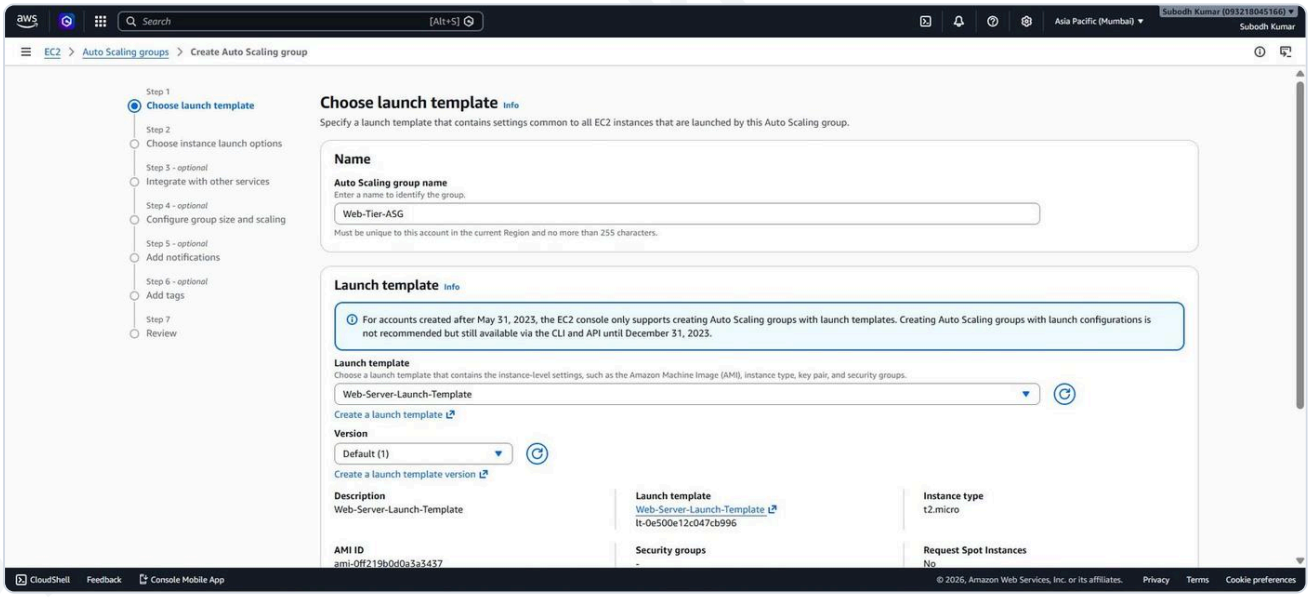
Keep other services as default and Create Auto scaling group.

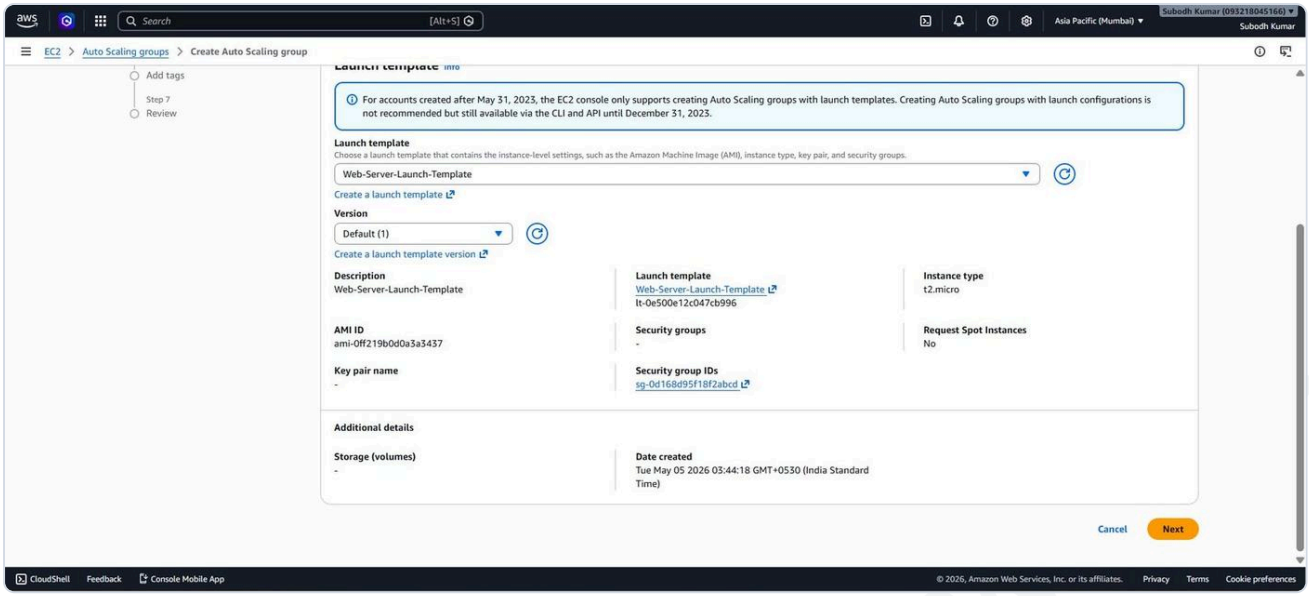
Now if you **check** the **instance list** you Can see that **two App Servers** are **creating**

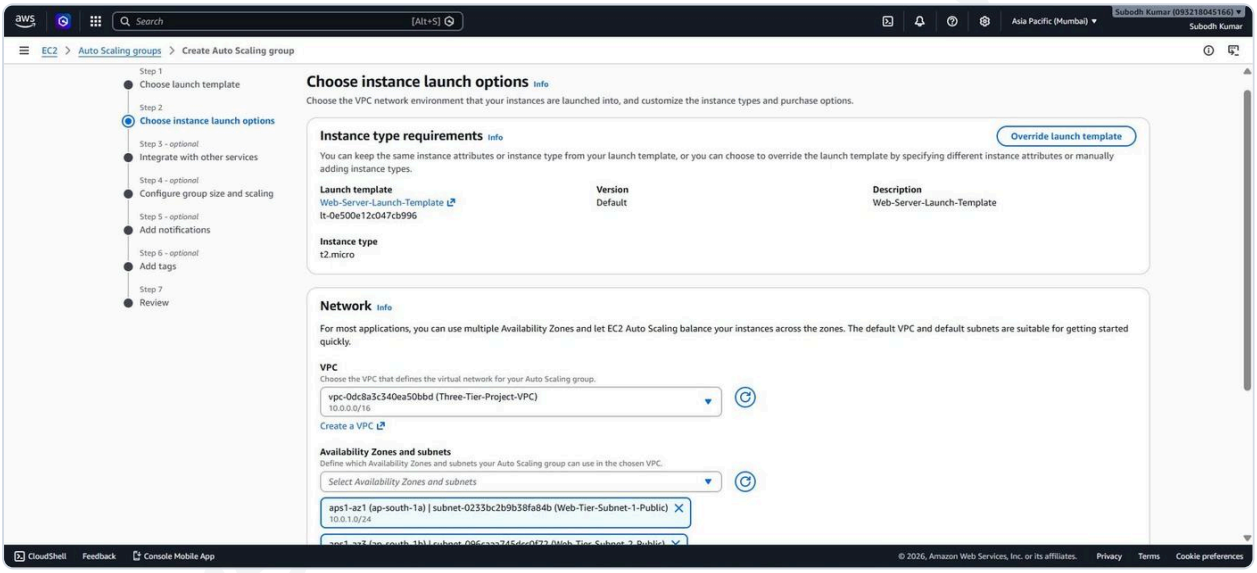


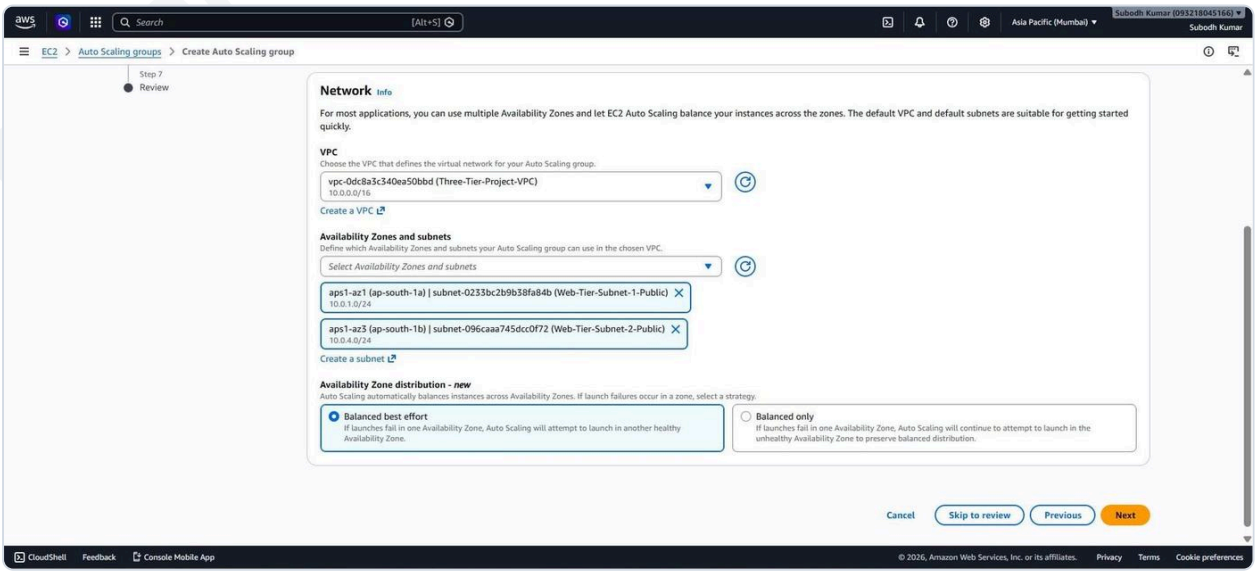
Auto Scaling Group for Web Tier

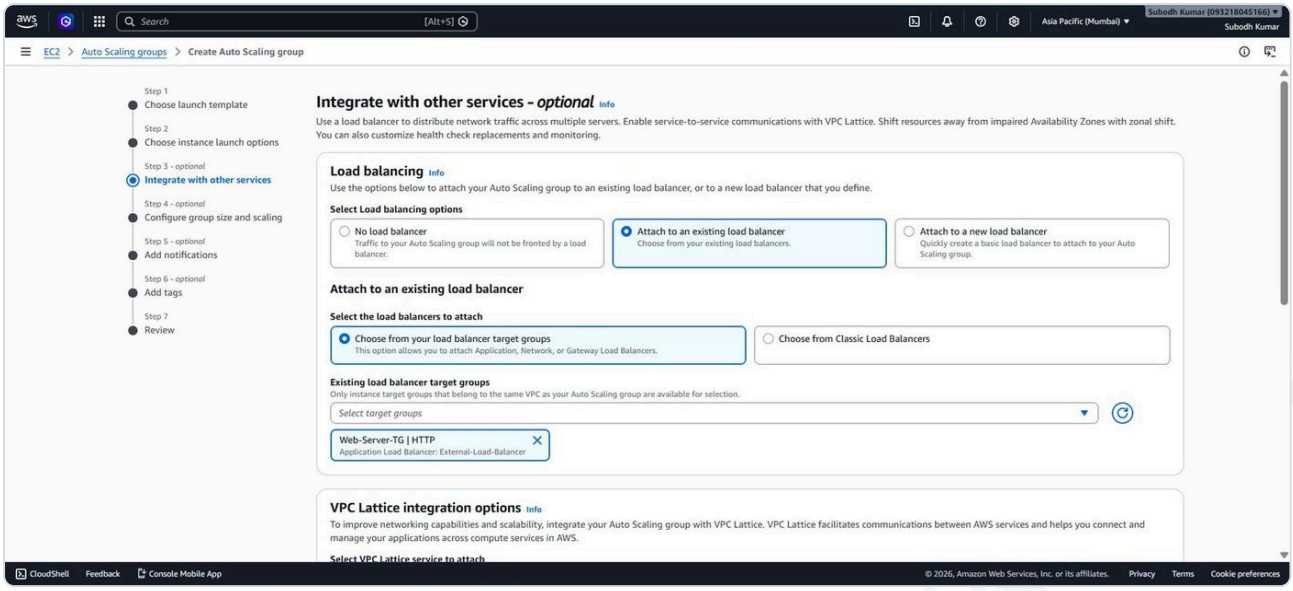
The process is the same. Choose components related to Web Tier.

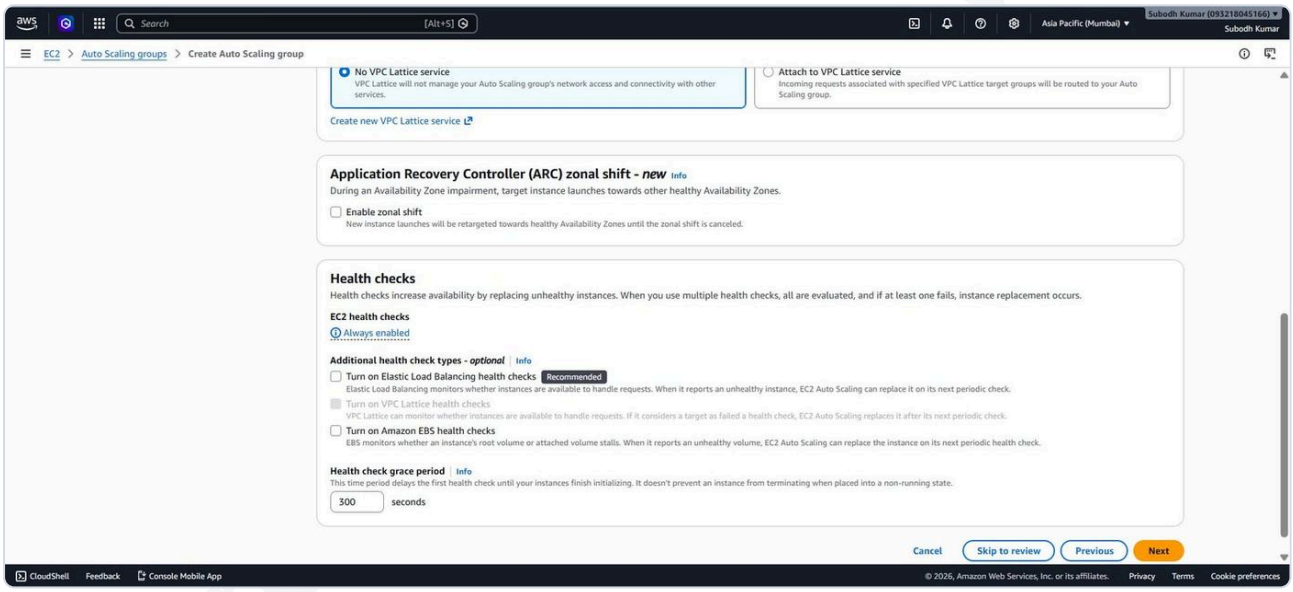


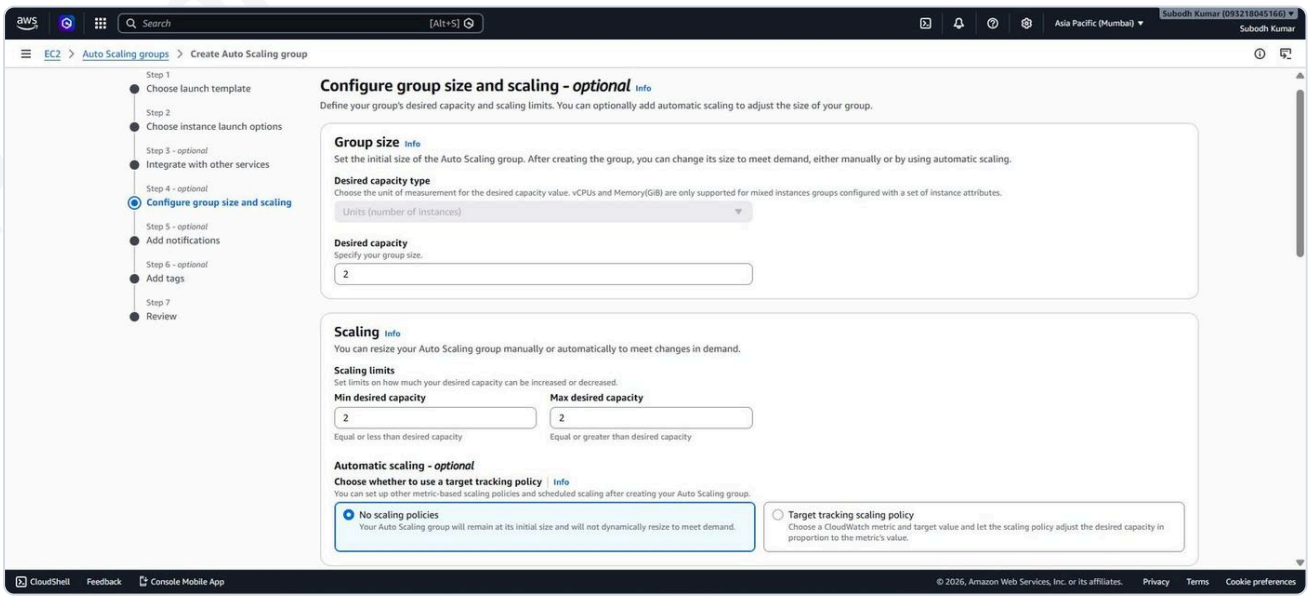

EC2 > Auto Scaling groups > Create Auto Scaling group
Add tags
Step 7
Review
For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.
Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
Web-Server-Launch-Template
Create a launch template
Version
Default (1)
Create a launch template version
Description
Web-Server-Launch-Template
Launch template
Web-Server-Launch-Template
lt-0e500e12c047cb996
Instance type
t2.micro
AMI ID
ami-0ff219b0d0a3a3437
Security groups
-
Request Spot Instances
No
Key pair name
-
Security group IDs
sg-0d168d95f18f2abcd
Additional details
Storage (volumes)
-
Date created
Tue May 05 2026 03:44:18 GMT+0530 (India Standard Time)
Cancel
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

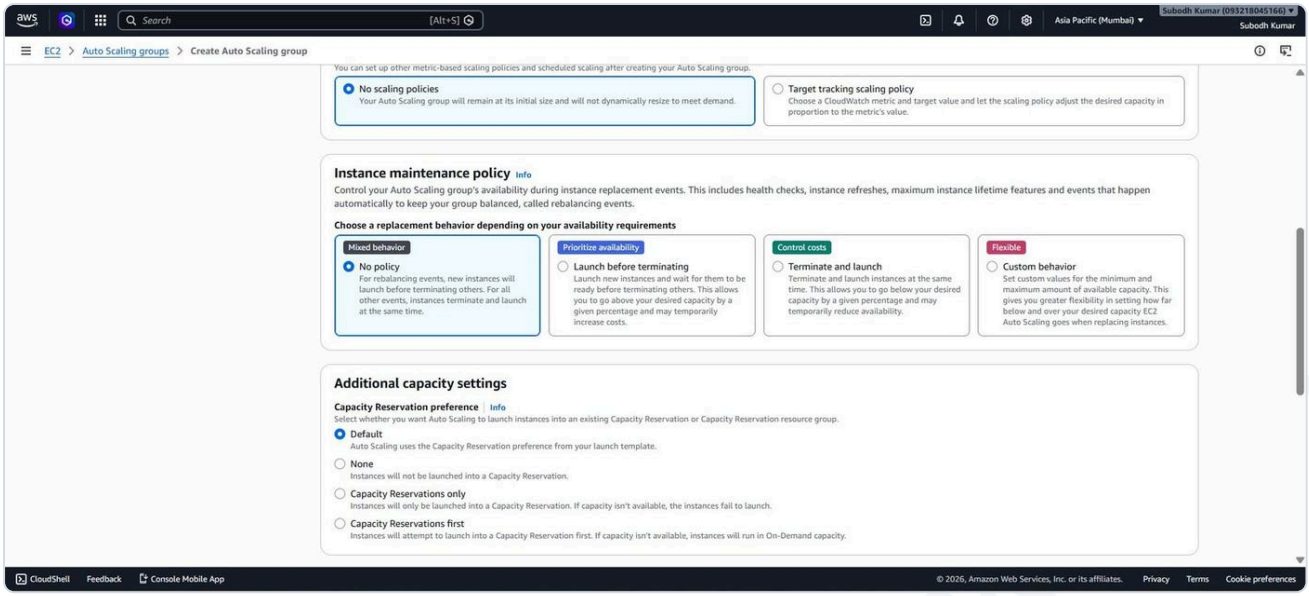

EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1
Choose launch template
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Choose instance launch options
Choose the VPC network environment that your instances are launched into, and customize the instance types and purchase options.
Instance type requirements
Override launch template
You can keep the same instance attributes or instance type from your launch template, or you can choose to override the launch template by specifying different instance attributes or manually adding instance types.
Launch template
Web-Server-Launch-Template
lt-0e500e12c047cb996
Version
Default
Description
Web-Server-Launch-Template
Instance type
t2.micro
Network
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
aps1-az1 (ap-south-1a) | subnet-0233bc2b9b38fa84b (Web-Tier-Subnet-1-Public)
10.0.1.0/24
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

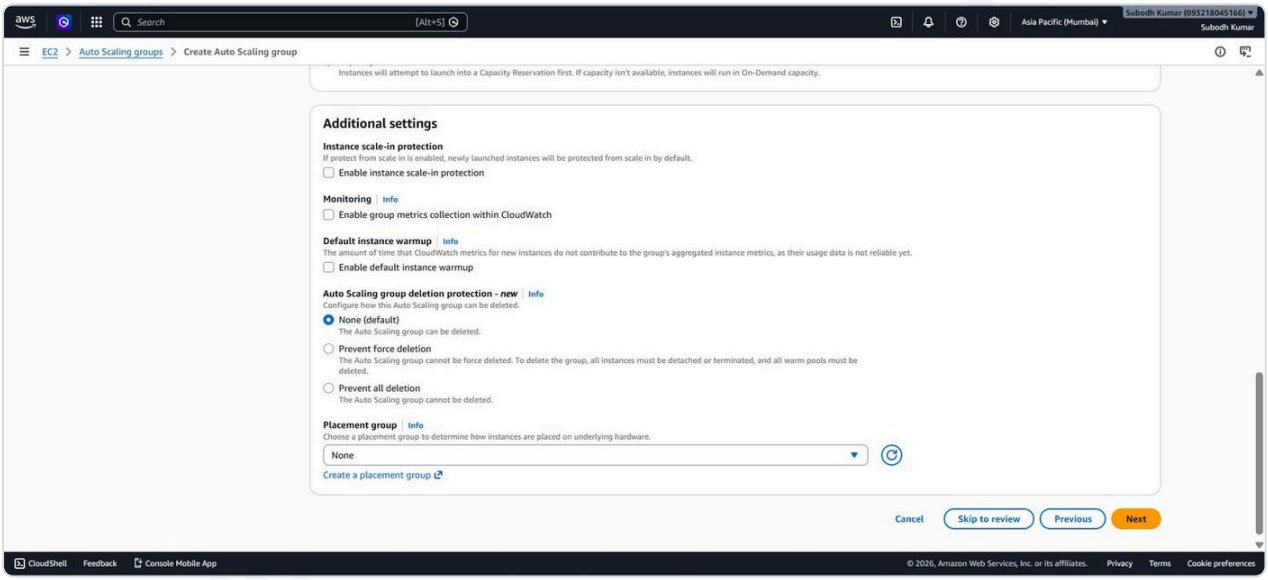

EC2 > Auto Scaling groups > Create Auto Scaling group
Step 7
Review
Network
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.
VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
10.0.0.0/16
Create a VPC
Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets
aps1-az1 (ap-south-1a) | subnet-0233bc2b9b38fa84b (Web-Tier-Subnet-1-Public)
10.0.1.0/24
aps1-az3 (ap-south-1b) | subnet-096caaa745dcc0f72 (Web-Tier-Subnet-2-Public)
10.0.4.0/24
Create a subnet
Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.
Balanced best effort
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.
Balanced only
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.
Cancel
Skip to review
Previous
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

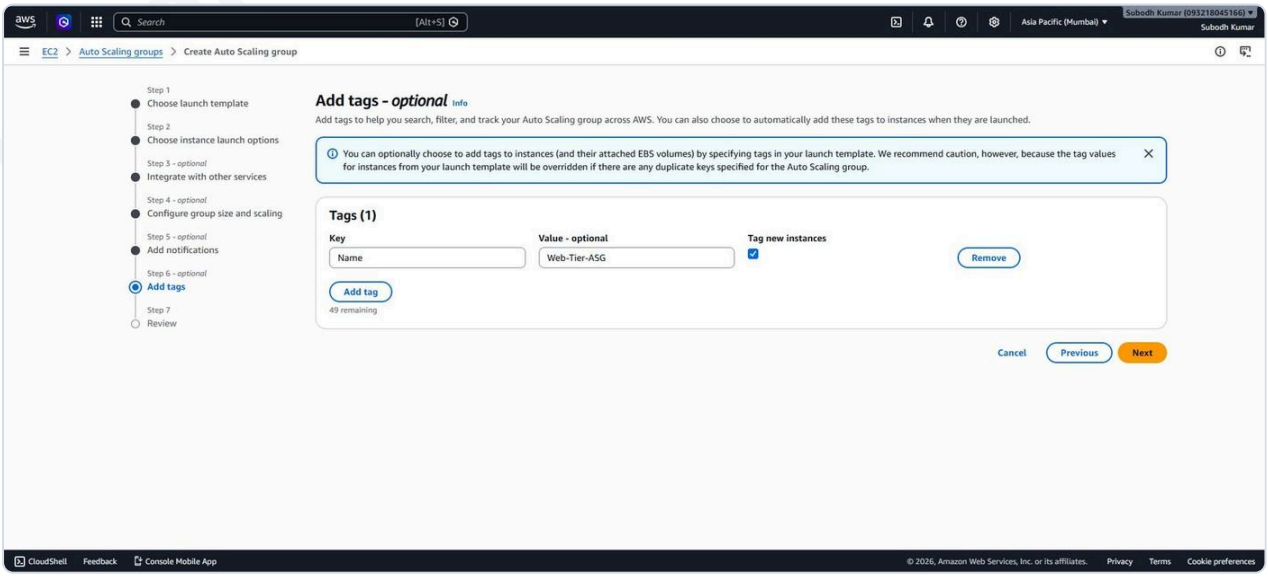

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review
Integrate with other services - optional Info
Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.
Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
Select Load balancing options
No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
Attach to an existing load balancer
Choose from your existing load balancers.
Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.
Choose from Classic Load Balancers
Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.
Select target groups
Web-Server-TG | HTTP
Application Load Balancer: External-Load-Balancer
VPC Lattice integration options Info
To improve networking capabilities and scalability, integrate your Auto Scaling group with VPC Lattice. VPC Lattice facilitates communications between AWS services and helps you connect and manage your applications across compute services in AWS.
Select VPC Lattice service to attach
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

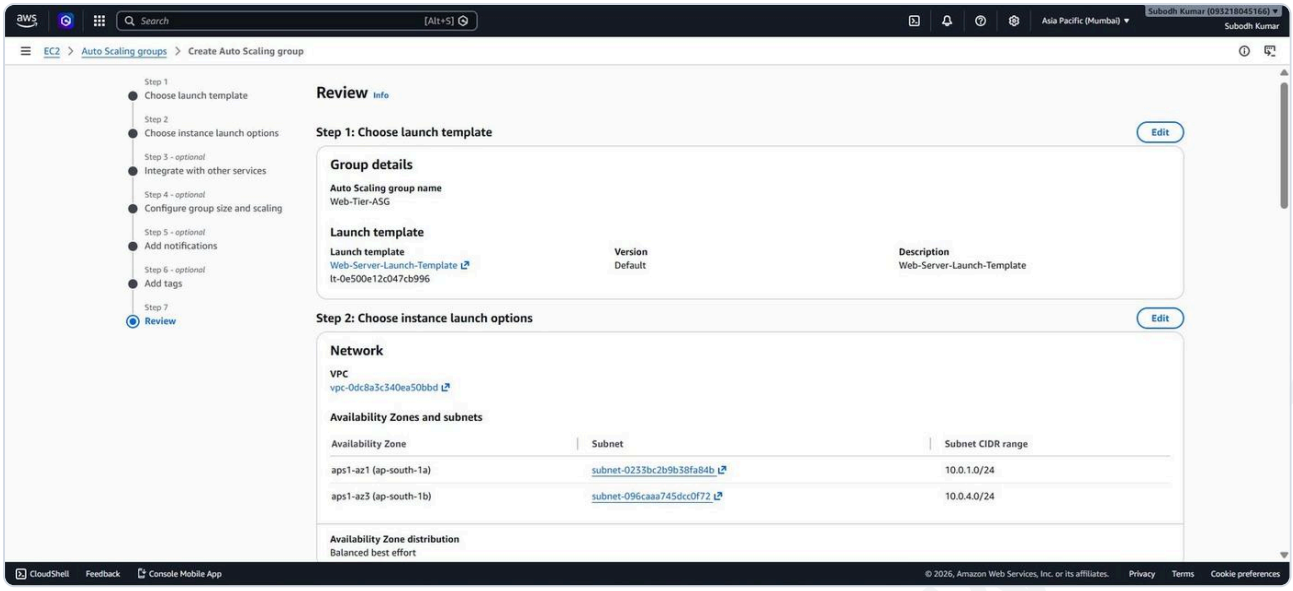

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
No VPC Lattice service
VPC Lattice will not manage your Auto Scaling group's network access and connectivity with other services.
Attach to VPC Lattice service
Incoming requests associated with specified VPC Lattice target groups will be routed to your Auto Scaling group.
Create new VPC Lattice service
Application Recovery Controller (ARC) zonal shift - new Info
During an Availability Zone impairment, target instance launches towards other healthy Availability Zones.
Enable zonal shift
New instance launches will be retargeted towards healthy Availability Zones until the zonal shift is canceled.
Health checks
Health checks increase availability by replacing unhealthy instances. When you use multiple health checks, all are evaluated, and if at least one fails, instance replacement occurs.
EC2 health checks
Always enabled
Additional health check types - optional Info
Turn on Elastic Load Balancing health checks Recommended
Elastic Load Balancing monitors whether instances are available to handle requests. When it reports an unhealthy instance, EC2 Auto Scaling can replace it on its next periodic check.
Turn on VPC Lattice health checks
VPC Lattice can monitor whether instances are available to handle requests. If it considers a target as failed a health check, EC2 Auto Scaling replaces it after its next periodic check.
Turn on Amazon EBS health checks
EBS monitors whether an instance's root volume or attached volume stalls. When it reports an unhealthy volume, EC2 Auto Scaling can replace the instance on its next periodic health check.
Health check grace period Info
This time period delays the first health check until your instances finish initializing. It doesn't prevent an instance from terminating when placed into a non-running state.
300 seconds
Cancel Skip to review Previous Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

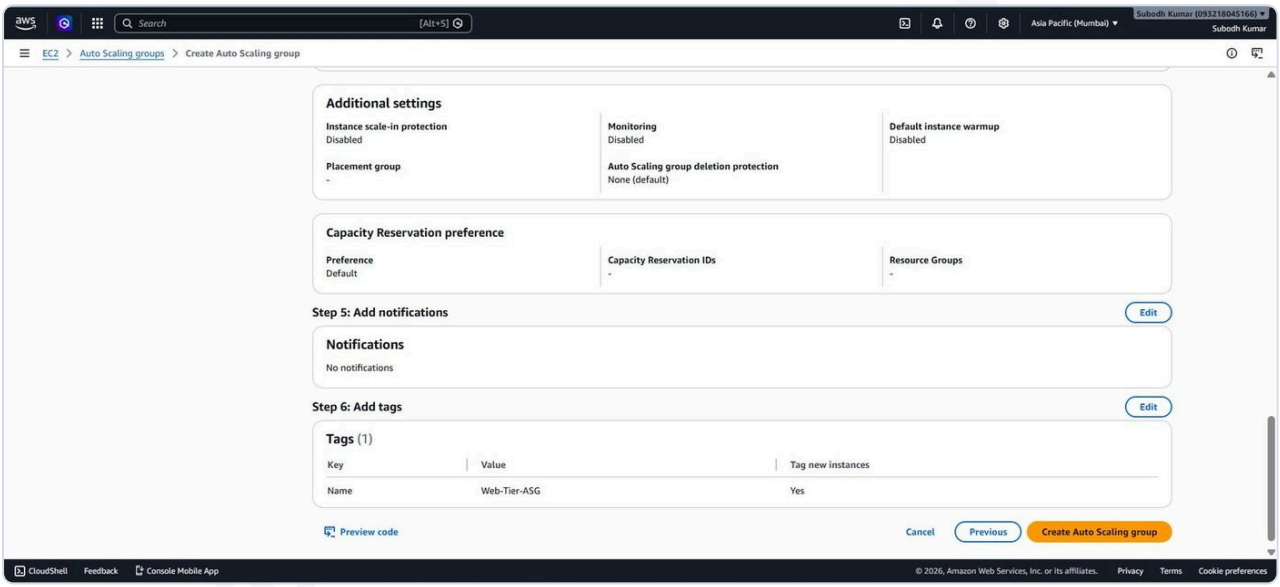

Search [Alt+S]
Asia Pacific (Mumbai)
Subodh Kumar
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review
Configure group size and scaling - optional Info
Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.
Group size Info
Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.
Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.
Units (number of instances)
Desired capacity
Specify your group size.
2
Scaling Info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.
Scaling limits
Set limits on how much your desired capacity can be increased or decreased.
Min desired capacity
2
Equal or less than desired capacity
Max desired capacity
2
Equal or greater than desired capacity
Automatic scaling - optional
Choose whether to use a target tracking policy Info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.
No scaling policies
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.
Target tracking scaling policy
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
No scaling policies
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.
Target tracking scaling policy
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.
Instance maintenance policy Info
Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.
Choose a replacement behavior depending on your availability requirements
Mixed behavior
No policy
For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.
Prioritize availability
Launch before terminating
Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.
Control costs
Terminate and launch
Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.
Flexible
Custom behavior
Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.
Additional capacity settings
Capacity Reservation preference Info
Select whether you want Auto Scaling to launch instances into an existing Capacity Reservation or Capacity Reservation resource group.
Default
Auto Scaling uses the Capacity Reservation preference from your launch template.
None
Instances will not be launched into a Capacity Reservation.
Capacity Reservations only
Instances will only be launched into a Capacity Reservation. If capacity isn't available, the instances fail to launch.
Capacity Reservations first
Instances will attempt to launch into a Capacity Reservation first. If capacity isn't available, instances will run in On-Demand capacity.
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

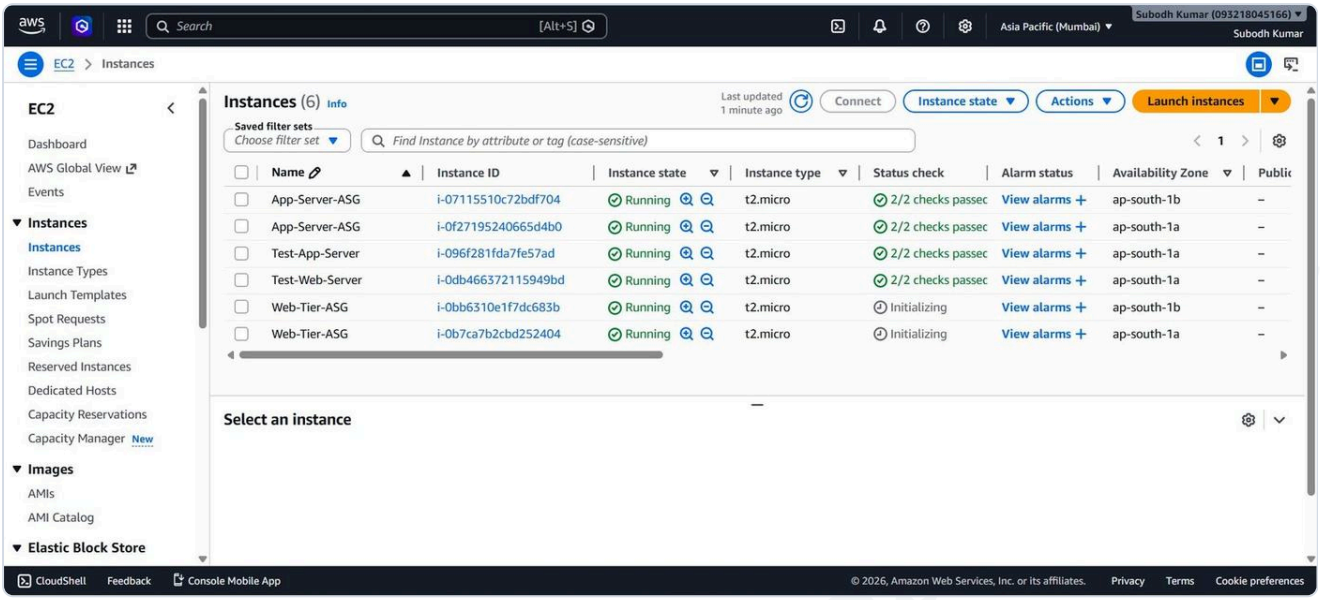

EC2 > Auto Scaling groups > Create Auto Scaling group
Additional settings
Instance scale-in protection
If protect from scale in is enabled, newly launched instances will be protected from scale in by default.
Enable instance scale-in protection
Monitoring Info
Enable group metrics collection within CloudWatch
Default instance warmup Info
The amount of time that CloudWatch metrics for new instances do not contribute to the group's aggregated instance metrics, as their usage data is not reliable yet.
Enable default instance warmup
Auto Scaling group deletion protection - new Info
Configure how this Auto Scaling group can be deleted.
None (default)
The Auto Scaling group can be deleted.
Prevent force deletion
The Auto Scaling group cannot be force deleted. To delete the group, all instances must be detached or terminated, and all warm pools must be deleted.
Prevent all deletion
The Auto Scaling group cannot be deleted.
Placement group Info
Choose a placement group to determine how instances are placed on underlying hardware.
None
Create a placement group
Cancel
Skip to review
Previous
Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1
Choose launch template
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Add tags - optional Info
Add tags to help you search, filter, and track your Auto Scaling group across AWS. You can also choose to automatically add these tags to instances when they are launched.
You can optionally choose to add tags to instances (and their attached EBS volumes) by specifying tags in your launch template. We recommend caution, however, because the tag values for instances from your launch template will be overridden if there are any duplicate keys specified for the Auto Scaling group.
Tags (1)
Key
Name
Value - optional
Web-Tier-ASG
Tag new instances
Remove
Add tag
49 remaining
Cancel
Previous
Next
CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review
Review Info
Step 1: Choose launch template
Edit
Group details
Auto Scaling group name
Web-Tier-ASG
Launch template
Launch template
Web-Server-Launch-Template
lt-0e500e12c047cb996
Version
Default
Description
Web-Server-Launch-Template
Step 2: Choose instance launch options
Edit
Network
VPC
vpc-0dc8a3c340ea50bbd
Availability Zones and subnets
Availability Zone
Subnet
Subnet CIDR range
aps1-az1 (ap-south-1a)
subnet-0233bc2b9b38fa84b
10.0.1.0/24
aps1-az3 (ap-south-1b)
subnet-096caaa745dcc0f72
10.0.4.0/24
Availability Zone distribution
Balanced best effort
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


EC2 > Auto Scaling groups > Create Auto Scaling group
Additional settings
Instance scale-in protection
Disabled
Monitoring
Disabled
Default instance warmup
Disabled
Placement group
-
Auto Scaling group deletion protection
None (default)
Capacity Reservation preference
Preference
Default
Capacity Reservation IDs
-
Resource Groups
-
Step 5: Add notifications
Edit
Notifications
No notifications
Step 6: Add tags
Edit
Tags (1)
Key
Value
Tag new instances
Name
Web-Tier-ASG
Yes
Preview code
Cancel
Previous
Create Auto Scaling group
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

After you are creating it you can see 4 instance and test instance on the list. Now you can delete the test instances .

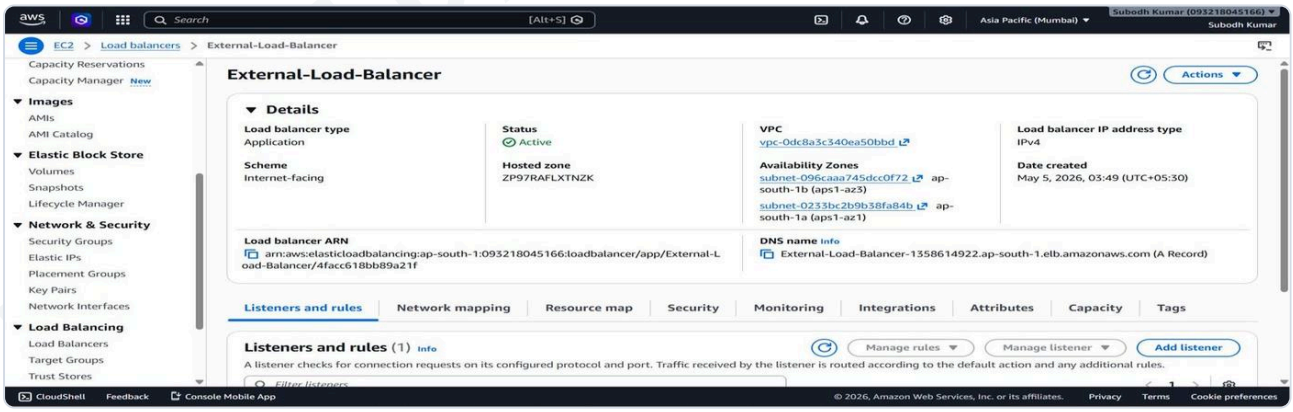


Cool You did a great job

Now We Can access our Web application.

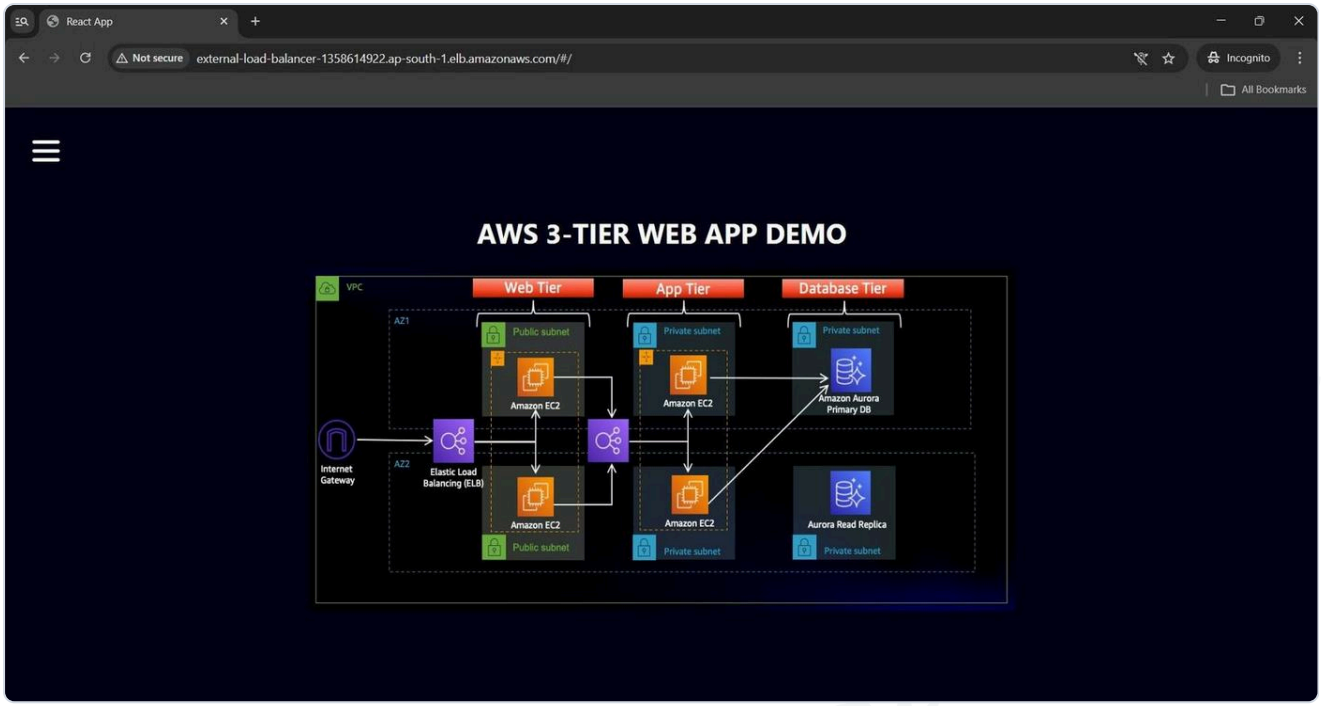
Step 14 Access the Application

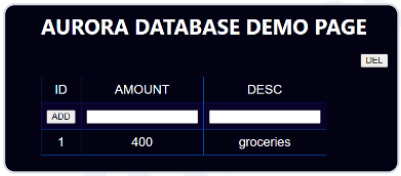
Go to **EC2 > Load Balancers**. Select **External Load Balancer**. Copy the **DNS name** of it.



Paste It into your **Browser**. You will **see** the **Application** is **running**

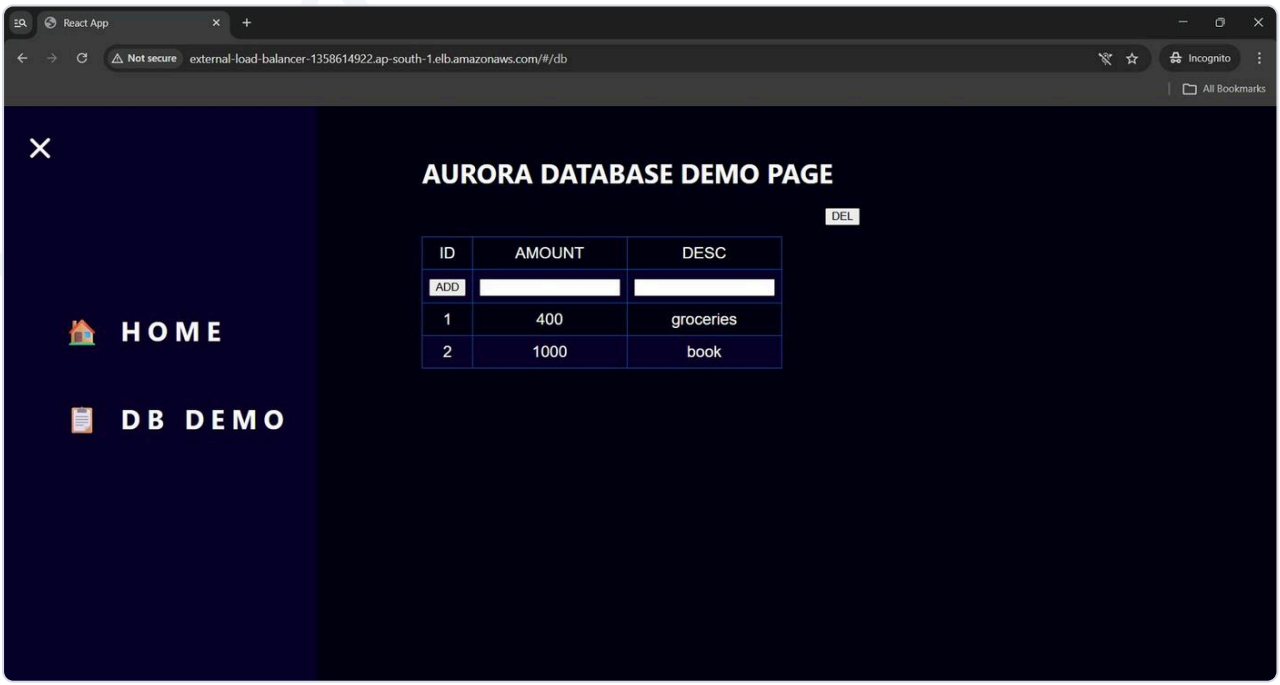
> **Note:** If the page does not open, please open it in Incognito mode.





Go to the **DB DEMO** section and check whether the DB is working.

Here you can see the example data we entered when setting up the DB, which means the application is receiving data from the DB.



You can add and delete items

Congratulations!!! You have successfully implemented Three Tier Web Application on AWS

> **Note:** In this guide we use the External Load Balancer DNS name as the URL, but it is not user friendly. We can purchase a domain name from a domain name service and set it up using **Route 53** to access the website with that domain name.

Step 15 Cleaning the Infrastructure

Since some services we used charge cost we have to delete our demo project.

Let's **delete** them according to the following order.

1. Auto Scaling Groups
2. In Application Load Balancers delete listeners first and then Load balancers
3. Target Groups
4. Launch Templates
5. AMIs
6. Snapshots
7. EC2 Instances
8. Aurora Database (First go to modify and disable delete protection. Then when deleting do not create snapshot. Tick I acknowledge)
9. NAT gateways
10. Elastic Ips
11. Route Tables (Remove all subnet associations first)
12. Internet Gateways
13. VPC
14. S3 Bucket
15. IAM Role

Prepared by Sukhada Wagale

THANK YOU!

for going through this project documentation

Throughout this project, I interacted with the following components on AWS.

1. EC2 — Amazon Elastic Compute Cloud (EC2)
2. S3 — Amazon Simple Storage Service (S3)
3. VPC — Virtual Private Cloud (VPC)
4. ASG — Auto Scaling Group (ASG)
5. ALB — Application Load Balancer (ALB)
6. IAM — Identity and Access Management (IAM)
7. RDS — Amazon Relational Database Service (RDS)
8. AMI — Amazon Machine Image (AMI)
9. LT — Launch Templates
10. TG — Target Groups
11. IGW — Internet Gateway (IGW)
12. NAT — NAT Gateway
13. RT — Route Table
14. SG — Security Groups

A big thank you to everyone who supported and guided me throughout this journey.
This project has been a valuable learning experience in building a **highly available, secure, scalable** and **fault tolerant** three-tier web application on AWS.

PRESENTED BY

Sukhada Wagale

Associate Cloud Engineer · AWS Certified Cloud Practitioner

AWS · Docker · Linux · Load Balancing · Auto Scaling · DevOps

GitHub: github.com/sukhada30github

LinkedIn: linkedin.com/in/sukhada-wagale-0961002b0

KEEP LEARNING · KEEP BUILDING · KEEP GROWING!